RKNN2 SDK Development Guide

ID: RK-KF-YF-417

Release Version: V2.5.0

Release Date: 2026-09-05

Security Level: □Top-Secret □Secret □Internal ■Public

Disclaimer

This document is provided on an "as is" basis. Rockchip Electronics Co., Ltd. ("the Company", hereinafter the same) makes no express or implied statements or warranties regarding the accuracy, reliability, completeness, merchantability, fitness for a particular purpose, or non-infringement of any statements, information, and content contained in this document. This document serves only as a reference for usage guidance.

Due to product version upgrades or other reasons, this document may be updated or modified from time to time without prior notice.

Trademark Statement

"Rockchip", "瑞芯微", "瑞芯" shall be Rockchip’s registered trademarks and owned by Rockchip. All the other trademarks or registered trademarks mentioned in this document shall be owned by their respective owners.

All rights reserved. ©2026. Rockchip Electronics Co., Ltd.

Beyond the scope of fair use, neither any entity nor individual shall extract, copy, or distribute this document in any form in whole or in part without the written approval of Rockchip.

Rockchip Electronics Co., Ltd.

No.18 Building, A District, No.89, software Boulevard Fuzhou, Fujian,PRC

Website: www.rock-chips.com

Customer service Tel: +86-4007-700-590

Customer service Fax: +86-591-83951833

Customer service e-Mail: fae@rock-chips.com

Preface

Overview

This document is intended for users of the RKNN2 SDK. It systematically introduces the SDK's model conversion, quantization, evaluation, and on-device deployment methods. The document is organized into five parts: fundamentals, conversion and evaluation, on-device deployment, advanced features, and diagnostics and optimization.

Intended Audience

This document is primarily intended for the following engineers:

- Technical support engineers
- Software development engineers

Revision History

Version	Modified By	Date	Description of Changes	Approved By
V1.6.0	HPC	2023-11-28	Initial version	熊伟
V2.0.0-beta0	HPC	2024-03-15	1. Add RK3576 content; 2. Add sparse inference; 3. Improve the Matmul API interface, adding quantization parameters, dynamic shape input, and iommu_domain_id functions	熊伟
V2.1.0	HPC	2024-08-02	1. Add RV1103B and RK2118 content; 2. Improve the rknn_server description; 3. Improve the Matmul API interface, adding more data types, layout types, and quantization methods	熊伟
V2.2.0	HPC	2024-09-04	1. Revised certain images and content descriptions	熊伟
V2.3.0	HPC	2024-11-06	1. Add support for installing RKNN-Toolkit2 via pip source; 2. Add support for the ARM64 version of RKNN-Toolkit2	熊伟
V2.3.2	HPC	2025-04-03	Add support for RV1126B	熊伟
V2.4.2	HPC	2026-03-19	Update the version number	熊伟
V2.5.0	HPC	2026-09-05	Update the encryption functionality	熊伟

Contents

RKNN2 SDK Development Guide

1. Overview and Development Preparation

1.1 RKNN2 SDK Overview

1.1.1 SDK Positioning and Typical Application Scenarios

1.1.1.1 Overview of the RKNN Software Stack

1.1.1.2 Introduction to RKNN-Toolkit2 Features

1.1.1.3 Introduction to RKNN Runtime Features

1.1.2 Supported Hardware Platforms and Deployment Modes

1.1.3 Terms, Abbreviations, and Document Conventions

1.2 Versions and Development Environment

1.2.1 Installing RKNN-Toolkit2

1.2.1.1 Installation via Docker

1.2.1.1.1 Install Docker

1.2.1.1.2 Image Preparation

1.2.1.1.3 Querying Image Information

1.2.1.1.4 Running the Image

1.2.1.1.5 Running the Demo

1.2.1.2 Installation via Pip

1.2.1.2.1 Install Python Environment

1.2.1.2.2 Install Conda Tool

1.2.1.2.3 Create RKNN-Toolkit2 Conda Environment

1.2.1.2.4 Install RKNN-Toolkit2

1.2.2 NPU Device Environment Setup

1.2.2.1 NPU Driver Version Confirmation

1.2.2.2 NPU Device Connection Environment Confirmation

1.2.2.3 RKNN Server Installation and Update

1.2.2.3.1 RK356X/RK3576/RK3588/RV1126B Platform

1.2.2.3.2 RV1103/RV1103B/RV1106/RV1106B Platform

1.2.2.4 View Detailed RKNN Server Logs

1.2.2.4.1 Android System

1.2.2.4.2 Linux System

1.2.3 Environment Verification

1.3 Development Process and Interface Selection

1.3.1 General Model Development Process
1.3.2 Interface Selection
1.3.3 Model Conversion and Deployment Artifacts
2. Model Conversion, Quantization, and Evaluation
2.1 General Model Conversion
2.1.1 Model Preparation
2.1.2 Conversion Configuration
2.1.3 Model Loading and Building
2.1.4 Exporting the RKNN Model
2.1.5 Conversion Result Verification
2.1.6 Model Conversion Tool: rknn_convert
2.1.7 Quantization Feature of RKNN-Toolkit2
2.1.8 Model Pruning
2.1.9 ONNX Model Editing
2.1.9.1 Interface description
2.1.9.2 Transformation formula description
2.1.9.3 Transformation formula example
2.2 Model Quantization
2.2.1 Basic Concepts of Quantization
2.2.1.1 Quantization Definition
2.2.1.2 Quantization Calculation
2.2.1.3 Quantization Granularity
2.2.1.4 Quantization Algorithms
2.2.2 Overview of the Quantization Process
2.2.3 Internal Quantization Process
2.2.3.1 Data Types
2.2.3.2 Internal Quantization Algorithms
2.2.3.3 Calibration Data Preparation
2.2.3.4 Concrete Example
2.2.4 Hybrid Quantization
2.2.4.1 Usage of Hybrid Quantization
2.2.4.2 Process of Hybrid Quantization Usage
2.2.4.3 Automatic Hybrid Quantization
2.2.5 Quantization-Aware Training (QAT)
2.2.5.1 Introduction to QAT

2.2.5.2 QAT Principle
2.2.5.3 QAT Usage Guidelines
2.2.5.4 QAT Implementation Example and Configuration Instructions
2.2.5.5 Supported Operators in QAT
2.2.5.6 Handling of Floating-Point Operators in QAT Model
2.2.5.7 Summary of QAT Experience
2.3 Model Evaluation
2.3.1 Model Inferencing
2.3.2 Model Accuracy Analysis
2.3.3 Model performance evaluation
2.3.4 Model memory evaluation
3. Board Deployment
3.1 Runtime Deployment Basics
3.1.1 Model and Context Lifecycle
3.1.2 Tensors and Data Layout Formats
3.1.3 Input/Output Memory
3.1.4 Synchronous and Asynchronous Inference
3.1.5 Error Handling and Resource Release
3.1.6 RKNN Runtime Zero-Copy Interface
3.1.6.1 Zero-Copy Introduction
3.1.6.2 C API Zero-Copy Process
3.1.6.3 C API Zero-Copy Usage
3.2 Deploying General Models with the C API
3.2.1 Complete Invocation Process
3.2.2 Model Initialization
3.2.3 Input/Output Preparation
3.2.4 Running Inference
3.2.5 Output Processing
3.2.6 Generating Deployment C Code
3.3 Toolkit Lite Python Deployment
3.3.1 Runtime Environment and Installation
3.3.2 Basic Deployment Process
3.3.3 Main Function Interfaces
3.3.4 Reference Example
4. Advanced Features and Extensions

4.1 Advanced Runtime Features

4.1.1 Matrix Multiplication API

4.1.1.1 Features and Usage

4.1.1.2 Matmul API Flow Chart

4.1.1.3 Advanced usage of Matmul API

4.1.1.3.1 Matrix multiplication with specified quantization parameters

4.1.1.3.2 Matrix multiplication of dynamic shape input

4.1.1.4 The data layout for high-performance

4.1.1.4.1 Matrix specification restrictions

4.1.2 NPU Multi-Core Configuration

4.1.2.1 Multi-Core Operation Configuration Method

4.1.2.2 Check Multi-Core Running Status

4.1.2.3 Multi-Core Performance Improvement Tips

4.1.3 Dynamic Shape

4.1.3.1 Dynamic Shape Introduction

4.1.3.2 RKNN SDK Version and Platform Requirements

4.1.3.3 Generate RKNN Model of Dynamic Shape

4.1.3.4 C API Deployment

4.1.3.4.1 General API

4.1.3.4.2 Zero-Copy API

4.1.4 Multi-Batch Usage

4.1.4.1 Multi-Batch Principle

4.1.4.2 Multi-Batch Usage

4.1.4.3 Multi-Batch Input and Output Settings

4.1.5 RK3588/RK3576 NPU SRAM

4.1.5.1 RKNPU Environmental Requirements

4.1.5.1.1 Kernel Environment Requirements

4.1.5.1.2 RKNN SDK Version Requirments

4.1.5.2 Usage

4.1.5.3 Debug

4.1.5.3.1 Query SRAM Enable

4.1.5.3.2 Query SRAM Usage

4.1.5.3.3 Query SRAM Size Through RKNN API

4.1.5.3.4 Query Model SRAM Occupancy

4.1.6 Cacheable Memory Consistency

4.1.6.1 Direction of cacheable memory synchronization

4.1.6.2 Synchronizing cacheable memory

4.1.7 Model Sparse Inference

4.1.7.1 Sparsity Principle

4.1.7.2 Training the sparse model

4.1.7.3 RKNN Sparse Inference Usage

4.1.7.4 RKNN sparse inference restrictions

4.2 Extensions and Security

4.2.1 Custom Operators

4.2.1.1 Introduction

4.2.1.2 Overall Process

4.2.1.2.1 Use RKNN-Toolkit2 to register custom operators and export RKNN models

4.2.1.2.2 C code implementation of the custom operator, load, register and execute through RKNN API.

4.2.1.2.3 Use RKNN-Toolkit2 device connected inference or accuracy analysis

4.2.1.3 Python Process

4.2.1.4 C API Deployment

4.2.1.4.1 Initialize Custom Operator Structure

4.2.1.4.1.1 Init Callback Function

4.2.1.4.1.2 Prepare Callback Function

4.2.1.4.1.3 Compute Callback Function

4.2.1.4.1.4 Destroy Callback Function

4.2.1.4.2 Register Custom Operator

4.2.1.4.3 Model Inference

4.2.1.4.4 Connected Device Accuracy Analysis

4.2.2 Model Encryption Deployment

4.2.2.1 Encryption on PC/Server

4.2.2.2 Loading Encrypted Models on Board

4.2.2.3 Compatibility of Encrypted Models with External Memory Allocation/Reuse

5. Troubleshooting and Optimization

5.1 Accuracy Troubleshooting

5.1.1 Simulator Accuracy Troubleshooting

5.1.1.1 Simulator FP16 Accuracy

5.1.1.2 Simulator Quantization Accuracy
5.1.2 Runtime Accuracy Troubleshooting
5.1.2.1 Board-Connected Accuracy
5.1.2.2 Runtime Accuracy
5.2 Performance Analysis and Optimization
5.2.1 Process of Model Performance Optimization Analysis
5.2.1.1 Checking the Runtime Environment
5.2.1.2 Deployment Time Analysis
5.2.2 Performance Analysis
5.2.2.1 Obtain Profile information
5.2.2.2 Analyzing the time-consuming layer by layer
5.2.2.3 Analyze the impact of CPU operators
5.2.2.4 Analysis of NPU operator performance bottlenecks
5.2.3 Quantitative acceleration
5.2.4 Graph optimization
5.2.4.1 Convert non-NPU OP to NPU OP
5.2.4.2 Use hardware Fuse feature to design network or graph optimization
5.2.4.3 Subgraph transformation
5.2.4.4 "Combining similar terms" and "Extracting common factors"
5.2.5 Operator level optimization
5.2.5.1 DDR-friendly OP size design (not mandatory)
5.2.5.2 Design of high-utilization operator
5.2.5.3 Subgraph fusion
5.3 Memory Analysis and Optimization
5.3.1 Introduction to memory composition and analysis methods
5.3.1.1 Memory composition during RKNN model runtime
5.3.1.2 Model memory analysis method
5.3.2 How to use externally allocated memory
5.3.2.1 Externally allocate input and output memory
5.3.2.2 Externally allocated model memory
5.3.3 Internal Memory Reuse
5.3.4 Multi-threaded Reuse RKNN Context
5.3.5 Multiple resolution models share the same weights
5.4 FAQs and Error Handling
5.4.1 NPU SDK environment problem

5.4.2 RKNN-Toolkit2 installation issue

5.4.3 Model conversion parameters description

5.4.4 Deep-learning framework FAQs

5.4.4.1 Deep learning frameworks and corresponding versions supported by RKNN-Toolkit2

5.4.4.2 OP support list for each framework

5.4.4.3 FAQs about ONNX model conversion

5.4.4.4 FAQs about Pytorch model conversion

5.4.4.5 FAQs about Tensorflow model conversion

5.4.5 Model Quantification Issues

5.4.6 Model conversion issues

5.4.7 Description of inference with simulator or device

5.4.8 Common issue about model evaluation

5.4.9 C API usage FAQ

5.5 Related Resources

1. Overview and Development Preparation

1.1 RKNN2 SDK Overview

1.1.1 SDK Positioning and Typical Application Scenarios

1.1.1.1 OVERVIEW OF THE RKNN SOFTWARE STACK

The RKNN software stack enables users to quickly deploy AI models to Rockchip chips. The overall framework is as follows:

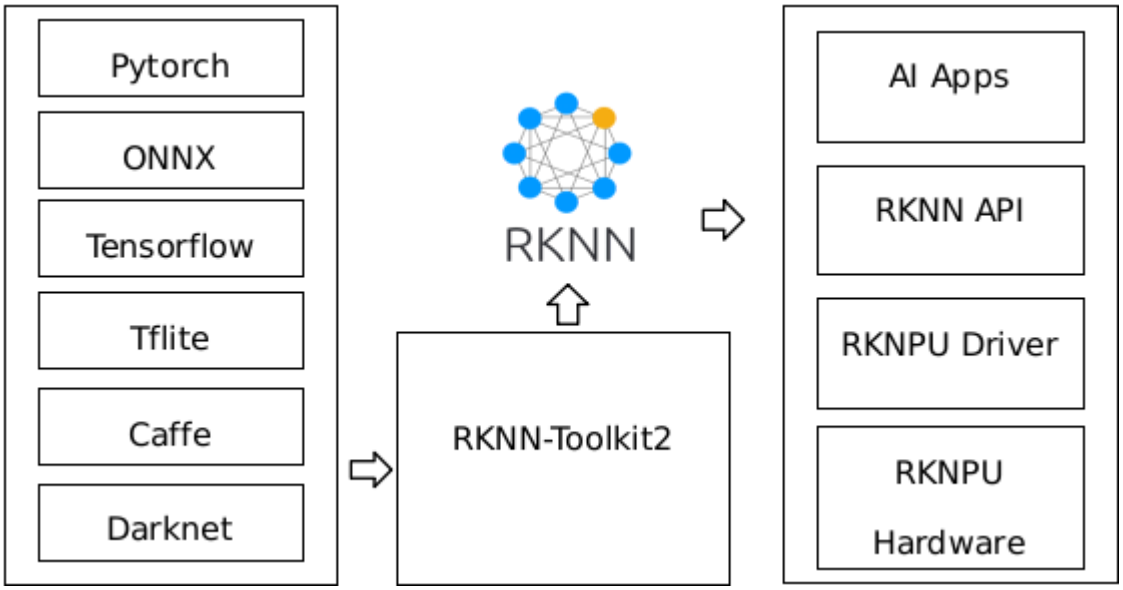


Figure 1-1 RKNN Software Stack

In order to use RKNPU, users need to first run the RKNN-Toolkit2 tool on the computer, convert the trained model into an RKNN format model, and then install it on the development board using the RKNN C API or Python.

1.1.1.2 INTRODUCTION TO RKNN-TOOLKIT2 FEATURES

RKNN-Toolkit2 is a development suite that allows users to convert, infer, and evaluate models on computer. The main diagram of RKNN-Toolkit2 is as follows:

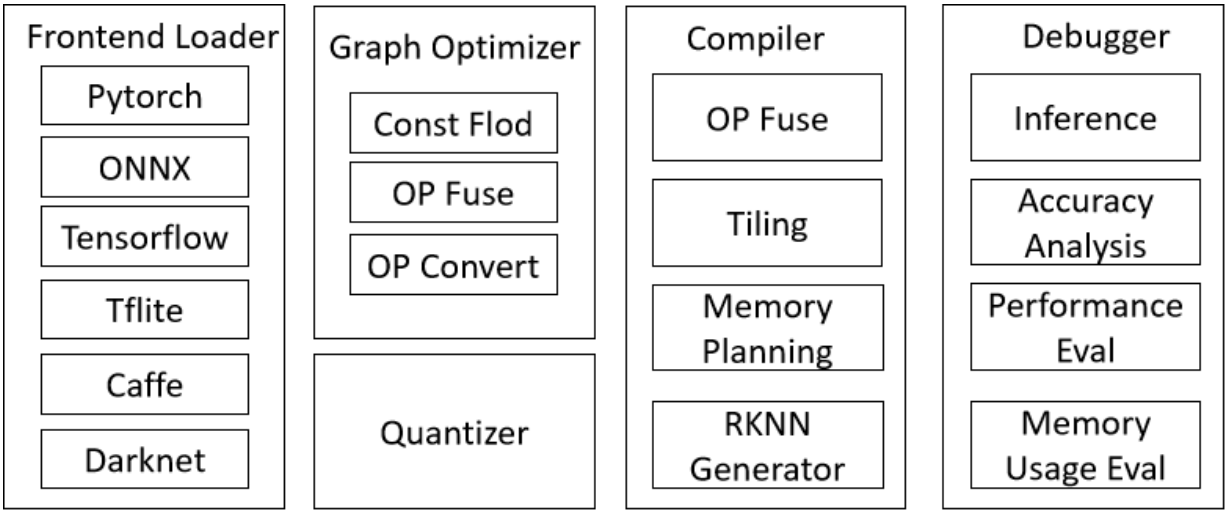


Figure 1-2 RKNN-Toolkit2 Software Diagram

The toolkit provides a convenient Python interface to perform the following functions:

1. Model Conversion: Support the conversion of PyTorch, ONNX, TensorFlow, TensorFlow Lite, Caffe, DarkNet, and other models into RKNN models.
2. Quantization Functionality: Support quantizing floating-point models into fixed-point models. It also supports mixed quantization functionality.
3. Model Inference: Distribute the RKNN model to the designated NPU device for inference and obtain the inference results; or simulate the NPU on the computer to run the RKNN model and obtain the inference results.
4. Performance and Memory Evaluation: Distributes the RKNN model to a designated NPU device to evaluate the performance and memory usage of the model when running on the actual device.
5. Quantization accuracy analysis: This function will give the cosine distance and Euclidean distance between the inference results of each layer of quantized model and that of floating-point model, so as to facilitate the analysis of how quantization errors occur and provide information for improving the accuracy of the quantization model.
6. Model Encryption: Encrypts the RKNN model using a hybrid encryption scheme of AES-256-CTR and RSA-OAEP to protect the model's intellectual property.

1.1.1.3 INTRODUCTION TO RKNN RUNTIME FEATURES

RKNN Runtime is responsible for loading the RKNN model and calling the NPU driver to infer the RKNN model on the NPU. When inferring the RKNN model, three processes are involved: pre-processing of the raw input data, running the model on the NPU, and post-processing of the output. Depending on the model input format and quantization method, RKNN Runtime provides two processing flows: the general API and the zero-copy API.

- General API inference: Provides a simple, low-barrier set of inference APIs that are easy to use. The process is shown in Figure 1-3. Data normalization, quantization, data layout format conversion, and dequantization are all performed on the CPU, while the model inference itself is executed on the NPU.

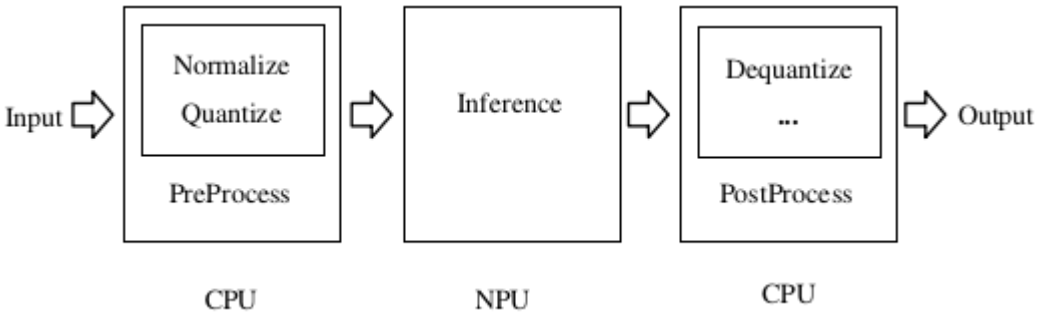


Figure 1-3 Data Processing Flow of the General API

- Zero-copy API inference: The process is shown in Figure 1-4. It optimizes the data processing flow of the general API. Normalization, quantization, and model inference are all performed on the NPU, while the data layout format conversion of the NPU output and the dequantization process are executed on the CPU or NPU. The zero-copy API is more efficient than the general API in handling the input data flow. It supports data flowing between different IP cores without any data copying, reducing CPU and DDR bandwidth consumption. For example, data obtained through a camera or from decoding can be imported into the NPU by zero-copy.

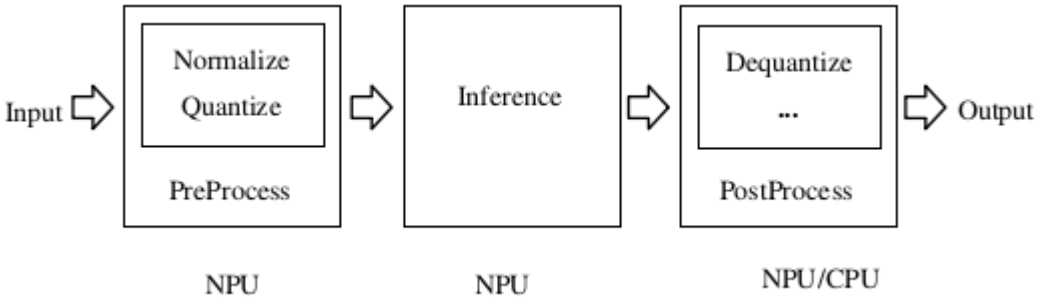


Figure 1-4 Data Processing Flow of the Zero-copy API

When the user's input data only has a virtual address, only the general API can be used; when the user's input data has a physical address or fd, both sets of APIs can be used.

1.1.2 Supported Hardware Platforms and Deployment Modes

This document applies to the following hardware platforms:

RV1103, RV1103B, RV1106, RV1106B, RK2118, RK3562, RK3566 series, RK3568 series, RK3576, RK3588 series, RV1126B

Note: RK3566_RK3568 is used to indicate RK3566/RK3568 series in this document. RK3588 is used to indicate RK3588/RK3588S series in this document.

1.1.3 Terms, Abbreviations, and Document Conventions

This section describes the terms and abbreviations commonly used in this document.

Hardware and Deployment

- **Board-connected inference**: The PC is connected to the development board via USB; inference is initiated by RKNN-Toolkit2 on the PC side and the model is executed on the NPU of the development board. It is used for model verification and accuracy analysis.

Models and Tensors

- **RKNN model**: A model file that runs on the RKNPU, with the `.rknn` extension.
- **Tensor**: A data structure representing multi-dimensional arrays, containing data memory and attribute information (shape / dtype / layout, etc.).
- **Data type (dtype)**: The data type of tensor elements, such as FP32 / FP16 / INT8 / UINT8.
- **Data layout (layout)**: The arrangement of a tensor in memory, such as NCHW / NHWC / NC1HWC2.
- **NC1HWC2**: An NPU-optimized memory layout format that stores the channel dimension in blocks to improve NPU access efficiency. The number of sub-channels on different platforms is described in 3.1.2 NPU layout.

Quantization

- **Quantized model**: A model running with low-precision data types such as INT8; scale and zero_point parameters are required for quantization/dequantization.
- **Non-quantized model**: A model running with the FP16 data type; no quantization parameters are required.

Document Conventions

- Lines starting with `#` in code blocks are comments.
- Content in `<>` is a parameter placeholder and should be replaced with actual values.
- Interface function names, parameter names, and file names are wrapped in backticks (`) .
- For detailed API parameter descriptions involved in this document, refer to the corresponding API reference manual.

1.2 Versions and Development Environment

RKNN-Toolkit2 supports two different version, one is for the x86, another one is for the ARM64. Both has the same installation setup except for ARM64 has not supported docker yet. Besides, ARM64 only supports models from the onnx and pytorch framework for conversion. Models from other frameworks does not support at the moment.

The difference between the version of x86 and ARM64:

Deepleanring Framework	**x86**	**ARM64**
Caffe	✔	✗
TensorFlow	✔	✗
TF Lite	✔	✗
ONNX	✔	✔
DarkNet	✔	✗
PyTorch	✔	✔

The difference in tools:

Dependent Tools	**x86**	**ARM64**
adb	✔	✗
rknn_server	✔	✗
rknn runtime	✔	✔

The difference when using x86 or ARM64:

The major difference is on using debugging function, such as accuracy analysis or inference, ARM64 is installed on ARM64 Linux, then using these functions mentioned above, only need to ensure that the `librknnrt.so` is located on '/usr/lib64'. Since the ARM64 does not support adb, it only works on current board. But x86 requires updating and utilizing the rknn_server and ensure that adb is available for using these debugging functions and connecting to other board if required.

Apart from that, x86 and ARM64 might have performance difference when it comes to accuracy analysis or quantizing.

Note: For RK2118 development environment preparation, please refer to 《01_RKNN2_RK2118_QuickStart_V2.5.0》.

1.2.1 Installing RKNN-Toolkit2

This chapter provides two methods for installing RKNN-Toolkit2: Docker and Pip. Users can choose either method according to their preferences.

1.2.1.1 INSTALLATION VIA DOCKER

1.2.1.1.1 Install Docker

Users who have already installed Docker can skip this step. For users who haven't installed Docker, please follow the official manual for installation. Official Docker installation manual link: https://docs.docker.com/install/linux/docker-ce/ubuntu/.

Note: Make sure to add the user to the docker group.

```
# Create docker group
sudo groupadd docker
# Add the current user to the docker group
sudo usermod -aG docker $USER
# Update and activate the docker group
newgrp docker
# Verify that sudo is not needed to execute docker commands
docker run hello-world
```

If it works, the results are as follows:

```
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
719385e32844: Pull complete
Digest: sha256:88ec0acaa3ec199d3b7eaf73588f4518c25f9d34f58ce9a0df68429c5af48e8d
Status: Downloaded newer image for hello-world:latest
Hello from Docker!
```

1.2.1.1.2 Image Preparation

This section introduces two methods for preparing the RKNN-Toolkit2 image environment, and either one can be chosen.

1. **Creating an image environment via Dockerfile**

In the docker/docker_file folder of the RKNN-Toolkit2 project, Dockerfiles for building the RKNN-Toolkit2 development environment are provided. Users create the image with the Docker command as follows.

```
# Note: xx and x.x.x below represent version numbers, please replace them with
actual values
cd docker/docker_file/ubuntu_xx_xx_cpxx
docker build -f Dockerfile_ubuntu_xx_xx_for_cpxx -t rknn-toolkit2:x.x.x-cpxx .
```

2. **Loading a Docker image with all development environments pre-packaged**

Download the corresponding version of the RKNN-Toolkit2 project file. After decompression, get the Docker image with all development environments pre-packaged in the docker/docker_image folder.

Network disk download link: https://console.zbox.filez.com/l/I00fc3. Extraction code: rknn

Execute the following command to load the image file corresponding to the Python version.

```
# Note: x.x.x represents the version number of RKNN-Toolkit2, and cpxx represents
the Python version number
docker load --input rknn-toolkit2-x.x.x-cpxx-docker.tar.gz
```

1.2.1.1.3 Querying Image Information

After the image is created or loaded successfully, check the Docker image information:

```
docker images
```

The relevant RKNN-Toolkit2 image information will be displayed:

```
REPOSITORY     TAG         IMAGE ID       CREATED      SIZE
rknn-toolkit2  x.x.x-cpxx  xxxxxxxxxxxx   1 hours ago  5.89GB
```

1.2.1.1.4 Running the Image

Execute the following command to run the Docker image, users will enter the bash environment of the image.

```
docker run -t -i --privileged -v /dev/bus/usb:/dev/bus/usb rknn-toolkit2:x.x.x-cpxx
/bin/bash
```

Users can map the "examples" folder code into the docker environment by appending the "-v \<host src folder>:\<image dst folder>" parameter.

```
docker run -t -i --privileged -v /dev/bus/usb:/dev/bus/usb -v /your/rknn-toolkit2-
x.x.x/examples:/examples rknn-toolkit2:x.x.x-cpxx /bin/bash
```

1.2.1.1.5 Running the Demo

```
cd examples/onnx/yolov5
python test.py
```

After successfully running the script, the results should be as follows:

```
class     score  xmin, ymin, xmax, ymax
-------------------------------------------------
person    0.884 [ 208,  244,  286,  506]
person    0.868 [ 478,  236,  559,  528]
person    0.825 [ 110,  238,  230,  534]
person    0.339 [  79,  353,  122,  516]
bus       0.705 [  92,  128,  554,  467]
```

1.2.1.2 INSTALLATION VIA PIP

1.2.1.2.1 Install Python Environment

If the Python environment is already installed, users can skip this step.

```
sudo apt-get update
sudo apt-get install python3 python3-dev python3-pip
sudo apt-get install libxslt1-dev zlib1g zlib1g-dev libglib2.0-0 libsm6 libgl1-
mesa-glx libprotobuf-dev gcc
```

If users want to install the RKNN-Toolkit2 in the local environment (non-Conda virtual environment), please proceed to step 2.1.2.4 after installing Python successfully.

1.2.1.2.2 Install Conda Tool

If there are multiple versions of Python installed on the system, it is recommended to use Conda to manage the Python environment.

Check whether Conda is installed and the version. If it is already installed, users can skip this step.

```
conda -V
# Prompt conda: command not found means conda is not installed.
# Successful installation will display the version such as: conda X.X.X
# X.X.X means version number, such as 23.9.0.
```

Download the Conda installation package:

```
wget -c https://github.com/conda-
forge/miniforge/releases/latest/download/Miniforge3-Linux-x86_64.sh
```

Install Conda:

```
chmod 777 Miniforge3-Linux-x86_64.sh
bash Miniforge3-Linux-x86_64.sh
```

ARM64 version:

Download the Conda installation package:

```
wget -c https://github.com/conda-
forge/miniforge/releases/latest/download/Miniforge3-Linux-aarch64.sh
```

Install Conda:

```
chmod 777 Miniforge3-Linux-aarch64.sh
bash Miniforge3-Linux-aarch64.sh
```

1.2.1.2.3 Create RKNN-Toolkit2 Conda Environment

Enter the Conda base environment:

```
source ~/miniforge3/bin/activate # Miniforge install directory path
# (base) xxx@xxx-pc:~$
```

Create a Conda environment named "RKNN-Toolkit2" with Python 3.8 version (recommand version):

```
conda create -n RKNN-Toolkit2 python=3.8
```

Enter the 'RKNN-Toolkit2' Conda environment:

```
conda activate RKNN-Toolkit2
# (RKNN-Toolkit2) xxx@xxx-pc:~$
```

1.2.1.2.4 Install RKNN-Toolkit2

After activating the 'RKNN-Toolkit2' Conda environment, you can install RKNN-Toolkit2 using either the pip source or a local wheel package: - Install via pip source

```
pip install rknn-toolkit2 -i https://pypi.org/simple

# If RKNN-Toolkit2 is already installed, you can upgrade it with the following
command:
pip install rknn-toolkit2 -i https://pypi.org/simple --upgrade
```

- Install via local wheel package

  ```
  # Please choose the appropriate requirements file based on the Python version
  and processor architecture:
  # where cpxx is the Python version
  pip install -r /rknn-toolkit2/packages/x86_64/requirements_cpxx.txt
  # pip install -r /rknn-toolkit2/packages/arm64/arm64_requirements_cpxx.txt

  # Install RKNN-Toolkit2
  # Please choose the appropriate wheel package file based on the Python version
  and processor architecture:
  # where x.x.x is the RKNN-Toolkit2 version number, cpxx is the Python version
  pip install /rknn-toolkit2/packages/x86_64/rknn_toolkit2-x.x.x-cpxx-cpxx-
  manylinux_2_17_x86_64.manylinux2014_x86_64.whl
  # pip install /rknn-toolkit2/packages/arm64/rknn_toolkit2-x.x.x-cpxx-cpxx-
  manylinux_2_17_aarch64.manylinux2014_aarch64.whl
  ```

If no error occurs after executing the following command, the installation is successful.

```
$ python
>>> from rknn.api import RKNN
```

1.2.2 NPU Device Environment Setup

1.2.2.1 NPU DRIVER VERSION CONFIRMATION

Query commands:

```
dmesg | grep -i rknpu
# or
cat /sys/kernel/debug/rknpu/version
# or
cat /sys/kernel/debug/rknpu/driver_version
# or
cat /proc/debug/rknpu/driver_version
```

Query results:

```
RKNPU driver: vX.X.X
```

X.X.X means version number such as 0.9.2.

Rockchip's firmware comes with RKNPU driver pre-installed. If the above commands cannot query the NPU driver version, it may be that the third-party firmware has not installed the RKNPU driver. Open the kernel config file with the "CONFIG_ROCKCHIP_RKNPU=y" option, recompile the kernel driver and flash it. It is recommended that the RKNPU driver version is >= 0.9.2.

1.2.2.2 NPU DEVICE CONNECTION ENVIRONMENT CONFIRMATION

The PC-to-board debugging function of RKNN-Toolkit2 requires the RKNN Server on the development side. It is a background proxy service running on the device, responsible for receiving commands and data transmitted from the computer via USB, executing the corresponding interfaces, and returning results to the computer. Therefore, it needs to be confirmed whether the device has started the RKNN Server.

Note: ARM64 does not need to use RKNN Server for debugging, skipping this step if you were using ARM64 for RKNN Toolkit2.

Check if there is an RKNN Server process running on the development board.

```
adb shell
ps | grep rknn_server
```

Query results:

```
702 root 1192 S grep rknn_server
```

If the RKNN Server process id is queried, it means RKNN Server starts normally. If the RKNN Server process id is not queried, execute the following command to manually start RKNN Server and then query.

Manually start RKNN Server on Android:

```
su
setenforce 0
nohup /vendor/bin/rknn_server >/dev/null&
```

RK356X/RK3576/RK3588/RV1126B Linux system to start RKNN Server manually:

```
nohup /usr/bin/rknn_server >/dev/null&
```

RV1103/RV1103B/RV1106/RV1106B Linux system to start RKNN Server manually:

```
nohup /oem/usr/bin/rknn_server >/dev/null&
```

Generally speaking, Rockchip's firmwares integrate RKNN Server which starts automatically during boot-up. If there is no automatic startup or no relevant files for manual startup, please install or update RKNN Server manually.

1.2.2.3 RKNN SERVER INSTALLATION AND UPDATE

The board-connected debugging feature of RKNN-Toolkit2 requires the RKNPU2 environment to be installed on the board and the RKNN Server service to be started. The following are two fundamental concepts of the RKNPU2 environment:

1. RKNN Server: A background proxy service running on the development board. It receives the data transmitted from the PC via USB, executes the corresponding interface of the on-board Runtime, and returns the results to the PC.

2. RKNPU2 Runtime library

- librknnrt.so: The Runtime library for RK3562 / RK3566 / RK3568 / RK3576 / RK3588 / RV1126B boards.
- librknnmrt.so: The Runtime library for RV1103 / RV1103B / RV1106 / RV1106B boards.

If RKNN Server is not installed on the board, the Runtime library does not exist, or the versions of RKNN Server and the Runtime library are inconsistent, you need to reinstall or update them.

Note:

1. If an RKNN model with dynamic dimension input is used, the versions of RKNN Server and the RKNPU2 Runtime library must be >= 1.5.0.

2. If a model larger than 2 GB is used, the rknn_server version must be >= 2.0.0b0.

3. On small-memory platforms such as RV1103 / RV1103B / RV1106 / RV1106B, it is recommended to use rknn_server version >= 2.1.0.

4. Make sure the versions of RKNN Server, the Runtime library, and RKNN-Toolkit2 are consistent; it is recommended to install the latest versions of all of them.

5. The RKNN Server in RK2118 is not used for the board-connected debugging feature of RKNN-Toolkit2; it is the service for the DSP to interact with the MCU.

6. The ARM64 version does not depend on RKNN Server, so there is no need to start or update it. However, the debugging feature depends on the on-board RKNPU2 Runtime library, so make sure this library exists on the board and is updated to the latest version.

1.2.2.3.1 RK356X/RK3576/RK3588/RV1126B Platform

1. Android system

Check the versions of the RKNN Server and the `librknnrt.so` library. If the version numbers are inconsistent, update them to the same version.

```
# Query the rknn_server version
strings /vendor/bin/rknn_server | grep -i "rknn_server version"
# Display the rknn_server version as X.X.X
# rknn_server version: X.X.X

# Query the librknnrt.so library version
# 64-bit system
strings /vendor/lib64/librknnrt.so | grep -i "librknnrt version"
# 32-bit system
strings /vendor/lib/librknnrt.so | grep -i "librknnrt version"
# Display the librknnrt library version as X.X.X
# librknnrt version: X.X.X
```

Update the RKNN Server and the `librknnrt.so` library.

```
adb root
adb remount

# 64-bit system:
adb push runtime/Android/rknn_server/arm64/rknn_server /vendor/bin/
adb push runtime/Android/librknn_api/arm64-v8a/librknnrt.so /vendor/lib64

# 32-bit system:
adb push runtime/Android/rknn_server/arm/rknn_server /vendor/bin/
adb push runtime/Android/librknn_api/armeabi-v7a/librknnrt.so /vendor/lib

# Restart the RKNN Server:
adb shell
su
chmod +x /vendor/bin/rknn_server
nohup /vendor/bin/rknn_server >/dev/null&
sync
reboot
```

2. Linux system

Check the versions of the RKNN Server and the `librknnrt.so` library. If they are inconsistent, update them to the same version.

```
# Query the rknn_server version
strings /usr/bin/rknn_server | grep -i "rknn_server version"
# Display the rknn_server version as X.X.X
# rknn_server version: X.X.X

# Query the librknnrt.so library version
strings /usr/lib/librknnrt.so | grep -i "librknnrt version"
# Display the librknnrt library version as X.X.X
# librknnrt version: X.X.X
```

Update the RKNN Server and the librknnrt.so library.

64-bit system:

```
adb push runtime/Linux/rknn_server/aarch64/usr/bin/* /usr/bin
adb push runtime/Linux/librknn_api/aarch64/librknnrt.so /usr/lib64
```

32-bit system:

```
adb push runtime/Linux/rknn_server/armhf/usr/bin/* /usr/bin
adb push runtime/Linux/librknn_api/armhf/librknnrt.so /usr/lib
```

Restart the RKNN Server:

```
adb shell
chmod +x /usr/bin/rknn_server
nohup /usr/bin/rknn_server >/dev/null&
restart_rknn.sh
```

1.2.2.3.2 RV1103/RV1103B/RV1106/RV1106B Platform

Check the versions of the RKNN Server and the librknnmrt.so library. If they are inconsistent, update them to the same version.

```
# Query the rknn_server version
strings /oem/usr/bin/rknn_server | grep -i "rknn_server version"
# Display the rknn_server version as X.X.X
# rknn_server version: X.X.X

# Query the librknnmrt.so library version
strings /oem/usr/lib/librknnmrt.so | grep -i "librknnmrt version"
# Display the librknnmrt library version as X.X.X
# librknnmrt version: X.X.X
```

Update the RKNN Server and the `librknnmrt.so` library. RV1103 / RV1103B and RV1106 / RV1106B use the same RKNN Server and librknnmrt.so library.

```
adb push runtime/Linux/rknn_server/armhf-uclibc/usr/bin/* /oem/usr/bin
adb push runtime/Linux/librknn_api/armhf-uclibc/librknnmrt.so /oem/usr/lib

# Restart the RKNN Server:
adb shell
chmod +x /oem/usr/bin/rknn_server
nohup /oem/usr/bin/rknn_server >/dev/null&
restart_rknn.sh
```

1.2.2.4 VIEW DETAILED RKNN SERVER LOGS

1.2.2.4.1 Android System

Enter the device terminal and set the log level:

```
adb shell
su
setenforce 0
setprop persist.vendor.rknn.server.log.level 5
```

Kill the current RKNN Server service process:

```
kill -9 `pgrep rknn_server`
```

If the RKNN Server does not restart automatically, users can start it manually and view detailed logs:

```
nohup /vendor/bin/rknn_server >/dev/null&
logcat
```

1.2.2.4.2 Linux System

Enter the device terminal and set the log level:

```
adb shell
export RKNN_SERVER_LOGLEVEL=5
```

Restart the RKNN Server to view detailed logs:

For the RK356X / RK3576 / RK3588 / RV1126B platform, run the following command:

```
nohup /usr/bin/rknn_server >/dev/null&
```

For the RV1103 / RV1103B / RV1106 / RV1106B platform, run the following commands:

```
nohup /oem/usr/bin/rknn_server >/dev/null&
```

1.2.3 Environment Verification

After the environment is set up, it is recommended to verify that all components are ready as follows:

1. Verify the RKNN-Toolkit2 installation

```
# Enter Python interactive mode and import the RKNN class
python
>>> from rknn.api import RKNN
# No error means the installation succeeded
```

2. Verify device connection

```
# Query adb-connected devices on the PC
adb devices
# The connection is normal if device IDs are listed
```

Note: The ARM64 version of Toolkit2 does not support adb; the debugging feature is used locally on the board only, so this step does not need to be verified.

3. Verify the on-board environment

Check whether the NPU driver version, RKNN Server process status, and Runtime library are ready. For checking, installation, and update methods, see 1.2.2 Device-side NPU Environment Preparation.

4. Query component versions

When troubleshooting, you can query the version information of each component as follows:

```
# Query the RKNN-Toolkit2 version
pip list | grep rknn-toolkit2

# Query the on-board rknn_server version (paths subject to 1.2.2)
strings /usr/bin/rknn_server | grep -i "rknn_server version"

# Query the on-board Runtime library version (RK356X / RK3576 / RK3588 / RV1126B
platforms, path subject to 1.2.2)
strings /usr/lib64/librknnrt.so | grep -i "librknnrt version"

# Query the on-board Runtime library version (RV1103 / RV1103B / RV1106 / RV1106B
platforms)
strings /oem/usr/lib/librknnmrt.so | grep -i "librknnmrt version"
```

> **Note**: The RKNN model file records the Toolkit2 version used during conversion. When troubleshooting abnormal on-board running, compare the model conversion version with the on-board Runtime library version to check whether they are consistent.

1.3 Development Process and Interface Selection

1.3.1 General Model Development Process

Users can refer to the following flowchart to understand the overall RKNN development steps. The process is mainly divided into 3 parts: model conversion, model evaluation, and on-board deployment and running.

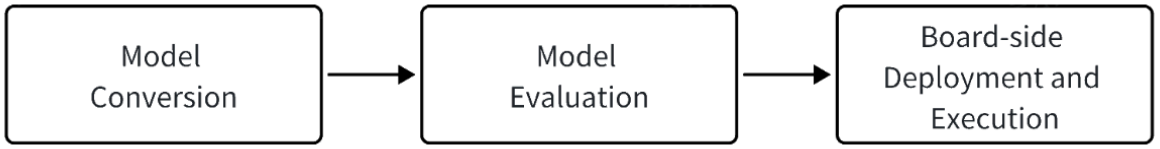


Figure 1-5 RKNN Development Flowchart

1. **Model Conversion**:

In this stage, the original deep learning model is converted into the RKNN format for efficient inference on RKNPU platforms. This stage includes the following 5 steps:

a. Obtain the original model: Obtain or train a deep learning model. It is recommended to use mainstream frameworks such as ONNX, PyTorch, or TensorFlow.

b. Model configuration: Make the necessary configurations in RKNN-Toolkit2, such as normalization parameters, quantization parameters, and target platform.

c. Model loading: Import the model into RKNN-Toolkit2 using the appropriate loading interface, and select the correct loading interface according to the model framework.

d. Model building: Build the RKNN model with the `rknn.build()` interface; you can choose whether to perform quantization to improve the model's inference performance on the hardware.

e. Model export: Export the RKNN model as a file (.rknn format) with the `rknn.export_rknn()` interface for subsequent deployment.

For details, see section 2.1 General Model Conversion.

2.**Model Evaluation**:

The main goal of this stage is to evaluate key metrics such as the accuracy of model inference results, inference performance, and memory usage. The main steps of evaluation are as follows:

a. Board-connected model debugging: Use Python to connect to the RKNPU platform to run inference on the model and verify the accuracy of the results. This stage covers pre-processing of the input data and post-processing of the output results, to ensure the model runs correctly on the board.

b. Accuracy evaluation: Use the `rknn.accuracy_analysis()` interface to compare the differences between the inference results of the quantized model and the floating-point model, in order to analyze the sources of quantization error.

c. Performance evaluation: Use the `rknn.eval_perf()` interface to analyze the inference performance of the model on the RKNPU platform, helping users further optimize the model structure and speed up inference performance.

d. Memory evaluation: Use the `rknn.eval_memory()` interface to understand the memory usage of the model on the RKNPU platform, helping users further optimize the model structure and minimize memory usage.

For details, see sections 2.3 Model Evaluation, 3.2 Deploying General Models with the C API, and 3.3 Toolkit Lite Python Deployment.

3. **On-board deployment and running**:

This stage covers the actual deployment and running of the model. It typically includes the following steps:

a. Model initialization: Load the RKNN model onto the RKNPU platform and prepare for pre-processing.

b. Model pre-processing: Load the data to be inferred onto the RKNPU platform and prepare for inference.

c. Model inference: Perform the inference operation, passing the input data to the model and obtaining the inference results.

d. Model post-processing: Obtain the inference results for post-processing, and pass the post-processed results to the application side.

e. Model release: After completing the inference process, release the model resources so that the RKNN model can be used by other tasks.

For details, see section 3.3 Toolkit Lite Python Deployment.

These three steps constitute the complete RKNN development process, ensuring that artificial intelligence models can be successfully converted, debugged, evaluated, and finally deployed efficiently on the RKNPU.

1.3.2 Interface Selection

The RKNN2 SDK provides two on-board invocation methods; users choose according to the scenario:

Method	Language	Applicable Scenarios
RKNN Runtime (C API)	C / C++	Production deployment, performance-sensitive scenarios
RKNN-Toolkit Lite2 (Python API)	Python	Quick on-board verification, prototype development

Differences between the two:

- **Performance**: The C API delivers the best performance; Toolkit Lite is slightly lower due to the overhead of the Python wrapper.
- **Development convenience**: Toolkit Lite is more convenient (Python programming, no cross-compilation required); the C API requires cross-compilation and integration into the on-board application.
- **Feature completeness**: The C API supports the full set of advanced features (zero-copy, NPU multi-core, matrix multiplication interface, dynamic shape, SRAM, model encryption, etc.); Toolkit Lite targets quick on-board verification and covers fewer advanced features.

Selection suggestions:

- **Production deployment / performance-sensitive scenarios**: Use the C API, which provides optimal performance and complete features, and supports security features such as encrypted model deployment.
- **On-board verification / prototype development**: Use Toolkit Lite; no cross-compilation is needed, and with the converted .rknn model, inference results can be quickly verified on the board.

> **Note**: For detailed usage of the two interfaces, see 3.2 Deploying General Models with the C API and 3.3 Toolkit Lite Python Deployment.

1.3.3 Model Conversion and Deployment Artifacts

The deployment artifacts generated after model conversion are as follows:

File/Directory	Description
`.rknn`	RKNN model file, containing the model structure and weights
C++ deployment example code (optional)	Generated via the `cpp_gen_cfg` parameter of `rknn.export_rknn()` or the `rknn.codegen()` interface; it can be used to test performance and verify accuracy directly, and can also be used for secondary development

> **Note**: The model exported by RKNN2 is a single `.rknn` file (model structure and weights integrated in one). During deployment, the `.rknn` model must be transferred to the board: C API deployment loads it via `rknn_init()`, and Toolkit Lite2 deployment loads it via `init()`. For a description of the artifacts in model encryption deployment scenarios, see 4.2.2 Model Encryption Deployment.

2. Model Conversion, Quantization, and Evaluation

2.1 General Model Conversion

RKNN-Toolkit2 provides rich functionality, including model conversion, performance analysis, deployment and debugging, etc. This section focuses on the model conversion function of RKNN-Toolkit2. Model conversion is one of the core functions of RKNN-Toolkit2. It allows users to convert deep learning models from various frameworks into RKNN format to run on RKNPU. The overall workflow is: model preparation → configuration of conversion parameters → loading and building the model → exporting the RKNN model → verifying the conversion result. Users can refer to the following model conversion flow chart to understand how to perform model conversion.

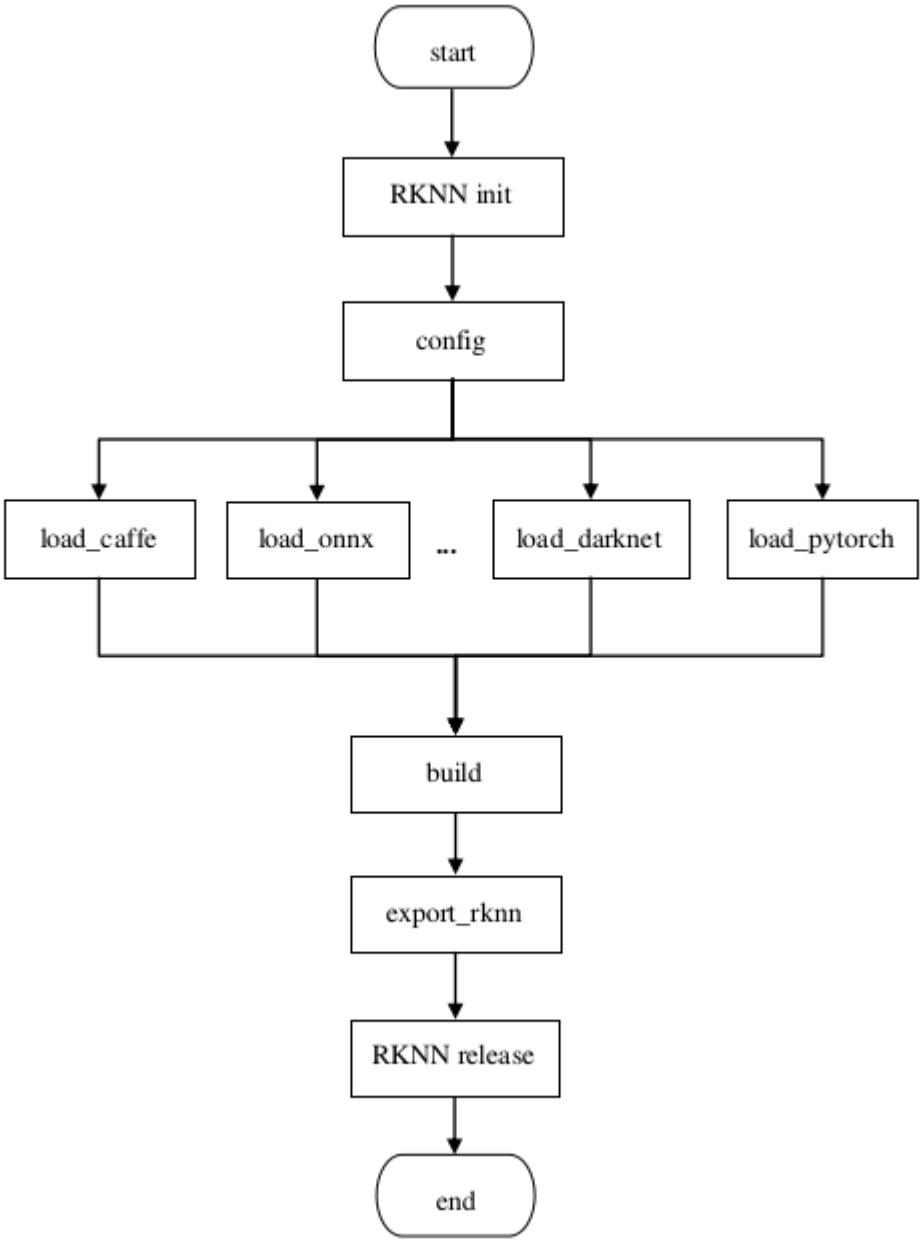


Figure 2-1 The flow chart of model conversion

Currently RKNN-Toolkit2 supports model conversion for multiple popular deep learning frameworks, including:

- Caffe
- TensorFlow
- TensorFlow Lite
- ONNX
- PyTorch
- Darknet

Users can use the above frameworks to train or obtain pre-trained models and convert them into RKNN format, making it easier and more efficient to deploy and run inference on the RKNPU platform.

2.1.1 Model Preparation

The input of model conversion is the original model trained or exported from each deep learning framework. Complete the following preparations before conversion:

- **Model source**: use a pre-trained model trained or obtained from a framework supported by RKNN-Toolkit2 (Caffe / TensorFlow / TensorFlow Lite / ONNX / PyTorch / DarkNet), and make sure the model infers correctly in the original framework.
- **Input/output information**: identify the model's input/output node names, input shapes, data types, etc. Some frameworks require this information explicitly when loading (e.g. TensorFlow's inputs / outputs / input_size_list, PyTorch's input_size_list); the subsequent configuration and loading APIs will use it.
- **Model check**: before conversion, it is recommended to inspect the model structure with tools such as Netron to confirm the model was exported correctly and its operators are supported; for ONNX models, onnxruntime can be used first to verify that the inference results match the original framework.

2.1.2 Conversion Configuration

Before model conversion, users need to set some configurations to ensure the correctness and performance of model conversion. Configuration parameters can be set through the rknn.config() interface, including setting the input mean, normalized value, whether to quantize, etc. Some commonly used configuration parameters are listed below:

- **mean_values** and **std_values** are used to set the mean and normalized values of the input. These values are used in the quantification process, and the images no longer need to be averaged and normalized during the C API inferencing to reduce deployment time.
- **quant_img_RGB2BGR** is used to control whether RGB to BGR conversion needs to be performed first when loading a quantization image during quantization. The default value is false. This configuration takes effect during quantification correction and will not take effect during the model inference stage. It needs to be processed by the users in pre-input processing. Please refer to Chapter 5.4.3 for detailed precautions. Note: Note: When

quant_img_RGB2BGR= True, the model's inference sequence is to perform RGB2BGR conversion first then perform mean_valuesand std_values operations later.

- **target_platform** is used to specify the target platform of the RKNN model, supporting RV1103, RV1103B, RV1106, RV1106B, RV1126B, RK2118, RK3562, RK3566, RK3568, RK3576 and RK3588 at the moment.
- **quantized_algorithm** is used to specify the quantization algorithm used when calculating the quantized parameters of each layer. You can choose normal, mmse, kl_divergence or gdq. The default algorithm is normal. For detailed description, see chapters 3.1.7, 6.1 and 6.2.
- **quantized_method** supports layer, channel or group{SIZE}, used to determine whether the weights of each layer share parameters. The default is channel. For details, see chapters 3.1.7, 6.1 and 6.2.
- **optimization_level** is used to modifying the model optimization level, you can turn off some or all of the optimization rules used in the model conversion process. The default value of this parameter is 3, which turns on all optimization options. When the value is 2 or 1, some optimization options that may affect the accuracy of some models are turned off. When the value is 0, all optimization options are turned off.
- **quantized_dtype** is used to specify the quantization type. Currently, INT8 for linear asymmetric quantization is supported, and the default is 'asymmetric_quantized-8'.
- **custom_string** is used to adds custom string information to the RKNN model, such as version information of the model`s own iteration, etc. This information can be obtained by querying RKNN_QUERY_CUSTOM_STRING which facilitates special processing according to different RKNN models during deployment. The default value is None.
- **dynamic_input** supports dynamic input and simulates the function of dynamic input according to multiple sets of input shapes specified by the users. The default value is None. See Chapter 4.1.3 for details.

For more specification of rknn.config(), please refer to the 《03_RKNN2_Toolkit_Python_API_Reference_V2.5.0》 manual. Only some common parameters are listed above.

Example code:

```
rknn.config(
mean_values=[[103.94, 116.78, 123.68]],
std_values=[[58.82, 58.82, 58.82]],
quant_img_RGB2BGR=False,
target_platform='rk3566')
```

2.1.3 Model Loading and Building

1. Initialize the RKNN object

In this stage, users need to initialize the RKNN object first, which is the first step of the entire workflow:

- Use the `RKNN()` constructor to initialize the RKNN object. Users can pass in the parameters `verbose` and `verbose_file`.

- The `verbose` parameter determines whether to display detailed log information on the screen.
- If the `verbose_file` parameter is set and `verbose` is `True`, the log information will also be written to the specified file.

Example code:

```
rknn = RKNN(verbose=True, verbose_file='./mobilenet_build.log')
```

2. Model loading interfaces

Users need to use the corresponding loading interface to import model files from different frameworks. RKNN-Toolkit2 provides different loading interfaces, including Caffe, TensorFlow, TensorFlow Lite, ONNX, PyTorch, etc. The following is a brief introduction to the model loading interfaces of each framework:

- Caffe model loading interface:
- Use the `rknn.load_caffe()` interface to load a Caffe model.
- You need to provide the model file (`.prototxt` suffix) path and the weight file (`.caffemodel` suffix) path.
- If the model has multiple input layers, you can specify the order of the input layer names.

Example code:

```
ret = rknn.load_caffe(model='./mobilenet_v2.prototxt',
blobs='./mobilenet_v2.caffemodel')
```

- TensorFlow model loading interface:
- Use the `rknn.load_tensorflow()` interface to load a TensorFlow model.
- You need to provide the TensorFlow model file (`.pb` suffix) path, the input node names, the input node shapes, and the output node names.

Example code:

```
ret = rknn.load_tensorflow(tf_pb='./ssd_mobilenet_v1_coco_2017_11_17.pb', inputs=
['Preprocessor/sub'], outputs=['concat', 'concat_1'], input_size_list=[[1, 300,
300, 3]])
```

- TensorFlow Lite model loading interface:
- Use the `rknn.load_tflite()` interface to load a TensorFlow Lite model.
- You need to provide the TensorFlow Lite model file (.tflite suffix) path.

Example code:

```
ret = rknn.load_tflite(model='./mobilenet_v1.tflite')
```

- ONNX model loading interface: - Use the `rknn.load_onnx()` interface to load an ONNX model. - You need to provide the ONNX model file (.onnx suffix) path.

Example code:

```
ret = rknn.load_onnx(model='./arcface.onnx')
```

- DarkNet model loading interface:
- Use the `rknn.load_darknet()` interface to load a DarkNet model.
- You need to provide the DarkNet model file (`.cfg` suffix) path and the weight file (`.weights` suffix) path.

Example code:

```
ret = rknn.load_darknet(model='./yolov3-tiny.cfg', weight='./yolov3.weights')
```

- PyTorch model loading interface: - Use the `rknn.load_pytorch()` interface to load a PyTorch model. - You need to provide the PyTorch model file (.pt suffix) path, and the model must be in torchscript format.

Example code:

```
ret = rknn.load_pytorch(model='./resnet18.pt', input_size_list=[[1, 3, 224, 224]])
```

Users can select the appropriate interface to load models from different frameworks, so as to ensure the correctness of model conversion.

3. Build the RKNN model

After loading the original model, the next step is to build the RKNN model through the `rknn.build()` interface. When building a model, users can choose whether to perform quantization, which can reduce the model size and improve performance on RKNPU. The parameters of the `rknn.build()` interface are as follows:

- do_quantization: this parameter controls whether to quantize the model. It is recommended to set it to `True`.
- dataset: this parameter specifies the dataset used for quantization calibration. The format of the dataset is a text file.

Example of dataset.txt:

```
./imgs/ILSVRC2012_val_00000665.JPEG
./imgs/ILSVRC2012_val_00001123.JPEG
./imgs/ILSVRC2012_val_00001129.JPEG
./imgs/ILSVRC2012_val_00001284.JPEG
./imgs/ILSVRC2012_val_00003026.JPEG
./imgs/ILSVRC2012_val_00005276.JPEG
```

Example code:

```
ret = rknn.build(do_quantization=True, dataset='./dataset.txt')
```

2.1.4 Exporting the RKNN Model

After building the RKNN model through the `rknn.build()` interface, users can export the RKNN model file (`.rknn` suffix) through the `rknn.export_rknn()` interface for subsequent model deployment. The parameters of the `rknn.export_rknn()` interface are as follows:

- export_path: the path of the exported model file.
- cpp_gen_cfg: this parameter determines whether to generate a C++ deployment example when exporting the model.

Example code:

```
ret = rknn.export_rknn(export_path='./mobilenet_v1.rknn')
```

After the conversion workflow is completed, call the `release()` interface to release the resources occupied by the RKNN object:

```
rknn.release()
```

2.1.5 Conversion Result Verification

After exporting the RKNN model, it is recommended to verify the conversion result:

- **Simulator inference verification**: on the PC, run inference on the model with the simulator via `rknn.init_runtime(target=None)` (`rknn.inference()`), and compare the output with the inference result of the original model in the original framework (cosine distance can be used to check consistency) to confirm that no error was introduced during conversion.
- **Board-connected inference verification**: connect the development board to the PC via USB and run board-connected inference via `rknn.init_runtime(target=xxx)` to verify the results on real NPU hardware.

> **Note**: For more detailed evaluation methods such as accuracy analysis, performance evaluation, and memory evaluation, see 2.3 Model Evaluation.

2.1.6 Model Conversion Tool: rknn_convert

`rknn_convert` is a set of commonly used model conversion tools provided by RKNN-Toolkit2. By encapsulating the above API interfaces, users only need to edit the `yml` configuration file corresponding to the model to convert the model through a command. The following is an example command of how to use the `rknn_convert` tool and the supported command parameters:

```
python -m rknn.api.rknn_convert -t rk3588 -i ./model_config.yml -o ./output_path
```

By using the above commands and parameters, users can convert models into RKNN format and save the converted models to the specified output path. The supported command parameters are described as follows:

- -i: the path of the model configuration file (.yml).
- -o: the output path of the converted model.
- -t: target_platform. The target platform can be rv1103, rv1103b, rv1106, rv1106b, rv1126b, rk2118, rk3562, rk3566, rk3568, rk3576 or rk3588.
- -e: (Optional) Evaluate the time consumption and memory usage of the model during board-connected runtime. Enter -e to enable it. Note: Be sure to connect the corresponding development board and set `target_platform` correctly, otherwise an error will be reported. When there are multiple devices, use the `-d` parameter to specify the device ID.
- -a: (Optional) Evaluate the accuracy of the generated rknn model. To enable simulator accuracy evaluation, enter -a "xx1.jpg xx2.jpg". To enable board-connected accuracy evaluation, use it together with the -d parameter.
- -v: (Optional) Specify whether to print detailed log information on the screen. Enter -v to enable printing mode.
- -d: (Optional) Use -d for a single adb device, and -d device_id for multiple adb devices. Query device_id through adb devices.

The following is a reference yml configuration file (`object_detection.yml`):

```yaml
models:
   # model output name
   name: object_detection
   # Original model framework
   platform: onnx
   # Model input file path
   model_file_path: ./object_detection.onnx
   # Describe information such as input and output shapes
   subgraphs:
     # model input tensor shape
     input_size_list:
       - 1,3,512,512
     # input tensor name
     inputs:
       - data
     # output tensor name
     outputs:
       - conv6-1
       - conv6-2
       - conv6-3
   # quantification flag
   quantize: true
   # Quantify dataset file path (relative yml path)
   dataset: ./dataset.txt
   configs:
     quantized_dtype: asymmetric_quantized-8
     # rknn.config mean_values
     mean_values: [127.5,127.5,127.5]
     # rknn.config std_values
     std_values: [128.0,128.0,128.0]
     # rknn.config quant_img_RGB2BGR
     quant_img_RGB2BGR: false
     # rknn.config quantized_algorithm
     quantized_algorithm: normal
```

This configuration file includes detailed information such as the model name, the framework used by the original model, the model file path, input and output information, and whether to perform quantization. Users can edit the corresponding configuration file according to the specific requirements of the model.

For the model conversion configuration, see the table below:

Table 2-1 Description of rknn_convert model conversion configuration parameters

Parameter	**Content to fill in**
-name	Model output name
-platform	The framework used by the original model, supporting tensorflow, tflite, caffe, onnx, pytorch, darknet
-model_file_path	Original model file path, applicable to single model file input, e.g.: tensorflow, tflite, onnx, pytorch
-quantize	Whether to enable quantization
-dataset	The quantization dataset file path (relative to the yml configuration file path). This item is required if accuracy_analysis is enabled
-prototxt_file_path	When platform is caffe, the prototxt file of the model

Parameter	Content to fill in
-caffemodel_file_path	When platform is caffe, the caffemodel file of the model
-darknet_cfg_path	When platform is darknet, the cfg file of the model
-darknet_weights_path	When platform is darknet, the weight file of the model
-subgraphs	Describes information such as input and output shapes. Except for specific frameworks, this parameter and its sub-parameters generally do not need to be filled in, and the model default values are used
----input_size_list (Subparameter)	The shape of the input tensor
----inputs (Subparameter)	The name of the input tensor
----outputs (Subparameter)	The name of the output tensor
-configs	The corresponding rknn.config() configuration
----quantized_dtype (Subparameter)	Quantization type. RKNN_toolkit2: can be set to [asymmetric_quantized-8]. If not entered, the default value is used
----mean_values (Subparameter)	The mean normalization value of the input. For a single-input RGB model, e.g., [123.675,116.28,103.53]; for multiple inputs, e.g., [[123,116,103], [255,255,255]]
----std_values (Subparameter)	The std normalization value of the input. For a single-input RGB model, e.g., [58.395,58.295,58.391]; for multiple inputs, e.g., [[127,127,127], [255,255,255]]
----quant_img_RGB2BGR (Subparameter)	Controls whether the quantization calibration images need to be converted from RGB to BGR first when loading them for quantization. The default value is False
----quantized_algorithm (Subparameter)	Quantization algorithm, selectable from ['normal', 'kl_divergence', 'mmse'], default is normal
----quantized_method (Subparameter)	Quantization method. RKNN_toolkit2 options are ['layer', 'channel'], default is channel
----optimization_level (Subparameter)	Sets the optimization level. The default is 3, which means using all default optimization options
----model_pruning (Subparameter)	Prune the model to reduce the model size. Defaults to false; set to true to enable
----quantize_weight (Subparameter)	When the quantize parameter is false, reduce the size of the rknn model by quantizing some weights. Defaults to false; set to true to enable
----single_core_mode (Subparameter)	Whether to generate a single-core model only, which can reduce the size and memory consumption of the RKNN model. The default value is False. Currently effective for RK3588/RK3576. The default value is False

2.1.7 Quantization Feature of RKNN-Toolkit2

RKNN-Toolkit2 provides multiple quantization methods and multiple quantization algorithms, selected via the quantized_algorithm and quantized_method parameters in rknn.config(). This section describes the commonly used Normal / MMSE / KL-Divergence quantization algorithms and the Layer / Channel quantization methods; the GDQ quantization algorithm and the Group

quantization method are described in 2.2.3 Internal Quantization Process. The following introduces these quantization algorithms and feature quantization methods:

- Quantization algorithms:
- Normal quantization algorithm: determines the maximum and minimum values of the quantization range directly from the maximum and minimum floating-point values of the model features (feature). It is fast, but performs poorly when the feature distribution is imbalanced. The recommended quantization dataset size is generally about 20-100 images.
- MMSE quantization algorithm: determines the maximum and minimum values of the quantization range by minimizing the mean-square-error loss between floating-point values and quantize-dequantize floating-point values, which can alleviate the quantization accuracy loss caused by large outliers to a certain extent. It uses brute-force iteration and is slower, but usually achieves higher accuracy than normal. The recommended quantization dataset size is generally about 20-50 images, which can be adjusted according to the quantization time.
- KL-Divergence quantization algorithm: abstracts the floating-point and fixed-point values of model features into two distributions, updates the distributions by adjusting thresholds, and determines the maximum and minimum values of the quantization range by measuring the similarity of the two distributions with KL divergence. It takes more time than normal but much less than mmse, and can achieve good improvement in some scenarios (uneven feature distribution). The recommended quantization dataset size is generally about 20-100 images.
- Quantization methods:
- Layer quantization method: quantizes all channels of the same network layer as a whole; all channels share the same quantization parameters.
- Channel quantization method: quantizes each output channel of the same network layer independently; each channel has its own quantization parameters. Generally, the Channel method achieves higher accuracy than the Layer method.
- According to the actual model quantization effect and requirements, users can choose a suitable quantization algorithm and feature quantization method. For more details on quantization, see Chapter 2.2 (Model Quantization).
- GDQ quantization algorithm: only valid for w4a16 and w8a16; it can effectively improve the weight quantization accuracy of w4a16 and w8a16. The recommended quantization dataset size is more than 200 images. See 2.2.3.2 Internal Quantization Algorithms for details.

2.1.8 Model Pruning

Model pruning can generally be divided into two methods: lossy pruning and lossless pruning. The model pruning of RKNN-Toolkit2 is lossless pruning, which means that it can reduce the size of the model and the amount of calculation without degrading floating point accuracy of the model. And the quantization accuracy of the model can even be improved.

However, not all models can be used for lossless pruning. Lossless pruning is based on the degree of sparseness of the model's weights to remove some weights and feature channels that have no impact on the model, results to reduce the size and the calculation of the model After the model pruning configuration is enabled (set the model_pruning parameter of rknn.config() to

True), the model will be automatically pruned according to the degree of weight sparsification during model conversion.

If the model is pruned successfully, the results will be printed, as follows:

```
I model_pruning results:

I Weight: -1.12145 MB (-6.9%)

I GFLOPs: -0.15563 (-13.4%)
```

Among Weight cut off 1.12145 MB (accounting for 6.9%), and the model calculation was reduced by 0.15563 GFLOPs (accounting for 13.4%).

If pruning fails due to insufficient model sparsity, no processing will be done (nor will the above information be printed), so this configuration will not affect normal model conversion.

2.1.9 ONNX Model Editing

Some models may have redundant ops after being converted to the RKNN model. It is common for redundant reshape and transpose ops to exist at the input and output nodes of the model, which affects the inference performance of the RKNN model. RKNN-Toolkit2 provides the onnx_edit interface, which is used to modify the input and output dimension definitions of the ONNX model, so that the adjusted onnx model can be converted into a RKNN model with better performance and reduce redundant reshape and transpose ops.

2.1.9.1 INTERFACE DESCRIPTION

Usage example:

```
from rknn.utils import onnx_edit
ret = onnx_edit(model = './concat_block.onnx',
export_path = './concat_block_edited.onnx',
inputs_transform = {'k_in': 'a,b,c,d->1,ad,b,c'},
outputs_transform = {'k_cache': 'a,b,c,d->1,ab,c,d'},
dataset = './dataset.txt'
)
```

- Model fills in the model to be modified, which is a required parameter.
- Export_path fills in the generation path of the new model, which is a required parameter.
- Inputs_transform fills in the transformation formula dictionary of the input node, the key is the node name, and the value is the transformation formula. It is an optional parameter and defaults to an empty dictionary.
- Outputs_transform fills in the transformation formula dictionary of the output node. It is an optional parameter and defaults to an empty dictionary.
- Dataset fills in the path set file of the input data. The file format requirements are consistent with the requirements of the rknn.build interface for dataset. After filling in, the onnx_edit interface will not only adjust the input and output definitions of the model, but also adjust the corresponding data in the dataset, and generate new dataset data in the directory at

the same level as export_path, which can be used for verification and quantification of the new model. It is an optional parameter and is empty by default.

2.1.9.2 TRANSFORMATION FORMULA DESCRIPTION

The inputs_transform and outputs_transform in the onnx_edit interface need to be filled with transformation formulas. The definition of the transformation formula is similar to the definition of the einsum operator. For example, the 'a,b,c,d->1,ad,b,c' formula refers to transforming the original dimensions into 'a,b,c,d'. followed by '1,ad,b,c'. The rules for filling in the transformation formula are as follows:

- Must be composed of two parts of characters, separated by '->'. The left side is the original character shape, and the right side is the transformed character shape.
- The symbols in the character shape are only allowed to be [a-z] or ',' '1'. Except for '1', other numeric shape characters are not supported. The reason is that the same numeric characters cannot determine the context of transpose.
- Characters [a-z] are considered independent and have no sequential relationship, that is, 'a,b->b,a' and 'c,a->a,c' represent the same transformation
- The original character shape on the left side of the equation is separated into n parts by ','. The number of n must match the dimensions of the model. For example, the model input definition is [32,4,1,64], and the input character can be 'a ,b,c,d' or 'a,b,1,d'
- Every symbol of the original character shape, except '1', must exist in the transformed character shape. For example, 'a,1,c,d->ac,d,1' is valid, 'a,1,c,d->ac,1' is invalid, 'd' is missing
- The transformed character shape allows the insertion of any number of '1's to achieve the dimension expansion effect, such as 'a,b,c-> a,1,cb,1,1'
- Original character shape allows multiple letters and assignment formulas to be used to express splitting the shape. For example, the original input definition is [32,4,1,64], 'ab,c,d,qk[a=2,k =8]->aq,cd,1,kb', which means split 32 into 2x16, split 64 into 8x8, and then perform transpose and reshape operations. The '[ ]' part is called the assignment formula, and multiple formulas are separated by ',' symbols. In addition, it is allowed that a certain character in the split is not assigned a value. At this time, the corresponding shape will be automatically inferred. For example, the assignment formula only gives a=2, and it is known that ab=32 in the model, then b=16 is automatically inferred; if The inferred shape exception will directly report an error. For example, ab=32. If a=5 is assigned, then b=6.4, and the dimension must be an integer. In this case, an exception error will be thrown.

2.1.9.3 TRANSFORMATION FORMULA EXAMPLE

- Change the 3-dimensional input to 4-dimensional input: 'a,b,c->a,b,1,c'
- Change the 5-dimensional input to 4-dimensional input: 'a,b,c,d,e->ab,c,d,e'
- Perform transpose(0,3,1,2) operation: 'a,b,c,d->a,d,b,c'
- Transpose and merge some dimensions: 'a,b,c,d->d,acb,1'
- Split dimensions, transpose, merge dimensions: 'a,bc,de,f[b=2,d=4]->ab,fe,dc,1'

2.2 Model Quantization

2.2.1 Basic Concepts of Quantization

2.2.1.1 QUANTIZATION DEFINITION

Model quantization refers to converting the floating-point parameters or activation values of a deep learning model into fixed-point representations, such as converting FP32 to INT8. Quantization reduces memory usage and enables model compression and inference acceleration, but it usually introduces a certain degree of accuracy loss.

According to the form of the mapping function between floating-point values and fixed-point numbers, quantization can be divided into linear quantization and non-linear quantization. Common non-linear quantization methods include exponential quantization, codebook quantization, and piecewise linear quantization. RKNN-Toolkit2 supports linear quantization only. In linear quantization, the intervals between fixed-point numbers are uniform; for example, INT8 linear quantization divides the quantization range evenly into 256 values, which is why it is also called uniform quantization. According to whether the quantization zero point is 0, linear quantization can be divided into symmetric quantization and asymmetric quantization. Symmetric quantization is a simplified form of asymmetric quantization; asymmetric quantization usually handles non-uniform data distributions better, so asymmetric quantization is more commonly used in practice.

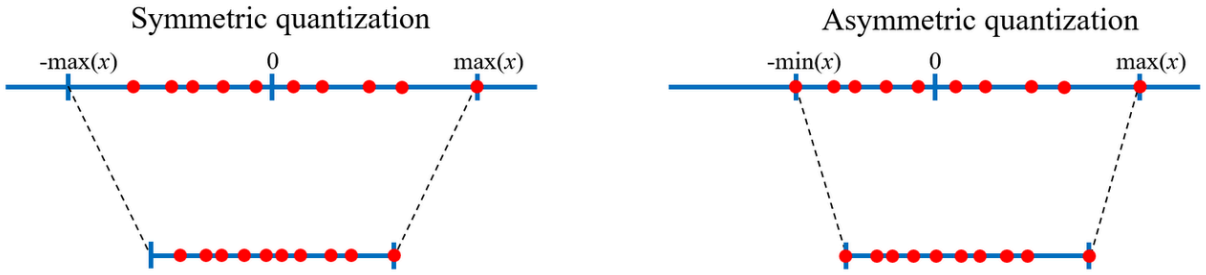


Figure 2-2 Linear Symmetric Quantization and Linear Asymmetric Quantization

2.2.1.2 QUANTIZATION CALCULATION

Taking linear asymmetric quantization as an example, the calculation of quantizing a floating-point number to a signed fixed-point number is as follows:

$$x_{int} = clamp(\lfloor \frac{x}{s} \rceil + z; -2^{b-1}, 2^{b-1} - 1) \tag{6-1}$$

where $x is the floating-point number, x_{int} is the quantized fixed-point number, $\lfloor \cdot \rceil$ is the round-to-nearest operation, s is the quantization scale factor, z is the quantization zero point, and b is the quantization bit width, e.g. b is 8 for the INT8 data type; $clamp$ is the truncation operation, defined as follows:

$$clamp(x; a, c) = \begin{cases} a, & x < a, \\ x, & a \le x \le c, \\ c, & x > c, \end{cases} \tag{6-2}$$

Converting a fixed-point number back to a floating-point number is called dequantization, defined as follows:

$$x \approx \widehat{x} = s(x_{int} - z) \tag{6-3}$$

where $\widehat{x}$ is also called the pseudo-quantized floating-point number. It is easy to see that, due to the rounding and truncation operations in the quantization calculation, $\widehat{x}$ is not exactly equal to x ; the difference between them is called the quantization error.

Let the quantization range be $(q_{\min}, q_{\max})$ and the truncation range be $(c_{\min}, c_{\max})$. The quantization parameters s and z are calculated as follows:

$$s = \frac{q_{\max} - q_{\min}}{c_{\max} - c_{\min}} = \frac{q_{\max} - q_{\min}}{2^b - 1} \tag{6-4}$$

$$z = c_{\max} - \lfloor \frac{q_{\max}}{s} \rceil \text{ or } z = c_{\min} - \lfloor \frac{q_{\min}}{s} \rceil \tag{6-5}$$

where the truncation range is determined by the quantized data type; for example, the truncation range of INT8 is (-128, 127). The quantization range is determined by the specific quantization algorithm. Setting $z = 0$ in the above definitions yields the calculation definition of linear symmetric quantization.

2.2.1.3 QUANTIZATION GRANULARITY

Quantization granularity defines which parameters in a model's floating-point tensor are quantized as a whole. RKNN-Toolkit2 supports three quantization granularities: Per-Layer / Per-Channel / Group. Per-Layer quantization quantizes all channels of a layer's weights or activations as a whole, with all channels sharing the same quantization parameters. Per-Channel quantization quantizes each output channel of a layer's weights independently, with each channel having its own quantization parameters. Group quantization applies only to linear-layer weights; on top of Per-Channel quantization, the elements of each output channel are split into groups, and each group is quantized independently.

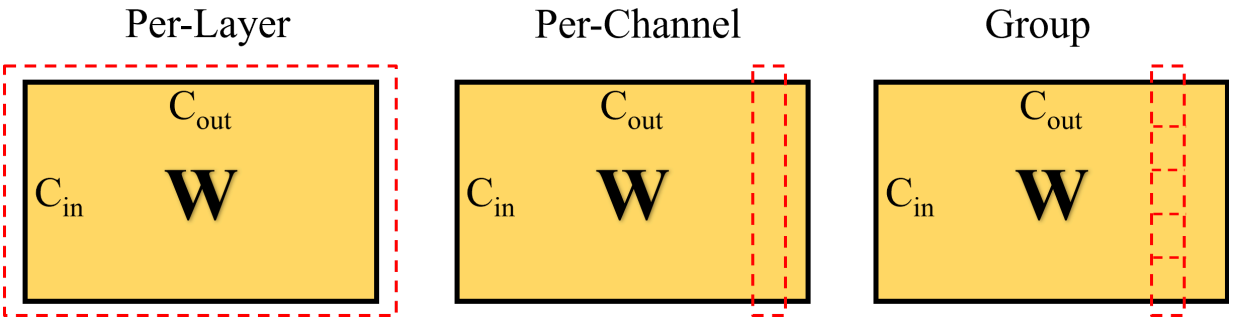


Figure 2-3 Per-Layer, Per-Channel and Group quantization

> **Note**: In RKNN-Toolkit2, both Group and Per-Channel quantization apply to weights only; activations and intermediate values still use Per-Layer quantization. Group quantization is only valid for linear layers under w4a16.

2.2.1.4 QUANTIZATION ALGORITHMS

A quantization algorithm is a scheme for solving the mapping from floating-point tensors to low-bit fixed-point representations and their quantization parameters under a given quantization form (e.g. linear quantization). By implementation, they fall into two categories: quantization-aware training and post-training quantization. Quantization-aware training fine-tunes model and quantization parameters during training to improve accuracy; it usually requires deep understanding of the training pipeline and a large amount of training data, and is therefore rarely used directly in model deployment toolchains. Post-training quantization usually requires only a small calibration dataset, or even none, and obtains quantization parameters at low cost. RKNN-Toolkit2 supports four post-training quantization algorithms: Normal / KL-Divergence / MMSE and GDQ.

2.2.2 Overview of the Quantization Process

RKNN-Toolkit2 uses post-training quantization (PTQ): quantization is completed together with model conversion, via the internal quantization process. The floating-point model is first loaded into the Toolkit, and quantization is performed and the RKNN model exported through `rknn.config()` ⟶ `rknn.load_xxx()` ⟶ `rknn.build()` ⟶ `rknn.export_rknn()`. The overall quantization process is:

```
Floating-point model ⟶ rknn.config() to configure quantization parameters ⟶
rknn.load_xxx() to load the model ⟶ rknn.build(do_quantization=True, dataset=...)
to quantize and build ⟶ rknn.export_rknn() to export the RKNN model
```

The key elements of the quantization process and where they are configured:

- **Quantization data type**: configured via `quantized_dtype` of `rknn.config()`; RKNN-Toolkit2 supports w8a8, w4a16, w8a16 and other quantization data types (see 2.2.3.1 Data Types).
- **Quantization algorithm**: configured via `quantized_algorithm` of `rknn.config()`; four post-training quantization algorithms are supported: Normal / KL-Divergence / MMSE / GDQ (principles in 2.2.1.4 Quantization Algorithms, details in 2.2.3.2 Internal Quantization Algorithms).
- **Quantization granularity**: configured via `quantized_method` of `rknn.config()`; three granularities are supported: layer (Per-Layer) / channel (Per-Channel) / group{SIZE} (Group).
- **Quantization calibration set**: specified via the `dataset` parameter of `rknn.build()`, used to compute the quantization parameters of activations (see 2.2.3.3 Calibration Data Preparation).
- **Hybrid quantization**: assigns higher-precision data types to quantization-sensitive layers to mitigate accuracy loss (see 2.2.4 Hybrid Quantization).
- **Quantization-aware training (QAT)**: inserts fake-quantization nodes during training and fine-tunes model parameters; suitable for scenarios where post-training quantization cannot meet accuracy requirements (see 2.2.5 Quantization-Aware Training).

2.2.3 Internal Quantization Process

2.2.3.1 DATA TYPES

The quantization data types supported by RKNN-Toolkit2 internal quantization are `w8a8`, `w4a16`, `w8a16`, `w4a8`, `w16a16i` and `w16a16i_dfp`, with a default value of `w8a8`. w4/w8/w16 means the weights are quantized to the corresponding bit width (int4/int8/int16), and a8/a16 means the activations are quantized to int8/float16. The quantization data types are described as follows:

- **w8a8** (default): weights in 8-bit asymmetric quantized precision, activations in 8-bit asymmetric quantized precision. Not supported on RK2118.
- **w4a16**: weights in 4-bit asymmetric quantized precision, activations in 16-bit floating-point precision. Only supported on RK3576 / RV1126B.
- **w8a16**: weights in 8-bit asymmetric quantized precision, activations in 16-bit floating-point precision. Only supported on RK3562.
- **w4a8**: weights in 4-bit asymmetric quantized precision, activations in 8-bit asymmetric quantized precision. Not supported yet.
- **w16a16i**: weights in 16-bit asymmetric quantized precision, activations in 16-bit asymmetric quantized precision. Only supported on RV1103 / RV1106.
- **w16a16i_dfp**: weights in 16-bit dynamic fixed-point quantization precision, activations in 16-bit dynamic fixed-point quantization precision. Only supported on RV1103 / RV1106.

2.2.3.2 INTERNAL QUANTIZATION ALGORITHMS

Internal quantization supports four quantization algorithms: Normal / KL-Divergence / MMSE / GDQ:

- **Normal**: determines the quantization range directly from the maximum and minimum floating-point values. It introduces no clipping error but is sensitive to outliers; it is fast and suitable for general scenarios. The quantization range is computed as:

$$q_{\min} = \min\ \mathbf{V} \tag{6-6}$$

$$q_{\max} = \max\ \mathbf{V} \tag{6-7}$$

where V is the floating-point tensor.

- **KL-Divergence**: determines the quantization range by comparing the distributions of floating-point and fixed-point values, adjusting thresholds and minimizing the KL divergence. It adapts better to non-uniform data distributions and mitigates the impact of a few outliers; slightly slower than Normal.

$$\underset{q_{\min},\, q_{\max}}{\arg\min}\, \mathrm{H}\left(\Psi(\mathbf{V}),\ \Psi\left(\mathbf{V}_{\text{int}}\right)\right) \tag{6-8}$$

where $H(\cdot,\cdot)$ is the KL divergence, $\Psi(\cdot)$ is the function mapping the data to a discrete distribution, and V_{int} is the quantized fixed-point tensor.

- **MMSE**: determines the quantization range by minimizing the mean-square error between floating-point values and dequantized floating-point values. It usually achieves higher

accuracy but requires more computation; it is slower and has higher memory overhead. Compared with KL-Divergence, it often performs better in outlier scenarios.

$$\arg\min_{q_{\min},\, q_{\max}} \left\| \mathbf{V} - \widehat{\mathbf{V}}\left(q_{\min},\, q_{\max}\right) \right\|_F^2 \tag{6-9}$$

where $\widehat{\mathbf{V}}\left(q_{\min},\, q_{\max}\right)$ is the quantize-dequantize form of $\mathbf{V}$, and $\|\cdot\|_F$ is the Frobenius norm.

- **GDQ**: only valid when quantized_dtype is w4a16 or w8a16; it can effectively improve the weight quantization accuracy of w4a16 and w8a16. The recommended quantization dataset size is more than 200 images. For details of the GDQ algorithm, refer to the 《03_RKNN2_Toolkit_Python_API_Reference_V2.5.0》 manual.

It is recommended to use the Normal algorithm first; when quantization accuracy issues arise, try KL-Divergence, MMSE or GDQ. For w4a16 / w8a16 quantization, try the GDQ algorithm first.

2.2.3.3 CALIBRATION DATA PREPARATION

In the internal quantization process, the calibration dataset is used to compute the quantization range of activations. It should cover the different data distributions of the model's real application scenarios; for example, the calibration dataset of a classification model should contain images of different categories from real scenarios. A calibration dataset of 20~200 images is generally recommended, adjustable according to the runtime of the quantization algorithm; when using the GDQ quantization algorithm, a calibration dataset of more than 200 images is recommended. For other quantization algorithms, increasing the calibration dataset size increases runtime but does not necessarily improve accuracy.

Internal quantization calibration data currently supports text file format: write the paths of calibration images (jpg or png) or npy files into a `.txt` file, one path per line:

```
a.jpg
b.jpg
```

For multiple inputs, separate the files of each input with spaces:

```
a0.jpg a1.jpg
b0.jpg b1.jpg
```

2.2.3.4 CONCRETE EXAMPLE

Example 1: ResNet50 internal quantization process example:

```
from rknn.api import RKNN

model_path = "resnet50-v2-7.onnx" ## Floating-point ONNX model exported in advance
output_path = 'resnet50-v2-7.rknn'
DATASET_PATH =
'../../../datasets/imagenet/ILSVRC2012_img_val_samples/dataset_20.txt'

# Create RKNN object
rknn = RKNN(verbose=False)

# Pre-process config
print('--> Config model')
rknn.config(mean_values=[[255*0.485, 255*0.456, 255*0.406]],
std_values=[[255*0.229, 255*0.224, 255*0.225]],
target_platform="rk3588",
quantized_dtype='w8a8',           ## "w4a16", "w8a16" etc. are optional (subject to
actual platform support)
quantized_algorithm='normal',     ## "MMSE", "KL-Divergence", "GDQ" are optional
quantized_method='channel',)
print('done')

# Load model
print('--> Loading model')
ret = rknn.load_onnx(model=model_path, inputs=['data'], input_size_list=[[1, 3,
224, 224]])

if ret != 0:
print('Load model failed!')
exit(ret)
print('done')

# Build model
print('--> Building model')
ret = rknn.build(do_quantization=True, dataset=DATASET_PATH)
if ret != 0:
print('Build model failed!')
exit(ret)
print('done')

# Export rknn model
print('--> Export rknn model')
ret = rknn.export_rknn(output_path)
if ret != 0:
print('Export rknn model failed!')
exit(ret)
print('done')
```

2.2.4 Hybrid Quantization

Hybrid quantization applies different quantization data types to different layers of the model, using higher-precision data types to represent layers that are not suitable for quantization, thereby mitigating the accuracy loss caused by model quantization. However, hybrid quantization introduces additional overhead and requires the user to determine the quantization data type of each layer.

2.2.4.1 USAGE OF HYBRID QUANTIZATION

To achieve a better balance between performance and accuracy, RKNN-Toolkit2 provides a hybrid quantization feature. Users can manually specify whether each layer should be quantized based on the output results of accuracy analysis.

Currently, the hybrid quantization feature supports the following usages:

1. Change specified quantized layers to non-quantized layers, e.g. compute them in FLOAT16. (Because non-quantized compute power on the NPU is lower, the inference speed will decrease to some extent.)
2. The quantization parameters of each layer can also be modified. (Modifying quantization parameters is not recommended.)

2.2.4.2 PROCESS OF HYBRID QUANTIZATION USAGE

When using the hybrid quantization feature, it involves four specific steps.

1. Load the original model and generate the quantization configuration file, temporary model file and data file. The specific interface calling process is as follows:

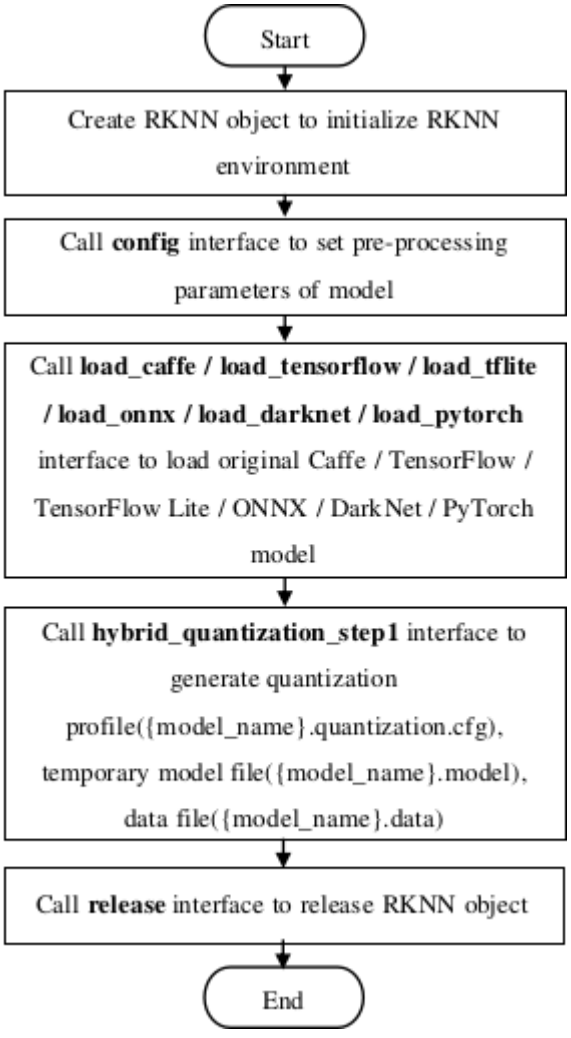


Figure 2-4 Hybrid quantization step 1

1. Modify the quantization configuration file generated in the first step.

After calling the hybrid_quantization_step1 interface in the first step, a configuration file named {model_name}.quantization.cfg will be generated in the current directory. The format of the configuration file is as follows:

```
custom_quantize_layers:
Conv__350:0: float16
Conv__358:0: float16
....
quantize_parameters:
....
FeatureExtractor/MobilenetV2/Conv/Relu6:0:
qtype: asymmetric_quantized
qmethod: layer
dtype: int8
min:
- 0.0
max:
- 6.0
scale:
- 0.023529411764705882
zero_point:
- -128
....
```

In the "custom_quantize_layers" section, users can add custom quantization layers by specifying the tensor name followed by the quantization type. This will change the operation type of the corresponding layer to the specified type. Currently, the available quantization data type is "float16".

The "quantize_parameters" section contains the quantization parameters for each tensor in the model. The quantization parameters for each tensor are presented in the format of "tensor_name: quantization_properties_and_parameters". The "min" and "max" represent the minimum and maximum values of the quantization range. Users can refer to the output results of precision analysis or use Netron to open the temporary model file named {model_name}.model to view the corresponding output tensor names.

1. Generate the RKNN model. The specific interface calling process is as follows:

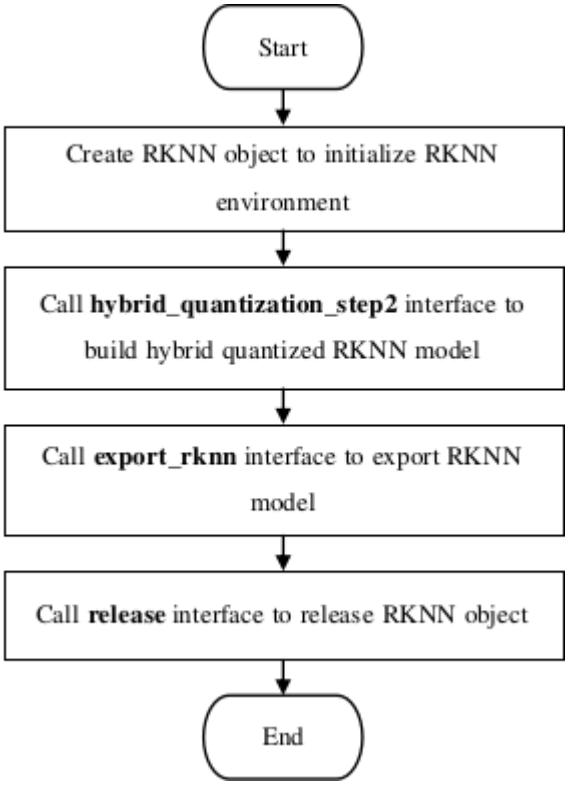


Figure 2-5 Hybrid quantization step 3

1. Perform inference using the RKNN model generated in the third step.

Note: In the RKNN-Toolkit2 project, under the "examples/functions/hybrid_quant" directory, there is an example of hybrid quantization. Users can refer to this example to perform hybrid quantization.

2.2.4.3 AUTOMATIC HYBRID QUANTIZATION

To make hybrid quantization easier to use and to solve model overflow issues, RKNN-Toolkit2 provides an automatic hybrid quantization feature. Through the `auto_hybrid` interface, users can automatically adjust the hybrid quantization model based on Euclidean distance and cosine distance thresholds during model quantization; when the model is non-quantized, it checks layer by layer whether fp16 overflow occurs and converts the overflowing layers to int16 computation.

Currently, the automatic hybrid quantization feature supports the following usages:

1. During model quantization, the hybrid quantization model is automatically adjusted based on Euclidean distance and cosine distance thresholds. Currently, only automatic adjustment from int8 quantization to fp16 is supported.

```
ret = rknn.build(do_quantization=True, dataset='./dataset.txt', auto_hybrid=True)
```

The thresholds for cosine distance and Euclidean distance can be set through `auto_hybrid_cos_thresh` and `auto_hybrid_euc_thresh` in the `config` interface. The cosine distance threshold defaults to 0.98, and the Euclidean distance threshold is not enabled by default. That is, by default only operators whose cosine distance accuracy is below the specified threshold are converted to fp16, in which case the Euclidean distance threshold has no effect. When the Euclidean distance threshold is enabled, operators whose cosine distance is below the cosine

distance threshold or whose Euclidean distance is above the Euclidean distance threshold are converted to fp16.

1. When the model is non-quantized, it checks layer by layer whether fp16 overflow occurs and converts the overflowing layers to int16 computation. In this case, a dataset needs to be provided to calculate the numerical range of each layer

   ```
   ret = rknn.build(do_quantization=False, dataset='./dataset.txt',
   auto_hybrid=True)
   ```

   The maximum and minimum values of each layer are calculated based on the given input. When the maximum or minimum value exceeds the upper or lower limit of fp16, the operators of the corresponding layer are converted to int16.

2.2.5 Quantization-Aware Training (QAT)

2.2.5.1 INTRODUCTION TO QAT

Quantization-Aware Training (QAT) is a training method aimed at addressing the accuracy loss issue in low-bit quantization. Low-bit quantization can result in accuracy loss because converting the value range from floating-point numbers to fixed-point numbers incurs precision loss. During QAT training, the quantization error is incorporated into the training loss function, allowing the training of a model with quantization parameters.

Contrasting with Post Training Quantization (PTQ) provided by RKNN-Toolkit2, the characteristics of these two quantization methods are as follows:

Quantization Method	Secondary training based on the original framework	Dataset	Adjust the weight parameters	Loss function	Performance
Post Training Quantization (PTQ)	No	Small amount of unlabeled data	No	Irrelevant	Optimal
Quantization-Aware Training (QAT)	Yes	Complete training dataset	Yes	Quantization loss is included in the training loss function.	When there are operators that do not support QAT, the performance is slightly weaker than PTQ.

2.2.5.2 QAT PRINCIPLE

During QAT training, all weights are stored and calculations are performed in floating-point format, which ensures that backpropagation functions properly and the model can be trained effectively. Unlike training a floating-point model, QAT inserts FakeQuantize modules at quantizable locations in the model to simulate precision loss when converting from floating-point numbers to fixed-point numbers. This allows the loss function to recognize and optimize the

quantization, ensuring accurate inference results when the model is converted to a fixed-point model.

QAT training is widely used and supported by mainstream inference frameworks. Users can refer to the links below for more detailed usage instructions.

Pytorch - https://pytorch.org/blog/quantization-in-practice/#quantization-aware-training-qat

Paddle - https://paddleslim.readthedocs.io/zh-cn/develop/api_cn/dygraph/quanter/qat.html

Tensorflow - https://www.tensorflow.org/model_optimization/guide/quantization/training

2.2.5.3 QAT USAGE GUIDELINES

Since QAT requires additional training code, and the code of some open-source repositories may conflict with the QAT feature, it is recommended to consider using QAT training for model quantization only when both of the following conditions are met:

1. Perform quantization accuracy troubleshooting as described in Chapter 5.1 to confirm that the PTQ feature of RKNN does not meet the accuracy requirements.
2. Try hybrid quantization as described in Chapter 2.2.4 to confirm that the hybrid quantization feature of RKNN does not meet the accuracy and performance requirements.

2.2.5.4 QAT IMPLEMENTATION EXAMPLE AND CONFIGURATION INSTRUCTIONS

Here are the documentations for QAT features in various frameworks. Please refer to the official documentation for actual usage.

PyTorch: https://pytorch.org/docs/stable/quantization.html (PyTorch has multiple sets of quantization interfaces. RKNN currently supports quantized models generated from the FX interface, specifically the prepare_qat_fx interface and its associated interfaces.)

Paddle: https://www.paddlepaddle.org.cn/tutorials/projectdetail/3949129#anchor-14

TensorFlow: https://www.tensorflow.org/model_optimization/guide/quantization/training

Taking PyTorch as an example to demonstrate the process of implementing QAT and some important considerations.

```
# for 1.10 <= torch <= 1.13
import torch
class M(torch.nn.Module):
def __init__(self):
super().__init__()
self.conv = torch.nn.Conv(3, 8, 3, 1)

def forward(self, x):
x = self.conv(x)
return x

# initialize a floating point model
float_model = M().train()

from torch.quantization import quantize_fx, QConfig, FakeQuantize,
MovingAverageMinMaxObserver, MovingAveragePerChannelMinMaxObserver

qconfig = QConfig(activation=FakeQuantize.with_args(observer=
MovingAverageMinMaxObserver,
quant_min=0,
quant_max=255,
reduce_range=False),   # reudece_range -> the default is True

weight=FakeQuantize.with_args(observer=
MovingAveragePerChannelMinMaxObserver,
quant_min=-128,
quant_max=127,
dtype=torch.qint8,
qscheme=torch.per_channel_affine,
#qscheme->the default is torch.per_channel_symmetric
reduce_range=False))
qconfig_dict = {"": qconfig}
model_qat = quantize_fx.prepare_qat_fx(float_model, qconfig_dict)

# define the training loop for quantization aware training
def train_loop(model, train_data):
model.train()
for image, target in data_loader:
...
# Run training
train_loop(model_qat, train_loop)

model_qat = quantize_fx.convert_fx(model_qat)
```

In the above code flow, except for the adjustments made to the qconfig configuration for RKNN hardware, all other operations are implemented according to the guidance provided in the official codes. There are two main modifications in the qconfig:

The activation quantization configuration specifies reduce_range as False. When reduce_range is False, the effective quantization range is from -128 to 127. When reduce_range is True, the effective quantization range is from -64 to 63, which results in poorer quantization performance. RKNN hardware supports reduce_range as False.

The weight quantization configuration specifies qscheme as torch.per_channel_affine. The default torch.per_channel_symmetric restricts zero_point to be fixed at 0, but RKNN hardware supports non-zero zero_point. Therefore, torch.per_channel_affine is chosen.

2.2.5.5 SUPPORTED OPERATORS IN QAT

Taking PyTorch as an example, during the QAT process, Conv and Linear operators with weight parameters are first quantized according to QAT rules. Then other operators are examined. If they meet the regular quantization rules, they will be quantized in regular quantization. If an operator does not meet the QAT and regular quantization rules, it will maintain the FP32 computation rule.

Different frameworks have different levels of support. Users can refer to the following links to find more detailed information about the supported quantization operators based on the framework and version they are using:

Pytorch:
https://github.com/pytorch/pytorch/blob/main/torch/ao/quantization/quantization_mappings.py

Paddle:
https://github.com/PaddlePaddle/Paddle/blob/86df789a567f1285101c57b6e3ada4b952c58f48/python/paddle/quantization/config.py

Tensorflow:
https://www.tensorflow.org/model_optimization/api_docs/python/tfmot/quantization/keras/QuantizeConfig

2.2.5.6 HANDLING OF FLOATING-POINT OPERATORS IN QAT MODEL

In a QAT model, when an operator cannot be quantized, floating-point computation is used for it. When such operators are converted to an RKNN model, there are two scenarios to consider.

1. Preceded and followed by quantizable operators:

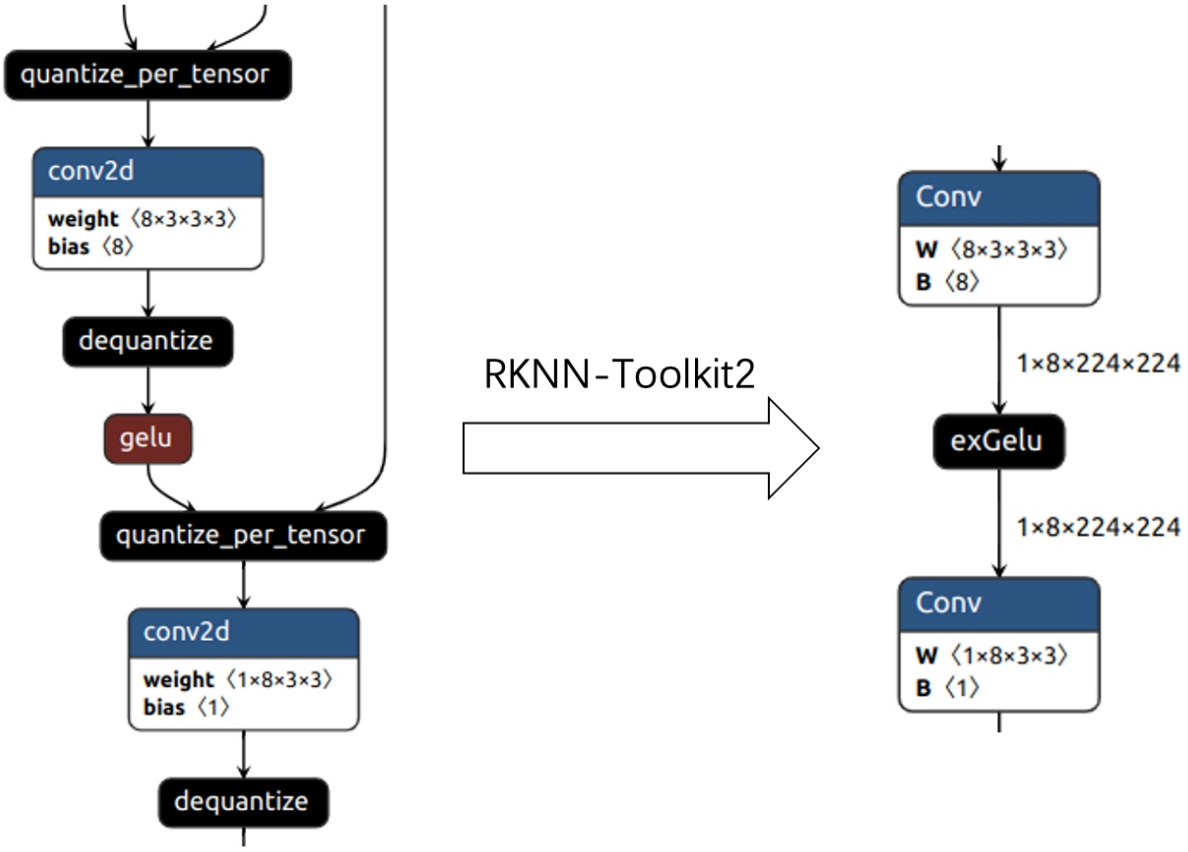


Figure 2-6 QAT OP preceded and followed by quantizable operators

As shown on the left of the figure above, the gelu operator in the original model is a floating-point operator, and the conv operators before and after it are quantized operators. When RKNN-Toolkit2 loads the model, it converts the floating-point operator between the two quantized operators into a quantized operator to improve inference performance, and this operation does not affect accuracy. After conversion to the RKNN model, the structure is as shown on the right of the figure above.

1. Non-quantizable operators exist before or after:

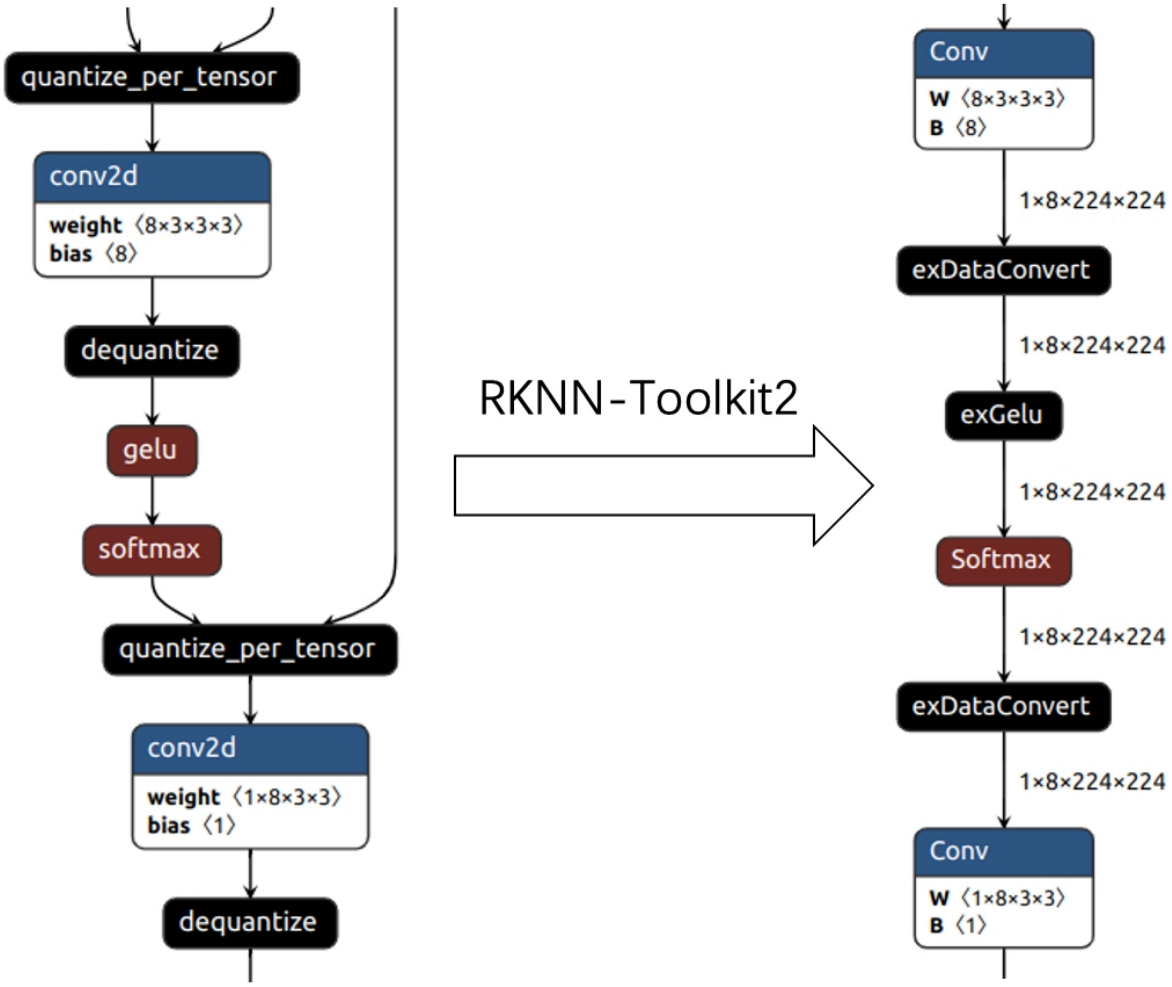


Figure 2-7 QAT OP preceded and followed by non-quantizable operators

As shown on the left of the figure above, the gelu and softmax operators in the original model are floating-point operators, and the conv operators before and after them are quantized operators. When RKNN-Toolkit2 loads the model, because the quantization parameters for gelu and softmax are missing, gelu and softmax remain in floating-point type. After conversion to the RKNN model, the structure is as shown on the right of the figure above: the RKNN model inserts a dequantization operator before gelu and a quantization operator after softmax, and these inserted quantization and dequantization operators all introduce additional time consumption.

2.2.5.7 SUMMARY OF QAT EXPERIENCE

1. QAT configuration

Depending on the characteristics of different hardware, QAT training often requires configuration adjustments to achieve better results. For RKNPU, it is recommended to configure the qconfig parameters by referring to the code instructions in 2.2.5.4 QAT Implementation Example and Configuration Instructions.

1. Quantization parameters saved in the model may require secondary adjustment

Take sigmoid as an example: the quantization parameters recorded by the sigmoid operator in the model may not be the ones actually used during inference. In the official code (https://github.com/pytorch/pytorch/blob/main/aten/src/ATen/native/quantized/cpu/qsigmoid.cpp),

the quantization parameters of sigmoid are adjusted during inference, setting min to 0 and max to 1.

For such operators, RKNN-Toolkit2 adjusts their quantization parameters during the model conversion stage when converting QAT models, making the inference results closer to the original PyTorch inference results.

2.3 Model Evaluation

2.3.1 Model Inferencing

After the original model is converted into an RKNN model, inference can be performed on the simulator or on the development board, and the inference results can be post-processed to check whether they are correct. If the inference results are incorrect, you can run accuracy analysis on the quantized model and check whether the pre-processing and post-processing flows are correct. The main interface parameters used in the model inference stage are described as follows:

The runtime initialization interface `rknn.init_runtime()` takes the following parameters:

- target: target hardware platform. The default value is `None`, and inference is performed on the simulator. To obtain the actual on-board inference results, this parameter must be set to the corresponding platform (RK3562 / RK3566 / RK3568 / RK3588 / RK3576 / RV1103 / RV1103B / RV1106 / RV1106B / RV1126B).
- device_id: device number. The default value is `None`. If target is set, the only device is selected for inference. If the computer is connected to multiple devices for board-connected inference, the corresponding device ID must be specified. If a device is connected over the network via ADB for model inference, the user needs to manually run the command `adb connect [IP]` to connect to the network device first and then fill in the device number, usually in the form of `[IP]` or `[IP:Port]`. The `adb devices` command can be used to list all connected devices.
- perf_debug: whether to enable `debug` mode during performance evaluation. The default value is `False`. Calling the `rknn.val_perf()` interface returns the total running time of the model. When set to `True`, the running time of each layer can be obtained; this is not supported on RV1103, RV1103B, RV1106 and RV1106B platforms.
- eval_mem: whether to enter memory evaluation mode. The default value is `False`. After setting it to `True` to enter memory evaluation mode, the `rknn.eval_memory()` interface can be called to obtain the memory usage of the model at runtime.

The inference interface `rknn.inference()` takes the following parameters:

- inputs: list of inputs to be inferred, in ndarray format.
- data_format: list of layouts of the input data, only valid for 4-dimensional inputs, in the format "`NCHW`" or "`NHWC`". The default value is `None`, which means the layout of all inputs is `NHWC`.

Example code:

```
ret = rknn.init_runtime(target=platform, device_id='515e9b401c060c0b')

# Preprocess
image_src = cv2.imread(IMG_PATH)
img = preprocess(image_src)

# Inference
outputs = rknn.inference(inputs=[img])

# Postprocess
outputs = postprocess(outputs)
```

Notes: 1.When performing inference on the RV1103 / RV1106 platforms, if the error "`E RKNN: failed to allocate fd, ret: -1, errno: 12`" is encountered, you can run `RkLunch-stop.sh` in the development board terminal to close other memory-consuming applications before performing board-connected inference. Updating `rknn_server` to version 2.1.0 can also mitigate this problem.

2.In the ARM64 version, `rknn.init_runtime()` does not need device_id

2.3.2 Model Accuracy Analysis

If there are problems with the inference results of a quantized model, you can use the `rknn.accuracy_analysis()` interface to perform accuracy analysis and check the accuracy of each layer of operators.

The accuracy analysis interface `rknn.accuracy_analysis()` takes the following parameters:

- inputs: list of input file paths (formats include `jpg`, `png`, `bmp` and `npy`).
- output_dir: directory where the analysis results are saved, default value '`./snapshot`'.
- target: target hardware platform. The default value is `None`, and the simulator is used for accuracy analysis. If target is set (RK3562 / RK3566 / RK3568 / RK3588 / RK3576 / RV1103 / RV1103B / RV1106 / RV1106B / RV1126B), the results of each layer at runtime will be obtained and accuracy analysis will be performed.
- device_id: device number. The default value is `None`. If target is set, the only device is selected for accuracy analysis. If the computer is connected to multiple devices, the corresponding device ID must be specified. If a device is connected over the network via ADB for accuracy analysis, the user needs to manually run the command `adb connect [IP]` to connect to the network device first and then fill in the device number, usually in the form of `[IP]` or `[IP]:[Port]`.

Notes:

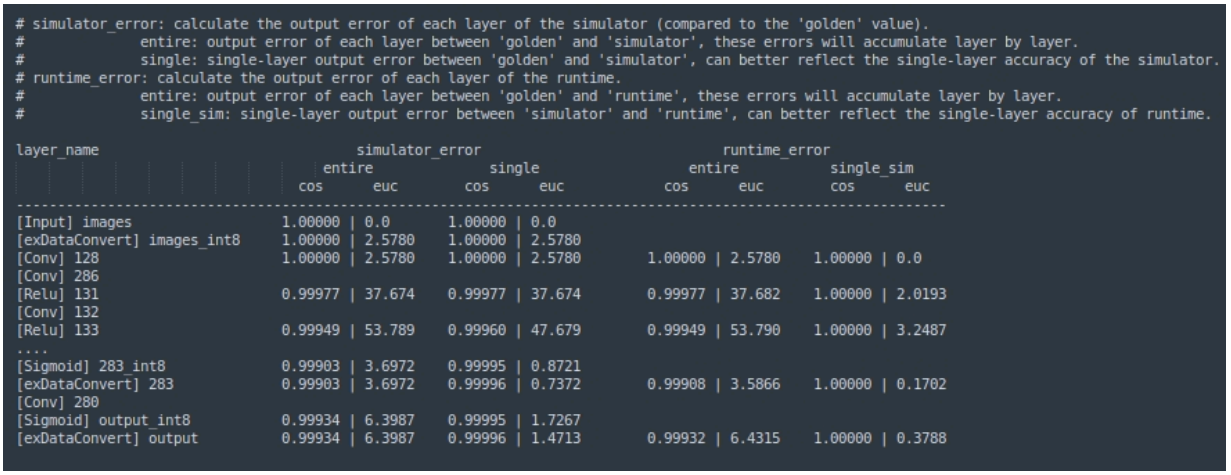
1.Accuracy analysis may fail because the model is too large and the storage capacity of the development board is insufficient. You can use the `df -h` command in the development board terminal to confirm the storage capacity. If there is insufficient space, please delete useless files to ensure that the partition corresponding to `output_dir` has enough storage space for saving the result files.
2.After loading an RKNN model via `rknn.load_rknn()`, the model accuracy analysis function cannot be used because the RKNN model lacks the original model information.

3.In the ARM64 version, `rknn.accuracy_analysis()` does not need device_id, and this version does not support analyzing accuracy on a device connected to another device via ADB.

Example code:

```
ret = rknn.accuracy_analysis(inputs=[img_path], target=platform,
device_id='515e9b401c060c0b')
```

The accuracy analysis results are shown in the figure below



```
# simulator_error: calculate the output error of each layer of the simulator (compared to the 'golden' value).
#             entire: output error of each layer between 'golden' and 'simulator', these errors will accumulate layer by layer.
#             single: single-layer output error between 'golden' and 'simulator', can better reflect the single-layer accuracy of the simulator.
# runtime_error: calculate the output error of each layer of the runtime.
#             entire: output error of each layer between 'golden' and 'runtime', these errors will accumulate layer by layer.
#             single_sim: single-layer output error between 'simulator' and 'runtime', can better reflect the single-layer accuracy of runtime.

layer_name                              simulator_error                                 runtime_error
                                      entire              single                    entire           single_sim
                                    cos      euc        cos      euc              cos      euc        cos      euc
-------------------------------------------------------------------------------------------------------------------
[Input] images                    1.00000 | 0.0      1.00000 | 0.0
[exDataConvert] images_int8       1.00000 | 2.5780   1.00000 | 2.5780
[Conv] 128                        1.00000 | 2.5780   1.00000 | 2.5780       1.00000 | 2.5780   1.00000 | 0.0
[Conv] 286
[Relu] 131                        0.99977 | 37.674   0.99977 | 37.674       0.99977 | 37.682   1.00000 | 2.0193
[Conv] 132
[Relu] 133                        0.99949 | 53.789   0.99960 | 47.679       0.99949 | 53.790   1.00000 | 3.2487
....
[Sigmoid] 283_int8                0.99903 | 3.6972   0.99995 | 0.8721
[exDataConvert] 283               0.99903 | 3.6972   0.99996 | 0.7372       0.99908 | 3.5866   1.00000 | 0.1702
[Conv] 280
[Sigmoid] output_int8             0.99934 | 6.3987   0.99995 | 1.7267
[exDataConvert] output            0.99934 | 6.3987   0.99996 | 1.4713       0.99932 | 6.4315   1.00000 | 0.3788
```

Figure 2-8 Accuracy analysis results

A complete accuracy analysis includes both simulator accuracy analysis results and on-board accuracy analysis results, and both the simulator and on-board accuracy analysis results are divided into whole-model inference result comparison and layer-by-layer inference result comparison. Details are as follows:

- simulator_error:
- entire: the cosine distance and Euclidean distance between the simulator result and the golden result from the input layer to the current layer.
- single: when the current layer uses the golden input, the cosine distance and Euclidean distance between the simulator result and the golden result.
- runtime_error:
- entire: the cosine distance and Euclidean distance between the actual on-board inference result and the golden result from the input layer to the current layer.
- single: when the current layer uses the golden input, the cosine distance and Euclidean distance between the actual on-board inference result of the current layer and the golden result.

2.3.3 Model performance evaluation

The interface `rknn.eval_perf()` outputs the current hardware frequencies and obtains the performance evaluation results of the model. The `fix_freq` parameter indicates whether to try to fix the frequency of the hardware (including CPU/NPU/DDR). Set it to `True` to fix the hardware frequency, otherwise set it to `False`; the default value is `True`. If the `perf_debug` parameter of `rknn.init_runtime()` is set to `True` during initialization, the time consumption of each

layer and the total time consumption will be output; if it is `False`, only the total time consumption will be output.

Note:

1.The RV1103 / RV1103B / RV1106 / RV1106B platforms do not support the `perf_debug` set to `True` mode and can only output the total time consumption of the model.

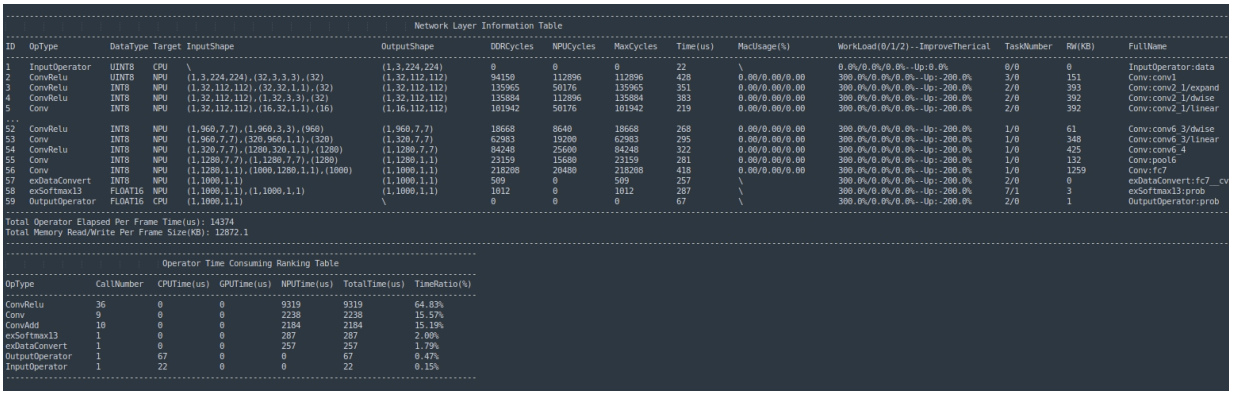
Example code:

```
ret = rknn.init_runtime(target=platform, perf_debug=True)
perf_detail = rknn.eval_perf()
```

Taking RK3588 as an example, the hardware frequencies output after executing eval_perf are as follows (the frequencies supported by different firmware may vary). Since the RK3588 CPU has a big.LITTLE core architecture, the CPU frequency column outputs multiple frequency values in KHz, while the NPU and DDR frequencies are both in Hz.

```
CPU Current Frequency List:
     - 1800000
     - 2256000
     - 2304000
NPU Current Frequency List:
     - 1000000000
DDR Current Frequency List:
     - 2112000000
```

The performance evaluation results are as follows:



Network Layer Information Table

ID	OpType	DataType	Target	InputShape	OutputShape	DDRCycles	NPUCycles	MaxCycles	Time(us)	MacUsage(%)	WorkLoad(0/1/2)--ImproveTherical	TaskNumber	RW(KB)	FullName
1	InputOperator	UINT8	CPU	\	(1,3,224,224)	0	0	0	22	\	0.0%/0.0%/0.0%--Up:0.0%	0/0	0	InputOperator:data
2	ConvRelu	UINT8	NPU	(1,3,224,224),(32,3,3,3),(32)	(1,32,112,112)	94150	112896	112896	428	0.00/0.00/0.00	300.0%/0.0%/0.0%--Up:-200.0%	3/0	151	Conv:conv1
3	ConvRelu	INT8	NPU	(1,32,112,112),(32,32,1,1),(32)	(1,32,112,112)	135965	50176	135965	351	0.00/0.00/0.00	300.0%/0.0%/0.0%--Up:-200.0%	2/0	393	Conv:conv2_1/expand
4	ConvRelu	INT8	NPU	(1,32,112,112),(1,32,3,3),(32)	(1,32,112,112)	135884	112896	135884	383	0.00/0.00/0.00	300.0%/0.0%/0.0%--Up:-200.0%	2/0	392	Conv:conv2_1/dwise
5	Conv	INT8	NPU	(1,32,112,112),(16,32,1,1),(16)	(1,16,112,112)	101942	50176	101942	219	0.00/0.00/0.00	300.0%/0.0%/0.0%--Up:-200.0%	2/0	392	Conv:conv2_1/linear
...														
52	ConvRelu	INT8	NPU	(1,960,7,7),(1,960,3,3),(960)	(1,960,7,7)	18668	8640	18668	268	0.00/0.00/0.00	300.0%/0.0%/0.0%--Up:-200.0%	1/0	61	Conv:conv6_3/dwise
53	Conv	INT8	NPU	(1,960,7,7),(320,960,1,1),(320)	(1,320,7,7)	62983	19200	62983	295	0.00/0.00/0.00	300.0%/0.0%/0.0%--Up:-200.0%	1/0	348	Conv:conv6_3/linear
54	ConvRelu	INT8	NPU	(1,320,7,7),(1280,320,1,1),(1280)	(1,1280,7,7)	84248	25600	84248	322	0.00/0.00/0.00	300.0%/0.0%/0.0%--Up:-200.0%	1/0	425	Conv:conv6_4
55	Conv	INT8	NPU	(1,1280,7,7),(1,1280,7,7),(1280)	(1,1280,1,1)	23159	15680	23159	281	0.00/0.00/0.00	300.0%/0.0%/0.0%--Up:-200.0%	1/0	132	Conv:pool6
56	Conv	INT8	NPU	(1,1280,1,1),(1000,1280,1,1),(1000)	(1,1000,1,1)	218208	20480	218208	418	0.00/0.00/0.00	300.0%/0.0%/0.0%--Up:-200.0%	1/0	1259	Conv:fc7
57	exDataConvert	INT8	NPU	(1,1000,1,1)	(1,1000,1,1)	509	0	509	257	\	300.0%/0.0%/0.0%--Up:-200.0%	2/0	0	exDataConvert:fc7__cv
58	exSoftmax13	FLOAT16	NPU	(1,1000,1,1),(1,1000,1,1)	(1,1000,1,1)	1012	0	1012	287	\	300.0%/0.0%/0.0%--Up:-200.0%	7/1	3	exSoftmax13:prob
59	OutputOperator	FLOAT16	CPU	(1,1000,1,1)	\	0	0	0	67	\	300.0%/0.0%/0.0%--Up:-200.0%	2/0	1	OutputOperator:prob

Total Operator Elapsed Per Frame Time(us): 14374
Total Memory Read/Write Per Frame Size(KB): 12872.1

Operator Time Consuming Ranking Table

OpType	CallNumber	CPUTime(us)	GPUTime(us)	NPUTime(us)	TotalTime(us)	TimeRatio(%)
ConvRelu	36	0	0	9319	9319	64.83%
Conv	9	0	0	2238	2238	15.57%
ConvAdd	10	0	0	2184	2184	15.19%
exSoftmax13	1	0	0	287	287	2.00%
exDataConvert	1	0	0	257	257	1.79%
OutputOperator	1	67	0	0	67	0.47%
InputOperator	1	22	0	0	22	0.15%

Figure 2-9 Performance evaluation results

The fields of the performance evaluation results are described in the following table:

Table 2-2 Description of the fields in the performance evaluation results

Field	**Description**
ID	Operator ID
OpType	Operator type
DataType	Input data type
Target	Hardware on which the operator runs (CPU/NPU/GPU)
InputShape	Input shape

Field	Description
OutputShape	Output shape
DDRCycles	Number of DDR read/write clock cycles
NPUCycles	Number of NPU computation clock cycles
MaxCycles	Maximum of DDR Cycles and NPU Cycles
Time(us)	Operator computation time (us)
MacUsage(%)	MAC usage
WorkLoad(0/1/2)	Load of cores 0/1/2 (RK3588 platform only)
WorkLoad(0/1)	Load of cores 0/1 (RK3576 platform only)
TaskNumber	Number of NPU tasks
RW(KB)	Total amount of data read and written (KB)
FullName	Full name of the operator
Total Operator Elapsed Per Frame Time(us)	Total time consumption per frame of model inference
Total Memory Read/Write Per Frame Size(KB)	Total bandwidth consumption per frame of model inference
CallNumber	Number of times the operator runs within a single frame
CPUTime(us)	Total time consumption of the operator on the CPU within a single frame
GPUTime(us)	Total time consumption of the operator on the GPU within a single frame
NPUTime(us)	Total time consumption of the operator on the NPU within a single frame
TotalTime(us)	Total time consumption of the operator within a single frame
TimeRatio(%)	Ratio of the total time consumption of the operator within a single frame to the total time consumption of the single frame

2.3.4 Model memory evaluation

The interface `rknn.eval_memory()` can obtain the memory consumption evaluation results of the model. If the `eval_mem` parameter of `rknn.init_runtime()` is set to `True` during initialization, the memory consumption of each part will be output.

Example code:

```
ret = rknn.init_runtime(target=platform, eval_mem=True)
memory_detail = rknn.eval_memory()
```

The memory evaluation results are as follows:

```
=====================================================
            Memory Profile Info Dump
=====================================================
NPU model memory detail(bytes):
    Weight Memory: 8.67 MiB
    Internal Tensor Memory: 7.42 MiB
    Other Memory: 3.03 MiB
    Total Memory: 19.12 MiB

INFO: When evaluating memory usage, we need consider
the size of model, current model size is: 11.86 MiB
=====================================================
```

The fields of the memory evaluation results are described as follows:

Table 2-3 Description of the fields in the memory evaluation results

Field	**Description**
Total Weight Memory	Memory usage of the weights in the model (MB)
Total Internal Tensor Memory	Memory usage of the intermediate tensors of the model (MB)
Other Memory	Other memory usage (e.g. register configuration, input and output tensors) (MB)
Total Memory	Total memory usage of the model (MB)

3. Board Deployment

3.1 Runtime Deployment Basics

This section describes the fundamental concepts of the RKNN Runtime and is the prerequisite for board deployment (with either the C API or Toolkit Lite). It covers the model and context lifecycle, tensors and data layout formats, input/output memory, synchronous and asynchronous inference, and error handling and resource release conventions.

3.1.1 Model and Context Lifecycle

`rknn_context` is the core handle of the RKNN Runtime, representing a runtime context instance. All model loading, attribute query, and inference operations are performed through this handle. During board deployment, strictly follow the "create → use → destroy" lifecycle management convention to avoid resource leaks or runtime exceptions.

- **Create**: initialize the context and load the RKNN model via `rknn_init()`. After successful initialization, the model weights and the internal memory required for running are ready.
- **Use**: after initialization, query the model's input/output attributes via `rknn_query()`, and run inference repeatedly through the general API (`rknn_inputs_set()` / `rknn_run()` / `rknn_outputs_get()`) or the zero-copy API (`rknn_set_io_mem()` + `rknn_run()`); the handle stays unchanged. On RK3588 / RK3576, the NPU cores used by the model can be specified via `rknn_set_core_mask()` before inference; see 4.1.2 NPU Multi-Core Configuration.
- **Destroy**: after all inference tasks are completed, destroy the context via `rknn_destroy()` to release all memory occupied by the model. The handle cannot be used after destruction.

3.1.2 Tensors and Data Layout Formats

The input and output of an RKNN model are managed through tensors. Before on-board deployment, use `rknn_query()` to query the tensor attributes (`rknn_tensor_attr`), which include the tensor name, shape, data type (dtype), data layout (layout), and quantization parameters (qnt_type / qnt_info), and then organize the input data and parse the output data accordingly.

- **Data type (dtype)**: includes FP32 / FP16 / INT8 / INT16 / UINT8, etc., determined by the quantization configuration at model conversion; quantized models default to INT8 (asymmetric quantization).
- **Data layout (layout)**: mainly `NCHW` , `NHWC` , `NC1HWC2` , and `UNDEFINE` ; `NC1HWC2` is the memory layout format dedicated to RKNPU hardware, see below for details.

Currently, there are four data layout formats of RKNN: `NHWC` , `NCHW` , `NC1HWC2` , and `UNDEFINE` .

The `NHWC` and `NCHW` data layouts are common data layouts in deep learning, and no additional explanation is given in this section. This section focuses on the storage and conversion of the `NC1HWC2` data format dedicated to RKNPU hardware.

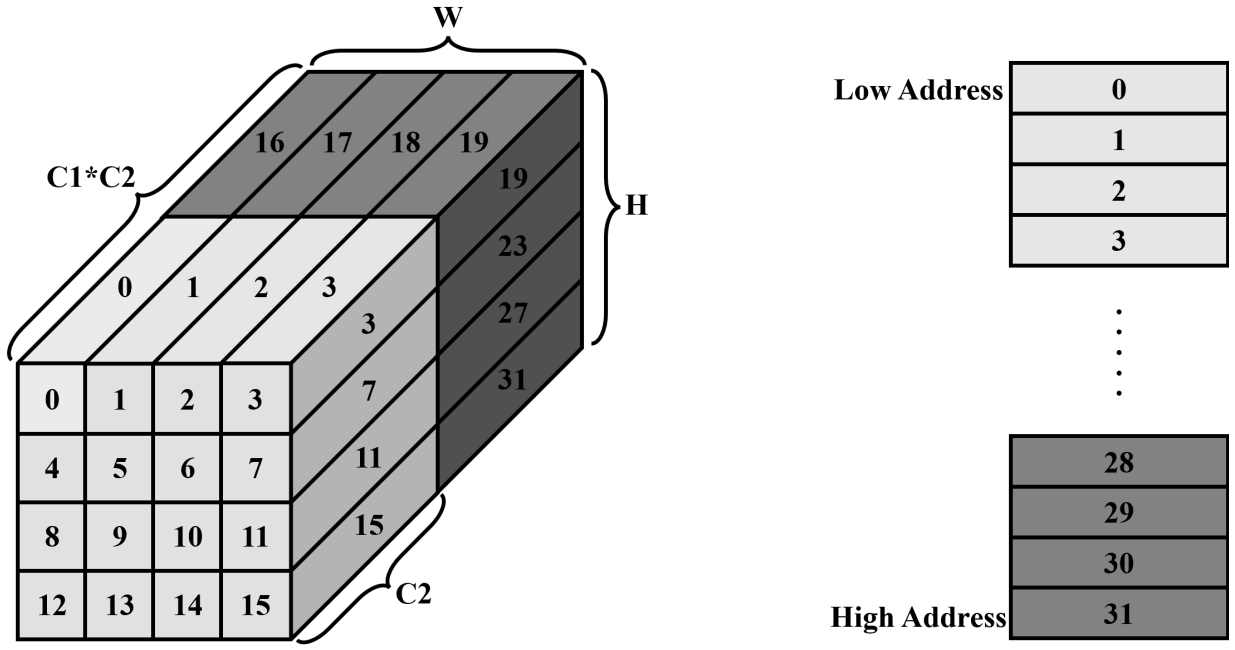


Figure 3-1 RKNPU NC1HWC2 data layout and storage

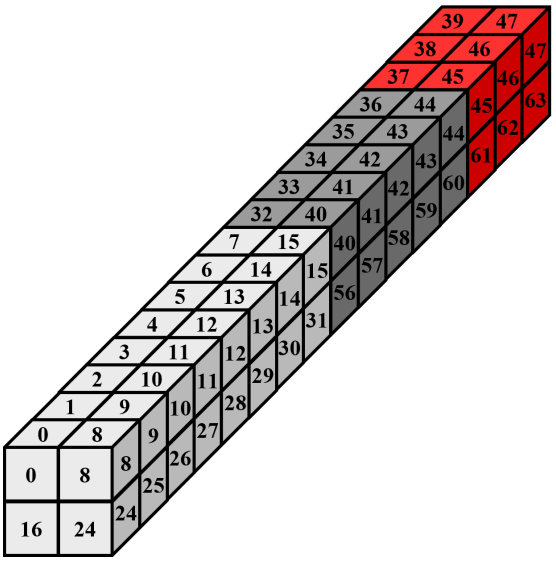
As shown in Figure 3-1, the number 0 represents one chunk of data, that is, C2 data elements are stored at a time, where C2 is determined by the platform. The rule constraints of C2 on different hardware platforms are shown in Table 3-1, and C1 is the rounded-up value of C/C2. The order in which `NC1HWC2` data is stored is consistent with the order of the increasing numbers in the figure: data 0-15 is stored first, and then data 16-31. Taking the RK3568 platform as an example, when the feature is (1, 13, 4, 4) int8 data, the corresponding `NC1HWC2` is (1, 2, 4, 4, 8); in this case, C2 is 8 and C1 is 2. For the feature data stored at positions 16-31 in memory, only the first 5 data of each corresponding C2 data block are valid, and the remaining 3 data are extra padded alignment data.

Table 3-1 C2 values for different hardware platforms

	RK2118	RK3566/RK3568	RK3588/RK3576/RV1126B	RV1103B/RV1106B	RV1103/RV1106	RK3562
int8	/	8	16	8	16	16
float16	4	4	8	/	8	8

Next, the process of converting the `NC1HWC2` data layout into `NCHW` and `NHWC` data in memory is described in detail.

Taking the feature (1, 13, 2, 2) on RK3568 as an example, for the conversion of the data in the memory layout, according to the alignment requirements above, the `NC1HWC2` corresponding to feature (1, 13, 2, 2) is (1, 2, 2, 2, 8). The storage of `NC1HWC2` is shown in the figure below, where the red parts are invalid data added for alignment.



Low Address								
	0	1	2	3	4	5	6	7
	8	9	10	11	12	13	14	15
	16	17	18	19	20	21	22	23
	24	25	26	27	28	29	30	31
	32	33	34	35	36	37	38	39
	40	41	42	43	44	45	46	47
	48	49	50	51	52	53	54	55
High Address	56	57	58	59	60	61	62	63

Figure 3-2 NC1HWC2 data layout expansion

Remove the invalid data and convert it into `NCHW`, that is, (1, 13, 2, 2) data, which is arranged in memory as follows:

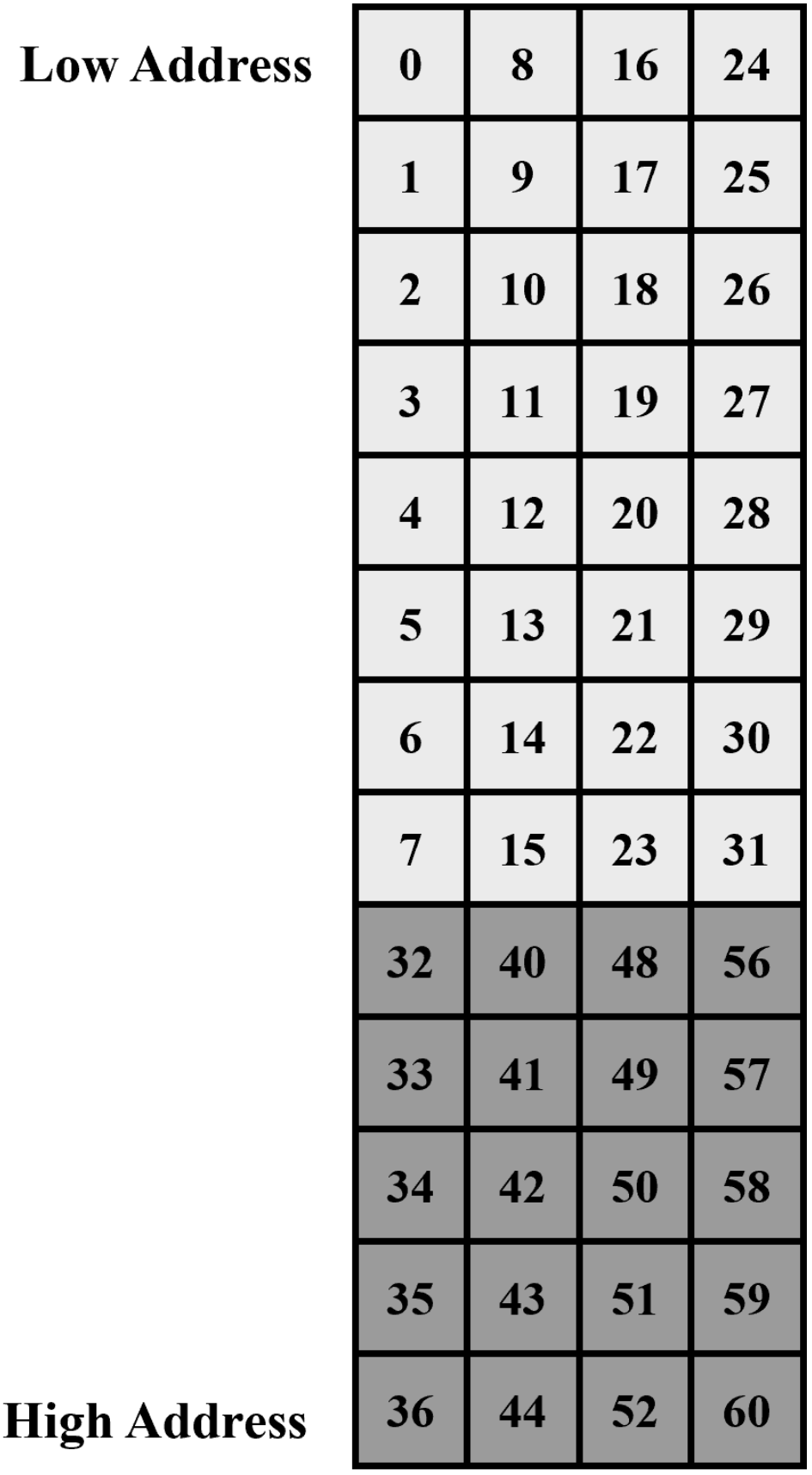


Figure 3-3 NCHW data layout

Remove the invalid data and convert it into `NHWC`, that is, (1, 2, 2, 13) data, which is arranged in memory as follows:

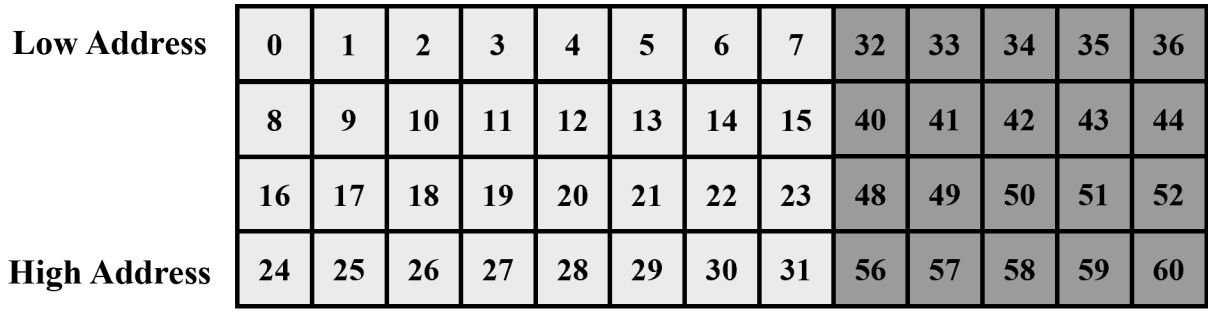


Figure 3-4 NHWC data layout

Conversion example code:

`NC1HWC2` to `NCHW`: converting an `NC1HWC2` with int8 data layout into an `NCHW` with int8 data layout is shown as follows:

```
/*
 *src:     represents the address of the NC1HWC2 input tensor
 *dst:     represents the address of the NCHW output tensor
 *dims:    represents the shape information of NC1HWC2
 *channel: represents the value of C in NCHW
 * h :     represents the value of h in NCHW
 * w:      represents the value of w in NCHW
 */
int NC1HWC2_to_NCHW(const int8_t* src, int8_t* dst, int* dims, int channel, int h,
int w)
{
  int batch  = dims[0];
  int C1     = dims[1];
  int C2     = dims[4];
  int hw_src = dims[2] * dims[3];
  int hw_dst = h * w;
  for (int i = 0; i < batch; i++) {
    src = src + i * C1 * hw_src * C2;
    dst = dst + i * channel * hw_dst;
    for (int c = 0; c < channel; ++c) {
      int plane  = c / C2;
      const int8_t* src_c  = plane * hw_src * C2 + src;
      int  offset = c % C2;
      for (int cur_h = 0; cur_h < h; ++cur_h)
        for (int cur_w = 0; cur_w < w; ++cur_w) {
          int cur_hw = cur_h * w + cur_w;
          dst[c * hw_dst + cur_h * w + cur_w] = src_c[C2 * cur_hw + offset];
        }
    }
  }
  return 0;
}
```

`NC1HWC2` to `NHWC`: converting an `NC1HWC2` with int8 data layout into an `NHWC` with int8 data layout is shown as follows:

```
/*
 *src:     represents the address of the NC1HWC2 input tensor
 *dst:     represents the address of the NCHW output tensor
 *dims:    represents the shape information of NC1HWC2
 *channel: represents the value of C in NHWC
 * h :     represents the value of h in NCHW
 * w:      represents the value of w in NCHW
 */
int NC1HWC2_to_NHWC(const int8_t* src, int8_t* dst, int* dims, int channel, int h,
int w)
{
  int batch  = dims[0];
  int C1     = dims[1];
  int C2     = dims[4];
  int hw_src = dims[2] * dims[3];
  int hw_dst = h * w;
  for (int i = 0; i < batch; i++) {
    src = src + i * C1 * hw_src * C2;
    dst = dst + i * channel * hw_dst;
    for (int cur_h = 0; cur_h < h; ++cur_h) {
      for (int cur_w = 0; cur_w < w; ++cur_w) {
        int cur_hw = cur_h * dims[3] + cur_w;
        for (int c = 0; c < channel; ++c) {
          int  plane  = c / C2;
          const auto* src_c   = plane * hw_src * C2 + src;
          int  offset  = c % C2;
          dst[cur_h * w * channel + cur_w * channel + c] = src_c[C2 * cur_hw +
offset];
        }
      }
    }
  }
  return 0;
}
```

3.1.3 Input/Output Memory

In the RKNN Runtime, the model's input/output data is passed through memory objects. There are two ways — the general API and the zero-copy API — with different memory management:

- **General API (internal-copy mode)**: the input buffer is prepared by the user (the `buf` member of `rknn_input` points to user memory); when `rknn_inputs_set()` is called, the Runtime copies the data into the model input. After inference, outputs are retrieved via `rknn_outputs_get()` (the Runtime allocates output memory and copies the results), and `rknn_outputs_release()` must be called after use to free that output memory. Type conversion, layout conversion, and normalization of inputs/outputs are all done inside the Runtime; it is simple to use and suitable for most scenarios.
- **Zero-copy API**: input/output memory accessible by the NPU is allocated via `rknn_create_mem()` (or externally allocated physical memory via `rknn_create_mem_from_fd()` / `rknn_create_mem_from_phys()`), and bound to tensors via `rknn_set_io_mem()`; the NPU reads/writes the memory directly, eliminating the internal data copy of the general API for better performance. The memory size, data type, and layout must strictly match the tensor attributes.

Selection suggestion: use the general API for routine deployment or when performance is not critical — it is simple and reliable; use the zero-copy API for maximum performance (eliminating input/output copy time), see 3.1.6 RKNN Runtime Zero-Copy Interface.

Memory release conventions:

1. Output memory of the general API is released via `rknn_outputs_release()` after post-processing; input/output memory of the zero-copy API is released via `rknn_destroy_mem()`.
2. Memory must be released before the context is destroyed (`rknn_destroy()`); otherwise resources may leak.
3. Released memory must not be used again; otherwise behavior is undefined.

3.1.4 Synchronous and Asynchronous Inference

The inference interface `rknn_run()` provided by the RKNN Runtime is a synchronous blocking call: the calling thread blocks until the NPU finishes the current inference and returns, after which the output results can be obtained without extra synchronization.

The current version of the RKNN Runtime does not provide an asynchronous inference interface. If other tasks need to run in parallel during inference (e.g. capturing and preprocessing the next frame), wrap it at the application level with multiple threads: for example, run `rknn_run()` in a dedicated inference thread while the main thread preprocesses data, passing tasks between producer and consumer through queues to form a pipeline and improve overall throughput.

Note: multiple threads must not call `rknn_run()` on the same context concurrently. For multi-threaded concurrent inference, refer to 5.3.4 Multi-threaded Reuse RKNN Context and create an independent context for each thread via `rknn_dup_context()`.

3.1.5 Error Handling and Resource Release

Error Handling

RKNN C API functions return an `int` status code, where `RKNN_SUCCESS` (0) indicates success and negative values indicate errors. During on-board deployment, the return value of every API call should be checked; on error, subsequent operations must not continue — jump to the resource cleanup path and exit, so as to avoid undefined behavior caused by executing in an abnormal state.

For the complete error code definitions, refer to《05_RKNN2_Runtime_C_API_Reference_V2.5.0》. During troubleshooting, logs can be used to locate the problem: setting the environment variable `RKNN_LOG_LEVEL=5` before running the application outputs detailed layer-by-layer debugging information. For the analysis and handling of common errors, see section 5.4 FAQs and Error Handling.

Example of error handling in the initialization phase (jump to the cleanup path on error):

```
int ret = rknn_init(&ctx, model, model_len, 0, NULL);
if (ret != RKNN_SUCCESS) {
    printf("rknn_init failed! ret=%d\n", ret);
    goto out;
}
```

Resource Release

The various resources in on-board deployment must be released in the correct order to avoid resource leaks. It is recommended to use `goto` to jump to a unified cleanup path, ensuring that resources are released correctly on both the normal path and the error path.

- **General API**: the release order is to first call `rknn_outputs_release()` to release the output data memory, and then call `rknn_destroy()` to destroy the context.

```
out:
    // Release the output data memory (general API)
    rknn_outputs_release(ctx, io_num.n_output, outputs);
    // Destroy the context
    if (ctx) rknn_destroy(ctx);
```

- **Zero-copy API**: the release order is to first call `rknn_destroy_mem()` to release the input and output memories one by one, and then destroy the context.

```
out:
    // Release the input/output memory (zero-copy API; input_mems/output_mems are
rknn_tensor_mem pointers returned by rknn_create_mem())
    for (int i = 0; i < io_num.n_input; i++) {
        rknn_destroy_mem(ctx, input_mems[i]);
    }
    for (int i = 0; i < io_num.n_output; i++) {
        rknn_destroy_mem(ctx, output_mems[i]);
    }
    // Destroy the context
    if (ctx) rknn_destroy(ctx);
```

> **Note**: After the context is destroyed, all resources allocated based on that context can no longer be accessed. Make sure that all resources have been released before the context is destroyed.

3.1.6 RKNN Runtime Zero-Copy Interface

3.1.6.1 ZERO-COPY INTRODUCTION

Currently, there are two groups of APIs available on RK3562 / RK3566 / RK3568 / RK3576 / RK3588 / RV1126B, namely the general API interface and the zero-copy process API interface, and the RV1103 series / RV1106 series support the zero-copy process API interface. For zero-copy example code, refer to the examples/rknn_zero_copy directory under the rknpu2 project. For detailed information and the code repository of rknpu2, please visit the link: https://github.com/airockchip/rknn-toolkit2/tree/master/rknpu2.

When performing inference on an RKNN model, the raw data goes through three major processes: input processing, NPU running the model, and output processing. Currently, depending on the model input format and quantization method, there are two processing flows inside the interface: the general API and the zero-copy API, as shown in Figure 3-5 and Figure 3-6. The main difference between the two groups of APIs is that every time the frame data is

updated, the general interface needs to copy the data allocated by the external module into the input memory of the NPU runtime, while the zero-copy process interface directly uses pre-allocated memory (including memory created by the NPU runtime or created by other external frameworks, such as the DRM framework), which reduces the overhead of memory copying, delivers better performance, and uses less bandwidth. When the user input data has only a virtual address, only the general API interface can be used; when the user input data has a physical address or fd, both groups of interfaces can be used. **The general API and the zero-copy API cannot be called in a mixed manner.**

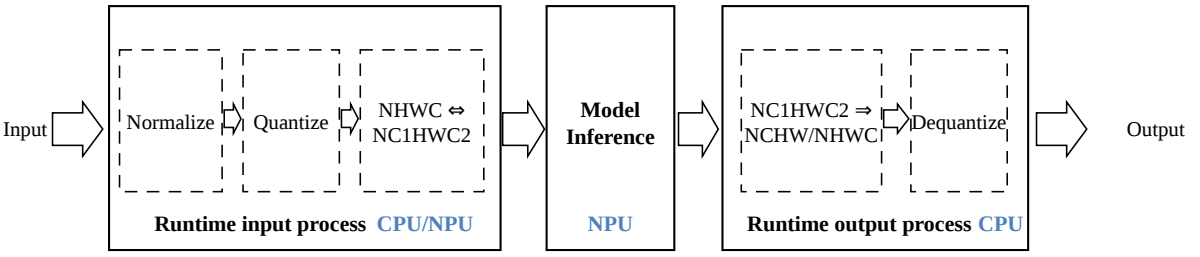


Figure 3-5 Data processing flow of the general API

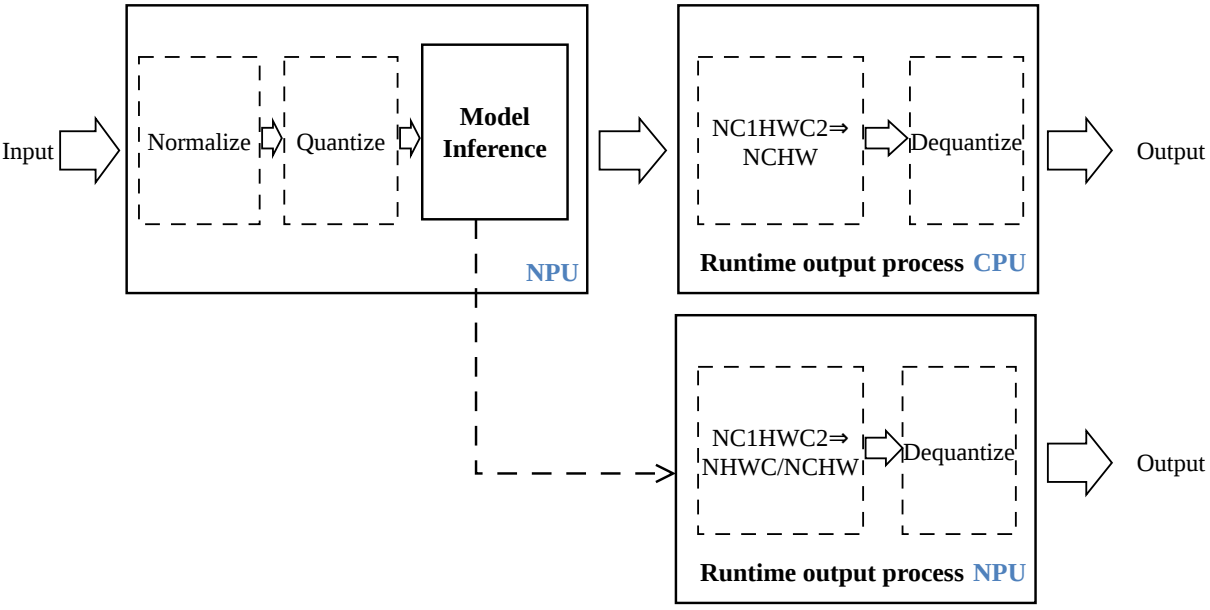


Figure 3-6 Data processing flow of the zero-copy API

1. General API

The flow of the general API is shown in Figure 3-5. The normalization, quantization, data layout conversion, and dequantization of the data run on the CPU (when the zero-copy input requirements are met, normalization and quantization run on the NPU, but the input data still needs to be copied by the CPU to the model's input buffer), while the inference of the model itself runs on the NPU.

1. Zero-copy API

The flow of the zero-copy API is shown in Figure 3-6. It optimizes the data processing flow of the general API: with the zero-copy API, normalization, quantization, and model inference all run on the NPU, while the output data layout conversion and the dequantization process run on the CPU or the NPU. The zero-copy API processes the input data flow more efficiently than the general API.

The usage conditions for the zero-copy scenario are shown in the following table:

Table 3-2 Zero-copy input requirements

Input dimensions	**Input alignment requirements**	
	RV1103B / RV1106B / RK3566 / RK3568	RK3562 / RK3576 / RK3588 / RV1103 / RV1106 / RV1126B
4 dimensions, number of channels is 1, 3, 4	Width 8-byte aligned	Width 16-byte aligned
Not 4 dimensions	Total size 8-byte aligned	Total size 16-byte aligned

3.1.6.2 C API ZERO-COPY PROCESS

The zero-copy API interface uses the `rknn_tensor_memory` structure, which needs to be created and set before inference, and the memory information in the structure is read after inference. Depending on whether the user needs to allocate the model's module memory (input/output/weights/intermediate results) themselves, and on the memory representation (file descriptor/physical address, etc.), there are the following three typical zero-copy calling processes, as shown in Figure 3-7 to Figure 3-9. The parts in red indicate the interfaces and data structures specially added for zero-copy, and the italics indicate the data structures passed between interface calls.

- Input/output memory allocated by the runtime

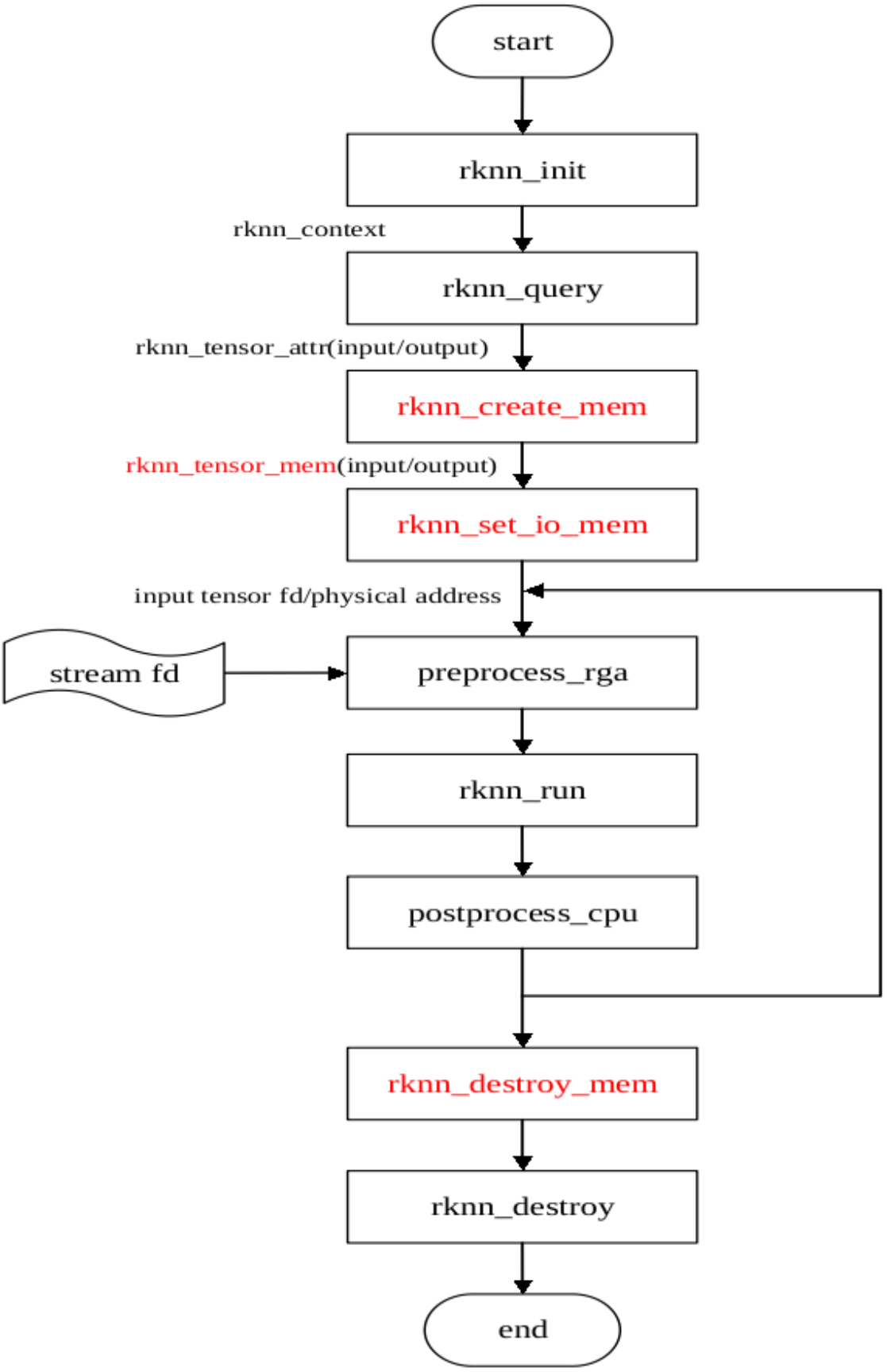


Figure 3-7 Zero-copy API interface calling process (input/output allocated internally)

As shown in Figure 3-7, when the input/output memory is allocated by the runtime, the `rknn_create_mem()` interface is called to create the `rknn_tensor_memory` structure, and `rknn_set_io_mem()` sets the input and output `rknn_tensor_memory` structures.

The input/output memory information structure created by the `rknn_create_mem()` interface contains the file descriptor member and the physical address. The RGA interface uses the memory information allocated by the NPU: `preprocess_rga()` represents the RGA interface, stream_fd represents the memory data of the RGA interface input source, and `postprocess_cpu()` represents the CPU implementation of post-processing.

- Input/output memory allocated externally

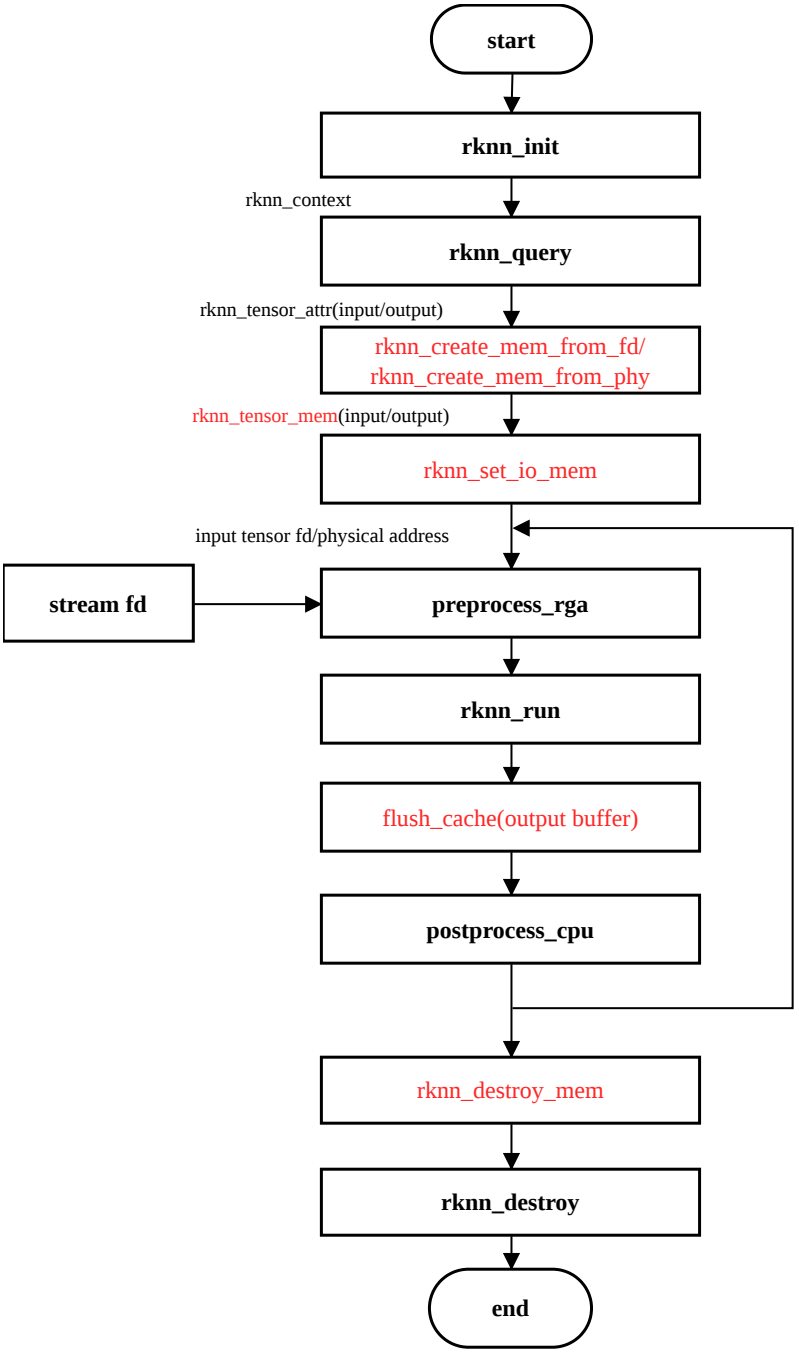


Figure 3-8 Zero-copy API interface calling process (input/output allocated externally)

As shown in Figure 3-8, when the input/output memory is allocated externally, the `rknn_create_mem_from_fd()` / `rknn_create_mem_from_phys()` interfaces are called to create the `rknn_tensor_memory` structure, and `rknn_set_io_mem()` sets the input and output `rknn_tensor_memory` structures. `flush_cache` indicates that the user needs to call the interface associated with the allocated memory type to flush the output cache.

- Input/output/weights/intermediate result memory allocated externally

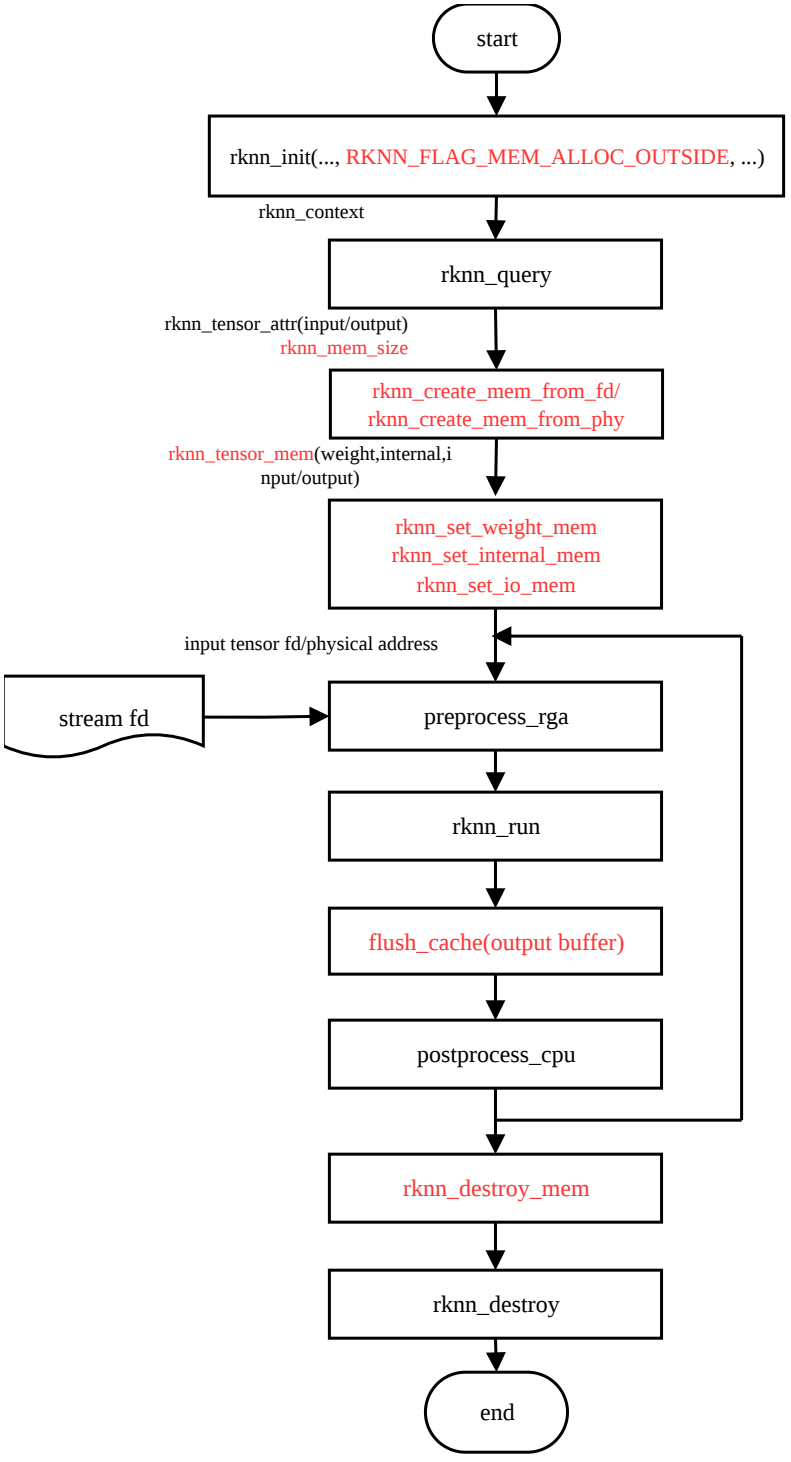


Figure 3-9 Zero-copy API interface calling process (input/output/weights/intermediate results allocated externally)

As shown in Figure 3-9, when the input/output/weights/intermediate result memory is allocated externally, the `rknn_create_mem_from_fd()` / `rknn_create_mem_from_phys()` interfaces are called to create the `rknn_tensor_memory` structure, `rknn_set_io_mem()` sets the input and output `rknn_tensor_memory` structures, and `rknn_set_weight_mem()` / `rknn_set_internal_mem()` set the weights/intermediate result `rknn_tensor_memory` structures.

3.1.6.3 C API ZERO-COPY USAGE

Taking Figure 3-7, the zero-copy API interface calling process (input/output allocated internally), as an example, the usage is as follows:

- rknn_query()

Input:

Use **RKNN_QUERY_NATIVE_INPUT_ATTR** to query the related attributes (note, not **RKNN_QUERY_INPUT_ATTR**). When the queried fmt (also called layout) differs, the way to pre-process the data in advance also differs. This method queries the input layout and type that are optimal for hardware efficiency.

The input cases of `rknn_query()` are as follows:

a. When the layout is `RKNN_TENSOR_NCHW`, the input is generally 4-dimensional and the data type is bool or int64. When passing data to the NPU, it also needs to be arranged in the NCHW format for the NPU.

b. When the layout is `RKNN_TENSOR_NHWC`, the input is generally 4-dimensional and the data type is float32/float16/int8/uint8, and the number of input channels is 1, 3, or 4. When passing data to the NPU, it also needs to be arranged in the `NHWC` format for the NPU. Note that when `pass_through=1`, the width may need to be stride-aligned, depending on the queried `w_stride` value.

c. When the layout is `RKNN_TENSOR_NC1HWC2`, the input is generally 4-dimensional and the data type is float16/int8, and the number of input channels is not 1, 3, or 4. When `pass_through=0`, the input data is arranged in the `NHWC` format, and the interface internally performs the CPU conversion from `NHWC` to `NC1HWC2`; when `pass_through=1`, the input data is arranged in the `NC1HWC2` format, and the user needs to convert it externally.

d. When the layout is `RKNN_TENSOR_UNDEFINED`, the input is generally not 4-dimensional. When passing data to the NPU, it needs to be passed to the NPU according to the input format of the ONNX model. The NPU does not perform any mean/std processing or layout conversion.

If the input configuration required by the user is different from the `rknn_tensor_attr` structure obtained by the query interface, the `rknn_tensor_attr` structure can be modified accordingly. The currently supported modifiable input data types are shown in Table 3-3. Special attention: if the queried data type is uint8 and the user wants to pass in float32 data, the size of the `rknn_tensor_attr` structure must be modified to four times the original size, and the data type in it must be modified to `RKNN_TENSOR_FLOAT32`. After modifying in this way, the hardware efficiency is no longer optimal, and the interface internally calls the CPU to perform the data type conversion.

Table 3-3 Input modifiable data type table

	The model data type obtained by rknn_query						
The data type modified by the user interface		bool	int8	float16	int16	int32	Int64
	bool	Y					

	The model data type obtained by rknn_query						
	int8		Y				
	uint8		Y	Y			
	float32		Y	Y			
	float16			Y			
	int16				Y		
	int32					Y	
	Int64						Y

Output:

Use **RKNN_QUERY_NATIVE_OUTPUT_ATTR** to query the related attributes (note, not **RKNN_QUERY_OUTPUT_ATTR**). When the queried fmt (also called layout) differs, the required post-processing also differs. This method queries the output layout and type that are optimal for hardware efficiency.

When the output is 4-dimensional and the user needs a 4-dimensional output in the `NHWC` layout, `RKNN_QUERY_NATIVE_NHWC_OUTPUT_ATTR` can be used to query the related attributes. In this way, an output in the NHWC layout can be obtained directly.

The output cases of `rknn_query()` are as follows:

a. When the layout is `RKNN_TENSOR_NC1HWC2`, the output is generally 4-dimensional and the data type is float16/int8. When the user needs the `NCHW` layout, the conversion from `NC1HWC2` to `NCHW` needs to be performed externally.

b. When the layout is `RKNN_TENSOR_UNDEFINED`, the output is generally not 4-dimensional and the data type is float16/int8. The user does not need to perform layout conversion externally.

c. When the layout is `RKNN_TENSOR_NCHW`, the output is generally 4-dimensional and the data type is float16/int8. The user does not need to perform layout conversion externally.

d. When the layout is `RKNN_TENSOR_NHWC`, the output is generally 4-dimensional and the data type is float16/int8. This case generally occurs when the user calls the `RKNN_QUERY_NATIVE_NHWC_OUTPUT_ATTR` interface to query the layout.

If the output configuration required by the user is different from the `rknn_tensor_attr` structure obtained by the query interface, the `rknn_tensor_attr` structure can be modified accordingly. The modifiable configuration information is shown in Table 3-5 and Table 3-4. Special attention: if the queried output data type is int8 and the user wants to obtain a float32 output, the size of the `rknn_tensor_attr` structure must be modified to four times the original size, and the data type in it must be modified to `RKNN_TENSOR_FLOAT32`. After modifying in this way, the hardware efficiency is no longer optimal, and the interface internally calls the CPU to perform the data type conversion.

Table 3-5 Output modifiable data type table

	The model data type obtained by rknn_query						
The data type modified by the user interface		bool	int8	float16	int16	int32	Int64
	bool	Y					
	int8		Y				
	uint8						
	float32		Y	Y			
	float16			Y			
	int16				Y		
	int32					Y	
	Int64						Y

Table 3-4 Output modifiable layout type table

	The model layout type obtained by rknn_query				
		NC1HWC2	NCHW	NHWC	UNDEFINE
The layout type set by the user interface	NC1HWC2	Y			
	NCHW	Y	Y		
	NHWC			Y	
	UNDEFINE				Y

The zero-copy interface NPU output configurations supported by RK3562/RK3566/RK3568/RK3576/RK3588/RV1126B are shown in Table 3-6, and the zero-copy interface NPU output configurations supported by RV1106/RV1106B/RV1103/RV1103B are shown in Table 3-7.

Table 3-6 Output configurations supported by the RK3562/RK3566/RK3568/RK3576/RK3588/RV1126B zero-copy interface NPU

Model type	**Output data type**	**Output dimensions**	**Supported output layout**
int8 model	int8/float16/float32	4-dimensional	NCHW/NC1HWC2/NHWC
		Non-4-dimensional	UNDEFINE
float16 model	float16/float32	4-dimensional	NCHW/NC1HWC2/NHWC
		Non-4-dimensional	UNDEFINE

Table 3-7 Output configurations supported by the RV1103/RV1103B/RV1106/RV1106B zero-copy interface NPU

Model type	**Output data type**	**Output dimensions**	**Supported output layout**
int8 model	int8/float16	4-dimensional	NCHW/NC1HWC2/NHWC
		Non-4-dimensional	UNDEFINE

- rknn_create_mem

The zero-copy API interface uses the `rknn_tensor_memory` structure, which needs to be created and set before inference, and the memory information in the structure is read after inference. When there is no need to modify the layout and type queried by `RKNN_QUERY_NATIVE_INPUT_ATTR` and `RKNN_QUERY_NATIVE_OUTPUT_ATTR`, simply create the memory with the default configured `size_with_stride`. If the corresponding layout and type have been modified, the memory must be created according to the corresponding size (for example, if the output data type is int8 and the user wants to obtain a float32 output, the size must be modified to four times the original size).

- rknn_set_io_mem

`rknn_set_io_mem()` is used to set the `rknn_tensor_mem` structure containing the model input/output memory information. Similar to `rknn_init()`, it only needs to be called once at the very beginning, and then `rknn_run()` can be executed repeatedly.

3.2 Deploying General Models with the C API

This chapter describes the deployment process of the general C API, covering the complete invocation process from model initialization, input/output preparation, and inference execution to output processing. It is suitable for production deployment and performance-sensitive scenarios. For the zero-copy C API invocation process, refer to 3.1.6 RKNN Runtime Zero-Copy Interface.

3.2.1 Complete Invocation Process

The typical calling sequence for deploying a general model with the C API is: model initialization → query input/output attributes → prepare and set inputs → run inference → get outputs → post-processing → release resources. The overall flow is shown in the figure below:

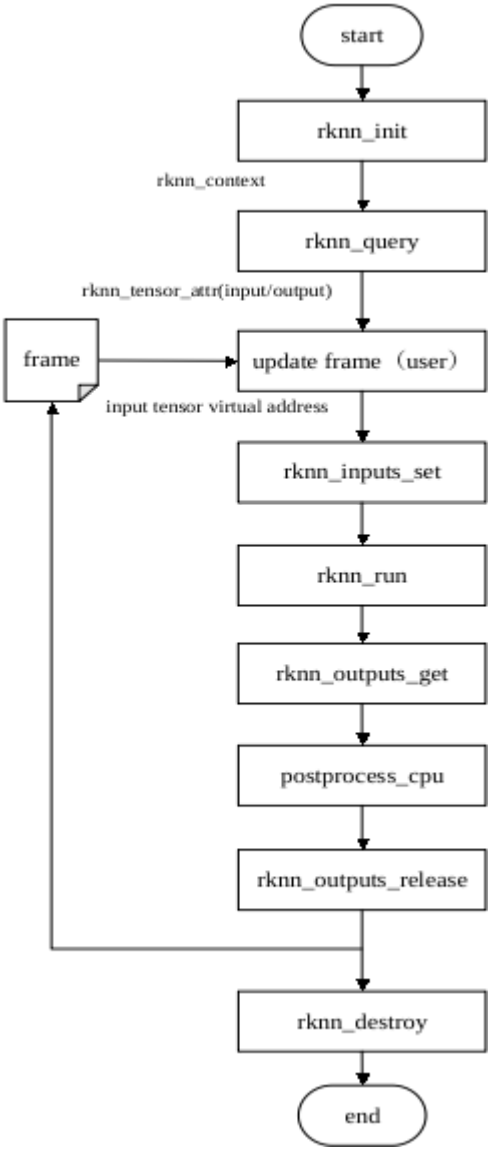


Figure 3-10 General API calling process

RKNN general API interface calling process:

1. Call `rknn_init()` to initialize the model;
2. Call `rknn_query()` to query the input and output attributes of the model;
3. Pre-process the input;
4. Call `rknn_inputs_set()` to configure the input data;
5. Call `rknn_run()` to run model inference;
6. Call `rknn_outputs_get()` to acquire the inference result data;
7. Post-process the output;
8. Call `rknn_outputs_release()` to release the output data memory;
9. Call `rknn_destroy()` to destroy RKNN;

The general API calling process is shown in Figure 3-10.

Example code of the general API calling process:

```
// Init RKNN model
ret = rknn_init(&ctx, model, model_len, 0, NULL);

// Get Model Input Output Number
rknn_input_output_num io_num;
ret = rknn_query(ctx, RKNN_QUERY_IN_OUT_NUM, &io_num, sizeof(io_num));

// Get Model Input Info
rknn_tensor_attr input_attrs[io_num.n_input];
memset(input_attrs, 0, sizeof(input_attrs));
for (int i = 0; i < io_num.n_input; i++)
{
    input_attrs[i].index = i;
    ret = rknn_query(ctx, RKNN_QUERY_INPUT_ATTR, &(input_attrs[i]),
sizeof(rknn_tensor_attr));
}

// Get Model Output Info
rknn_tensor_attr output_attrs[io_num.n_output];
memset(output_attrs, 0, sizeof(output_attrs));
for (int i = 0; i < io_num.n_output; i++)
{
    output_attrs[i].index = i;
    ret = rknn_query(ctx, RKNN_QUERY_OUTPUT_ATTR, &(output_attrs[i]),
sizeof(rknn_tensor_attr));
}

rknn_input inputs[io_num.n_input];
rknn_output outputs[io_num.n_output];
memset(inputs, 0, sizeof(inputs));
memset(outputs, 0, sizeof(outputs));

// Pre-process
// Read Image
image_buffer_t src_image;
memset(&src_image, 0, sizeof(image_buffer_t));
ret = read_image(image_path, &src_image);

// Set Input Data
inputs[0].index = 0;
inputs[0].type = RKNN_TENSOR_UINT8;
inputs[0].fmt = RKNN_TENSOR_NHWC;
inputs[0].size = src_image.size;
inputs[0].buf = src_image.virt_addr;

ret = rknn_inputs_set(rknn_ctx, io_num.n_input, inputs);

// Run
ret = rknn_run(rknn_ctx, nullptr);

// Get Output Data
ret = rknn_outputs_get(rknn_ctx, io_num.n_output, outputs, NULL);

// Post-process
post_process(outputs, results);

// Release RKNN Output
rknn_outputs_release(rknn_ctx, io_num.n_output, outputs);

if (rknn_ctx != 0)
```

```
{
    rknn_destroy(rknn_ctx);
}
```

3.2.2 Model Initialization

Model initialization is done via the `rknn_init()` API, which loads the RKNN model and creates the runtime context; it is the prerequisite of all subsequent operations.

`rknn_init()` supports two model loading modes:

- **Load from file path**: pass the `.rknn` model file path string via the `model` parameter with `size` set to 0; the Runtime reads the model file itself.
- **Load from memory**: read the `.rknn` model file into memory and pass the data pointer via `model` with the data length via `size`. Suitable for scenarios where the model is preloaded in memory, or decrypted in application code after model encryption.

The `flag` parameter specifies initialization flags (e.g. runtime performance collection `RKNN_FLAG_COLLECT_PERF_MASK`, user-allocated memory `RKNN_FLAG_MEM_ALLOC_OUTSIDE`); pass 0 for default initialization. The `ext` parameter carries extension information; pass NULL in general.

```
rknn_context ctx;
int ret = rknn_init(&ctx, "./model.rknn", 0, 0, NULL);
if (ret != RKNN_SUCCESS) {
    printf("rknn_init failed! ret=%d\n", ret);
    goto out;
}
```

On RK3588 / RK3576, the NPU cores used by the model can be specified via `rknn_set_core_mask()` after initialization; it must match the configuration used during model conversion. See 4.1.2 NPU Multi-Core Configuration.

3.2.3 Input/Output Preparation

After model initialization, query the model's input/output attributes and prepare the input data accordingly.

1. Query input/output attributes

Query the number and attributes of the model's input/output tensors via `rknn_query()`: first query `RKNN_QUERY_IN_OUT_NUM` for the counts, then query each tensor's attributes (`RKNN_QUERY_INPUT_ATTR` / `RKNN_QUERY_OUTPUT_ATTR`) to get shape, dtype, layout, quantization parameters, etc., which are used to organize input data and parse output results.

```
// Query the number of inputs and outputs
rknn_input_output_num io_num;
ret = rknn_query(ctx, RKNN_QUERY_IN_OUT_NUM, &io_num, sizeof(io_num));

// Query each input tensor's attributes
rknn_tensor_attr input_attrs[io_num.n_input];
memset(input_attrs, 0, sizeof(input_attrs));
for (int i = 0; i < io_num.n_input; i++)
{
    input_attrs[i].index = i;
    ret = rknn_query(ctx, RKNN_QUERY_INPUT_ATTR, &(input_attrs[i]),
sizeof(rknn_tensor_attr));
}
```

2. Prepare and set input data

After preprocessing the input data (resize, color-space conversion, etc.), fill in the `rknn_input` structure and pass it to the Runtime via `rknn_inputs_set()`:

- `index`: input index.
- `buf` / `size`: input data pointer and size in bytes; size equals the number of elements times the per-element byte count.
- `type`: input data type (e.g. `RKNN_TENSOR_UINT8`); if it differs from the model input type, the Runtime converts it automatically.
- `fmt`: input data layout (e.g. `RKNN_TENSOR_NHWC`); 4-dimensional inputs should specify the correct layout, non-4-dimensional inputs use `RKNN_TENSOR_UNDEFINED`.
- `pass_through`: whether to skip the Runtime's internal preprocessing (normalization, type conversion, layout conversion).

```
rknn_input inputs[io_num.n_input];
memset(inputs, 0, sizeof(inputs));
inputs[0].index = 0;
inputs[0].type = RKNN_TENSOR_UINT8;
inputs[0].fmt = RKNN_TENSOR_NHWC;
inputs[0].size = src_image.size;
inputs[0].buf = src_image.virt_addr;

ret = rknn_inputs_set(ctx, io_num.n_input, inputs);
```

> **Note**:
>
> - If `mean_values` and `std_values` were configured via `rknn.config()` during model conversion, the Runtime performs normalization automatically during general-API inference; no mean/std handling is needed in the application, which effectively reduces deployment time.
> - The channel order (RGB / BGR) of input data must match the model's training configuration. If `quant_img_RGB2BGR` was used during quantization, note that it does not take effect at inference time; channel conversion is the user's responsibility.
> - Input preparation differs for the zero-copy API: allocate memory via `rknn_create_mem()` and bind it with `rknn_set_io_mem()`; the memory size, dtype, and layout must strictly match the queried tensor attributes. See 3.1.6 RKNN Runtime Zero-Copy Interface for details.

3.2.4 Running Inference

After the inputs are set, run inference via `rknn_run()`. It is a synchronous blocking call: when it returns, the current inference is complete.

```
ret = rknn_run(ctx, nullptr);
if (ret < 0) {
    printf("rknn_run failed! ret=%d\n", ret);
    goto out;
}
```

On the multi-core RK3588 / RK3576 platforms, set the core mask via `rknn_set_core_mask()` before inference (see 3.2.2 Model Initialization). For methods of measuring inference time (`RKNN_QUERY_PERF_RUN` query, `RKNN_LOG_LEVEL` logs), see 5.2 Performance Analysis and Optimization.

3.2.5 Output Processing

After inference, retrieve the output data via `rknn_outputs_get()`, run post-processing, and call `rknn_outputs_release()` to free the output memory after use.

```
rknn_output outputs[io_num.n_output];
memset(outputs, 0, sizeof(outputs));
outputs[0].want_float = 1;
ret = rknn_outputs_get(ctx, io_num.n_output, outputs, NULL);
if (ret < 0) {
    printf("rknn_outputs_get failed! ret=%d\n", ret);
    goto out;
}

// Post-processing (e.g. argmax for classification, NMS for detection)
post_process(outputs, results);

// Release output memory
rknn_outputs_release(ctx, io_num.n_output, outputs);
```

Notes when reading output data:

- **Data type**: with `want_float = 1`, the Runtime converts outputs to float32 before returning; with `want_float = 0`, the model's original output type is returned (INT8 quantized data for quantized models, float16 for non-quantized models), and dequantization to floating-point values must be done with the quantization parameters (scale / zero_point) from the queried tensor attributes.
- **Data layout**: the general API returns outputs in the model's original layout; 4-dimensional outputs are generally `NCHW`, which post-processing should parse accordingly.
- **Memory release**: the output memory obtained via `rknn_outputs_get()` is allocated by the Runtime and must be released via `rknn_outputs_release()` to avoid leaks; the zero-copy API reads the bound memory directly and has no such step.

3.2.6 Generating Deployment C Code

RKNN-Toolkit2 version 2.0.0 adds a new Codegen interface, which is used to generate model deployment code and simplify the difficulty for developers to get started. Codegen is re-encapsulated based on the CAPI zero-copy interface, and the interface style is consistent with the demo in RKNN_Model_Zoo. The generated deployment code can be used to directly test performance and verify inference accuracy, and developers can also conduct secondary development based on the generated code.

Usage example:

```
ret = rknn.codegen(output_path='./rknn_app_demo',
                   inputs=['../../caffe/mobilenet_v2/dog_224x224.jpg'],
overwrite=True)
```

- Before calling the codegen interface, you must first call the rknn.export interface to save the RKNN model.
- output_path is the output folder directory, the user can configure the directory name
- Inputs fills in the path list of model input. It is allowed to leave it blank. The valid file format is jpg/png/npy. When npy files are used as input, the dimension information of npy data should be consistent with the dimension information of the model input.
- When overwrite is set to True, the files in the directory specified by output_path will be overwritten. The default value is False
- After generating the deployment code, please refer to the README.md document instructions in the generation directory to compile and test
- If inputs are filled with valid values, after inference, the deployment code example will evaluate the difference in inference results between the CAPI interface and the RKNN-Toolkit2 simulator. The evaluation method is to compare the cosine similarity of each output.
- No NPU hardware platform restrictions, the board system is required to be Linux or Android
- Supports quantitative and non-quantitative models

3.3 Toolkit Lite Python Deployment

RKNN-Toolkit Lite2 provides users with a Python interface for board-side model inference, making it convenient for users to use Python language for AI application development.

Note: Before using RKNN-Toolkit Lite2 to develop AI applications, you need to convert the models exported by each deep learning framework into RKNN models through RKNN-Toolkit2. Please refer to Chapter 2.1 for detailed model conversion methods.

3.3.1 Runtime Environment and Installation

RKNN-Toolkit Lite2 runs on the development board. The supported hardware platforms, operating systems, Python versions, and dependent libraries depend on the RKNN-Toolkit Lite2 release package of the corresponding version. For the specific support scope, refer to the "Applicable Versions and Platforms" chapter in 《04_RKNN2_Toolkit_Lite_Python_API_Reference_V2.5.0》.

RKNN-Toolkit Lite2 is installed through a Python Wheel package. The SDK provides installation packages for the corresponding platforms and Python versions, located in the `rknn-toolkit-lite2/packages` directory. You can select the corresponding Wheel according to the Python version of the development board and complete the installation with `pip3 install`. For example:

```
pip3 install rknn_toolkit_lite2-x.y.z-cpxx-cpxx-linux_aarch64.whl
```

Before installation, make sure the development board is already equipped with Python, pip, and the related dependency environment. For complete installation steps and dependency instructions, refer to the "Installation and Environment Deployment" chapter in 《04_RKNN2_Toolkit_Lite_Python_API_Reference_V2.5.0》.

3.3.2 Basic Deployment Process

RKNN-Toolkit Lite2 uses the `RKNNLite` object as the main entry of the Python API. The typical on-board deployment process is as follows:

```
Create RKNNLite object
       ↓
  load_rknn()
       ↓
 init_runtime()
       ↓
Prepare input and complete pre-processing
       ↓
  inference()
       ↓
  Process inference results
       ↓
  release()
```

The main corresponding interfaces include:

- `RKNNLite()`: create an RKNN-Toolkit Lite2 object;
- `load_rknn()`: load the RKNN model;
- `init_runtime()`: initialize the Runtime environment;

- `inference()`: feed input data and run model inference;
- `release()`: release the RKNNLite object and related resources.

Before calling `inference()`, the application needs to complete data reading and pre-processing according to the model input requirements; after the inference is completed, post-processing such as classification or detection is performed according to the model type.

3.3.3 Main Function Interfaces

In addition to the basic model deployment process, RKNN-Toolkit Lite2 provides interfaces for querying runtime information and supported platforms. Common interfaces are as follows:

Interface	Function
`RKNNLite()`	Create an RKNNLite object, with configurable logging
`load_rknn()`	Load an RKNN model
`init_runtime()`	Initialize the Runtime environment required for model inference
`inference()`	Run model inference and return the results
`get_sdk_version()`	Query the versions of the Runtime, driver, and model
`list_support_target_platform()`	Query the target chip platforms supported by the RKNN model
`release()`	Release runtime resources

`init_runtime()` can configure the NPU core working mode according to the supported platform; `inference()` accepts model inputs as a list of `ndarray` objects and allows specifying the data layout as needed. For parameter values, usage restrictions, and return value conventions, refer to the interface descriptions in 《04_RKNN2_Toolkit_Lite_Python_API_Reference_V2.5.0》.

3.3.4 Reference Example

The following is a basic inference example of RKNN-Toolkit Lite2, showing the complete calling process of creating the `RKNNLite` object, loading the RKNN model, initializing the Runtime, running inference, and releasing resources. In the example, `input_data` represents the input data that has already been pre-processed according to the model requirements.

```
rknn_lite = RKNNLite(verbose=True)

# 1. Load RKNN model
ret = rknn_lite.load_rknn('./model.rknn')
if ret != 0:
    print('Load RKNN model failed')
    exit(ret)

# 2. Initialize Runtime
ret = rknn_lite.init_runtime(core_mask=RKNNLite.NPU_CORE_AUTO)
if ret != 0:
    print('Init runtime environment failed')
    exit(ret)

# 3. Run inference
outputs = rknn_lite.inference(inputs=[input_data])

# 4. Post-process outputs according to the model type
# ...

# 5. Release resources
rknn_lite.release()
```

The SDK's `examples/inference_with_lite` directory also provides a complete image classification example based on ResNet18. After deploying the example, the RKNN model, and the test data to a development board with RKNN-Toolkit Lite2 installed, you can enter the example directory and run:

```
cd examples/inference_with_lite
python3 test.py
```

This example covers input pre-processing, model inference, and Top-5 classification result processing, and can be used as a reference for actual application development. For the parameters, return values, and usage restrictions of interfaces such as `RKNNLite`, `load_rknn()`, `init_runtime()`, `inference()`, and `release()`, refer to 《04_RKNN2_Toolkit_Lite_Python_API_Reference_V2.5.0》.

4. Advanced Features and Extensions

4.1 Advanced Runtime Features

4.1.1 Matrix Multiplication API

Matmul API is a separate set of C API provided by Runtime, which is used to run matrix multiplication operations on the NPU (RK2118 / RV1103 series / RV1106 series are not supported yet). Matrix multiplication is an important operation in linear algebra. The operation is defined as follows: **C=AB**. Where A is an M×K matrix, B is a K×N matrix, and C is an M×N matrix.

4.1.1.1 FEATURES AND USAGE

The Matmul API is mostly used for parameter calculation tasks in deep learning. For example, matrix multiplication is used extensively for computation in the main modules of the currently widely used Transformer model structure (the self-attention mechanism and the feedforward neural network layers), so the efficiency of matrix multiplication is crucial to the overall performance of Transformer models. The Matmul API has the following characteristics:

- **Efficient**: The underlying layer is implemented with RKNPU, offering the advantages of high performance and low power consumption.
- **Flexible**: No need to load an RKNN model. It supports int4, int8 and float16, the three input data types commonly used for edge-side computing, and provides a separate memory allocation interface or a mechanism for using external memory, so users can manage and reuse the input and output memory of the matrices.

4.1.1.2 MATMUL API FLOW CHART

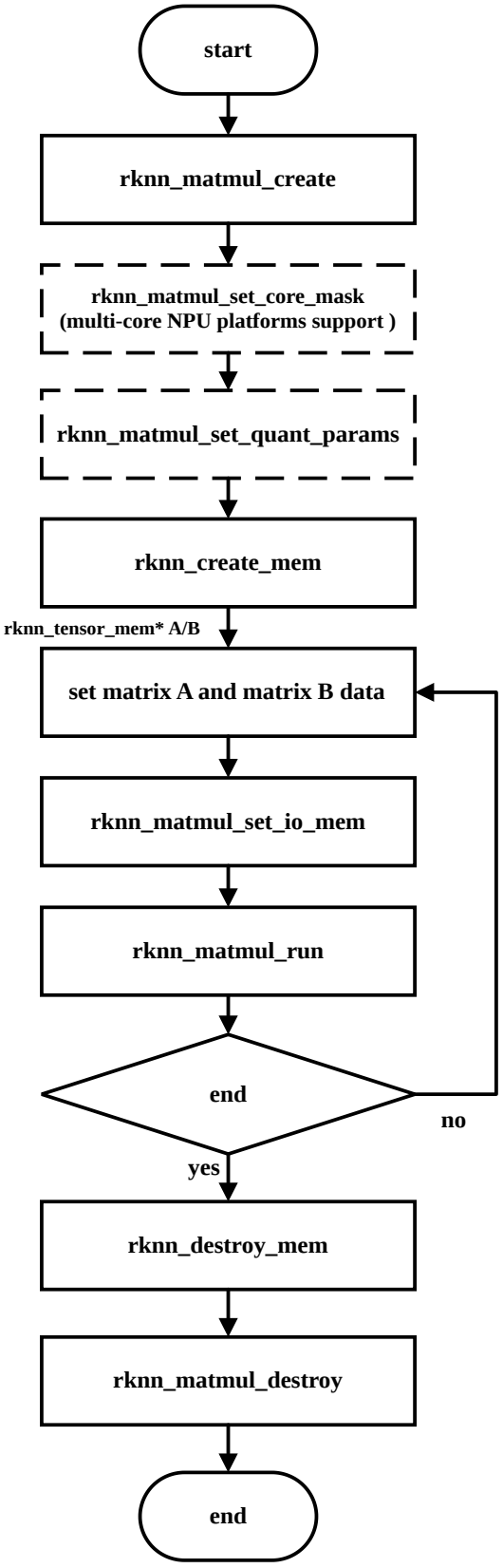


Figure 4-1 The flow chart of basic Matmul API

First, the structures and interfaces of the Matmul API are located in the rknn_matmul_api.h header file, which must be included when compiling the developer's program. The flow of basic

Matmul API usage is shown in the figure above.

Using the Matmul API typically involves the following steps:

1. Create the context: set the rknn_matmul_info structure, which contains M, K, N, the data types of the input and output matrices, the data layout used by the input and output matrices, and other information; then, call the rknn_matmul_create interface to initialize the context. After initialization, obtain the rknn_matmul_io_attr structure pointer, which contains the tensor information of the input and output matrices.
2. Specify the NPU core to run on (only valid on chip platforms with a multi-core NPU): call rknn_matmul_set_core_mask and set the mask to specify a certain NPU core to perform the computation. On chip platforms with a multi-core NPU, the default behavior when this interface is not called is to automatically select an idle core.
3. Set the quantization parameters of matrices A, B and C: call rknn_matmul_set_quant_params to pass in the rknn_quant_params structure containing the quantization parameter values of the corresponding matrices. The default behavior when this interface is not called is scale=1.0, zero_point=0 for each matrix.
4. Create input and output memory: call the rknn_create_mem interface to create memory according to the sizes in the input and output matrix tensor information.
5. Fill in the input data: fill in the data of input matrices A and B according to the shape and data type, and set the quantization parameters with the quantized values.
6. Set input and output memory: call rknn_matmul_set_io_mem to record the input matrix filled with data into the context, and the output memory is also recorded into the context. Besides recording the memory addresses, this interface also rearranges the data according to the configured layout, so it must be called after the input data is filled in or updated. This behavior differs from that of the rknn_set_io_mem interface in the zero-copy API.
7. Run the matrix multiplication: after the input and output memory is set up, call rknn_matmul_run to perform the matrix multiplication.
8. Process the output: after the matrix multiplication is performed, read the result from the output memory according to the configured layout.
9. Destroy the resources: after the execution is finished, call rknn_destroy_mem and rknn_matmul_destroy to destroy the memory and context resources respectively.

4.1.1.3 ADVANCED USAGE OF MATMUL API

When creating the context, the user is required to set the `rknn_matmul_info` structure. `rknn_matmul_info` represents the specification information used to perform matrix multiplication; it contains the scale of the matrix multiplication, as well as the data types and data layouts of the input and output matrices. Among them, B_layout and AC_layout are used to set the high-performance data layouts. The specific structure definition is shown in the following table:

Table 4-1 Definition of the rknn_matmul_info structure

Member	Data Type	Description
M	int32_t	The number of rows of matrix A
K	int32_t	The number of columns of matrix A

Member	Data Type	Description
N	int32_t	The number of columns of matrix B
type	rknn_matmul_type	The data types of the input and output matrices: RKNN_FLOAT16_MM_FLOAT16_TO_FLOAT32: indicates that matrices A and B are of type float16, and matrix C is of type float32; RKNN_INT8_MM_INT8_TO_INT32: indicates that matrices A and B are of type int8, and matrix C is of type int32; RKNN_INT8_MM_INT8_TO_INT8: indicates that matrices A, B and C are of type int8; RKNN_FLOAT16_MM_FLOAT16_TO_FLOAT16: indicates that matrices A, B and C are of type float16; RKNN_FLOAT16_MM_INT8_TO_FLOAT32: indicates that matrix A is of type float16, matrix B is of type int8, and matrix C is of type float32; RKNN_FLOAT16_MM_INT8_TO_FLOAT16: indicates that matrix A is of type float16, matrix B is of type int8, and matrix C is of type float16; RKNN_FLOAT16_MM_INT4_TO_FLOAT32: indicates that matrix A is of type float16, matrix B is of type int4, and matrix C is of type float32; RKNN_FLOAT16_MM_INT4_TO_FLOAT16: indicates that matrix A is of type float16, matrix B is of type int4, and matrix C is of type float16; RKNN_INT8_MM_INT8_TO_FLOAT32: indicates that matrices A and B are of type int8, and matrix C is of type float32; RKNN_INT4_MM_INT4_TO_INT16: indicates that matrices A and B are of type int4, and matrix C is of type int16; RKNN_INT8_MM_INT4_TO_INT32: indicates that matrix A is of type int8, matrix B is of type int4, and matrix C is of type int32
B_layout	int16_t	The data layout of matrix B. RKNN_MM_LAYOUT_NORM: indicates that matrix B is arranged in its original shape, i.e., the KxN shape; RKNN_MM_LAYOUT_NATIVE: indicates that matrix B is arranged in a high-performance shape; RKNN_MM_LAYOUT_TP_NORM: indicates that matrix B is arranged in the shape after Transpose, i.e., the NxK shape
B_quant_type	int16_t	The quantization type of matrix B. RKNN_QUANT_TYPE_PER_LAYER_SYM: indicates that matrix B is symmetrically quantized in the Per-Layer manner; RKNN_QUANT_TYPE_PER_LAYER_ASYM: indicates that matrix B is asymmetrically quantized in the Per-Layer manner; RKNN_QUANT_TYPE_PER_CHANNEL_SYM: indicates that matrix B is symmetrically quantized in the Per-Channel manner; RKNN_QUANT_TYPE_PER_CHANNEL_ASYM: indicates that matrix B is asymmetrically quantized in the Per-Channel manner; RKNN_QUANT_TYPE_PER_GROUP_SYM: indicates that matrix B is symmetrically quantized in the Per-Group manner; RKNN_QUANT_TYPE_PER_GROUP_ASYM: indicates that matrix B is asymmetrically quantized in the Per-Group manner

Member	Data Type	Description
AC_layout	int16_t	The data layout of matrices A and C. RKNN_MM_LAYOUT_NORM: indicates that matrices A and C are arranged in their original shapes; RKNN_MM_LAYOUT_NATIVE: indicates that matrices A and C are arranged in high-performance shapes; For a detailed description of the layouts, please refer to 4.1.1.4 High-Performance Data Layouts
AC_quant_type	int16_t	The quantization type of matrices A and C. RKNN_QUANT_TYPE_PER_LAYER_SYM: indicates that matrices A and C are symmetrically quantized in the Per-Layer manner; RKNN_QUANT_TYPE_PER_LAYER_ASYM: indicates that matrices A and C are asymmetrically quantized in the Per-Layer manner
iommu_domain_id	int32_t	The index of the IOMMU address space domain where the matrix context resides. IOMMU address spaces correspond to contexts one-to-one, and each IOMMU address space is 4GB in size. This parameter is mainly used when the specifications of matrices A, B and C are large, and the memory allocated by the NPU in a domain exceeds 4GB and another domain needs to be switched to.
group_size	int16_t	The number of elements in a group. It takes effect only after group quantization is enabled.
reserved	int8_t[]	Reserved field

Among them, the original shape of matrix A is MxK, the original shape of matrix B is KxN, and the original shape of matrix C is MxN.

4.1.1.3.1 Matrix multiplication with specified quantization parameters

If the high-bit data of the matrix product needs to be quantized into low-bit data according to the quantization parameters, for example, matrices A and B are of type int8 and matrix C is of type int8, the quantization parameters of the matrices need to be set. The quantization parameters consist of scale and zero_point, represented by the rknn_quant_params structure. The quantization parameters of matrices A and C are set through the rknn_matmul_set_quant_params interface. After the quantization parameters are set successfully, the quantization parameters of matrix B in Per-Channel quantization mode can be read through the rknn_matmul_get_quant_params interface. The definition of the rknn_quant_params data structure is shown in the following table:

Table 4-2 Definition of the rknn_quant_params structure

Member	Data Type	Description
name	char[]	The name of the matrix
scale	float*	The first address of the quantization parameter scale array
scale_len	int32_t	The length of the quantization parameter scale array
zp	int32_t*	The first address of the quantization parameter zero_point array
zp_len	int32_t	The length of the quantization parameter zero_point array

Matmul supports three quantization methods: Per-Layer, Per-Channel and Per-Group, and the differences of the quantization parameters are as follows:

- Per-Layer method: the length of the scale and zp arrays is 1.
- Per-Channel method: the length of the scale and zp arrays is N.
- Per-Group method: the length of the scale and zp arrays is N*K/group_size. The number of elements in each group is specified by group_size, and elements in the same group share the same quantization parameters. In terms of the storage order of the quantization parameter arrays, the quantization parameters of K/group_size groups are stored first, and then N quantization parameters are stored.

Each quantization method can be either symmetric quantization or asymmetric quantization: symmetric quantization means that zp is all 0, and asymmetric quantization means that zp is not all 0.

4.1.1.3.2 Matrix multiplication of dynamic shape input

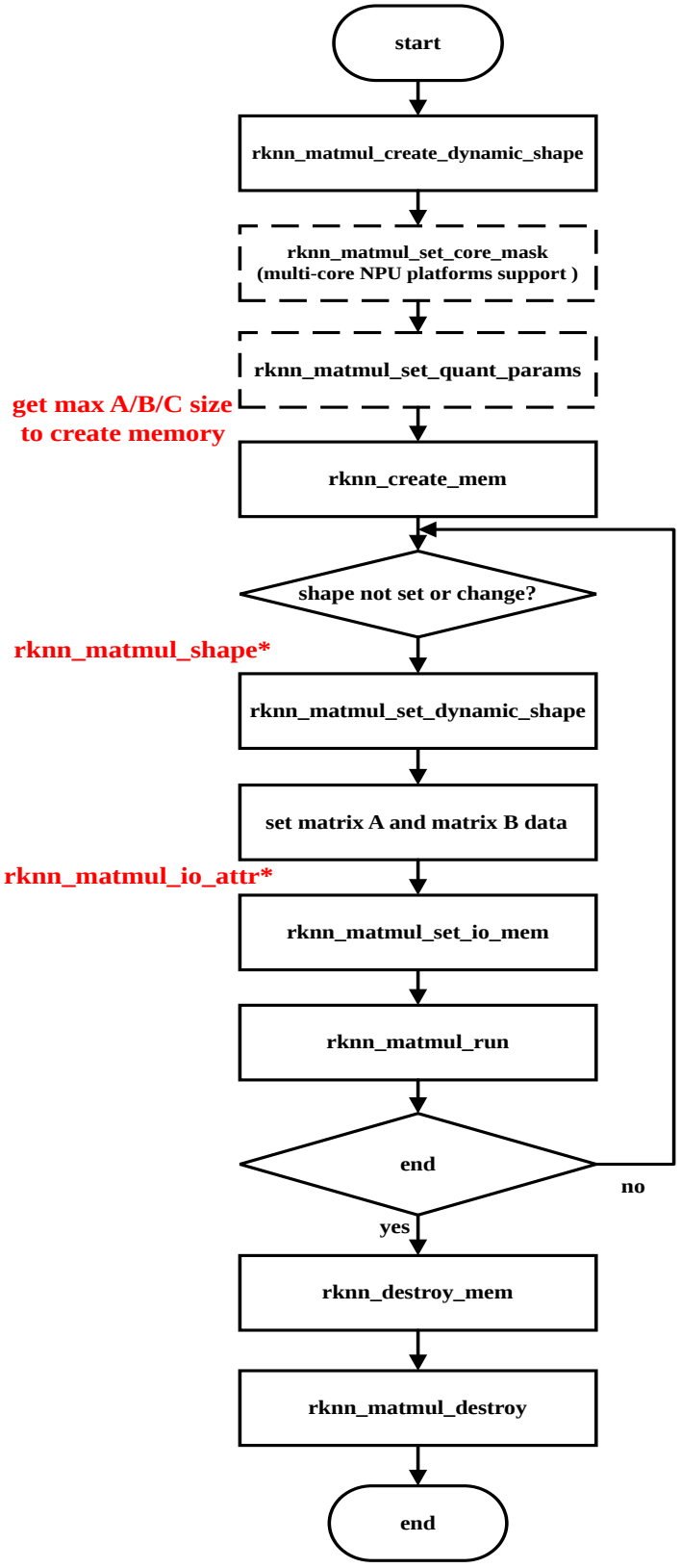


Figure 4-2 Matmul API calling process for dynamic shape input

The flow of Matmul API usage for dynamic shape input is shown in the figure above. The differences between using the dynamic shape Matmul API and the basic Matmul API are as follows:

1. Create the context: configure the number of shapes and the required shapes, and set the rknn_matmul_info structure, which contains the data types of the input and output matrices, the data layout used by the input and output matrices, and other information. **Note that M, K and N in the dynamic shape Matmul interface parameter rknn_matmul_info do not need to be set**. Then, call the rknn_matmul_create_dynamic_shape interface to initialize the context. After initialization, obtain the rknn_matmul_io_attr structure array, which contains the input and output matrix tensor information under all shape configurations.
2. Create input and output memory: first obtain the maximum tensor size from the rknn_matmul_io_attr structure array, then call the rknn_create_mem interface to create memory using the maximum input and output matrix tensor sizes.
3. Set the input shape: by calling the rknn_matmul_set_dynamic_shape interface and passing in the rknn_matmul_shape pointer, set the shape used in the current inference process.
4. Set input and output memory: call rknn_matmul_set_io_mem to record the input matrix filled with data into the context, and the output memory is also recorded into the context. The passed-in rknn_tensor_mem *parameter is the input and output memory of the maximum size created previously, and the rknn_matmul_tensor_attr* parameter is the matrix attribute information corresponding to the current shape.

4.1.1.4 THE DATA LAYOUT FOR HIGH-PERFORMANCE

Since the NPU is a dedicated hardware architecture, reading data in original shapes such as MxK and KxN is not the most efficient; likewise, writing the C matrix in the MxN shape is not the most efficient either. Users can achieve higher performance by using special data layouts. **The AC_layout parameter controls whether matrices A and C use the high-performance data layout, and the B_layout parameter controls whether matrix B uses the high-performance data layout.** Assume that the original shape of matrix A is MxK, the original shape of matrix B is KxN, and the original shape of matrix C is MxN. The required data layouts are as follows:

1. If AC_layout=RKNN_MM_LAYOUT_NORM and B_layout=RKNN_MM_LAYOUT_NORM, the shape of matrix A is [M,K], the shape of matrix B is [K,N], and the shape of matrix C is [M,N].
2. If AC_layout=RKNN_MM_LAYOUT_NATIVE and B_layout=RKNN_MM_LAYOUT_NATIVE, the high-performance data layouts of matrices A, B and C under different chip platforms and data types are shown in the following table (the division results in the table are all rounded up, and the extra parts are padded with 0):
3. The difference between B_layout=RKNN_MM_LAYOUT_NORM and B_layout=RKNN_MM_LAYOUT_TP_NORM is that in the former, matrix B is in the shape of [K,N], while in the latter, matrix B is in the shape of [N,K]; the latter offers better execution efficiency.

Table 4-3 High-performance shapes of matrices A, B and C on each chip platform

	RK3562	**RK3566/RK3568**	**RK3576/RV1126B**	**RK3588**
A shape (int4)	not supported yet	not supported yet	[K/32,M,32]	[K/32,M,32]
B shape (int4)	not supported yet	not supported yet	[N/64,K/32,64,32]*	[N/64,K/32,64,32]
C shape (int16)	not supported yet	not supported yet	[N/8,M,8]	[N/8,M,8]

	RK3562	RK3566/RK3568	RK3576/RV1126B	RK3588
A shape (int8)	[K/16,M,16]	[K/8,M,8]	[K/16,M,16]	[K/16,M,16]
B shape (int8)	[N/16,K/32,16,32]	[N/16,K/32,16,32]	[N/32,K/32,32,32]*	[N/32,K/32,32,32]
C shape (int32)	[N/4,M,4]	[N/4,M,4]	[N/4,M,4]	[N/4,M,4]
A shape (float16)	[K/8,M,8]	[K/4,M,4]	[K/8,M,8]	[K/8,M,8]
B shape (float16)	[N/8,K/32,8,32]	[N/8,K/16,8,16]	[N/16,K/32,16,32]	[N/16,K/32,16,32]
C shape (float32)	[N/4,M,4]	[N/4,M,4]	[N/4,M,4]	[N/4,M,4]

> [!NOTE]
>
> The data layouts in the table above are the high-performance shapes for the "**same-type**" case of A and B (i.e., A and B have the same data type). RK3576/RV1126B supports "**heterogeneous**" inputs, i.e., A and B are allowed to have different data types. In the "heterogeneous" input case, the high-performance data layout of B is the same as the shape of B in the "same-type" case corresponding to the type of C. For example, if A is of type int4, B is of type float16, and C is of type float32, then the high-performance data layout of B is [N/16,K/32,16,32]. The `rknn_B_normal_layout_to_native_layout` interface provides the functionality of converting the original shape of B into the high-performance layout.

Taking the RK3566/RK3568 platform and the int8 data type as an example, the element correspondence of matrix A transformed from [M,K] to [K/8,M,8] is shown in the figure below:

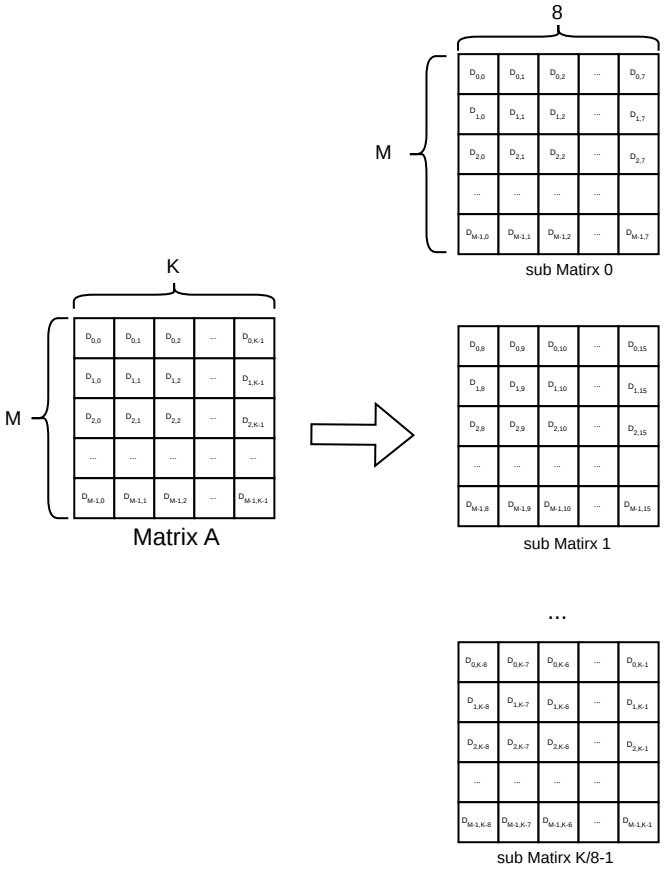


Figure 4-3 The element correspondence diagram of int8 type matrix A transformed from [M,K] to [K/8,M,8]

In the figure, the data corresponding to the (i,j)-th element of matrix A is Di,j; the left figure is a matrix with shape=[M,K], and each small matrix in the right figure has shape=[M,8], K/8 of them from top to bottom.

The example code for converting matrix A or C from the original shape to the high-performance shape is as follows:

```
template <typename Ti, typename To>
void norm_layout_to_perf_layout(Ti *src, To *dst, int32_t M, int32_t K, int32_t
subK){
    int outter_size = (int)std::ceil(K * 1.0f / subK);
    for (int i = 0; i < outter_size; i++)
    {
        for (int m = 0; m < M; m++)
        {
            for (int j = 0; j < subK; j++)
            {
                int ki = i * subK + j;
                if (ki >= K)
                {
                  dst[i * M * subK + m * subK + j] = 0;
                }
                else
                {
                  dst[i * M * subK + m * subK + j] = src[m * K + ki];
                }
            }
        }
    }
}
```

Taking the RK3566/RK3568 platform and the int8 data type as an example, the element correspondence of matrix B transformed from [K,N] to [N/16,K/32,16,32] is shown in the figure below:

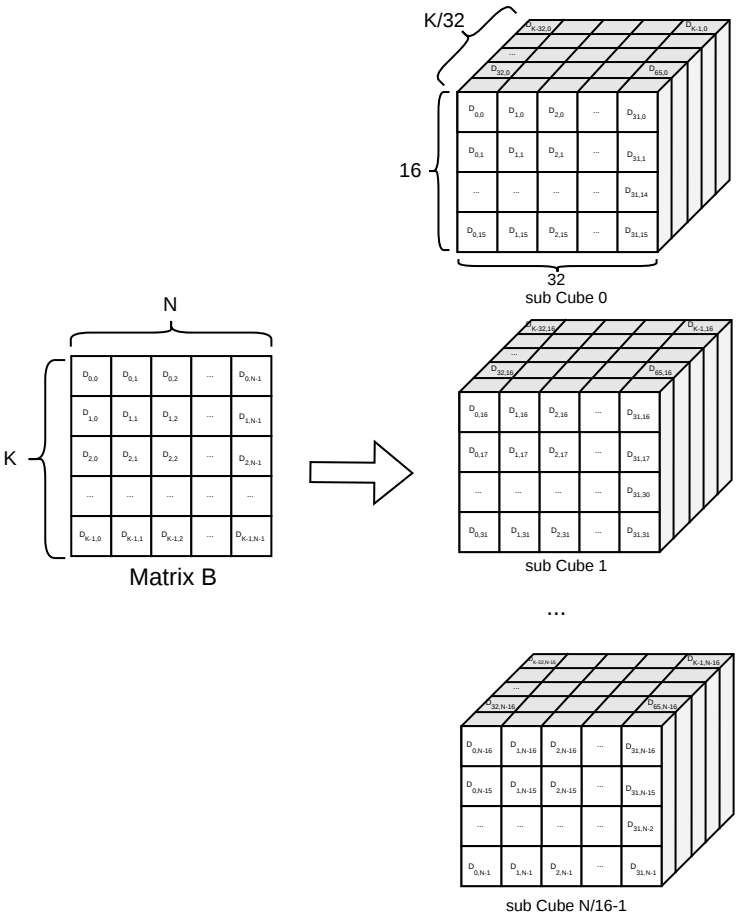


Figure 4-4 The element correspondence diagram of the int8 type matrix B transformed from [K, N] to [N/16, K/32, 16, 32]

In the figure, the data corresponding to the (i,j)-th element of matrix B is Di,j; the left figure is a matrix with shape=[K,N], and each small cube in the right figure has shape=[K/32,16,32], N/16 of them from top to bottom.

The example code for converting matrix B from the original shape to the high-performance shape is as follows:

```
template <typename Ti, typename To>
voi norm_layout_to_native_layout(Ti *src, To *dst, int32_t K, int32_t N, int32_t
subN, int32_t subK)
{
    int N_remain = (int)std::ceil(N * 1.0f / subN);
    int K_remain = (int)std::ceil(K * 1.0f / subK);
    for (int i = 0; i < N_remain; i++)
    {
        for (int j = 0; j < K_remain; j++)
        {
            for (int n = 0; n < subN; n++)
            {
                int ni = i * subN + n;
                for (int k = 0; k < subK; k++)
                {
                    int ki = j * subK + k;
                    if (ki < K && ni < N)
                    {
                        dst[((i * K_remain + j) * subN + n) * subK + k] = src[ki *
N + ni];
                    }
                    else
                    {
                        dst[((i * K_remain + j) * subN + n) * subK + k] = 0;
                    }
                }
            }
        }
    }
}
```

> [!NOTE]
>
> For the complete Matmul API example code, see https://github.com/airockchip/rknn-toolkit2/tree/master/rknpu2/examples/rknn_matmul_api_demo.

4.1.1.4.1 Matrix specification restrictions

The Matmul API is implemented based on the NPU hardware architecture and is subject to hardware specifications. It has the following restrictions on the parameters:

- AC_layout and B_layout can be set independently. AC_layout and B_layout support RKNN_MM_LAYOUT_NORM or RKNN_MM_LAYOUT_TP_NORM on all hardware platforms; except for the RK3566/RK3568 platform, all other platforms support RKNN_MM_LAYOUT_TP_NORM.
- The Matmul interface supports a variety of input and output data bit widths. The matrix data type combinations currently supported by each platform are shown in the following table:

Table 4-4 Data type combinations of matrices A, B and C supported by the Matmul interface

Serial number	RK356X	RK3576/RV1126B	RK3588
1	RKNN_INT8_MM_INT8_TO_INT32	RKNN_FLOAT16_MM_FLOAT16_TO_FLOAT16	RKNN_FLOAT16_MM_FLOAT16_TO_FLOAT16
2	RKNN_FLOAT16_MM_FLOAT16_TO_FLOAT32	RKNN_FLOAT16_MM_FLOAT16_TO_FLOAT32	RKNN_FLOAT16_MM_FLOAT16_TO_FLOAT32
3		RKNN_INT8_MM_INT8_TO_INT32	RKNN_INT8_MM_INT8_TO_INT32
4		RKNN_FLOAT16_MM_INT8_TO_FLOAT32	RKNN_INT8_MM_INT8_TO_FLOAT32
5		RKNN_FLOAT16_MM_INT4_TO_FLOAT16	RKNN_INT8_MM_INT8_TO_INT8
6		RKNN_FLOAT16_MM_INT4_TO_FLOAT32	RKNN_INT4_MM_INT4_TO_INT16
7		RKNN_INT8_MM_INT4_TO_INT32	

Where RK356X refers to platforms such as RK3562/RK3566/RK3568. The float16 floating-point format follows the IEEE-754 standard; for the specific format, please refer to IEEE-754 half.

- The size restrictions of K and N in Matmul are as follows:

Table 4-5 Size restrictions of K and N on each chip platform

	RK3562	RK3566/RK3568	RK3576/RV1126B	RK3588
K size limit (int4)	not supported yet	not supported yet	*	32-aligned
K size limit (int8)	K <= 10240 and K mod 32 = 0	K <= 10240 and K mod 32 = 0	32-aligned	32-aligned
K size limit (float16)	K <= 10240 and K mod 32 = 0	K <= 10240 and K mod 32 = 0	32-aligned	32-aligned
N size limit (int4)	not supported yet	not supported yet	*	N <= 8192 and N mod 64 = 0
N size limit (int8)	N <= 4096 and N mod 16 = 0	N <= 4096 and N mod 16 = 0	N <= 4096 and N mod 32 = 0	N <= 4096 and N mod 32 = 0
N size limit (float16)	N <= 4096 and N mod 16 = 0	N <= 4096 and N mod 16 = 0	N <= 4096 and N mod 32 = 0	N <= 4096 and N mod 32 = 0

- > [!NOTE]
  >
  > * indicates that the size and alignment restrictions are the same as those of B corresponding to the data type of A on the same platform. For example, the size and alignment restrictions of B for the RKNN_FLOAT16_MM_INT4_TO_FLOAT32 combination type are the same as those for the RKNN_FLOAT16_MM_FLOAT16_TO_FLOAT32 combination.

For RK3588:

When K > 8192, B will be split into T segments,

```
int T = std::ceil(K / 8192);
```

For example: normal layout -> native layout, K = 20488, N = 4096, T = 3, the data will be split into 3 segments,

```
subN = rknn_matmul_io_attr.B.dims[2];
subK = rknn_matmul_io_attr.B.dims[3];
```

The transformation process of K,N is as follows:

```
                                          (8196, 4096)   (4096 / subN, 8196 / subK,
subN, subK)
 (K, N) = (20488, 4096)->(8196, 4096)->(4096 / subN, 8196 / subK, subN, subK)
                 normal layout  (4096, 4096)    (4096 / subN, 4096 / subK, subN,
subK)
                                      T normal layout                  T native
layout
```

It is recommended to use the rknn_B_normal_layout_to_native_layout interface for direct data conversion.

For RK3576/RV1126B: When K > 4096, B will be split into T segments,

```
int T = std::ceil(K / 4096);
```

For example: normal layout -> native layout, K = 10240, N = 2048, T = 3, the data will be split into 3 segments,

```
subN = rknn_matmul_io_attr.B.dims[2];
subK = rknn_matmul_io_attr.B.dims[3];
```

The transformation process of K,N is as follows:

```
                                          (4096, 2048)   (2048 / subN, 4096 / subK,
subN, subK)
(K, N) = (10240, 2048) ->(4096, 2048)->(2048 / subN, 4096 / subK, subN, subK)
                normal layout   (2048, 2048)    (2048 / subN, 2048 / subK, subN,
subK)
                                       T normal layout                   T native
layout
```

It is recommended to use the rknn_B_normal_layout_to_native_layout interface for direct data conversion.

4.1.2 NPU Multi-Core Configuration

RK3588 provides stronger computing power through its 3-core NPU, and RK3576 through its 2-core NPU. This section describes in detail the configuration method of the multi-core NPU to improve the inference efficiency of models.

Note: Multi-core operation is suitable for networks with large per-layer computation workloads. For small networks the improvement is limited, and performance may even degrade due to switching between single-core and multi-core modes (this switching requires CPU intervention).

4.1.2.1 MULTI-CORE OPERATION CONFIGURATION METHOD

If you use Python as the application development language, you can set the NPU core for the model to run on through the "`core_mask`" parameter in the `init_runtime()` interface of RKNN-Toolkit2 or RKNN-Toolkit Lite2. The detailed description of this parameter is as follows:

Table 4-6 init_runtime interface core_mask parameter description

Parameter	Description
core_mask	This parameter is used to set the NPU core for the model to run on. The available values and the corresponding descriptions are as follows: NPU_CORE_AUTO: automatic scheduling mode; the model automatically runs on the currently idle NPU core in single-core mode. NPU_CORE_0: the model runs on NPU Core0. NPU_CORE_1: the model runs on NPU Core1. NPU_CORE_2: the model runs on NPU Core2. NPU_CORE_0_1: the model runs on NPU Core0 and NPU Core1 at the same time. NPU_CORE_0_1_2: the model runs on NPU Core0, Core1 and Core2 at the same time. NPU_CORE_ALL: the number of NPU cores is automatically configured according to the platform. The default value is NPU_CORE_AUTO. Note: When setting this parameter on RKNN-Toolkit Lite2, RKNNLite must be prefixed to the value, for example RKNNLite.NPU_CORE_AUTO; when setting this parameter on RKNN-Toolkit2, RKNN must be prefixed to the value, for example RKNN.NPU_CORE_AUTO.

To set the NPU core with RKNN-Toolkit2, the reference code is as follows:

```
## Python
......
# Init runtime environment
print('--> Init runtime environment')
ret = rknn.init_runtime(target='rk3588', core_mask=RKNN.NPU_CORE_0)
if ret != 0:
    print('Init runtime environment failed!')
    exit(ret)
```

To set the NPU core with RKNN-Toolkit Lite2, the reference code is as follows:

```
## Python
......
# Init runtime environment
print('--> Init runtime environment')
ret = rknn_lite.init_runtime(core_mask=RKNNLite.NPU_CORE_0)
if ret != 0:
    print('Init runtime environment failed')
    exit(ret)
print('done')
```

If you use C/C++ as the application development language, you can call the `rknn_set_core_mask()` interface to set the NPU core for the model to run on. The detailed description of the `core_mask` parameter of this interface is as follows:

Table 4-7 rknn_set_core_mask interface core_mask parameter description

Parameter	Description
core_mask	This parameter is used to set the NPU core for the model to run on. The available values and the corresponding descriptions are as follows: RKNN_NPU_CORE_AUTO: automatic scheduling mode; the model automatically runs on the currently idle NPU core in single-core mode. RKNN_NPU_CORE_0: the model runs on NPU Core0. RKNN_NPU_CORE_1: the model runs on NPU Core1. RKNN_NPU_CORE_2: the model runs on NPU Core2. RKNN_NPU_CORE_0_1: the model runs on NPU Core0 and NPU Core1 at the same time. RKNN_NPU_CORE_0_1_2: the model runs on NPU Core0, Core1 and Core2 at the same time. RKNN_NPU_CORE_ALL: the number of NPU cores is automatically configured according to the platform.

To set the NPU core for the model to run on with the C/C++ API, the reference code is as follows:

```
// C++
// rknn_context ctx;
rknn_core_mask core_mask = RKNN_NPU_CORE_0;
int ret = rknn_set_core_mask(ctx, core_mask);
```

4.1.2.2 CHECK MULTI-CORE RUNNING STATUS

This section describes in detail the effect when the RKNN model runs in multi-core mode.

If you use RKNN-Toolkit2 connected to a development board for model inference, you need to set the `perf_debug` parameter to `True` when calling the `rknn.init_runtime()` interface, and then call the `rknn.eval_perf()` interface to print the running information of each layer. The reference code is as follows:

```
#Python
# Init runtime environment
ret = rknn.init_runtime(target='rk3588', device_id='29d5dd97766a5c27',
perf_debug=True)
if ret != 0:
    print('Init runtime environment failed!')
    exit(ret)

# Eval performance
rknn.eval_perf()
```

If model inference is performed directly on the board, you need to set RKNN_LOG_LEVEL to 4 or above before running the application, and the running information of each layer of the model will be printed. The setting method is as follows:

```
# Python
# When using the Python interface provided by RKNN-Toolkit Lite2, simply set
verbose to True when creating the RKNNLite object
rknnlite = RKNNLite(verbose=True)

# When using the C/C++ interface, the following environment variable must be set
before running the binary program
export RKNN_LOG_LEVEL=4
```

Taking the lenet model as an example, after the above settings, the terminal will print a log similar to the following (for the convenience of display, fields such as InputShape, OutputShape, DDR Cycles, NPU Cycles, Total Cycles, Time(us), MacUsage(%), Task Number, Lut Number, RW(kb), FullName have been removed):

```
ID   OpType           DataType Target WorkLoad(0/1/2)-ImproveTherical
1    InputOperator    UINT8    CPU    100.0%/0.0%/0.0% - Up:0.0%
2    Conv               UINT8    NPU    50.0%/50.0%/0.0% - Up:50.0%
3    MaxPool            INT8    NPU    100.0%/0.0%/0.0% - Up:0.0%
4    Conv                INT8    NPU    50.0%/50.0%/0.0% - Up:50.0%
5    MaxPool            INT8    NPU    100.0%/0.0%/0.0% - Up:0.0%
6    ConvRelu           INT8    NPU    48.1%/51.9%/0.0% - Up:48.1%
7    Conv                INT8    NPU    100.0%/0.0%/0.0% - Up:0.0%
8    Softmax            INT8    CPU    0.0%/0.0%/0.0% - Up:0.0%
9    OutputOperator   FLOAT16 CPU    0.0%/0.0%/0.0% - Up:0.0%
Total Operator Elapsed Time(us): 591
Total Memory RW Amount(MB): 0
```

The "WorkLoad(0/1/2)-ImproveTherical" column in the running information of each layer of the model is printed only on multi-core NPUs. It records how the tasks of each layer of the model are allocated on the NPU cores and the theoretical performance improvement. For example, "50.0%/50.0%/0.0% - Up:50.0%" means that the computation of this layer is allocated with Core0 responsible for 50% and Core1 responsible for 50%; compared with single-core operation, the performance of this layer can theoretically be improved by 50%. If the performance of a certain layer has not improved, such as "100.0%/0.0%/0.0% - Up:0.0%", the following situations may exist:

The workload of this layer is too small, smaller than the granularity of NPU multi-core task allocation, so this layer runs on a single core;

This type of operator does not implement multi-core task segmentation in the NPU driver, and it will be supported in subsequent versions. The operators that currently support multi-core task segmentation include: **Conv, DepthwiseConvolution, Add, Concat, Relu, Clip, Relu6, ThresholdedRelu, PRelu, LeakyRelu**.

4.1.2.3 MULTI-CORE PERFORMANCE IMPROVEMENT TIPS

You can try the following methods to get higher multi-core running performance:

- Set CPU/DDR/NPU frequency to the highest
- Bind the application to the CPU big core
- Bind the NPU interrupt to the corresponding CPU big core above

The fixed frequency commands corresponding to different firmware are different, please refer to Chapter 5.2.1.1.

Take binding the application to the CPU4 core as an example. For the last two points mentioned above, you can refer to the following script:

```
interrupts=$(cat /proc/interrupts | grep npu)
interrupts_array=($interrupts)

irq1=$(echo ${interrupts_array[0]} | awk -F ':' '{print $1}')
irq2=$(echo ${interrupts_array[14]} | awk -F ':' '{print $1}')
irq3=$(echo ${interrupts_array[28]} | awk -F ':' '{print $1}')

for irq in $irq1 $irq2 $irq3; do
  echo 4 > /proc/irq/$irq/smp_affinity_list
done

taskset 10 ./rknn_benchmark lenet.rknn "" 10 3 # The taskset mask value
corresponding to CPU4 is 0x10
```

The above script will perform the following operations:

- Execute the cat /proc/interrupts | grep npu command and parse out three interrupt numbers (remove colons)
- Use a loop to set the smp_affinity_list of each interrupt number to 4 (the corresponding ID of CPU4 is 4)
- Finally execute the taskset 10 ./rknn_benchmark lenet.rknn "" 10 3 command. The taskset parameter corresponding to CPU4 is 10 (For specific usage of taskset, please refer to: https://man7.org/linux/man-pages/man1/taskset.1.html)

Through the above operation, the NPU interrupt and the application "rknn_benchmark" will run on CPU4, which can eliminate the core switching overhead of NPU interrupt processing.

4.1.3 Dynamic Shape

4.1.3.1 DYNAMIC SHAPE INTRODUCTION

Dynamic shape means that the shape of the model input data can change at runtime. It can help handle situations where the input data size is not fixed and increase the flexibility of the model. In the case of the RKNN model that only supported static shape before, if the user needs to use multiple input shapes, the traditional approach is to generate multiple RKNN models and initialize multiple contexts to perform inference separately when deploying the model. However, after the introduction of dynamic shape, can only keep a copy of the dynamic shape RKNN model that is similar in size to the static shape RKNN model, and use one context for inferenceing, thus saving Flash and DDR occupancy. Dynamic shape plays an important role in image processing and sequence model reasoning. Typical application scenarios include:

- Models with changing sequence lengths, common in NLP models, such as BERT, GPT
- Models that vary in spatial dimensions, such as segmentation and style transfer
- The model with batch, the batch dimension changes
- Object detection model with variable output number

4.1.3.2 RKNN SDK VERSION AND PLATFORM REQUIREMENTS

- RKNN-Toolkit2 version>=1.5.0
- RKNPU Runtime library (librknnrt.so) version >=1.5.0
- The NPU of the RK3562/RK3566/RK3568/RK3576/RK3588/RK3588S/RV1126B platforms supports this function

4.1.3.3 GENERATE RKNN MODEL OF DYNAMIC SHAPE

This section introduces the steps to generate an RKNN model with dynamic shape using the Python interface of RKNN-Toolkit2:

1. Confirm that the model supports dynamic shape

If the model file itself is not of dynamic shape, RKNN-Toolkit2 supports expanding it into an RKNN model with dynamic shape. First, the user must confirm that the model itself does not contain operators or subgraph structures that limit dynamic shape. For example, the shape of a constant cannot be changed, and the RKNN-Toolkit2 tool will report an error during the conversion process. If dynamic shape expansion is not supported, the user should modify the model structure according to the error message and retrain the model to support dynamic shape. It is recommended to use a model that is originally of dynamic shape.

1. Set the input shapes to be used

Due to NPU hardware characteristics, a dynamic shape RKNN model does not support arbitrary changes of input shapes, and users are required to set a limited number of input shapes. For models with multiple inputs, the number of shapes for each input must be the same. For example, when using RKNN-Toolkit2 to convert a Caffe model, the Python code example is as follows:

```
# Python
dynamic_input = [
    [[1,3,224,224]],    # set the first shape for all inputs
    [[1,3,192,192]],    # set the second shape for all inputs
    [[1,3,160,160]],    # set the third shape for all inputs
]

# Pre-process config
rknn.config(mean_values=[103.94, 116.78, 123.68], std_values=[58.82, 58.82, 58.82],
quant_img_RGB2BGR=True, dynamic_input=dynamic_shapes)
```

The above interface configuration will generate a dynamic shape RKNN model that supports three shapes: [1,3,224,224], [1,3,192,192] and [1,3,160,160].

The shape in `dynamic_input` is consistent with the layout of the original model framework. For example, for classifying RGB images of the same 224x224 size, the TensorFlow/TFLite model input is [1,224,224,3], while the ONNX model input is [1,3,224,224].

1. Quantization

After setting the input shapes, if quantization is needed, the quantization calibration dataset must be set. The tool will read the maximum resolution input set by the user for quantization (the group of shapes with the largest sum of all input sizes). For example, a model has two inputs, one input has shapes [1,224] and [1,112], and the other input has shapes [1,40] and [1,80]. The sum of all input sizes for the first group of shapes is 1*224+1*40=264, and the sum of all input sizes for the second group of shapes is 1*112+1*80=192. The sum of all input sizes for the first group of shapes is larger, so the two inputs are quantized with the shapes [1,224] and [1,40] respectively.

- If the quantization calibration dataset is in jpg/png image format, the user can use images of different resolutions for quantization, because the tool will resize the images to the maximum resolution using the resize method of opencv before quantization.
- If the quantization calibration dataset is in npy format, the user must use the shape of the maximum resolution input. After quantization, all shapes in the model use the same set of quantization parameters for inference at runtime.

In addition, the maximum resolution input shape will also be printed out when calling rknn.config, as follows:

```
W config: The 'dynamic_input' function has been enabled, the MaxShape is
dynamic_input[0] = [[1,224],[1,40]]!
The following functions are subject to the MaxShape:
1. The quantified dataset needs to be configured according to MaxShape
2. The eval_perf or eval_memory return the results of MaxShape
```

4. Inference evaluation or accuracy analysis

When performing inference or accuracy analysis with a dynamic shape RKNN model, the user must provide inputs with one of the groups of shapes set in step 2. The interface usage is consistent with the static shape RKNN model scenario, and will not be described in detail here.

For a complete example of creating a dynamic shape RKNN model, please refer to (https://github.com/airockchip/rknn-toolkit2/tree/master/rknn-toolkit2/examples/functions/dynamic_shape)

4.1.3.4 C API DEPLOYMENT

After obtaining the dynamic shape RKNN model, use the RKNPU2 C API to deploy it. According to the interface form, it is divided into general API and zero-copy API deployment processes.

4.1.3.4.1 General API

The process of deploying a dynamic shape RKNN model using the general API is shown in the figure below:

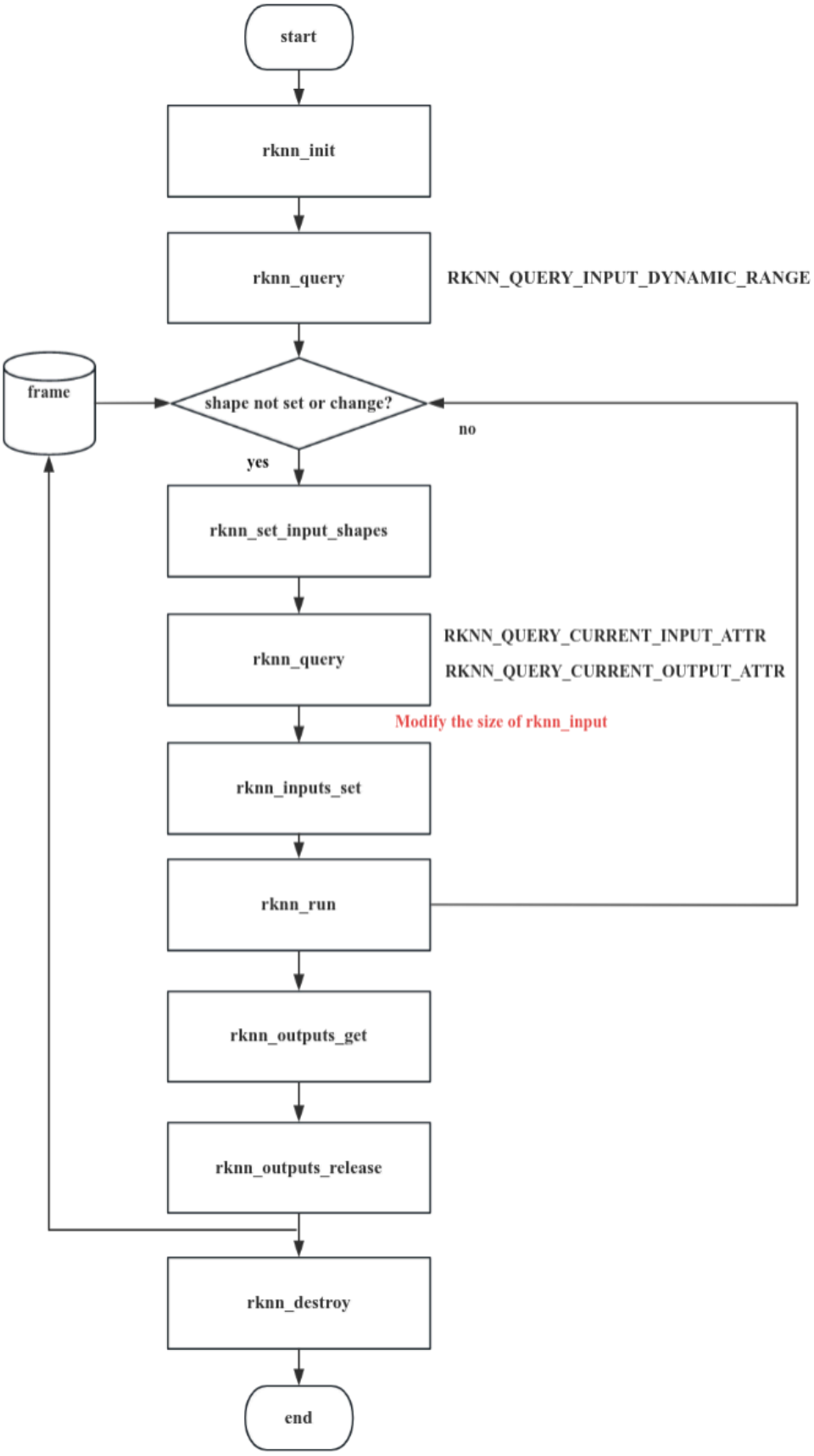


Figure 4-5 General API calling process of dynamic shape input interface

After loading a dynamic shape RKNN model, the input shape can be dynamically modified at runtime. First, the list of input shapes supported by the RKNN model can be queried through `rknn_query()`. The shape list information supported by each input is returned in the form of the

`rknn_input_range` structure, which contains the name, data layout information, number of shapes, and specific shapes of each input. Then, by calling the `rknn_set_input_shapes()` interface and passing in the `rknn_tensor_attr` array pointer containing the shape information of each input, the shape used for the current inference can be set. After setting the input shape, `rknn_query()` can be called again to query the input and output shapes after the current setting is successful.

Finally, complete the inference according to the general API process. Each time the input shape is switched, a new shape needs to be set again, data of the new shape size prepared, and the `rknn_inputs_set()` interface called again. If there is no need to switch the input shape before inference, there is no need to call the `rknn_set_input_shapes()` interface repeatedly.

1. Initialization

Call the `rknn_init()` interface to initialize the dynamic shape RKNN model.

For dynamic shape RKNN models, there are the following restrictions when initializing the context:

- The weight sharing function is not supported (initialization with the `RKNN_FLAG_SHARE_WEIGHT_MEM` flag).
- The context reuse function is not supported (see the `rknn_dup_context` interface for details).

2. Query the input shape combinations supported by the RKNN model

After successful initialization, the list of input shapes supported by the RKNN model can be queried through `rknn_query()`. The shape list information supported by each input is returned in the form of the `rknn_input_range` structure, which contains the name, layout information, number of supported shapes, and specific shapes of each input. The C code example is as follows:

```
// Query the input shapes supported by the model
rknn_input_range dyn_range[io_num.n_input];
memset(dyn_range, 0, io_num.n_input * sizeof(rknn_input_range));
for (uint32_t i = 0; i < io_num.n_input; i++)
{
    dyn_range[i].index = i;
    ret = rknn_query(ctx, RKNN_QUERY_INPUT_DYNAMIC_RANGE, &dyn_range[i],
sizeof(rknn_input_range));
    if (ret != RKNN_SUCC)
    {
        fprintf(stderr, "rknn_query error! ret=%d\n", ret);
        return -1;
    }
    dump_input_dynamic_range(&dyn_range[i]);
}
```

Note: For multi-input models, the shapes of all inputs correspond one-to-one in order. For example, for an RKNN model with two inputs and multiple shapes, the first shape of the first input combined with the first shape of the second input is valid, and there are no intersecting shape combinations. For example, a model has two inputs A and B, where the shapes of A are [1,224] and [1,112], and the shapes of B are [1,40] and [1,80]. In this case, only the following two groups of input shapes are supported:

- A shape = [1,224], B shape=[1,40]
- A shape = [1,112], B shape=[1,80]

3. Set input shape

When input data is set for the first time or the shape of the input data changes, the `rknn_set_input_shapes()` interface needs to be called to dynamically modify the input shape. After loading a dynamic shape RKNN model, the input shape can be dynamically modified at runtime. By calling the `rknn_set_input_shapes()` interface and passing in the `rknn_tensor_attr` array of all inputs, the three members `dims`, `n_dims` and `fmt` in each `rknn_tensor_attr` represent the shape used for the current inference. The C code example is as follows:

```
/**
dynamic inputs shape range:
index=0, name=data, shape_number=2, range=[[1, 224, 224, 3],[1, 112, 224, 3]], fmt
= NHWC
**/
input_attrs[0].dims[0] = 1;
input_attrs[0].dims[1] = 224;
input_attrs[0].dims[2] = 224;
input_attrs[0].dims[3] = 3;
input_attrs[0].fmt=RKNN_TENSOR_NHWC;
ret = rknn_set_input_shapes(ctx, io_num.n_input, input_attrs);
if (ret < 0)
{
    fprintf(stderr, "rknn_set_input_shapes error! ret=%d\n", ret);
    return -1;
}
```

Among them, `io_num.n_input` is the number of inputs, and `input_attrs` is the `rknn_tensor_attr` structure array of the model inputs.

Note: The shape set here must be included in the shape list queried in step 2.

After setting the input shape, `rknn_query` can be called again to query the input and output shapes after the current setting is successful. The C code example is as follows:

```
// Get the input and output shapes of the current inference
rknn_tensor_attr cur_input_attrs[io_num.n_input];
memset(cur_input_attrs, 0, io_num.n_input * sizeof(rknn_tensor_attr));
for (uint32_t i = 0; i < io_num.n_input; i++)
{
    cur_input_attrs[i].index = i;
    ret = rknn_query(ctx, RKNN_QUERY_CURRENT_INPUT_ATTR, &(cur_input_attrs[i]),
sizeof(rknn_tensor_attr));
    if (ret < 0)
    {
        printf("rknn_init error! ret=%d\n", ret);
        return -1;
    }
    dump_tensor_attr(&cur_input_attrs[i]);
}
rknn_tensor_attr cur_output_attrs[io_num.n_output];
memset(cur_output_attrs, 0, io_num.n_output * sizeof(rknn_tensor_attr));
for (uint32_t i = 0; i < io_num.n_output; i++)
{
    cur_output_attrs[i].index = i;
    ret = rknn_query(ctx, RKNN_QUERY_CURRENT_OUTPUT_ATTR, &(cur_output_attrs[i]),
sizeof(rknn_tensor_attr));
    if (ret != RKNN_SUCC)
    {
        printf("rknn_query fail! ret=%d\n", ret);
        return -1;
    }
    dump_tensor_attr(&cur_output_attrs[i]);
}
```

Notes:

- The `rknn_set_input_shape` interface requires that fmt use `NHWC` when the input tensor's shape is 4-dimensional, and `UNDEFINED` when it is not 4-dimensional.
- Before `rknn_set_input_shapes` is called, the shape information queried using commands with the `RKNN_QUERY_CURRENT` prefix is invalid.

4. Inference

After setting the current input shape, assume that the shape information of the input Tensor is stored in the `cur_input_attrs` array. Taking the general API interface as an example, the C code example is as follows:

```
// Set input information
rknn_input inputs[io_num.n_input];
memset(inputs, 0, io_num.n_input * sizeof(rknn_input));
for (int i = 0; i < io_num.n_input; i++)
{
    int height = cur_input_attrs[i].fmt == RKNN_TENSOR_NHWC ?
cur_input_attrs[i].dims[1] : cur_input_attrs[i].dims[2];
    int width = cur_input_attrs[i].fmt == RKNN_TENSOR_NHWC ?
cur_input_attrs[i].dims[2] : cur_input_attrs[i].dims[3];
    cv::resize(imgs[i], imgs[i], cv::Size(width, height));
    inputs[i].index = i;
    inputs[i].pass_through = 0;
    inputs[i].type = RKNN_TENSOR_UINT8;
    inputs[i].fmt = RKNN_TENSOR_NHWC;
    inputs[i].buf = imgs[i].data;
    inputs[i].size = imgs[i].total() * imgs[i].channels();
}

// Convert the input data into the correct format and put it into the input buffer
ret = rknn_inputs_set(ctx, io_num.n_input, inputs);
if (ret < 0)
{
    printf("rknn_input_set fail! ret=%d\n", ret);
    return -1;
}

// Run inference
printf("Begin perf ...\n");
double total_time = 0;
for (int i = 0; i < loop_count; ++i)
{
    int64_t start_us = getCurrentTimeUs();
    ret = rknn_run(ctx, NULL);
    int64_t elapse_us = getCurrentTimeUs() - start_us;
    if (ret < 0)
    {
        printf("rknn run error %d\n", ret);
        return -1;
    }
    total_time += elapse_us / 1000.f;
    printf("%4d: Elapse Time = %.2fms, FPS = %.2f\n", i, elapse_us / 1000.f, 1000.f
* 1000.f / elapse_us);
}
printf("Avg FPS = %.3f\n", loop_count * 1000.f / total_time);

// Get output results
rknn_output outputs[io_num.n_output];
memset(outputs, 0, io_num.n_output * sizeof(rknn_output));
for (uint32_t i = 0; i < io_num.n_output; ++i)
{
    outputs[i].want_float = 1;
    outputs[i].index = i;
    outputs[i].is_prealloc = 0;
}

ret = rknn_outputs_get(ctx, io_num.n_output, outputs, NULL);
if (ret < 0)
{
    printf("rknn_outputs_get fail! ret=%d\n", ret);
    return ret;
```

```
}

// Release the output buffer
ret = rknn_outputs_release(ctx, io_num.n_output, outputs);
```

4.1.3.4.2 Zero-Copy API

For the zero-copy API, after successful initialization, the list of input shapes supported by the RKNN model can be queried through `rknn_query()`, and the input and output memory are allocated by calling the `rknn_create_mem()` interface. Next, by calling the `rknn_set_input_shapes()` interface and passing in the `rknn_tensor_attr` array pointer containing the shape information of each input, the shape used for the current inference can be set. After setting the input shape, `rknn_query()` can be called again to query the input and output shapes after the successful setting. Finally, call the `rknn_set_io_mem()` interface to set the required input and output memory. Each time the input shape is switched, a new shape needs to be set again, data of the new shape size prepared, and the `rknn_set_io_mem()` interface called again. If there is no need to switch the input shape before inference, there is no need to call the `rknn_set_input_shapes()` interface repeatedly. The typical usage process is shown in the figure below:

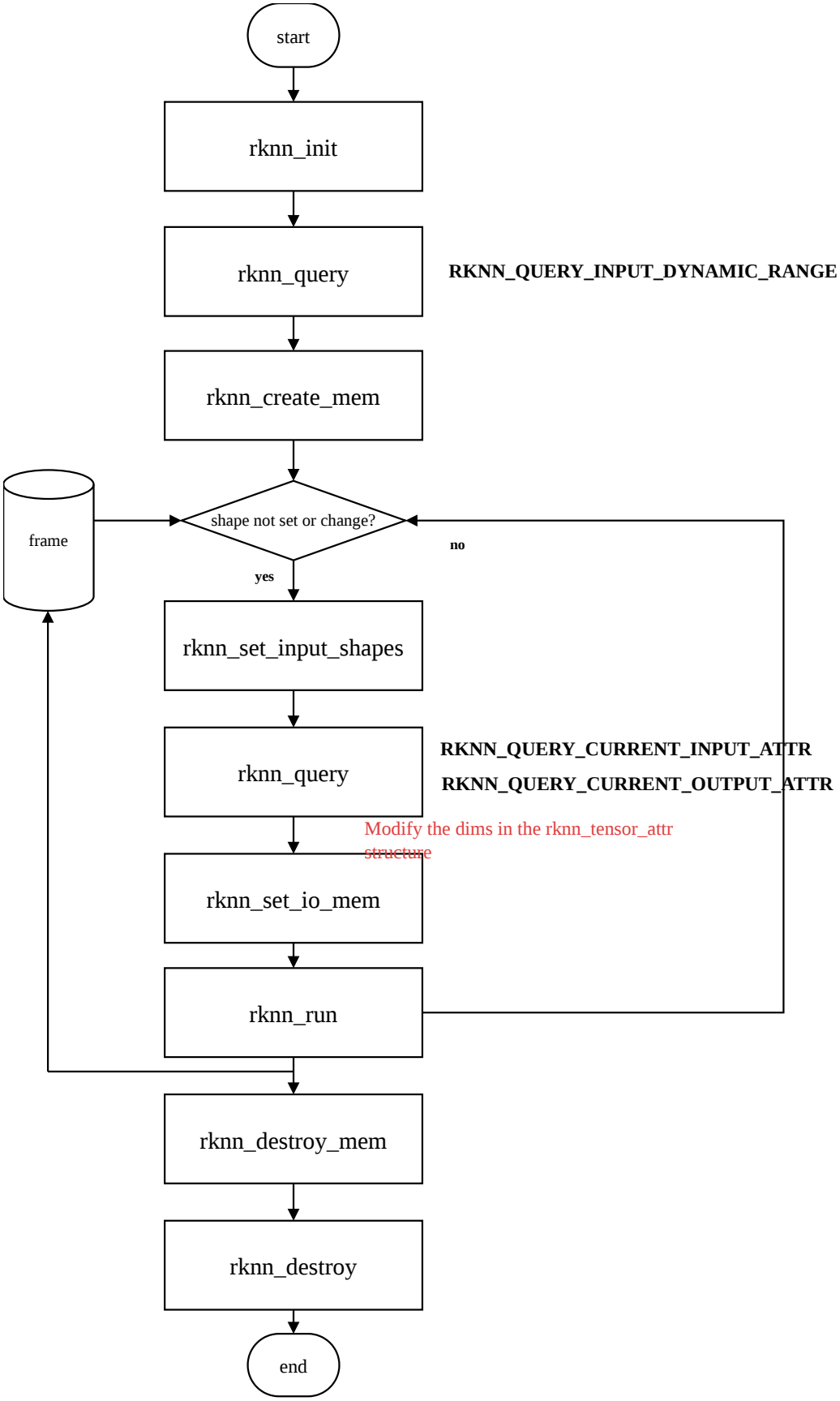


Figure 4-6 Zero-copy API calling process of dynamic shape input interface

Initialization, querying the input shape combinations supported by the RKNN model, and setting the input shape are the same as the above general API, and will not be described in detail here.

The difference is that the interfaces used after setting the input shape are different. The zero-copy inference C code example is as follows:

```
// Create the largest input tensor memory
rknn_tensor_mem *input_mems[io_num.n_input];
for (int i = 0; i < io_num.n_input; i++) {
    // default input type is int8 (normalize and quantize need compute in outside)
    // if set uint8, will fuse normalize and quantize to npu
    input_attrs[i].type = RKNN_TENSOR_UINT8;
    // default fmt is NHWC, npu only support NHWC in zero copy mode
    input_attrs[i].fmt = RKNN_TENSOR_NHWC;
    input_mems[i] = rknn_create_mem(ctx, input_attrs[i].size_with_stride);
...
// Create the largest output tensor memory
rknn_tensor_mem *output_mems[io_num.n_output];
for (uint32_t i = 0; i < io_num.n_output; ++i) {
    // default output type is depend on model, this require float32 to compute top5
    // allocate float32 output tensor
    int output_size = output_attrs[i].size * sizeof(float);
    output_mems[i] = rknn_create_mem(ctx, output_size);
...
// Load the input and set the model input shape, which is called once each time the
input shape is switched
for (int s = 0; s < shape_num; ++s) {
    for (int i = 0; i < io_num.n_input; i++) {
        for (int j = 0; j < input_attrs[i].n_dims; ++j) {
            input_attrs[i].dims[j] = shape_range[i].dyn_range[s][j];
    .....
    ret = rknn_set_input_shapes(ctx, io_num.n_input, input_attrs);
    if (ret < 0) {
        fprintf(stderr, "rknn_set_input_shape error! ret=%d\n", ret);
        return -1;
    ....
    // Get the input and output shapes of the current inference
    printf("current input tensors:\n");
    rknn_tensor_attr cur_input_attrs[io_num.n_input];
    memset(cur_input_attrs, 0, io_num.n_input * sizeof(rknn_tensor_attr));
    for (uint32_t i = 0; i < io_num.n_input; i++) {
        cur_input_attrs[i].index = i;
        // query info
        ret = rknn_query(ctx, RKNN_QUERY_CURRENT_INPUT_ATTR, &(cur_input_attrs[i]),
sizeof(rknn_tensor_attr));
        if (ret < 0) {
            printf("rknn_init error! ret=%d\n", ret);
            return -1;
        dump_tensor_attr(&cur_input_attrs[i]);
        ...
        printf("current output tensors:\n");
    rknn_tensor_attr cur_output_attrs[io_num.n_output];
    memset(cur_output_attrs, 0, io_num.n_output * sizeof(rknn_tensor_attr));
    for (uint32_t i = 0; i < io_num.n_output; i++) {
        cur_output_attrs[i].index = i;
        // query info
        ret = rknn_query(ctx, RKNN_QUERY_CURRENT_OUTPUT_ATTR, &
(cur_output_attrs[i]), sizeof(rknn_tensor_attr));
        if (ret != RKNN_SUCC) {
            printf("rknn_query fail! ret=%d\n", ret);
            return -1;
        ...
        dump_tensor_attr(&cur_output_attrs[i]);
    // Specify the number of NPU cores, only 3588/3576 is supported
    rknn_set_core_mask(ctx, (rknn_core_mask)core_mask);
    // Set input information
```

```
    rknn_input inputs[io_num.n_input];
    memset(inputs, 0, io_num.n_input * sizeof(rknn_input));
    std::vector<cv::Mat> resize_imgs;
    resize_imgs.resize(io_num.n_input);
    for (int i = 0; i < io_num.n_input; i++) {
        int height = cur_input_attrs[i].fmt == RKNN_TENSOR_NHWC ?
cur_input_attrs[i].dims[1] : cur_input_attrs[i].dims[2];
        int width = cur_input_attrs[i].fmt == RKNN_TENSOR_NHWC ?
cur_input_attrs[i].dims[2] : cur_input_attrs[i].dims[3];
        int stride = cur_input_attrs[i].w_stride;
        cv::resize(imgs[i], resize_imgs[i], cv::Size(width, height));
        int input_size = resize_imgs[i].total() * resize_imgs[i].channels();
        // Copy external data to the zero-copy input buffer
        if (width == stride)
            memcpy(input_mems[i]->virt_addr, resize_imgs[i].data, input_size);
        else {
            int height = cur_input_attrs[i].dims[1];
            int channel = cur_input_attrs[i].dims[3];
            // copy from src to dst with stride
            uint8_t *src_ptr = resize_imgs[i].data;
            uint8_t *dst_ptr = (uint8_t *)input_mems[i]->virt_addr;
            // width-channel elements
            int src_wc_elems = width * channel;
            int dst_wc_elems = stride * channel;
            for (int b = 0; b < cur_input_attrs[i].dims[0]; b++) {
                for (int h = 0; h < height; ++h) {
                    memcpy(dst_ptr, src_ptr, src_wc_elems);
                    src_ptr += src_wc_elems;
                    dst_ptr += dst_wc_elems;
            ....
    // Update the input zero-copy buffer memory
    for (int i = 0; i < io_num.n_input; i++) {
        cur_input_attrs[i].type = RKNN_TENSOR_UINT8;
        ret = rknn_set_io_mem(ctx, input_mems[i], &cur_input_attrs[i]);
        if (ret < 0) {
            printf("rknn_set_io_mem fail! ret=%d\n", ret);
            return -1;
     ....
    // Update the output zero-copy buffer memory
    for (uint32_t i = 0; i < io_num.n_output; ++i) {
        // default output type is depend on model, this require float32 to compute
top5
        cur_output_attrs[i].type = RKNN_TENSOR_FLOAT32;
        cur_output_attrs[i].fmt = RKNN_TENSOR_NCHW;
        // set output memory and attribute
        ret = rknn_set_io_mem(ctx, output_mems[i], &cur_output_attrs[i]);
        if (ret < 0) {
            printf("rknn_set_io_mem fail! ret=%d\n", ret);
            return -1;
      ....
    // Inference
    printf("Begin perf ...\n");
    double total_time = 0;
    for (int i = 0; i < loop_count; ++i) {
        int64_t start_us = getCurrentTimeUs();
        ret = rknn_run(ctx, NULL);
        int64_t elapse_us = getCurrentTimeUs() - start_us;
        if (ret < 0) {
            printf("rknn run error %d\n", ret);
            return -1;
        ...
```

```
        total_time += elapse_us / 1000.f;
        printf("%4d: Elapse Time = %.2fms, FPS = %.2f\n", i, elapse_us / 1000.f,
1000.f * 1000.f / elapse_us);
    printf("Avg FPS = %.3f\n", loop_count * 1000.f / total_time);
```

Notes:

1. For the `rknn_set_io_mem()` interface in the dynamic shape case, the description of the input buffer's shape and size:
2. After initialization is completed and before calling the `rknn_set_input_shapes()` interface, the shapes of the input and output Tensors queried by the `rknn_query()` interface using `RKNN_QUERY_INPUT_ATTR` and `RKNN_QUERY_OUTPUT_ATTR` are usually the largest. Users can use the sizes obtained from these two commands to allocate the input and output memory. For multi-input models, the shapes of some inputs may not be the largest; in this case, you need to search for the largest specification among the supported shapes and allocate the largest input and output memory.
3. If the input is non-4-dimensional, use `fmt=UNDEFINED` and pass in the buffer of the original model input shape, whose size is calculated from the input shape and type.
4. If the input is 4-dimensional, using `fmt=NHWC` or `NC1HWC2` is supported, passing in the buffer of the `NHWC` or `NC1HWC2` shape and the corresponding size (obtain the shape and size by querying the corresponding fields via `rknn_query`).
5. In the `rknn_query()` interface, when the flags are `RKNN_QUERY_CURRENT_INPUT_ATTR` and `RKNN_QUERY_CURRENT_OUTPUT_ATTR`, the shape of the original model input/output is obtained, whose format is `NHWC` or `UNDEFINED`; when the flags are `RKNN_QUERY_CURRENT_NATIVE_INPUT_ATTR` and `RKNN_QUERY_CURRENT_NATIVE_OUTPUT_ATTR`, the shape of the model input/output when the NPU reads data with optimal performance is obtained, whose format is `NHWC` or `NC1HWC2`.
6. When the buffer arrangement format used in the `rknn_set_io_mem()` interface is `NHWC`, the shape and fmt in `rknn_tensor_attr` must be set according to the information queried by `RKNN_QUERY_CURRENT_INPUT_ATTR`; if the buffer arrangement format used is `NC1HWC2`, they must be set according to the information queried by `RKNN_QUERY_CURRENT_NATIVE_INPUT_ATTR`.

For the complete dynamic shape C API Demo, please refer to (https://github.com/airockchip/rknn-toolkit2/tree/master/rknpu2/examples/rknn_dynamic_shape_input_demo)

4.1.4 Multi-Batch Usage

4.1.4.1 MULTI-BATCH PRINCIPLE

There are 3 cores inside the RK3588 NPU and 2 cores inside the RK3576 NPU. In order to utilize the multi-core performance more efficiently, a multi-batch inference function is provided. When multi-batch inference is enabled, `rknn_dup_context` is called internally to copy the `context` (`rknn_dup_context` only copies the Internal of the `context`, and the Weight is reused). When `rknn_batch_size=2`, one copy is made; when `rknn_batch_size >=3`, two copies are made (at most 3 cores work at the same time, and only two copies are made to avoid memory waste). **Each context core_mask will be set to 0 to allow automatic scheduling inside the**

multiple cores. When `rknn_run()` is executed, a thread pool is created internally, and 3 threads are called at the same time to perform inference on the 3 `contexts`.

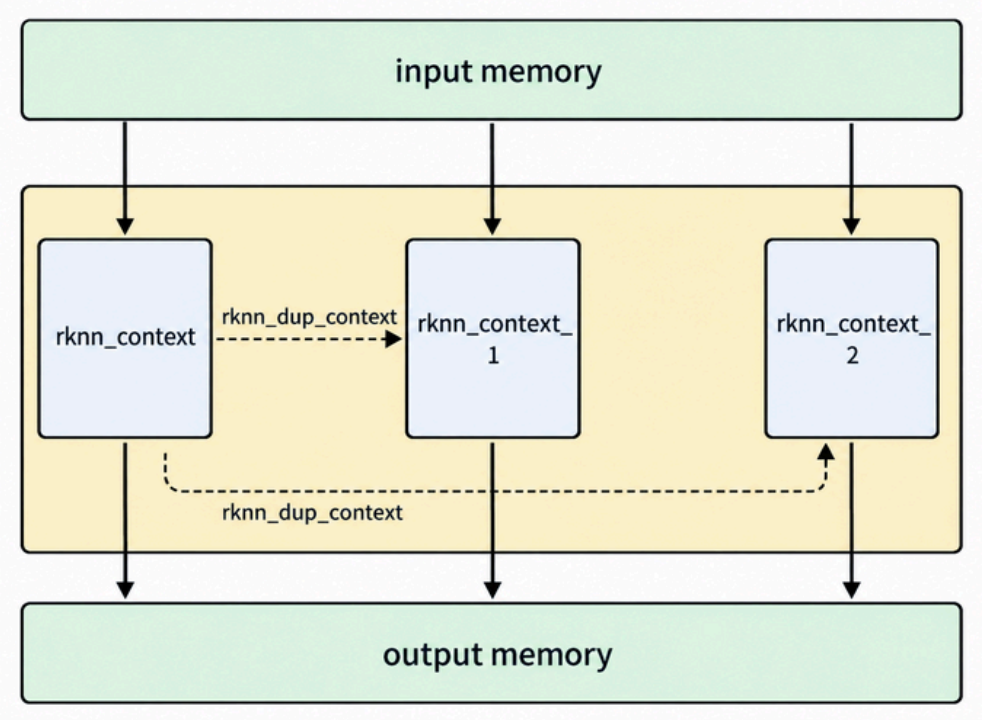


Figure 4-7 Multi-batch internal principle diagram

4.1.4.2 MULTI-BATCH USAGE

The usage of multi-batch is as follows:

1. Enable multi-batch settings on Python:

   ```
   ret = rknn.build(do_quantization=True, dataset='./dataset.txt',
   rknn_batch_size=3)
   ```

   In order to achieve optimal performance, it is recommended that rknn_batch_size be a multiple of 3 for RK3588, and a multiple of 2 for RK3576.

2. It is recommended to use zero-copy interface

4.1.4.3 MULTI-BATCH INPUT AND OUTPUT SETTINGS

When the multi-batch function is enabled, the queried input and output sizes are `rknn_batch_size` times those when it is not enabled. Each internal `context` calculates its own input offset, takes the input data for inference according to this input offset, then calculates its own output offset, and writes to its own output according to this output offset. Taking the second batch as an example, the input offset is the queried `input_size` divided by `rknn_batch_size`, and the output offset is the queried `output_size` divided by `rknn_batch_size`.

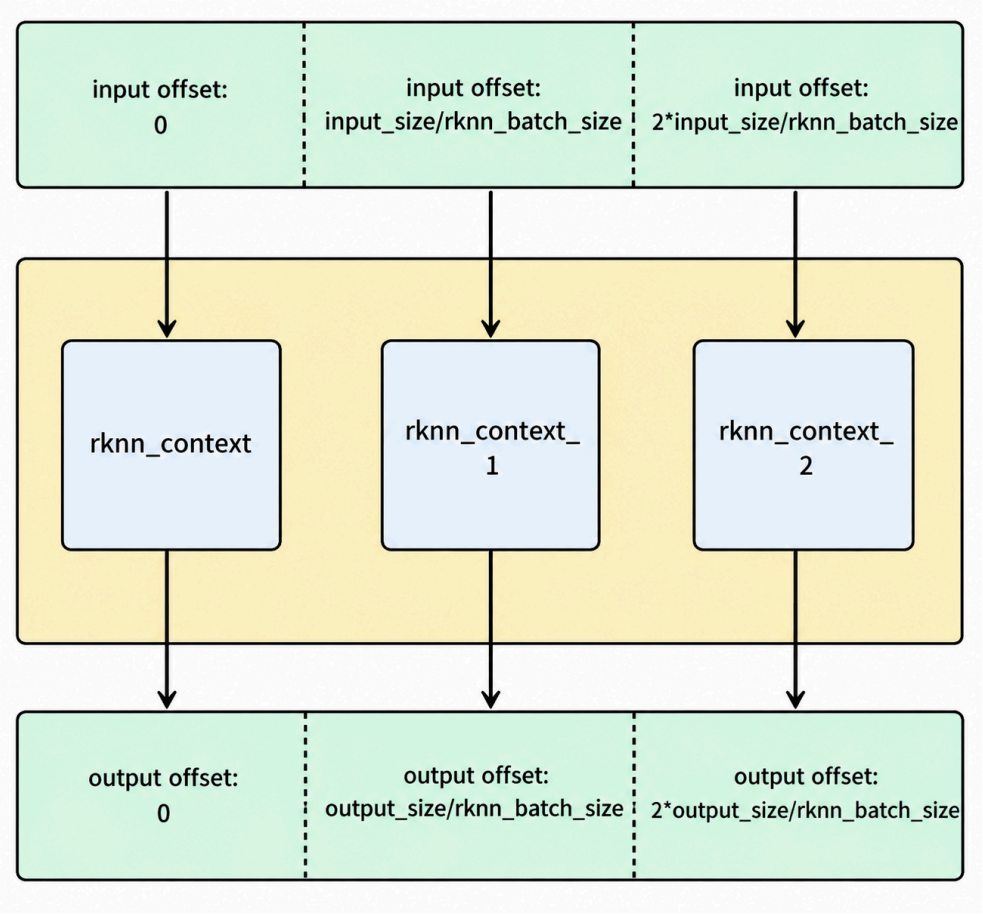


Figure 4-8 Multi-batch internal input and output address offset diagram

4.1.5 RK3588/RK3576 NPU SRAM

- RK3588 SOC contains 1MB of SRAM internally, of which 956KB can be used by various IPs on the SOC. It supports designated allocation for RKNPU.
- RK3576 SOC contains 512KB of SRAM internally, of which 480KB can be used by various IPs on the SOC. It supports designated allocation for RKNPU.
- SRAM can help RKNPU applications reduce DDR bandwidth pressure, but it may have a certain impact on inference time.

4.1.5.1 RKNPU ENVIRONMENTAL REQUIREMENTS

4.1.5.1.1 Kernel Environment Requirements

- RKNPU driver version>=0.8.0
- Kernel config needs to enable CONFIG_ROCKCHIP_RKNPU_SRAM=y
  - The Android system config path is as follows:

    ```
    <path-to-your-kernel>/arch/arm64/configs/rockchip_defconfig
    ```

  - The Linux system config path is as follows:

```
<path-to-your-kernel>/arch/arm64/configs/rockchip_linux_defconfig
```

- The corresponding DTS of the kernel needs to be allocated to RKNPU from the system SRAM.
  - Allocate the required size of SRAM from the system to RKNPU. The maximum size that can be allocated to RK3588 is 956KB, and the maximum size that can be allocated to RK3576 is 480KB, and the size needs to be 4K aligned.
  - Note: SRAM may have been allocated to other IPs in the default system, such as the codec module. The SRAM areas allocated to each IP cannot overlap, otherwise there will be data confusion caused by simultaneous reading and writing.
  - The following is an example of allocating 956KB to RK3588 and 480KB to RK3576 for RKNPU

RK3588:

```
syssram: sram@ff001000 {
    compatible = "mmio-sram";
    reg = <0x0 0xff001000 0x0 0xef000>;

    #address-cells = <1>;
    #size-cells = <1>;
    ranges = <0x0 0x0 0xff001000 0xef000>;
    /* Allocate RKNPU SRAM /
    / start address and size should be 4k algin */
    rknpu_sram: rknpu_sram@0 {
        reg = <0x0 0xef000>; // 956KB
    };
};
```

- Mount the allocated SRAM to the RKNPU node and modify the dtsi file as shown below:

```
<path-to-your-kernel>/arch/arm64/boot/dts/rockchip/rk3588s.dtsi
```

```
rknpu: npu@fdab0000 {
    compatible = "rockchip,rk3588-rknpu";
    /* ... /
    / Add reference RKNPU sram */
    rockchip,sram = <&rknpu_sram>;
    status = "disabled";
};
```

RK3576:

```
sram: sram@3ff88000 {
    compatible = "mmio-sram";
    reg = <0x0 0x3ff88000 0x0 0x78000>;

    #address-cells = <1>;
    #size-cells = <1>;
    ranges = <0x0 0x0 0x3ff88000 0x78000>;
    /* Allocate RKNPU SRAM /
    / start address and size should be 4k algin */
    rknpu_sram: rknpu-sram@0 {
            reg = <0x0 0x78000>; // 480KB
    };
};
```

- Mount the allocated SRAM to the RKNPU node and modify the dtsi file as shown below:

  ```
  <path-to-your-kernel>/arch/arm64/boot/dts/rockchip/rk3576.dtsi
  ```

```
rknpu: npu@27700000 {
    compatible = "rockchip,rk3576-rknpu";
    /* ... /
    / Add reference RKNPU sram */
    rockchip,sram = <&rknpu_sram>;
    status = "disabled";
};
```

4.1.5.1.2 RKNN SDK Version Requirments

- RKNPU Runtime library (librknnrt.so) version >=1.6.0

4.1.5.2 USAGE

Specify RKNN_FLAG_ENABLE_SRAM in the flags parameter of the rknn_init() interface to enable SRAM in this context.

For example:

```
ret = rknn_init(&ctx, rknn_model, size, RKNN_FLAG_ENABLE_SRAM, NULL);
```

When RKNN_FLAG_ENABLE_SRAM is set, as much memory as possible will be allocated from the system's available SRAM as the model's Internal Tensor memory.

Note:

- When SRAM is occupied by a certain rknn context, other rknn contexts do not support reusing the SRAM of this segment. The RKNN_FLAG_SHARE_SRAM function in rknn_api.h has not yet been implemented.
- When a certain rknn context does not occupy all the SRAM, the remaining SRAM can be used by other rknn contexts.

4.1.5.3 DEBUG

4.1.5.3.1 Query SRAM Enable

Check whether SRAM is enabled through the boot kernel log, which contains the address range and size information of the SRAM specified for the RKNPU, as shown below:

```
rk3588_s:/ # dmesg | grep rknpu -i
RKNPU fdab0000.npu: RKNPU: sram region: [0x00000000ff001000, 0x00000000ff0f0000),
sram size: 0xef000
```

4.1.5.3.2 Query SRAM Usage

- SRAM usage can be queried through nodes
- The following is a bit chart of unused SRAM, each point represents 4K size

```
rk3588_s:/ # cat /sys/kernel/debug/rknpu/mm
SRAM bitmap: "*" - used, "." - free (1bit = 4KB)
[000] [................................]
[001] [................................]
[002] [................................]
[003] [................................]
[004] [................................]
[005] [................................]
[006] [................................]
[007] [................]
SRAM total size: 978944, used: 0, free: 978944
# Unit is Byte
```

- The following is a bit chart after using 512KB SRAM

```
rk3588_s:/ # cat /sys/kernel/debug/rknpu/mm
SRAM bitmap: "*" - used, "." - free (1bit = 4KB)
[000] [********************************]
[001] [********************************]
[002] [********************************]
[003] [********************************]
[004] [................................]
[005] [................................]
[006] [................................]
[007] [................]
SRAM total size: 978944, used: 524288, free: 454656
# Unit is Byte
```

4.1.5.3.3 Query SRAM Size Through RKNN API

- Query SRAM size information through the RKNN_QUERY_MEM_SIZE interface of rknn_query()

```
typedef struct _rknn_mem_size {
    uint32_t total_weight_size;
    uint32_t total_internal_size;
    uint64_t total_dma_allocated_size;
    uint32_t total_sram_size;
    uint32_t free_sram_size;
    uint32_t reserved[10];
} rknn_mem_size;
```

- Among total_sram_size indicates the total size of SRAM allocated by the system to RKNPU, unit is Byte.
- free_sram_size indicates the size of SRAM that can be used by the remaining RKNPU, unit is Byte

4.1.5.3.4 Query Model SRAM Occupancy

- In the RKNPU device environment, set the following environment variables before running the RKNN application to print the SRAM usage prediction:

```
export RKNN_LOG_LEVEL=3
```

- Layer occupancy of Internal allocated SRAM, as shown in the following log:

```
--------------------------------------------------------------------------------
Total allocated Internal SRAM Size: 524288, Addr: [0xff3e0000, 0xff460000)
--------------------------------------------------------------------------------
--------------------------------------------------------------------------+---------
-------------------------+-----------
ID  User           Tensor   DataType OrigShape      NativeShape      |
[Start       End)       Size |    SramHit
--------------------------------------------------------------------------+---------
-------------------------+-----------
1   ConvRelu       input0   INT8     (1,3,224,224)  (1,1,224,224,3)  |
0xff3b0000 0xff3d4c00 0x00024c00 | \
2   ConvRelu       output2  INT8     (1,32,112,112) (1,2,112,112,16) |
0xff404c00 0xff466c00 0x00062000 | 0x0005b400
3   ConvRelu       output4  INT8     (1,32,112,112) (1,4,112,112,16) |
0xff466c00 0xff52ac00 0x000c4000 | 0x00000000
4   ConvRelu       output6  INT8     (1,64,112,112) (1,4,112,112,16) |
0xff52ac00*0xff5eec00 0x000c4000 | 0x00000000
5   ConvRelu       output8  INT8     (1,64,56,56)   (1,4,56,56,16)   |
0xff3e0000 0xff411000 0x00031000 | 0x00031000
6   ConvRelu       output10 INT8     (1,128,56,56)  (1,8,56,56,16)   |
0xff411000 0xff473000 0x00062000 | 0x0004f000
7   ConvRelu       output12 INT8     (1,128,56,56)  (1,8,56,56,16)   |
0xff473000 0xff4d5000 0x00062000 | 0x00000000
8   ConvRelu       output14 INT8     (1,128,56,56)  (1,8,56,56,16)   |
0xff3e0000 0xff442000 0x00062000 | 0x00062000
9   ConvRelu       output16 INT8     (1,128,28,28)  (1,8,28,28,16)   |
0xff442000 0xff45a800 0x00018800 | 0x00018800
10  ConvRelu       output18 INT8     (1,256,28,28)  (1,16,28,28,16)  |
0xff3e0000 0xff411000 0x00031000 | 0x00031000
11  ConvRelu       output20 INT8     (1,256,28,28)  (1,16,28,28,16)  |
0xff411000 0xff442000 0x00031000 | 0x00031000
12  ConvRelu       output22 INT8     (1,256,28,28)  (1,16,28,28,16)  |
0xff3e0000 0xff411000 0x00031000 | 0x00031000
13  ConvRelu       output24 INT8     (1,256,14,14)  (1,16,14,14,16)  |
0xff411000 0xff41d400 0x0000c400 | 0x0000c400
14  ConvRelu       output26 INT8     (1,512,14,14)  (1,32,14,14,16)  |
0xff3e0000 0xff3f8800 0x00018800 | 0x00018800
15  ConvRelu       output28 INT8     (1,512,14,14)  (1,32,14,14,16)  |
0xff3f8800 0xff411000 0x00018800 | 0x00018800
16  ConvRelu       output30 INT8     (1,512,14,14)  (1,32,14,14,16)  |
0xff3e0000 0xff3f8800 0x00018800 | 0x00018800
17  ConvRelu       output32 INT8     (1,512,14,14)  (1,32,14,14,16)  |
0xff3f8800 0xff411000 0x00018800 | 0x00018800
18  ConvRelu       output34 INT8     (1,512,14,14)  (1,32,14,14,16)  |
0xff3e0000 0xff3f8800 0x00018800 | 0x00018800
19  ConvRelu       output36 INT8     (1,512,14,14)  (1,32,14,14,16)  |
0xff3f8800 0xff411000 0x00018800 | 0x00018800
20  ConvRelu       output38 INT8     (1,512,14,14)  (1,32,14,14,16)  |
0xff3e0000 0xff3f8800 0x00018800 | 0x00018800
21  ConvRelu       output40 INT8     (1,512,14,14)  (1,32,14,14,16)  |
0xff3f8800 0xff411000 0x00018800 | 0x00018800
22  ConvRelu       output42 INT8     (1,512,14,14)  (1,32,14,14,16)  |
0xff3e0000 0xff3f8800 0x00018800 | 0x00018800
23  ConvRelu       output44 INT8     (1,512,14,14)  (1,32,14,14,16)  |
0xff3f8800 0xff411000 0x00018800 | 0x00018800
24  ConvRelu       output46 INT8     (1,512,14,14)  (1,32,14,14,16)  |
0xff3e0000 0xff3f8800 0x00018800 | 0x00018800
25  ConvRelu       output48 INT8     (1,512,7,7)    (1,33,7,7,16)    |
0xff3f8800 0xff3ff000 0x00006800 | 0x00006800
26  ConvRelu       output50 INT8     (1,1024,7,7)   (1,67,7,7,16)    |
```

```
0xff3e0000 0xff3ed000 0x0000d000 | 0x0000d000
27  ConvRelu       output52 INT8     (1,1024,7,7)   (1,67,7,7,16)    |
0xff3ed000 0xff3fa000 0x0000d000 | 0x0000d000
28  AveragePool    output54 INT8     (1,1024,7,7)   (1,67,7,7,16)    |
0xff3e0000 0xff3ed000 0x0000d000 | 0x0000d000
29  Conv           output55 INT8     (1,1024,1,1)   (1,64,1,1,16)    |
0xff3ed000 0xff3ed400 0x00000400 | 0x00000400
30  Softmax        output56 INT8     (1,1000,1,1)   (1,64,1,1,16)    |
0xff3e0000 0xff3e0400 0x00000400 | 0x00000400
31  OutputOperator output57 FLOAT    (1,1000,1,1)   (1,1000,1,1)     |
0xff3ae000 0xff3aefa0 0x00000fa0 | \
-------------------------------------------------------------------------+---------
------------------------+-----------
----------------------------------------
Total Weight Memory Size: 4260864
Total Internal Memory Size: 2157568
Predict Internal Memory RW Amount: 11068320
Predict Weight Memory RW Amount: 4260832
Predict SRAM Hit RW Amount: 6688768
----------------------------------------
```

- The SRAM Hit in the text chart above is the SRAM size occupied by the current layer tensor, which means that compared with not turning on SRAM, the amount of DDR read and write data corresponding to size will be saved.
- Predict SRAM Hit RW Amount is the read and write prediction of the entire model SRAM, indicating the amount of DDR read and write data that can be saved in each frame compared with SRAM not turned on.
- Note: Linux environment logs are redirected to the terminal, and Android environment logs are redirected to logcat.

4.1.6 Cacheable Memory Consistency

The Cacheable memory consistency problem means that when both the CPU and NPU devices access the same memory with cache flag, the CPU will cache the data into the data cache. If the DDR data accessed by the NPU is inconsistent with the CPU cache, it will cause data reading errors. Therefore, it is necessary to call the refresh cache interface to ensure that the DDR memory data accessed by the CPU and NPU is consistent. This chapter introduces the direction of synchronized data and how to use the rknn_mem_sync interface to refresh the cache.

4.1.6.1 DIRECTION OF CACHEABLE MEMORY SYNCHRONIZATION

When the CPU writes data to the cacheable memory, and then when the NPU accesses the memory, it must ensure that the data in the CPU cache is synchronized to the DDR. The direction of synchronization at this time is from the CPU to the NPU device; when the NPU finishes writing the data, the CPU begins to access the memory. When accessing memory, ensure that the DDR data is consistent with the data in the CPU cache. At this time, the direction of synchronization is from the NPU device to the CPU. RKNN C API provides the rknn_mem_sync_mode enumeration type to indicate the direction of cacheable memory synchronization. The data structure is as follows:

```
/*
    The mode to sync cacheable rknn memory.
*/
typedef enum _rknn_mem_sync_mode {
RKNN_MEMORY_SYNC_TO_DEVICE = 0x1, /* the mode used for consistency of device access
after CPU accesses data. */
RKNN_MEMORY_SYNC_FROM_DEVICE = 0x2, /* the mode used for consistency of CPU access
after device accesses data. */
RKNN_MEMORY_SYNC_BIDIRECTIONAL =
    RKNN_MEMORY_SYNC_TO_DEVICE | RKNN_MEMORY_SYNC_FROM_DEVICE, /* the mode used for
consistency of data access
between device and CPU in both directions. */
} rknn_mem_sync_mode;
```

- RKNN_MEMORY_SYNC_TO_DEVICE: Indicates that the data synchronization direction is from CPU to NPU device
- RKNN_MEMORY_SYNC_FROM_DEVICE: Indicates that the data synchronization direction is from NPU device to CPU
- RKNN_MEMORY_SYNC_BIDIRECTIONAL: Indicates that data is synchronized in both directions between NPU and CPU. This enumeration can be used when the user is not sure about the direction of synchronized data.

When clarifying the data synchronization direction, it is recommended to use the one-way CPU cache refresh mode to avoid performance losses caused by redundant CPU cache refresh actions.

4.1.6.2 SYNCHRONIZING CACHEABLE MEMORY

After clarifying the data synchronization direction, you can use the rknn_mem_sync interface to synchronize the cacheable memory. The interface form is as follows:

```
int rknn_mem_sync(rknn_context context, rknn_tensor_mem* mem, rknn_mem_sync_mode
mode);
```

Among them, context is the context (set to NULL by default on non-RV1103/RV1103B/RV1106/RV1106B platforms), mem is the rknn_tensor_mem* pointer type returned by the rknn_create_mem interface, and mode refers to the direction of synchronized data.

4.1.7 Model Sparse Inference

This function is used by the Torch model to automatically sparse the model weights during the training phase and use RKNN for sparse reasoning, thereby reduce model inference time. Currently the function only supports RK3576 and RK1126B.

4.1.7.1 SPARSITY PRINCIPLE

The model weights are sparsified (set to zero) in a user-defined manner during the training stage. The specific methods include 4:2 input direction sparsification, 4:2 output direction sparsification, 16:4 input and output sparsification, and 16:4 output and input sparsification. The

implementation principle of 4:2 input direction sparsification is shown in Figure 4-9: along the input direction of the model weights, two of every four consecutive values are selected and set to zero. Note that when the specified direction of the model weights is not 4-aligned, a padding operation will be performed.

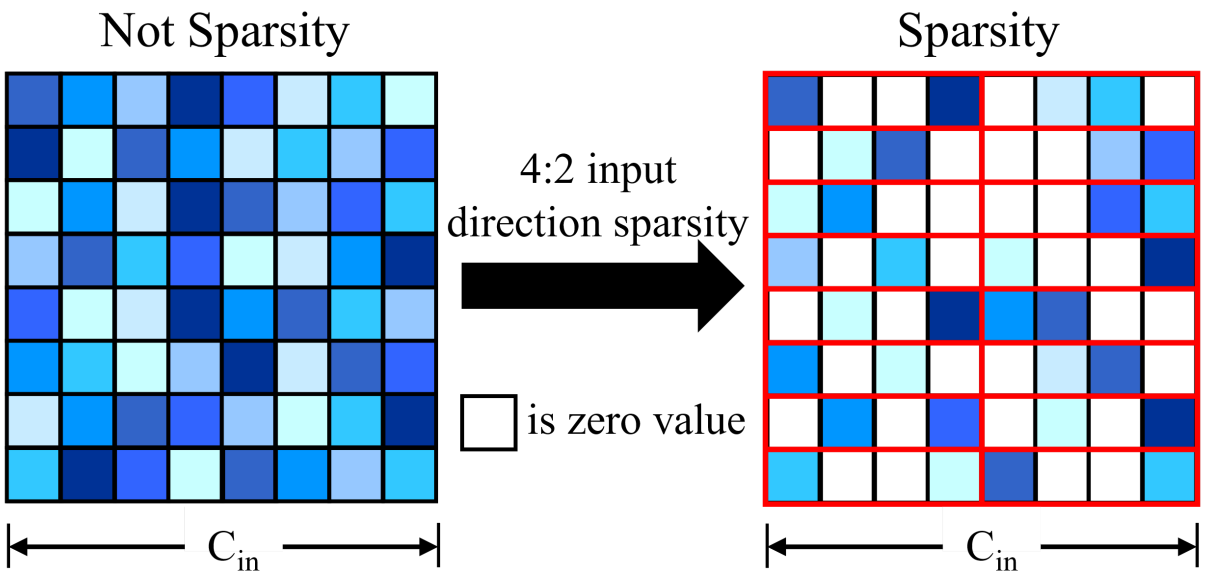


Figure 4-9 4:2 input direction sparsity principle diagram

The descriptions of the four weight sparsification methods are shown in Table 4-8, where the input and output directions refer to the directions of the weights. For example, for a 2-dimensional convolution with shape $C_{out}\times C_{in}\times K_h\times K_w$, the input direction is C_{in} and the output direction is C_{out}. For the methods with a sparsity rate of 75%, the pre-trained model can be either an unsparsified model or a sparse model. It is recommended to use a single-direction sparse model as the pre-trained model, which can reduce the accuracy loss after sparsification.

Table 4-8 Model weight sparsification description

Sparsification method	Sparsity rate	Description
4:2 input direction sparsification	50%	Sparsify the weights along the input direction
4:2 output direction sparsification	50%	Sparsify the weights along the output direction
16:4 input and output sparsification	75%	First sparsify 4:2 along the input direction, then sparsify 4:2 along the output direction
16:4 output and input sparsification	75%	First sparsify 4:2 along the output direction, then sparsify 4:2 along the input direction

4.1.7.2 TRAINING THE SPARSE MODEL

1. First, make sure that cuda is available and install the autosparsity package for the corresponding Python version

```
pip install autosparsity-1.0-cp38-cp38m-linux_x86_64.whl
```

The autosparsity installation package path is: https://github.com/airockchip/rknn-toolkit2/tree/master/autosparsity/packages

1. Take the 4:2 input direction sparsification of resnet50 in torchvision as an example to perform weight sparsification training

```
import torch
import torchvision.models as models
from autosparsity.sparsity import sparsity_model

if __name__ == "__main__":

    model = models.resnet50(pretrained=True).cuda()
    optimizer = None
    mode = 0
    sparsity_model(model, optimizer, mode)
    model.eval()
    x = torch.randn((1,3,224,224)).cuda()
    torch.onnx.export(
        model, x, 'resnet50.onnx', input_names=['inputs'], output_names=['outputs']
    )
```

To sparsify a custom model, simply add the `sparsity_model` function before model training. The reference example is as follows:

```
# insert model autosparsity code before training
    import torch
    import torchvision.models as models
    from autosparsity.sparsity import sparsity_model

    ...

    model = models.resnet34(pretrained=True).cuda()
    mode = 0
    sparsity_model(model, optimizer, mode)

# normal training
    x, y = DataLoader(args)
    for epoch in range(epochs):
        y_pred = model(x)
        loss = loss_func(y_pred, y)
        loss.backward()
        optimizer.step()
    ...
```

Note: Sparse training of Attention models relies on a well-trained front-end model. Therefore, before performing sparsification training, please first train a model with 'good' accuracy, and then perform sparsification training based on that model.

The descriptions of the parameters of the `sparsity_model` function are as follows:

Table 4-9 Description of parameters of sparsity_model function

Parameters	**Description**
model	Original training model

Parameters	Description
optimizer	Original optimizer, default is None
mode	Sparsification method, optional values are 0, 1, 2, 3, default is 0 0: 4:2 input direction sparsification (50% sparsity rate) 1: 4:2 output direction sparsification (50% sparsity rate) 2: 16:4 input and output sparsification (75% sparsity rate) 3: 16:4 output and input sparsification (75% sparsity rate)
verbose	Log level, optional values are 0, 1, 2, 3, default is 2 0: Errors 1: Errors and Warnings 2: Errors, warnings and info
whitelist	List of modules supported by sparsification, supporting 1d conv, 2d conv, 3d conv, linear, MultiheadAttention, default is [torch.nn.Linear, torch.nn.Conv2d]
allowed_layer_names	Layer names allowed to be sparsified. When configured by the user, only the specified layers will be sparsified, default is None
disallowed_layer_names	Layer names not allowed to be sparsified. When configured by the user, these layers will be skipped, default is []
fast	Set to True to use a fast method to calculate the mask, default is False (the default mask calculation method fails for some models; if a sparsification error is reported, try setting this parameter to True)

4.1.7.3 RKNN SPARSE INFERENCE USAGE

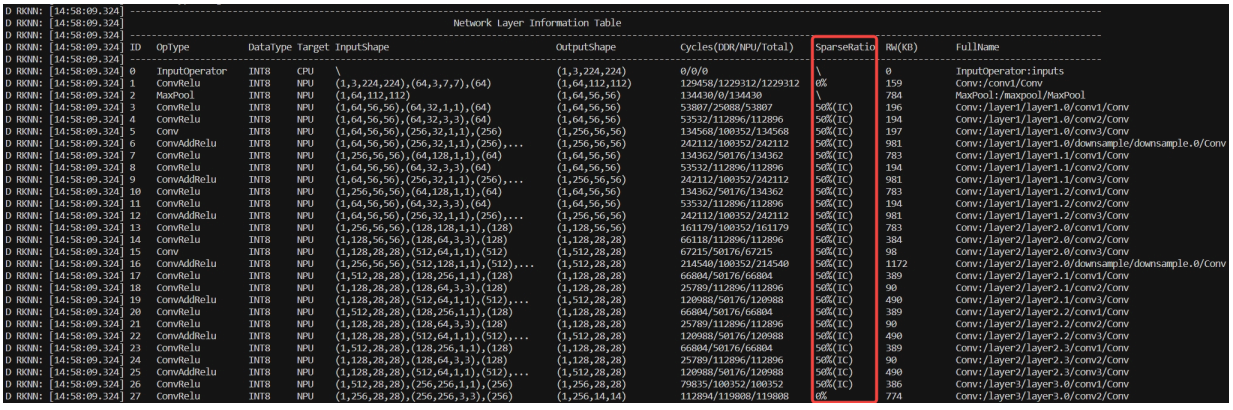
Model sparsification can be enabled or disabled through the "sparse_infer" parameter of the `config()` interface in RKNN-Toolkit2. The corresponding parameter values are `True/False`, and the default value is `False`. The reference code for enabling sparse inference is as follows:

```
rknn.config(target_platform='rk3576', sparse_infer=True)
```

The complete sparse inference Python code can be found at: https://github.com/airockchip/rknn-toolkit2/tree/master/autosparsity/examples

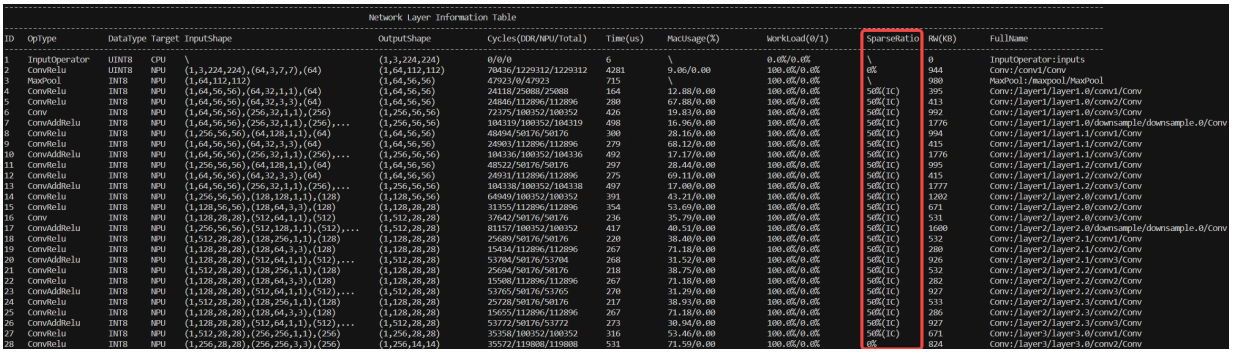
When deploying with the C API, first use RKNN-Toolkit2 to set the "`sparse_infer`" parameter to True in the `config()` interface to generate an RKNN model with sparse inference, and then call the general API interface process or the zero-copy API interface process normally.

When using Python code for inference, you can set `verbose=True` when building the RKNN object to enable the log that prints the sparsification status of each layer; when using the C API for inference, set the environment variable `RKNN_LOG_LEVEL=4` to enable the log that prints the sparsification status of each layer. The log information is as follows:



```
D RKNN: [14:58:09.324] ------------------------------------------------------------------------------------------------------------------------------------------------------
D RKNN: [14:58:09.324]                                                Network Layer Information Table
D RKNN: [14:58:09.324] ------------------------------------------------------------------------------------------------------------------------------------------------------
D RKNN: [14:58:09.324] ID   OpType          DataType Target InputShape                                    OutputShape          Cycles(DDR/NPU/Total)     SparseRatio  RW(KB)     FullName
D RKNN: [14:58:09.324] ------------------------------------------------------------------------------------------------------------------------------------------------------
D RKNN: [14:58:09.324] 0    InputOperator   INT8     CPU    \                                             (1,3,224,224)        0/0/0                     \            0          InputOperator:inputs
D RKNN: [14:58:09.324] 1    ConvRelu        INT8     NPU    (1,3,224,224),(64,3,7,7),(64)                 (1,64,112,112)       129458/1229312/1229312    0%           159        Conv:/conv1/Conv
D RKNN: [14:58:09.324] 2    MaxPool         INT8     NPU    (1,64,112,112)                                (1,64,56,56)         134430/0/134430           \            784        MaxPool:/maxpool/MaxPool
D RKNN: [14:58:09.324] 3    ConvRelu        INT8     NPU    (1,64,56,56),(64,32,1,1),(64)                 (1,64,56,56)         53807/25088/53807         50%(IC)      196        Conv:/layer1/layer1.0/conv1/Conv
D RKNN: [14:58:09.324] 4    ConvRelu        INT8     NPU    (1,64,56,56),(64,32,3,3),(64)                 (1,64,56,56)         53532/112896/112896       50%(IC)      194        Conv:/layer1/layer1.0/conv2/Conv
D RKNN: [14:58:09.324] 5    Conv            INT8     NPU    (1,64,56,56),(256,32,1,1),(256)               (1,256,56,56)        134568/100352/134568      50%(IC)      197        Conv:/layer1/layer1.0/conv3/Conv
D RKNN: [14:58:09.324] 6    ConvAddRelu     INT8     NPU    (1,64,56,56),(256,32,1,1),(256),...           (1,256,56,56)        242112/100352/242112      50%(IC)      981        Conv:/layer1/layer1.0/downsample/downsample.0/Conv
D RKNN: [14:58:09.324] 7    ConvRelu        INT8     NPU    (1,256,56,56),(64,128,1,1),(64)               (1,64,56,56)         134362/50176/134362       50%(IC)      783        Conv:/layer1/layer1.1/conv1/Conv
D RKNN: [14:58:09.324] 8    ConvRelu        INT8     NPU    (1,64,56,56),(64,32,3,3),(64)                 (1,64,56,56)         53532/112896/112896       50%(IC)      194        Conv:/layer1/layer1.1/conv2/Conv
D RKNN: [14:58:09.324] 9    ConvAddRelu     INT8     NPU    (1,64,56,56),(256,32,1,1),(256),...           (1,256,56,56)        242112/100352/242112      50%(IC)      981        Conv:/layer1/layer1.1/conv3/Conv
D RKNN: [14:58:09.324] 10   ConvRelu        INT8     NPU    (1,256,56,56),(64,128,1,1),(64)               (1,64,56,56)         134362/50176/134362       50%(IC)      783        Conv:/layer1/layer1.2/conv1/Conv
D RKNN: [14:58:09.324] 11   ConvRelu        INT8     NPU    (1,64,56,56),(64,32,3,3),(64)                 (1,64,56,56)         53532/112896/112896       50%(IC)      194        Conv:/layer1/layer1.2/conv2/Conv
D RKNN: [14:58:09.324] 12   ConvAddRelu     INT8     NPU    (1,64,56,56),(256,32,1,1),(256),...           (1,256,56,56)        242112/100352/242112      50%(IC)      981        Conv:/layer1/layer1.2/conv3/Conv
D RKNN: [14:58:09.324] 13   ConvRelu        INT8     NPU    (1,256,56,56),(128,128,1,1),(128)             (1,128,56,56)        161179/100352/161179      50%(IC)      783        Conv:/layer2/layer2.0/conv1/Conv
D RKNN: [14:58:09.324] 14   ConvRelu        INT8     NPU    (1,128,56,56),(128,64,3,3),(128)              (1,128,28,28)        66118/112896/112896       50%(IC)      384        Conv:/layer2/layer2.0/conv2/Conv
D RKNN: [14:58:09.324] 15   Conv            INT8     NPU    (1,128,28,28),(512,64,1,1),(512)              (1,512,28,28)        67215/50176/67215         50%(IC)      98         Conv:/layer2/layer2.0/conv3/Conv
D RKNN: [14:58:09.324] 16   ConvAddRelu     INT8     NPU    (1,256,56,56),(512,128,1,1),(512),...         (1,512,28,28)        214540/100352/214540      50%(IC)      1172       Conv:/layer2/layer2.0/downsample/downsample.0/Conv
D RKNN: [14:58:09.324] 17   ConvRelu        INT8     NPU    (1,512,28,28),(128,256,1,1),(128)             (1,128,28,28)        66804/50176/66804         50%(IC)      389        Conv:/layer2/layer2.1/conv1/Conv
D RKNN: [14:58:09.324] 18   ConvRelu        INT8     NPU    (1,128,28,28),(128,64,3,3),(128)              (1,128,28,28)        25789/112896/112896       50%(IC)      90         Conv:/layer2/layer2.1/conv2/Conv
D RKNN: [14:58:09.324] 19   ConvAddRelu     INT8     NPU    (1,128,28,28),(512,64,1,1),(512),...          (1,512,28,28)        120988/50176/120988       50%(IC)      490        Conv:/layer2/layer2.1/conv3/Conv
D RKNN: [14:58:09.324] 20   ConvRelu        INT8     NPU    (1,512,28,28),(128,256,1,1),(128)             (1,128,28,28)        66804/50176/66804         50%(IC)      389        Conv:/layer2/layer2.2/conv1/Conv
D RKNN: [14:58:09.324] 21   ConvRelu        INT8     NPU    (1,128,28,28),(128,64,3,3),(128)              (1,128,28,28)        25789/112896/112896       50%(IC)      90         Conv:/layer2/layer2.2/conv2/Conv
D RKNN: [14:58:09.324] 22   ConvAddRelu     INT8     NPU    (1,128,28,28),(512,64,1,1),(512),...          (1,512,28,28)        120988/50176/120988       50%(IC)      490        Conv:/layer2/layer2.2/conv3/Conv
D RKNN: [14:58:09.324] 23   ConvRelu        INT8     NPU    (1,512,28,28),(128,256,1,1),(128)             (1,128,28,28)        66804/50176/66804         50%(IC)      389        Conv:/layer2/layer2.3/conv1/Conv
D RKNN: [14:58:09.324] 24   ConvRelu        INT8     NPU    (1,128,28,28),(128,64,3,3),(128)              (1,128,28,28)        25789/112896/112896       50%(IC)      90         Conv:/layer2/layer2.3/conv2/Conv
D RKNN: [14:58:09.324] 25   ConvAddRelu     INT8     NPU    (1,128,28,28),(512,64,1,1),(512),...          (1,512,28,28)        120988/50176/120988       50%(IC)      490        Conv:/layer2/layer2.3/conv3/Conv
D RKNN: [14:58:09.324] 26   ConvRelu        INT8     NPU    (1,512,28,28),(256,256,1,1),(256)             (1,256,28,28)        79835/100352/100352       50%(IC)      386        Conv:/layer3/layer3.0/conv1/Conv
D RKNN: [14:58:09.324] 27   ConvRelu        INT8     NPU    (1,256,28,28),(256,256,3,3),(256)             (1,256,14,14)        112894/119808/119808      0%           774        Conv:/layer3/layer3.0/conv2/Conv
```

Figure 4-10 4:2 input direction sparsification Python printing log



Network Layer Information Table

ID	OpType	DataType	Target	InputShape	OutputShape	Cycles(DDR/NPU/Total)	Time(us)	MacUsage(%)	WorkLoad(0/1)	SparseRatio	RW(KB)	FullName
1	InputOperator	UINT8	CPU	\	(1,3,224,224)	0/0/0	6	\	0.0%/0.0%	\	0	InputOperator:inputs
2	ConvRelu	UINT8	NPU	(1,3,224,224),(64,3,7,7),(64)	(1,64,112,112)	70436/1229312/1229312	4281	9.06/0.00	100.0%/0.0%	0%	944	Conv:/conv1/Conv
3	MaxPool	INT8	NPU	(1,64,112,112)	(1,64,56,56)	47923/0/47923	715	\	100.0%/0.0%	\	980	MaxPool:/maxpool/MaxPool
4	ConvRelu	INT8	NPU	(1,64,56,56),(64,32,1,1),(64)	(1,64,56,56)	24118/25088/25088	164	12.88/0.00	100.0%/0.0%	50%(IC)	395	Conv:/layer1/layer1.0/conv1/Conv
5	ConvRelu	INT8	NPU	(1,64,56,56),(64,32,3,3),(64)	(1,64,56,56)	24846/112896/112896	280	67.88/0.00	100.0%/0.0%	50%(IC)	413	Conv:/layer1/layer1.0/conv2/Conv
6	Conv	INT8	NPU	(1,64,56,56),(256,32,1,1),(256)	(1,256,56,56)	72375/100352/100352	426	19.83/0.00	100.0%/0.0%	50%(IC)	992	Conv:/layer1/layer1.0/conv3/Conv
7	ConvAddRelu	INT8	NPU	(1,64,56,56),(256,32,1,1),(256),...	(1,256,56,56)	104319/100352/104319	498	16.96/0.00	100.0%/0.0%	50%(IC)	1776	Conv:/layer1/layer1.0/downsample/downsample.0/Conv
8	ConvRelu	INT8	NPU	(1,256,56,56),(64,128,1,1),(64)	(1,64,56,56)	48494/50176/50176	300	20.16/0.00	100.0%/0.0%	50%(IC)	994	Conv:/layer1/layer1.1/conv1/Conv
9	ConvRelu	INT8	NPU	(1,64,56,56),(64,32,3,3),(64)	(1,64,56,56)	24903/112896/112896	279	68.12/0.00	100.0%/0.0%	50%(IC)	415	Conv:/layer1/layer1.1/conv2/Conv
10	ConvAddRelu	INT8	NPU	(1,64,56,56),(256,32,1,1),(256),...	(1,256,56,56)	104336/100352/104336	492	17.17/0.00	100.0%/0.0%	50%(IC)	1776	Conv:/layer1/layer1.1/conv3/Conv
11	ConvRelu	INT8	NPU	(1,256,56,56),(64,128,1,1),(64)	(1,64,56,56)	48522/50176/50176	297	28.44/0.00	100.0%/0.0%	50%(IC)	995	Conv:/layer1/layer1.2/conv1/Conv
12	ConvRelu	INT8	NPU	(1,64,56,56),(64,32,3,3),(64)	(1,64,56,56)	24931/112896/112896	275	69.11/0.00	100.0%/0.0%	50%(IC)	415	Conv:/layer1/layer1.2/conv2/Conv
13	ConvAddRelu	INT8	NPU	(1,64,56,56),(256,32,1,1),(256),...	(1,256,56,56)	104338/100352/104338	497	17.00/0.00	100.0%/0.0%	50%(IC)	1777	Conv:/layer1/layer1.2/conv3/Conv
14	ConvRelu	INT8	NPU	(1,256,56,56),(128,128,1,1),(128)	(1,128,56,56)	64949/100352/100352	391	43.21/0.00	100.0%/0.0%	50%(IC)	1202	Conv:/layer2/layer2.0/conv1/Conv
15	ConvRelu	INT8	NPU	(1,128,56,56),(128,64,3,3),(128)	(1,128,28,28)	31355/112896/112896	354	53.69/0.00	100.0%/0.0%	50%(IC)	671	Conv:/layer2/layer2.0/conv2/Conv
16	Conv	INT8	NPU	(1,128,28,28),(512,64,1,1),(512)	(1,512,28,28)	37642/50176/50176	236	35.79/0.00	100.0%/0.0%	50%(IC)	531	Conv:/layer2/layer2.0/conv3/Conv
17	ConvAddRelu	INT8	NPU	(1,256,56,56),(512,128,1,1),(512),...	(1,512,28,28)	81157/100352/100352	417	40.51/0.00	100.0%/0.0%	50%(IC)	1600	Conv:/layer2/layer2.0/downsample/downsample.0/Conv
18	ConvRelu	INT8	NPU	(1,512,28,28),(128,256,1,1),(128)	(1,128,28,28)	25689/50176/50176	220	38.40/0.00	100.0%/0.0%	50%(IC)	532	Conv:/layer2/layer2.1/conv1/Conv
19	ConvRelu	INT8	NPU	(1,128,28,28),(128,64,3,3),(128)	(1,128,28,28)	15434/112896/112896	267	71.18/0.00	100.0%/0.0%	50%(IC)	280	Conv:/layer2/layer2.1/conv2/Conv
20	ConvAddRelu	INT8	NPU	(1,128,28,28),(512,64,1,1),(512),...	(1,512,28,28)	53704/50176/53704	268	31.52/0.00	100.0%/0.0%	50%(IC)	926	Conv:/layer2/layer2.1/conv3/Conv
21	ConvRelu	INT8	NPU	(1,512,28,28),(128,256,1,1),(128)	(1,128,28,28)	25694/50176/50176	218	38.75/0.00	100.0%/0.0%	50%(IC)	532	Conv:/layer2/layer2.2/conv1/Conv
22	ConvRelu	INT8	NPU	(1,128,28,28),(128,64,3,3),(128)	(1,128,28,28)	15508/112896/112896	267	71.18/0.00	100.0%/0.0%	50%(IC)	282	Conv:/layer2/layer2.2/conv2/Conv
23	ConvAddRelu	INT8	NPU	(1,128,28,28),(512,64,1,1),(512),...	(1,512,28,28)	53765/50176/53765	270	31.29/0.00	100.0%/0.0%	50%(IC)	927	Conv:/layer2/layer2.2/conv3/Conv
24	ConvRelu	INT8	NPU	(1,512,28,28),(128,256,1,1),(128)	(1,128,28,28)	25728/50176/50176	217	38.93/0.00	100.0%/0.0%	50%(IC)	533	Conv:/layer2/layer2.3/conv1/Conv
25	ConvRelu	INT8	NPU	(1,128,28,28),(128,64,3,3),(128)	(1,128,28,28)	15655/112896/112896	267	71.18/0.00	100.0%/0.0%	50%(IC)	286	Conv:/layer2/layer2.3/conv2/Conv
26	ConvAddRelu	INT8	NPU	(1,128,28,28),(512,64,1,1),(512),...	(1,512,28,28)	53772/50176/53772	273	30.94/0.00	100.0%/0.0%	50%(IC)	927	Conv:/layer2/layer2.3/conv3/Conv
27	ConvRelu	INT8	NPU	(1,512,28,28),(256,256,1,1),(256)	(1,256,28,28)	35358/100352/100352	316	53.46/0.00	100.0%/0.0%	50%(IC)	671	Conv:/layer3/layer3.0/conv1/Conv
28	ConvRelu	INT8	NPU	(1,256,28,28),(256,256,3,3),(256)	(1,256,14,14)	35572/119808/119808	531	71.59/0.00	100.0%/0.0%	0%	824	Conv:/layer3/layer3.0/conv2/Conv

Figure 4-11 4:2 input direction sparsification C API printing log

Here, `SparseRation` represents the sparsity rate: 4:2 input direction sparsification corresponds to 50% (IC), 4:2 output direction sparsification corresponds to 50% (OC), and 16:4 input and output sparsification and 16:4 output and input sparsification correspond to 75%. 0% means no sparsification has been performed.

4.1.7.4 RKNN SPARSE INFERENCE RESTRICTIONS

Sparse inference is implemented based on the NPU hardware architecture. Limited by hardware specifications, the sparse inference of RK3576 currently only supports 2d Conv with the group parameter set to 1. The other restrictions are as follows:

Table 4-10 RKNN sparse inference restrictions

	Number of channels	**Data type**
4:2 input direction sparsification	Input is 32-aligned, output is 32-aligned	Only Int8 is supported, Float16 is not supported
4:2 output direction sparsification	Input is 32-aligned, output is 32-aligned	Only Int8 is supported, Float16 is not supported
16:4 input and output sparsification	Input is 32-aligned, output is 32-aligned	Only Int8 is supported, Float16 is not supported
16:4 output and input sparsification	Input is 32-aligned, output is 32-aligned	Only Int8 is supported, Float16 is not supported

4.2 Extensions and Security

4.2.1 Custom Operators

4.2.1.1 INTRODUCTION

The RKNN SDK provides a mechanism for custom operators, which allows developers to define and execute custom operators in the inference stage of RKNN models. By implementing custom operators, developers can extend model functionality and optimize for specific hardware (CPU or GPU) to make full use of hardware resources and increase inference speed. At the same time, developing custom operators requires a deep understanding of deep learning computing principles and the characteristics of the target hardware platform to ensure correctness and performance. Currently, only custom operators for ONNX models are supported.

RKNN custom operators mainly include two major steps:

- Use RKNN-Toolkit2 to register custom operators and export the RKNN model.
- Write the C code implementation of the custom operator, and load, register and execute it through the RKNN API.

The overall process is shown in the following figure:

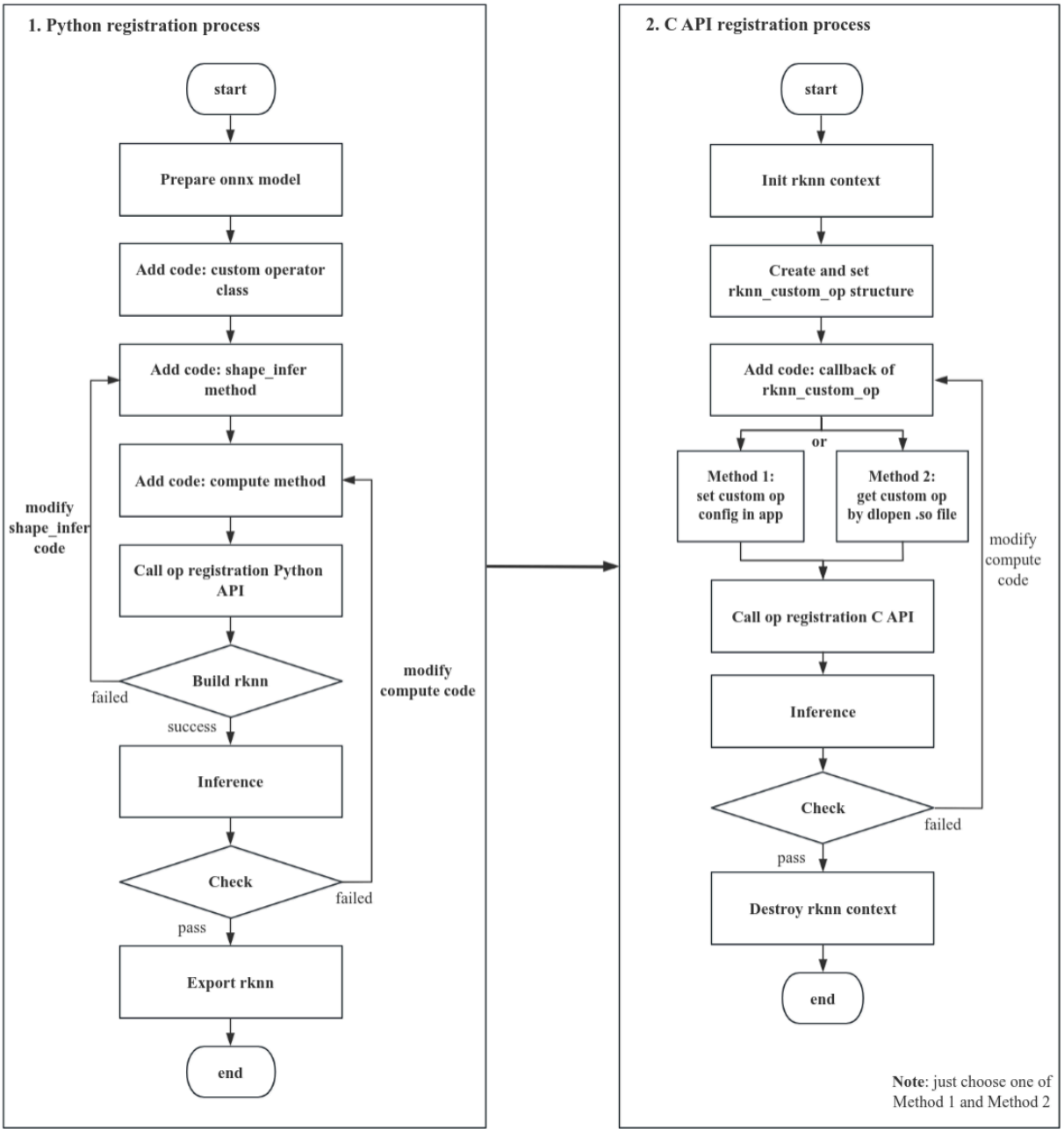


Figure 4-12 Complete process of registering custom operators

4.2.1.2 OVERALL PROCESS

4.2.1.2.1 Use RKNN-Toolkit2 to register custom operators and export RKNN models

1. Prepare the onnx model: According to the ONNX specification, the user designs the op type, name, attributes, input/output number of the custom operator, and inserts the operator into the topology graph position in onnx. The user designs and exports onnx models using the API provided by the onnx package.

2. Implement the custom operator Python class, which mainly includes two function interfaces: shape_infer() and compute(), and call rknn.reg_custom_op() to register the operator.
3. If rknn.build() is executed successfully, inference can be performed, otherwise you need to check the shape_infer() code implementation in step 2. Then run the simulation. If the simulation results are correct, you can call the rknn.export_rknn() interface to export the RKNN model. Otherwise, you need to check the compute() code in step 2.

4.2.1.2.2 C code implementation of the custom operator, load, register and execute through RKNN API.

1. Based on the rknn_custom_op class of rknn_custom_op.h, write the C code implementation of the custom operator. After the writing is completed, fill in the information of the rknn_custom_op class.
2. Call rknn_register_custom_ops() to register the information of the rknn_custom_op class.
3. Refer to the process of the general API or zero-copy API to build and infer the model. You can turn on the model detailed log and dump function to confirm the correctness of the custom operator implementation.

4.2.1.2.3 Use RKNN-Toolkit2 device connected inference or accuracy analysis

If the user needs to perform device connected accuracy analysis on a model containing a custom operator, need to compile the callback function implementation code of the custom operator into .so, place it in the specified path, and restart rknn_server. Please refer to Chapter 4.2.1.4.4 for details.

4.2.1.3 PYTHON PROCESS

Currently, only the onnx model supports custom operators, allowing users to **add Non-ONNX standard operators supported by RKNN**.

Adding an Non-ONNX standard custom operator is used to add a new operator that does not exist in the ONNX operator list. In addition to meeting the ONNX spec specification, the operator must also meet the following rules:

- The op_type of the operator cannot be the same as the ONNX standard operator. It is recommended to start with the "cst" character.
- The operator must have a connection relationship with other operators, including the shape of each input/output, data type, etc.
- Operator input attributes support single values or arrays of types bool, int32, float32, and int64.
- Operator constant input supports bool, int32, float32, and int64 types. This type refers to the data type of the unquantized ONNX model.

Because the Non-ONNX standard operator is not a standard operator in the ONNX SPEC. Therefore, users need to build and export an ONNX model containing the Non-ONNX standard operator through the ONNX API or the API of other frameworks.

For convenience, a simple modification of the definition of Softmax is used as an example to build an ONNX model containing the Non-ONNX standard operator cstSoftmax. The modification

method is as follows:

```
import onnx

path="test_softmax.onnx"
model=onnx.load(path)

for node in model.graph.node:
    if node.op_type =="Softmax":
        node.op_type = "cstSoftmax"
        ...   # Modify the attribute definition of cstSoftmax, etc
onnx.save(model, "./test_softmax_custom.onnx")
```

After building an onnx model containing custom operators, you can use Netron to open the ONNX model and check whether the custom operator meets the specifications. Then you can implement the custom operator class, as follows (taking custom Softmax as an example):

```
import numpy as np
from rknn.api.custom_op import get_node_attr
class cstSoftmax:
    op_type = 'cstSoftmax'
    def shape_infer(self, node, in_shapes, in_dtypes):
        out_shapes = in_shapes.copy()
        out_dtypes = in_dtypes.copy()
        return out_shapes, out_dtypes
    def compute(self, node, inputs):
        x = inputs[0]
        axis = get_node_attr(node, 'axis')
        x_max = np.max(x, axis=axis, keepdims=True)
        tmp = np.exp(x - x_max)
        s = np.sum(tmp, axis=axis, keepdims=True)
        outputs = [tmp / s]
        return outputs
```

- The custom operator must be a Python class.
- The custom operator class must contain a string variable named op_type, which is consistent with the name of the custom operator type in the ONNX.
- The custom operator class must contain the member function shape_infer(self, node, in_shapes, in_dtypes). The function name and parameter name must be consistent, otherwise an error will be reported. This function is used for shape inference of custom operators. Among node is the operator node object of ONNX, which contains the attributes and input and output information of the custom operator; in_shapes is the shape information of all inputs of the operator, the format is [shape_0, shape_1, ...], and the type of the shape in the list is a list; in_dtypes is the dtype information of all inputs to the operator, the format is [dtype_0, dtype_1, ...], and the type of the dtype in the list is the numpy's dtype. In addition, this function needs to return all the output shape information and dtype information of the operator, and the format is consistent with in_shapes and in_dtypes.
- The custom operator class must contain the member function compute(self, node, inputs), and the function name and parameter name must be consistent, otherwise an error will be reported. This function is used for inference of custom operators. Among node is the

operator node object of ONNX, which contains the attributes and input and output information of the custom operator; inputs is the input data of the operator, and the format is [array_0, array_1, ...], and the type of the dtype in the list is the numpy's ndarray. In addition, this function needs to return all output data of the operator, in the same format as inputs.

- If the custom operator contains custom attributes, you can obtain the attribute value of the custom operator through from rknn.api.custom_op import get_node_attr.

After completing the custom operator class, you can register the operator class through rknn.reg_custom_op(). After registration, you can call rknn.build() to convert and generate the RKNN model. Custom operator classes can ensure normal functions such as model conversion and inference. The specific implementation is as follows (taking custom Softmax as an example):

```
from rknn.api import RKNN
# Create RKNN object
rknn = RKNN(verbose=True)

# Pre-process config
print('--> Config model')
rknn.config(mean_values=[103.94, 116.78, 123.68], std_values=[58.82, 58.82, 58.82],
quant_img_RGB2BGR=True, target_platform='rk3566')
print('done')

print('--> Register cstSoftmax op')
ret = rknn.reg_custom_op(cstSoftmax())
if ret != 0:
    print('Register cstSoftmax op failed!')
    exit(ret)
print('done')

print('--> Loading model')
ret = rknn.load_onnx(model='mobilenet_v2.onnx')
if ret != 0:
    print('Load model failed!')
    exit(ret)
print('done')

# Build model
print('--> Building model')
ret = rknn.build(do_quantization=True, dataset='./dataset.txt')
if ret != 0:
    print('Build model failed!')
    exit(ret)
print('done')
```

rknn.reg_custom_op() needs to be called between rknn.config() and rknn.load_xxx().

4.2.1.4 C API DEPLOYMENT

After obtaining the RKNN model with custom operators, start calling the C API for deployment. First, the structure and interface of the custom operators are located in the `rknn_custom_op.h` header file, and the developer program needs to include this header file. The process of registering and using custom operators is shown in the figure below:

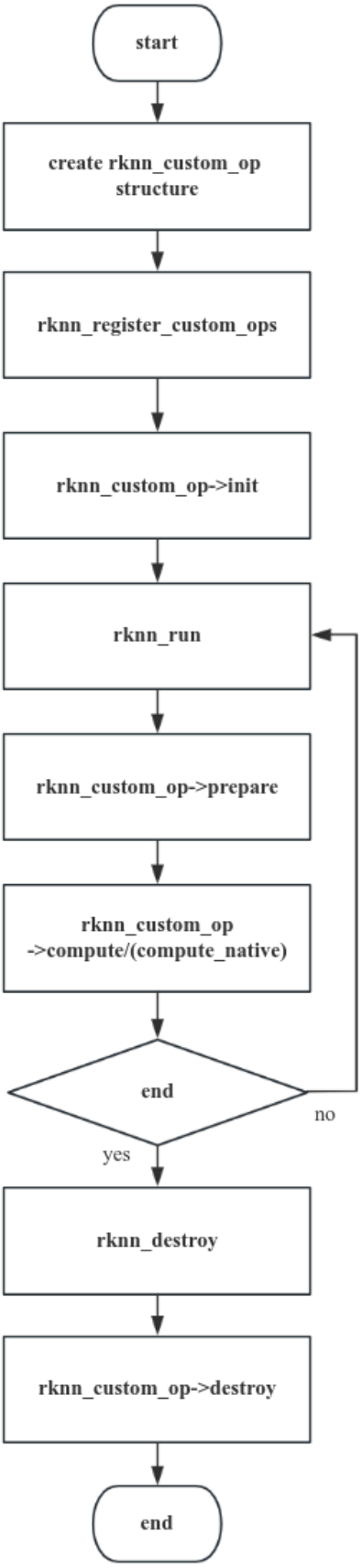


Figure 4-13 C API calling process for registering custom operators

4.2.1.4.1 Initialize Custom Operator Structure

After creating the rknn context, developers need to create the rknn_custom_op structure and set custom operator information. Operator information includes the following:

- version: The version of the operator.
- target: The execution backend of the operator, currently supports CPU and GPU.
- op_type: The type of operator, which is the same as the type in the ONNX model.
- cl_kernel_name: cl_kernel function name of OpenCL code. Must be configured when registering a GPU operator.
- cl_kernel_source: The full path of the .cl file of the custom operator or the OpenCL kernel string. When cl_source_size=0, it represents the full path of the .cl file; when cl_source_size>0, it represents the OpenCL kernel code string. Must be configured when registering a GPU operator.
- cl_source_size: The size of cl_kernel_source. Size equal to 0 is a special case, it means cl_kernel_source is the path.
- cl_build_options: OpenCL kernel compilation options, passed in as a string. Must be configured when registering a GPU operator.
- init: optional, called once in rknn_register_custom_ops.
- prepare: optional, it is a preprocessing callback function. Each time rknn_run will execute prepare and compute/compute_native callback functions, the order of execution is prepare first, compute/compute_native last.
- compute: Must be implemented, operator computation callback function, its input/output are NCHW float32 format data (onnx model if the input/output is int64 data type, it will be int64 format data)
- compute_native: reserved, please set to NULL.
- destroy: optional, executed once in rknn_destory.
- The init/prepare/compute callback function parameter definition specifications are as follows:
  - rknn_custom_op_context* op_ctx: context information of op callback function
  - rknn_custom_op_tensor* inputs: op input tensor data and information
  - uint32_t n_inputs: number of op inputs
  - rknn_custom_op_tensor* outputs: op output tensor data and information
  - uint32_t n_outputs: number of op outputs
- The destroy callback function only has one parameter: rknn_custom_op_context* op_ctx.

rknn_custom_op_context

Contains target (execution backend device), GPU context, custom operator private context and priv_data, of which priv_data is managed by the developer (assignment, reading, writing, and destroy). The GPU context includes cl_context, cl_command_queue, and cl_kernel pointers, which can be passed forced type conversion to obtain the corresponding OpenCL object.

priv_data is a pointer that the user can choose to configure or not. The usual usage is that the user creates a resource in the init() callback function, points priv_data to the memory address and applys in the prepare()/compute() callback function, finally in destroy resources within the destroy() callback function.

rknn_custom_op_tensor

Represents input/output tensor information, including tensor name, shape, size, quantization parameters, virtual base address, fd, data offset and other information.

The user does not need to create the input and output Tensor memory of the operator in the callback compute() callback function. The data corresponding to the virtual address is ready when entering the compute() callback function. The calculation formula of virtual address is **Tensor's effective address = virtual base address + data offset**. The virt_addr of the mem member represents the virtual base address, and the offset of the mem member represents the data offset (in bytes). The user can read the effective address of the input tensor in the callback function, which points to the output data calculated by the previous layer operator, and the effective address of the output tensor points for the input that will be sent to the next layer operator.

rknn_custom_op_attr

The developer obtains the attribute information by calling the rknn_custom_op_get_op_attr() interface and passing in the attribute field. The attribute information is represented by rknn_custom_op_attr. The void type buffer, dtype and number of elements in rknn_custom_op_attr represent a memory segment. The developers can obtain an array of the corresponding numerical type by forcefully casting the buffer pointer type in C/C++ based on the dtype.

4.2.1.4.1.1 Init Callback Function

It is used for parsing operator information or initializing temporary buffers or input/output buffers, such as for allocating a temporary buffer. The sample code of the init callback function is as follows:

- CPU Operator:

```
/**
* cpu kernel init callback for custom op
*/
int custom_op_init_callback(rknn_custom_op_context* op_ctx,
rknn_custom_op_tensor* inputs, uint32_t n_inputs,
                            rknn_custom_op_tensor* outputs, uint32_t n_outputs)
{
    printf("custom_op_init_callback\n");
    // create tmp buffer
    float* tmp_buffer = (float*)malloc(inputs[0].attr.n_elems * sizeof(float));
    op_ctx->priv_data = tmp_buffer;
    return 0;
}
```

- GPU Operator:

```
/**
* opencl kernel init callback for custom op
* */
int relu_init_callback_gpu(rknn_custom_op_context* op_ctx,
rknn_custom_op_tensor* inputs, uint32_t n_inputs,
                           rknn_custom_op_tensor* outputs, uint32_t n_outputs)
{
    printf("relu_init_callback_gpu\n");
    // 获取opencl context
    cl_context cl_ctx = (cl_context)op_ctx->gpu_ctx.cl_context;

    // create tmp cl buffer
    cl_mem* memObject = (cl_mem*)malloc(sizeof(cl_mem) * 2);
    memObject[0]      = clCreateBuffer(cl_ctx, CL_MEM_READ_WRITE,
inputs[0].attr.size, NULL, NULL);
    memObject[1]      = clCreateBuffer(cl_ctx, CL_MEM_READ_WRITE,
outputs[0].attr.size, NULL, NULL);
    op_ctx->priv_data = memObject;
    return 0;
}
```

4.2.1.4.1.2 Prepare Callback Function

This callback function is called for each frame inference and is currently reserved for future.

4.2.1.4.1.3 Compute Callback Function

It is the computation function of a custom operator. Developers must complete the kernel function whose inputs and outputs are float32 data in `NCHW` or `UNDEFINED` format.

1. `compute` callback (CPU). Assuming the developer wants to implement a custom layer that performs the softmax function, an example of the CPU operator `compute` function is as follows:

```
/**
 * float32 kernel implemetation sample for custom op
 * */
int compute_custom_softmax_float32(rknn_custom_op_context* op_ctx,
rknn_custom_op_tensor* inputs, uint32_t n_inputs,
                                   rknn_custom_op_tensor* outputs, uint32_t
n_outputs)
{
  unsigned char*      in_ptr   = (unsigned char*)inputs[0].mem.virt_addr +
inputs[0].mem.offset;
  unsigned char*      out_ptr  = (unsigned char*)outputs[0].mem.virt_addr +
outputs[0].mem.offset;
  int                 axis     = 0;
  const float*        in_data  = (const float*)in_ptr;
  float*              out_data = (float*)out_ptr;
  std::string         name     = "";
  rknn_custom_op_attr op_name;
  rknn_custom_op_get_op_attr(op_ctx, "name", &op_name);
  if (op_name.n_elems > 0 && op_name.dtype == RKNN_TENSOR_UINT8) {
    name = (char*)op_name.data;
  }

  rknn_custom_op_attr op_attr;
  rknn_custom_op_get_op_attr(op_ctx, "axis", &op_attr);
  if (op_attr.n_elems == 1 && op_attr.dtype == RKNN_TENSOR_INT64) {
    axis = ((int64_t*)op_attr.data)[0];
  }

  printf("op name = %s, axis = %d\n", name.c_str(), axis);
  float* tmp_buffer = (float*)op_ctx->priv_data;
  // kernel implemetation for custom op
  {
    int inside  = 1;
    int outside = 1;
    int channel = 1;

    while (axis < 0) {
      axis += inputs[0].attr.n_dims;
    }

    for (int i = 0; i < axis; i++) {
      outside *= inputs[0].attr.dims[i];
    }
    channel = inputs[0].attr.dims[axis];
    for (int i = axis; i < inputs[0].attr.n_dims; i++) {
      inside *= inputs[0].attr.dims[i];
    }

    for (int y = 0; y < outside; y++) {
      const float* src_y    = in_data + y * inside;
      float*       dst_y    = out_data + y * inside;
      float        max_data = -FLT_MAX;
      float        sum_data = 0.0f;

      for (int i = 0; i < inside; ++i) {
        max_data = fmaxf(max_data, src_y[i]);
      }
      for (int i = 0; i < inside; ++i) {
        tmp_buffer[i] = expf(src_y[i] - max_data);
        sum_data += tmp_buffer[i];
```

```
        }
        for (int i = 0; i < inside; ++i) {
          dst_y[i] = tmp_buffer[i] / sum_data;
        }
      }
    }
    return 0;
}
```

1. `compute` callback function (GPU)

For GPU operators, developers can complete the following steps in the callback function:

- The developer retrieves the OpenCL `cl_context`, `cl_command_queue` and `cl_kernel` objects from `gpu_ctx` in `rknn_custom_op_context`. This process requires the developer to perform data type conversions.
- If necessary, the user creates `cl_mem` object buffers for the op inputs or outputs.
- Set the function arguments of the `cl_kernel`.
- The input buffer data of the OpenCL kernel function arguments currently only supports float; other types are not supported yet.
- When zero-copy is used, calling `clImportMemoryARM` can help the user map the memory of the input tensor into the OpenCL `cl_mem` structure. The input tensor already contains the input data, so the user does not need to copy it again. This process can also be handled in the init callback function, where the `cl_mem` structure is recorded in the `priv_data` member, and finally `priv_data` is read and used in the `compute` callback.
- Run the `cl_kernel` in a blocking manner.
- The input data inside the CL kernel is provided in `NCHW` layout.
- If the developer needs the CPU to access the data after the GPU computation is finished, the cache of the output tensor must be flushed by calling the `rknn_mem_sync` function before reading the data.

Assuming the developer wants to implement a custom layer that performs the relu function, an example of the GPU operator compute function is as follows:

```
/**
 * opencl kernel init callback for custom op
 * */
int compute_custom_relu_float32(rknn_custom_op_context* op_ctx,
rknn_custom_op_tensor* inputs, uint32_t num_inputs,
                                rknn_custom_op_tensor* outputs, uint32_t
num_outputs)
{
  std::string         name = "";
  rknn_custom_op_attr op_name;
  rknn_custom_op_get_op_attr(op_ctx, "name", &op_name);
  if (op_name.n_elems > 0 && op_name.dtype == RKNN_TENSOR_UINT8) {
    name = (char*)op_name.data;
  }

  // get context
  cl_context cl_ctx = (cl_context)op_ctx->gpu_ctx.cl_context;

  // get command queue
  cl_command_queue queue = (cl_command_queue)op_ctx->gpu_ctx.cl_command_queue;

  // get kernel
  cl_kernel kernel = (cl_kernel)op_ctx->gpu_ctx.cl_kernel;

  // import input/output buffer
  const cl_import_properties_arm props[3] = {
    CL_IMPORT_TYPE_ARM,
    CL_IMPORT_TYPE_DMA_BUF_ARM,
    0,
  };

  cl_int status;
  cl_mem inObject = clImportMemoryARM(cl_ctx, CL_MEM_READ_WRITE, props,
&inputs[0].mem.fd,
                                      inputs[0].mem.offset + inputs[0].mem.size,
&status);
  if (status != CL_SUCCESS) {
    printf("Tensor: %s clImportMemoryARM failed\n", inputs[0].attr.name);
  }
  cl_mem outObject = clImportMemoryARM(cl_ctx, CL_MEM_READ_WRITE, props,
&outputs[0].mem.fd,
                                       outputs[0].mem.offset + outputs[0].mem.size,
&status);
  if (status != CL_SUCCESS) {
    printf("Tensor: %s clImportMemoryARM failed\n", outputs[0].attr.name);
  }

  int          in_type_bytes  = get_type_bytes(inputs[0].attr.type);
  int          out_type_bytes = get_type_bytes(outputs[0].attr.type);
  int          in_offset      = inputs[0].mem.offset / in_type_bytes;
  int          out_offset     = outputs[0].mem.offset / out_type_bytes;
  unsigned int elems          = inputs[0].attr.n_elems;

  // set kernel args
  int argIndex = 0;
  clSetKernelArg(kernel, argIndex++, sizeof(cl_mem), &inObject);
  clSetKernelArg(kernel, argIndex++, sizeof(cl_mem), &outObject);
  clSetKernelArg(kernel, argIndex++, sizeof(int), &in_offset);
  clSetKernelArg(kernel, argIndex++, sizeof(int), &out_offset);
  clSetKernelArg(kernel, argIndex++, sizeof(unsigned int), &elems);
```

```
  // set global worksize
  const size_t global_work_size[3] = {elems, 1, 1};

  // enqueueNDRangeKernel
  clEnqueueNDRangeKernel(queue, kernel, 1, NULL, global_work_size, NULL, 0, NULL,
NULL);

  // finish command queue
  clFinish(queue);

  // //cpu access data after sync to device
  // rknn_mem_sync(&outputs[0].mem, RKNN_MEMORY_SYNC_FROM_DEVICE);
  // // save output npy
  // char output_path[PATH_MAX];
  // sprintf(output_path, "%s/cpu_output%d.npy", ".", 0);
  // unsigned char* out_data = (unsigned
char*)outputs[0].mem.virt_addr+outputs[0].mem.offset;
  // save_npy(output_path, (float*)out_data, &inputs[0].attr);
  return 0;
}
```

4.2.1.4.1.4 Destroy Callback Function

It is used to destroy temporary buffers or input/output buffers of custom operators. The sample code to destroy the temporary buffer is as follows:

- CPU Operator

```
/**
* cpu kernel destroy callback for custom op
*/
int custom_op_destroy_callback(rknn_custom_op_context* op_ctx)
{
    printf("custom_op_destroy_callback\n");
    // clear tmp buffer
    free(op_ctx->priv_data);
    return 0;
}
```

- GPU Operator

```
/**
* opencl kernel destroy callback for custom op
*/
int relu_destroy_callback_gpu(rknn_custom_op_context* op_ctx)
{
    // clear tmp buffer
    printf("relu_destroy_callback_gpu\n");
    cl_mem* memObject = (cl_mem*)op_ctx->priv_data;
    clReleaseMemObject(memObject[0]);
    clReleaseMemObject(memObject[1]);
    free(memObject);
    return 0;
}
```

4.2.1.4.2 Register Custom Operator

After setting the rknn_custom_op structure, you need to call rknn_register_custom_ops() to register it in rknn_context. This interface supports registering multiple custom operators at the same time.

After completing the CPU's compute callback function, the code example for registering a CPU custom operator named "cstSoftmax" and "ArgMax" is as follows:

- CPU Operator

```
// CPU operators
rknn_custom_op user_op[2];
memset(user_op, 0, 2 * sizeof(rknn_custom_op));
strncpy(user_op[0].op_type, "cstSoftmax", RKNN_MAX_NAME_LEN - 1);
user_op[0].version = 1;
user_op[0].target  = RKNN_TARGET_TYPE_CPU;
user_op[0].init    = custom_op_init_callback;
user_op[0].compute = compute_custom_softmax_float32;
user_op[0].destroy = custom_op_destroy_callback;

strncpy(user_op[1].op_type, "ArgMax", RKNN_MAX_NAME_LEN - 1);
user_op[1].version = 1;
user_op[1].target  = RKNN_TARGET_TYPE_CPU;
user_op[1].init    = custom_op_init_callback;
user_op[1].compute = compute_custom_argmax_float32;
user_op[1].destroy = custom_op_destroy_callback;

ret = rknn_register_custom_ops(ctx, user_op, 2);
if (ret < 0) {
    printf("rknn_register_custom_ops fail! ret = %d\n", ret);
    return -1;
}
```

- GPU Operator

  For GPU operators, OpenCL kernel registration is supported in two ways: constant string or file path. When cl_source_size in the rknn_custom_op structure is equal to 0, cl_kernel_source represents the file path of the OpenCL kernel. When cl_source_size is greater than 0, cl_kernel_source represents the OpenCL kernel function string. The sample code of the OpenCL kernel with relu function saved as a string is as follows:

```
char* cl_kernel_source = "#pragma OPENCL EXTENSION cl_arm_printf : enable"
    "#pragma OPENCL EXTENSION cl_khr_fp16 : enable "
    " __kernel void relu_float(__global const float* input, __global float*
output, int "
    "in_offset, int out_offset, const unsigned int elems) "
    "{"
    "  int gid = get_global_id(0);"
    "  if (gid < elems) {"
    "    float in_value           = input[in_offset + gid];"
    "    output[out_offset + gid] = in_value >= 0.f ? in_value : 0.f;"
    "  }"
    "}"
    "__kernel void relu_half(__global const half* input, __global half* output,
int in_offset, "
    "int out_offset, const unsigned int elems)"
    "{"
    "  int gid = get_global_id(0);"
    "  if (gid < elems) {"
    "    half in_value            = input[in_offset + gid];"
    "    output[out_offset + gid] = in_value >= 0.f ? in_value : 0.f;"
    "  }"
    "}";
```

After completing the OpenCL kernel function and the GPU compute callback function, you can set the rknn_custom_op structure array and register the GPU operator. The sample code for registering the GPU operator is as follows:

```
// GPU operators
rknn_custom_op user_op[1];
memset(user_op, 0, sizeof(rknn_custom_op));
strncpy(user_op->op_type, "cstSoftmax", RKNN_MAX_NAME_LEN - 1);
user_op->version = 1;
user_op->target  = RKNN_TARGET_TYPE_GPU;
user_op->init    = relu_init_callback_gpu;
user_op->compute = compute_custom_relu_float32;
user_op->destroy = relu_destroy_callback_gpu;
#ifdef LOAD_FROM_PATH
user_op->cl_kernel_source = "./custom_op.cl";
user_op->cl_source_size   = 0;
#else
user_op->cl_kernel_source = cl_kernel_source;
user_op->cl_source_size   = strlen(cl_kernel_source);
#endif
strcpy(user_op->cl_kernel_name, "relu_float");

ret = rknn_register_custom_ops(ctx, user_op, 1);
if (ret < 0) {
    printf("rknn_register_custom_ops fail! ret = %d\n", ret);
    return -1;
}
```

Before registering and calling this interface, you must clearly define the op_type of the custom operator, prepare the operator information and configure the rknn_custom_op array. Each type of custom operator must call the registration interface once, and operators of the same type on the network must be called only once.

4.2.1.4.3 Model Inference

After registering all operators, you can use the general API or zero-copy API process to complete inference.

4.2.1.4.4 Connected Device Accuracy Analysis

The board-connected debugging function of custom operators requires that the rknn_server version >= 1.6.0.

During board-connected debugging, RKNN Server will use dlopen to open the user-compiled custom operator plugin library from a specific directory to obtain operator information. For registering custom operators via the plugin library approach, the user must implement a function named `get_rknn_custom_op`.

If the user needs to perform connected device accuracy analysis on a model containing custom operators, the specific steps are as follows:

1. Implement a `get_rknn_custom_op()` function and the necessary callback functions, and compile them into a library for the corresponding system. The compiled plugin library name must be prefixed with "librkcst_", for example, the library name is `librkcst_relu.so`.
2. Place the plugin in `/vendor/lib64/` (Android arm64-v8a) or `/usr/lib/rknpu/op_plugins` (Linux)
3. Use the Python interface of RKNN-Toolkit2 on the host or upper computer to perform connected device accuracy analysis.

The sample code for the `get_rknn_custom_op` function is as follows:

```
std::vector<std::string> get_all_plugin_paths(std::string plugin_dir)
{
  std::vector<std::string> plugin_paths;
  if (access(plugin_dir.c_str(), 0) != 0) {
    fprintf(stderr, "Can not access plugin directory: %s, please check it!\n",
plugin_dir.c_str());
  }

  DIR*           dir;
  struct dirent* ent;
  const char*    prefix = RKNN_CSTOP_PLUGIN_PREFIX; // All library file names
should start with this prefix

  if ((dir = opendir(plugin_dir.c_str())) != NULL) {
    while ((ent = readdir(dir)) != NULL) {
      if (ent->d_type == DT_REG) {
        const char* filename = ent->d_name;
        size_t      len      = strlen(filename);

        if (len > 10 && strncmp(filename, prefix, strlen(prefix)) == 0) {
          printf("Found plugin: %s file in %s\n", filename, plugin_dir.c_str());
          plugin_paths.push_back(plugin_dir + "/" + filename);
        }
      }
    }
    closedir(dir);
  } else {
    fprintf(stderr, "Unable to open directory");
  }

  return plugin_paths;
}

// the default path of the custom operator plugin libraries
std::string plugin_dir =
#if defined(__ANDROID__)
#  if defined(__aarch64__)
    "/vendor/lib64/";
#  else
    "/vendor/lib/";
#  endif // __aarch64__
#elif defined(__linux__)
    "/usr/lib/rknpu/op_plugins/";
#endif

std::vector<std::string> plugin_paths = get_all_plugin_paths(plugin_dir);
std::vector<void*> so_handles;
for (auto path : plugin_paths) {
printf("load plugin %s\n", path.c_str());
void* plugin_lib = dlopen(path.c_str(), RTLD_NOW);
char* error      = dlerror();
if (error != NULL) {
    fprintf(stderr, "dlopen %s fail: %s.\nPlease try to set 'export
LD_LIBRARY_PATH=$LD_LIBRARY_PATH:%s'\n",
            path.c_str(), error, plugin_dir.c_str());
    dlclose(plugin_lib);
    return -1;
}
printf("dlopen %s successfully!\n", path.c_str());
get_custom_op_func custom_op_func = (get_custom_op_func)dlsym(plugin_lib,
```

```
"get_rknn_custom_op");
error                            = dlerror();
if (error != NULL) {
    fprintf(stderr, "dlsym fail: %s\n", error);
    dlclose(plugin_lib);
    return -1;
}

rknn_custom_op* user_op = custom_op_func();
ret                     = rknn_register_custom_ops(ctx, user_op, 1);
if (ret < 0) {
    printf("rknn_register_custom_ops fail! ret = %d\n", ret);
    return -1;
}
so_handles.push_back(plugin_lib);
}
```

The plugin library has the following notes:

- A custom operator plugin library can only register one custom operator. If multiple custom operators need to be registered, multiple plugin libraries must be created.
- The name of the plugin library must start with "librkcst_" and end with .so.
- If the plugin library is implemented in C++ code, in order to correctly open the function symbols in the C interface, the following macros must be added before and after the function definition:

```
#ifdef __cplusplus
extern "C" {
#endif

//code

#ifdef __cplusplus
} // extern "C"
#endif
```

- If dlopen fails to load the plugin library, you need to check whether the `LD_LIBRARY_PATH` environment variable is set, and check whether the directory containing the plugin library is in the path specified by the environment variable.

4.2.2 Model Encryption Deployment

The RKNN SDK supports offline encryption of RKNN models and decryption at runtime.

> Model encryption is only one part of model protection and cannot completely eliminate the risk of cracking; users are advised to avoid direct third-party access to their models as much as possible and to strengthen overall protection at the system level.
>
> **Note**: The legacy `rknn.export_encrypted_rknn_model()` / `crypt_level` scheme is still compatible but no longer recommended. Please use the hybrid encryption scheme described in this section.

This scheme adopts a **Hybrid Encryption** architecture:

- **AES-256-CTR**: symmetric encryption, used to encrypt the model body (high performance, supports random-access decryption)，only support 256bit
- **RSA-OAEP**: asymmetric encryption, used to protect the AES key (secure key distribution)

Key System:

Key	Type	Owner	Purpose
User key	AES-256 symmetric key	Generated by user	Encrypt/decrypt the model body
RK key pair	RSA-2048 asymmetric key pair	Pre-installed by RK	Public key encrypts the user key; the private key built into the chip decrypts the user key

Encrypted file format:

Offset	Length	Content
0 – 63	64 bytes	RKNN file header (plaintext), with the RKCE encryption indicator written at offset 24–47
64+	body	AES-256-CTR encrypted model body

The key envelope file (`user_key_iv.enc`) has the format `RKKE header (16 bytes) + RSA-OAEP(AES key || AES IV)` .

4.2.2.1 ENCRYPTION ON PC/SERVER

For the conversion script, examples, and RK public key, refer to the encryption example in the rknpu2 project: `rknpu2/examples/rknn_encrypt_demo` .

Use `rknn_model_encrypt.py` to encrypt a converted `.rknn` model:

```
python3 rknn_model_encrypt.py --model model.rknn \
                                  --rk-pubkey rk_public_key.pem \
                                  [--output-dir ./encrypted] \
                                  [--aes-bits 256]
```

Output files:

File	Description
`model.rknn`	The encrypted model (plaintext RKNN header + RKCE indicator + AES-CTR ciphertext)
`model_key.bin`	The RSA-OAEP encrypted key envelope (with RKKE header), deployed alongside the model

> **Public key**: `rk_public_key.pem` is provided by Rockchip. After the user key is encrypted by the RK public key, only the RSA private key built into the chip can decrypt it.

4.2.2.2 LOADING ENCRYPTED MODELS ON BOARD

At runtime, the key envelope is passed in via `rknn_init_extend.user_key` , and `rknn_init()` performs decryption internally. The subsequent inference flow is identical to that of a plaintext

model. An encrypted model can be loaded either from a file path (size set to 0) or as an in-memory buffer.

```c
#include "rknn_api.h"

/* 1. Load the key envelope into memory */
size_t key_env_size = 0;
void*  key_env = load_file("model_key.bin", &key_env_size);
if (!key_env) {
    printf("load key envelope failed!\n");
    return -1;
}

/* 2. Pass the key envelope via rknn_init_extend.user_key */
rknn_init_extend      init_ext;
rknn_decrypt_key_mem  key_mem;
memset(&init_ext, 0, sizeof(rknn_init_extend));
key_mem.key_data  = key_env;
key_mem.key_size  = (uint32_t)key_env_size;
init_ext.user_key = &key_mem;

/* 3. rknn_init decrypts internally; subsequent usage is the same as a plaintext
model */
rknn_context ctx = 0;
int ret = rknn_init(&ctx, (void*)"model.rknn", 0, 0, &init_ext);
if (ret < 0) {
    if (ret == RKNN_ERR_KEY_INVALID)
        printf("key envelope invalid / RSA unwrap failed\n");
    else if (ret == RKNN_ERR_DECRYPT_FAILED)
        printf("AES-CTR model body decryption failed\n");
    else if (ret == RKNN_ERR_SOC_BIND_FAILED)
        printf("current SoC is not in the bound platform list\n");
    memset(key_env, 0, key_env_size);
    free(key_env);
    return -1;
}

/* 4. The user-side key memory can be released after rknn_init returns */
memset(key_env, 0, key_env_size);
free(key_env);

/* 5. Proceed with the normal inference flow: rknn_query / rknn_inputs_set /
rknn_run / ... */
```

> **Memory ownership**: The `rknn_decrypt_key_mem` structure and the buffer pointed to by `key_data` are owned by the caller. The runtime copies the key envelope inside `rknn_init()`, so the caller can release it after `rknn_init()` returns. It is recommended to zero out the key memory (`memset` to 0) before releasing it to reduce the risk of key leakage.

- Supported platforms: RK3562/RK3566/RK3568/RK3576/RK3588/RV1126B/RV1106/RV1103B

4.2.2.3 COMPATIBILITY OF ENCRYPTED MODELS WITH EXTERNAL MEMORY ALLOCATION/REUSE

When initializing the context, encrypted models are **only compatible with Internal memory reuse (external allocation of Internal memory)**: initialize with the `RKNN_FLAG_INTERNAL_ALLOC_OUTSIDE` flag and set the Internal memory through `rknn_set_internal_mem()`. Multiple models can reuse the same block of Internal memory. For details, see the Internal Memory Reuse section in Chapter 5.3.3.

Encrypted models are **not compatible** with the following features:

- Weight sharing: initialization with the `RKNN_FLAG_SHARE_WEIGHT_MEM` flag is not supported.
- External allocation of weights: setting external weight memory through `rknn_set_weight_mem()` is not supported.
- Context reuse: reuse of contexts through the `rknn_dup_context()` interface is not supported.

5. Troubleshooting and Optimization

5.1 Accuracy Troubleshooting

Model accuracy issue troubleshooting is generally carried out from two aspects: simulator accuracy troubleshooting and board-side runtime accuracy troubleshooting. Correct simulator inference results are the prerequisite for correct board-side runtime inference. Therefore, it is essential to first ensure that the simulator inference results are correct, and then troubleshoot board-side runtime accuracy issues. Overall, follow the progressive troubleshooting flow of "simulator FP16 accuracy → simulator quantization accuracy → board-connected accuracy → runtime accuracy", where the correctness of each preceding stage is the prerequisite for the correctness of the next stage.

Therefore, this chapter provides troubleshooting suggestions and solutions for the two aspects of **simulator accuracy troubleshooting** and **runtime accuracy troubleshooting**. The following figure shows the specific troubleshooting steps:

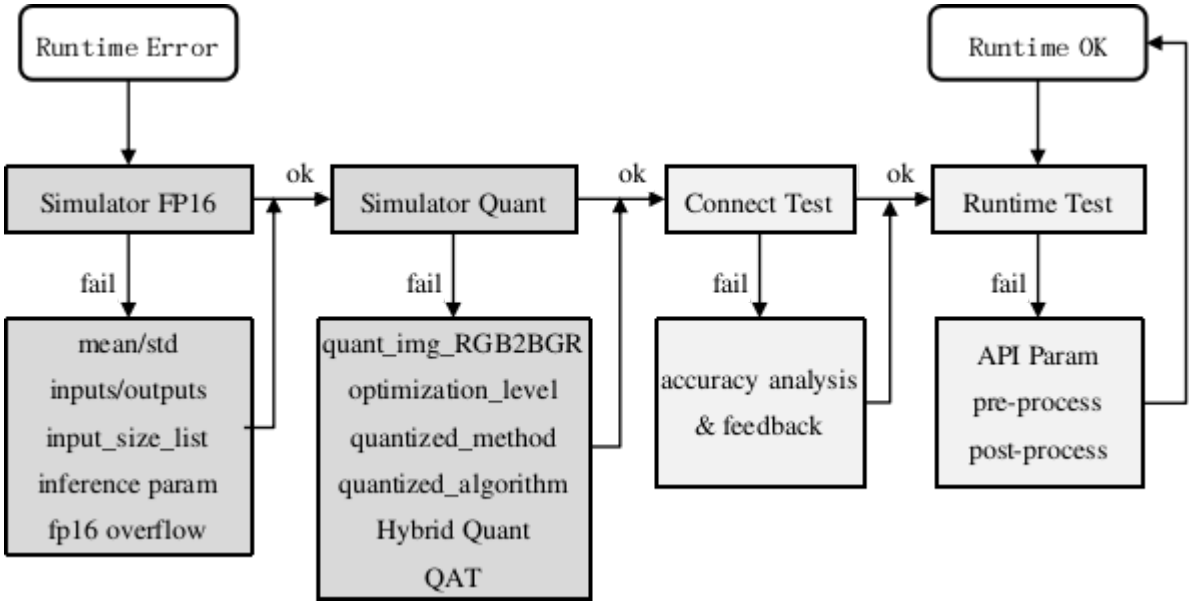


Figure 5-1 The step of accuracy analysis

- Simulator accuracy troubleshooting is mainly divided into two sub-steps: **simulator FP16 accuracy** troubleshooting and **simulator quantization accuracy** troubleshooting (the dark-colored boxes in the above figure).
- Runtime accuracy troubleshooting is mainly divided into two sub-steps: **board-connected accuracy** troubleshooting and **runtime accuracy** troubleshooting (the light-colored boxes in the above figure).

Cosine distance can be used as the basic criterion for judging accuracy, but the final impact of quantization on accuracy still needs to be validated on the dataset.

5.1.1 Simulator Accuracy Troubleshooting

Correct simulator inference results are the prerequisite for correct board-side runtime inference, so priority must be given to ensuring that the simulator inference results are correct.

Simulator inference on RKNN-Toolkit2 is divided into **FP16 inference** and **quantization inference**, depending on whether the model is quantized. Correct FP16 inference results are the prerequisite for correct quantization inference results. Therefore, when there are accuracy issues with quantization inference, first verify the correctness of FP16 inference, and then troubleshoot the accuracy issues of quantization inference.

5.1.1.1 SIMULATOR FP16 ACCURACY

RKNPU currently does not support FP32 calculations, so the simulator defaults to FP16 computation when quantization is not enabled. To convert the original model to an FP16 RKNN model, use the rknn.build() interface with the do_quantization parameter set False. Then, call rknn.init_runtime(target=None) and rknn.inference()interfaces for FP16 simulation inference and to obtain the output results.

If the FP16 inference output is incorrect, users can perform the following troubleshooting:

- Configuration Error

  The configuration information of the model is mainly concentrated in the rknn.config() interface. At the same time, there is also a small amount of configuration information in other APIs that may affect the accuracy of FP16. The main parameters are as follows:

  mean_values / std_values: Normalization parameters of the model. Generally, the input normalization operation of the original model is implemented in the model's input preprocessing. However, for RKNN models during inference, this normalization operation can be included (when quantization is enabled, the quantization correction dataset will undergo normalization first). Therefore, when the original model has a normalization step, ensure that these parameters are consistent with the normalization parameters used in the original model.

  input_size_list: If the input node shape information of the rknn.load_tensorflow(), rknn.load_pytorch() and rknn.load_onnx() interfaces is configured incorrectly, it will lead to incorrect inference results.

  inputs / outputs: If the input and output node names of the rknn.load_tensorflow() and rknn.load_onnx() interfaces are configured incorrectly, it will lead to incorrect inference results.

  inference interface parameters: The input parameters for the rknn.inference() interface mainly include inputs and data_format.

  - Generally in the Python environment, image data is read using cv2.imread(). In this case, it is important to note that the image format read by cv2.imread() is BGR. If the original model's input is BGR (such as in most caffe models), there is no need to adjust the RGB order. However, if the original model's input is RGB, users need to use cv2.cvtColor(img, cv2.COLOR_BGR2RGB) to convert the image data to RGB. Additionally, the shape dimensions of the image read by cv2.imread() are 3 dimensions, but the input shape of most models is 4 dimensions. Therefore, users need to use np.expand_dims(img, 0) to expand the input shape to 4 dimensions before passing it to the rknn.inference() interface. The layout of the image read by cv2.imread() is NHWC, and the default value of data_format is also NHWC, so there is no need to set the data_format parameter.

- If the model's input data is not read by cv2.imread(), it is necessary to know the layout of the input data and set the data_format parameter correctly. Also, ensure that the shape of the input data is consistent with the original model. If the input data is image data, ensure that its RGB order is consistent with the original model.

Checking parameter configurations is a crucial step and is the main reason many users encounter incorrect FP16 inference results. The specific troubleshooting steps are as follows:

a. Perform inference using the original model in the original inference framework and save the inference results.

b. Use the RKNN-Toolkit2 to convert and infer the original model. Use the same input data as in the previous step and set the FP16 inference mode (set do_quantization to False in rknn.build()). Also, the target parameter of rknn.init_runtime() is set to None to call the RKNN-Toolkit2 simulator for inference, and save the inference results.

c. Compare the results of the two inferences. If the results are relatively consistent (cosine distance can be used to judge consistency), it indicates that the above configurations are correct.

d. If the results are inconsistent, check whether the above parameters are correct.

If it is confirmed that the above parameter configurations are correct and the results are still inconsistent, it may be due to tensors in the model exceeding the FP16 expression range.

- Exceeding the FP16 Expression Range

  When converted from FP32 to FP16, the intermediate tensors of the model may encounter overflow issues. Since the typical data type for model inference is FP32, if tensors in the model during inference have values exceeding the FP16 expression range (-65504～65504), the tensor will overflow, resulting in abnormal model inference results.

  For overflow issues, users can use the rknn.accuracy_analysis(..., target=None) interface (refer to Model Accuracy Analysis) for simulator FP16 accuracy analysis. If the entire column or single column of simulator_error in the analysis results has an abnormal value (If words such as 'inf' appear), an FP16 overflow may occur. In this case, users can try modifying the model structure to ensure that all tensors in the model do not overflow in FP16 (such as adding some BN layers, etc.).

  If it is confirmed that the above parameter configurations are correct and there is no FP16 overflow, but the results are still inconsistent, it may be an issue with the internal implementation of the simulator. Please provide the reproduction file of the model to Rockchip NPU team for analysis and resolution.

5.1.1.2 SIMULATOR QUANTIZATION ACCURACY

After ruling out FP16 accuracy issues, the model can be quantized (when using the rknn.build() interface, set the do_quantization parameter to True), and then call the rknn.accuracy_analysis(..., target=None) interface (refer to Model Accuracy Analysis) to conduct simulator quantization accuracy analysis.

If in the analysis results, users find that the accuracy of the entire column of simulator_error has dropped significantly, and no layer shows a particularly large accuracy decrease in the single column of simulator_error, investigate mainly in the following areas:

- **Configuration errors**

Similar to configuration issues with FP16 inference, incorrect configurations can also lead to quantization inference accuracy problems. Therefore, on the basis of ensuring the correct configuration for FP16 inference, the following quantization configuration parameters still need to be checked.

quant_img_RGB2BGR: Indicates whether to perform the RGB2BGR operation when loading quantization images, typically used for caffe models. For more detailed information, refer to the description of the quant_img_RGB2BGR parameter. This parameter must be consistent with the channel order of the images used during training; an incorrect configuration can also cause a considerable decrease in quantization accuracy.

optimization_level: The choice of optimization level. The default is 3, which means speed priority. In this case, some optimization rules that are beneficial to performance but slightly affect accuracy are enabled. Lowering this configuration (e.g., changing it to 0) disables these optimization rules.

dataset: The quantization calibration dataset configuration for the rknn.build() interface, used to calculate appropriate quantization parameters (scale / zero_point) for each tensor during the quantization process. If a calibration dataset that differs significantly from the actual deployment scenario is selected, accuracy degradation may occur. In addition, having too many or too few calibration images will also affect accuracy (generally 20~200 images are chosen).

To check quantization parameter configuration issues specifically, generally follow these steps:

a. Directly perform quantization inference, and then check the inference results and compare them with the inference results of the original model under the original inference framework. If the difference in results is not very large, it can be considered that the quant_img_RGB2BGR and dataset parameters are basically correct.

b. If the difference in results is still obvious:

- If the input image format of the original model is BGR (common in caffe models), modify quant_img_RGB2BGR to True. As for the RGB order of the model input, it can actually be determined from the input data processing code in the previous FP16 inference accuracy verification step.
- First use a single image for quantization (leave only one line in dataset.txt), and also use this image for inference. If the accuracy of this single image improves considerably, it indicates that the previously selected quantization calibration dataset was a poor choice, and images that better match the deployment scenario can be re-selected (if the improvement is not obvious, it may not be a problem with the dataset).
- If only one image was originally used for quantization (there is only one line in dataset.txt), try using more images for quantization, which can be increased to about 20~200 images.

After the above configuration checks, there should not be a case where the quantization results are completely wrong. If completely wrong results occur, please recheck the above configurations. If the accuracy of the model is still not sufficient after confirming that the configurations are correct, try modifying related configurations such as the quantization method and algorithm.

- **Quantization methods and quantization algorithms**

  Some models are inherently unfriendly to quantization. In such cases, try switching to different quantization methods and quantization algorithms. There are mainly three quantization methods: layer, channel, and group (group is only effective for linear layers, see 2.2.1.3 Quantization Granularity), which can be set through the quantized_method parameter in the rknn.config() interface (default is channel). There are mainly four quantization algorithms: normal, kl_divergence, mmse, and gdq (see 2.2.3.2 Internal Quantization Algorithms), which can be set through the quantized_algorithm parameter in the rknn.config() interface (default is normal). The steps are as follows:

  a. If the layer quantization method was originally used, switch to the channel quantization method. In general, the accuracy of the channel quantization method is much higher than that of the layer quantization method.

  b. If the quantization method is already channel but the accuracy still cannot meet the requirements, change the quantization algorithm from normal to kl_divergence or mmse. This will significantly increase the quantization time, but delivers better accuracy than normal, while the runtime performance is not affected.

  The characteristics of the quantization algorithms and the recommended amount of calibration data are summarized as follows:

Quantization algorithm	Characteristics	Recommended amount of calibration data
normal (default)	Conventional calibration, fast	20~100 images
kl_divergence	Good improvement when the feature distribution is uneven, slightly more time-consuming than normal	20~100 images
mmse	Iteratively searches the quantization range, usually higher accuracy, but greatly increases time and memory	20~50 images
gdq	Only effective under w4a16 and w8a16, can effectively improve weight quantization accuracy	More than 200 images

  It is recommended to start with normal, and then try kl_divergence, mmse, or gdq if the accuracy is unsatisfactory; the quantization algorithm only affects the model conversion stage and does not affect the on-board runtime performance.

  If after using the above methods, the accuracy of the entire column of simulator_error in the analysis results is still not good, and some layers in the single column of simulator_error show a considerable accuracy drop, it may be that the weight value distribution of these layers is poor, causing a large accuracy decrease after quantization. For example, when the weight distribution of a Conv layer is highly uneven, consider using **hybrid quantization** to further improve the model accuracy. The steps are as follows:

  a. First use the accuracy analysis interface to analyze the accuracy, find out the layers that cause the considerable accuracy decrease, and record the output tensor names of the corresponding layers. (Note that, because errors accumulate layer by layer, the earlier a layer is, the greater its impact on the final accuracy. Therefore, consider not only the accuracy loss in the single column of simulator_error, but also the position of the layer in the model.)

b. Use the hybrid quantization method and write the tensor names obtained in the previous step into the hybrid quantization configuration file (refer to Hybrid Quantization).

c. Complete the hybrid quantization steps and test the accuracy (the accuracy analysis interface can continue to be used to observe accuracy changes).

Generally, the accuracy of the model can be improved after hybrid quantization. If the improvement is not obvious or not sufficient, try applying hybrid quantization to more layers, but this will also cause a decrease in inference performance. Therefore, users need to balance accuracy and speed when applying hybrid quantization. Another special approach is that, when the Op with the accuracy drop is in the last layer, users can also choose to move the execution of that Op into post-processing, which likewise effectively avoids the accuracy problem of that layer.

- **QAT (Quantization-Aware Training)**

  If the accuracy is still insufficient after hybrid quantization, or if the accuracy is sufficient but the performance cannot meet the requirements due to hybrid quantization, try using quantization-aware training to retrain the model, and export the model with quantization parameters (e.g., in onnx/pt/pb/tflite formats). For more information on quantization-aware training, please refer to Quantization-Aware Training (QAT).

5.1.2 Runtime Accuracy Troubleshooting

When the accuracy of the simulator is normal, abnormal inference results may still occur when deploying the board-side C API. There are generally three reasons for this problem. The first is caused by a bug in the runtime of the board. The second is incorrect usage of the RKNPU2 C API when programming the interface. The third is incorrect pre-processing or post-processing of the model.

When encountering such problems, users can first use the board-connected function to quickly check whether it is caused by a bug in the runtime of the board. If there are no issues with the board-connected, then check the C API deployment problem.

5.1.2.1 BOARD-CONNECTED ACCURACY

1. After configuring the board-connected debugging environment (refer to NPU Device Environment Setup for the environment setup), connect the board to the computer via USB, and then use RKNN-Toolkit2 to perform board-connected inference (set the target parameter of rknn.init_runtime(), Such as target='rk3566'), and check whether the inference results are roughly correct (because the simulator does not strictly simulate NPU hardware, the results may not be completely consistent with the simulator).
2. If the inference results in the above steps are significantly different from the simulator inference results, it can be initially determined that there is a bug in the runtime of the board. In this case, users can use the accuracy analysis interface (refer to Model Accuracy Analysis) to perform board-connected accuracy analysis (use rknn.accuracy_analysis() interface, and set the target parameter, such as target='rk3566'). After the accuracy analysis is completed, the analysis results of each layer will be output.
3. Check the single_sim column of runtime_error in the analysis results. If its cos cosine distance is low or euc Euclidean distance is high (displayed in yellow or red), resulting in an

increasing difference between the entire column of runtime_error and the entire column of simulator_error, it is possible that the runtime has accuracy loss or anomalies when implementing this layer. In this case, the analysis results and reproduced models can be fed back to the Rockchip NPU team for repair.

5.1.2.2 RUNTIME ACCURACY

If there is no problem with the board-connected accuracy but the accuracy is still problematic, the problem may lie in the C/C++ code itself that the user uses to call the RKNN C API for programming. In this case, the user needs to carefully check whether the interface configuration of the RKNN C API is correct, and whether the pre-processing and post-processing flows of the model are correct (they need to be completely consistent with the flows on the simulator side). Users can check by following these steps:

1. **Check input configuration and data**

   Check whether the input of the C API is configured correctly. For example, if RKNN-Toolkit2 has already configured the mean and variance when converting the RKNN model, no normalization is needed in the C/C++ code. For 3-channel input, the channel order is consistent with the input channel set during model training; for 4-dimensional input shapes, fmt=NHWC; for non-4-dimensional input, fmt=UNDEFINED. If users use the general API, the size of the input buffer is equal to the number of input tensor elements * the number of bytes of each element. If users use the zero-copy API, refer to 3.1.6 RKNN Runtime Zero-Copy Interface for the memory size and input data format created by the rknn_create_mem interface.

   After confirming that the configuration is correct, users need to check the data of the input layer. Users can set RKNN_LOG_LEVEL=5 before running the application, and then run the application. The layer-by-layer results during inference will be saved as numpy format files in the /data/dumps (Android system) or /userdata/dumps (Linux system) directory. Check whether the numpy file containing the InputOperator field meets expectations. If users use the general API, it is the result of input normalization; if users use the zero-copy API, it is the data before normalization.

2. **Check output configuration and data**

   After ensuring that the input is correct, check whether the output in the code is configured correctly. For example, if users use the general API, when want_float=1 is set, the output is a float32 type result; when want_float=0 is set, the output is a quantized data type or float16 type (non-quantized data type). If users use the zero-copy API, refer to 3.1.6 RKNN Runtime Zero-Copy Interface for the memory size and input data format created by the rknn_create_mem() interface.

   To check the data of the output layer, similarly, after the above layer-by-layer numpy files are generated, open the numpy file containing the OutputOperator field and check whether the data is correct. If the input results are confirmed to be correct but the output is still wrong, the Runtime may have a problem handling specific input/output types. In this case, feed the analysis results and a reproduction model back to the Rockchip NPU team for repair.

5.2 Performance Analysis and Optimization

When deploying a model, after getting the results running correctly, users will have further performance optimization needs. This section describes how to tune the model from the perspective of the complete model performance optimization flow. It then elaborates on the operations most commonly used by users: model performance analysis, graph-level optimization, and operator-level optimization. The complete optimization flow is shown in the following figure:

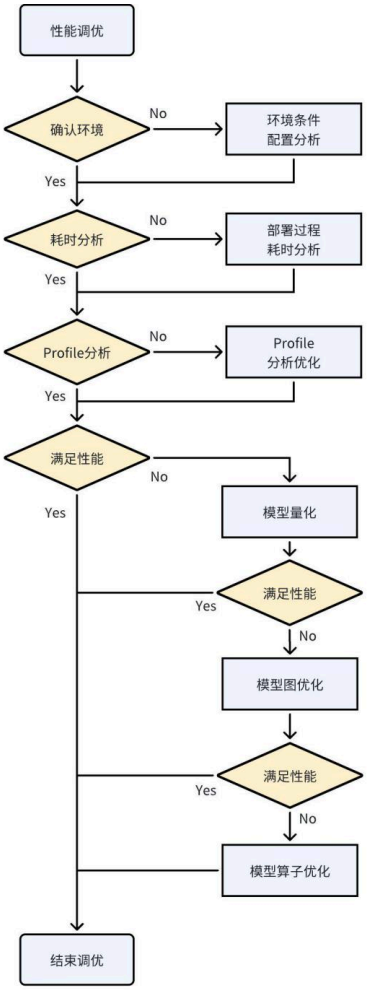


Figure 5-2 Model performance optimization flow

5.2.1 Process of Model Performance Optimization Analysis

5.2.1.1 CHECKING THE RUNTIME ENVIRONMENT

At the beginning of all performance analysis and optimization, the baseline of the test environment should be established first. Only when the baselines are the same is the measured performance data meaningful. For example, the inference performance of the same model tested without fixed frequencies fluctuates considerably, so the time consumption of a batch of tests cannot be taken as the average single-frame time.

There are several aspects to consider when querying and setting the conditions and configuration of the test environment:

- **Querying and setting the CPU, DDR, and NPU frequencies**

The fixed frequency directly affects the execution speed; the higher the frequency, the relatively better the performance. The relationship between frequency changes and performance improvement is not linear. Increasing the frequency relatively increases power consumption as well. Refer to the following link for fixed frequency commands: https://github.com/airockchip/rknn_model_zoo/blob/main/scaling_frequency.sh

Alternatively, you can simply use the following command to set the performance mode:

```
echo performance | tee $(find /sys/ -name *governor) /dev/null || true
```

- **Checking the NPU kernel driver version**

The performance optimization items of some features or operators are related to the kernel driver version, and newer kernel driver versions can apply the latest underlying optimization implementations. Therefore, please check whether a relatively recent kernel driver version is in use. Currently, it is recommended to update to a version later than 0.9.2. This step is not a mandatory update.

The commands for checking the kernel driver version are as follows:

```
cat /sys/kernel/debug/rknpu/version     # for
RK3566/RK3568/RK3588/RK3562/RK3576
cat /proc/rknpu/version                         # for RV1106 series/RV1103
series/RV1126B
```

- **Checking the NPU load**

The NPU load is the percentage of time the NPU spends executing tasks within a unit of time. The load reflects how busy the NPU is. If the queried load is low, it indicates that the NPU is waiting for task submissions for a longer time, and it is necessary to check the time consumed by data input/output copies, the optimization of pre-processing and post-processing in the application, and so on. Alternatively, use a multi-threaded processing approach in the application to increase the NPU load.

Also note that the NPU load does not represent the actual MAC utilization. MAC utilization reflects the execution efficiency of the running tasks in the NPU hardware units.

The commands for querying the NPU load are as follows:

```
cat /sys/kernel/debug/rknpu/load
# or
cat /proc/debug/rknpu/load
```

5.2.1.2 DEPLOYMENT TIME ANALYSIS

The entire deployment process can be divided into three aspects in terms of time consumption: user application time, input-output data copy time, and model inference time. Analyzing the time distribution of each step can help identify optimization priorities. The time consumption of these steps can be measured by timestamping in the application.

- User application time:

User application time mainly refers to the non-NPU-related time during the inference process. Typically, it includes pre-processing, post-processing, and the execution time of logical code. Users have full control over this part. If the application time is significantly high in proportion to the overall time, besides optimizing and simplifying the code, user can try to accelerate some operations using dedicated hardware.

For example, matrix multiplication and addition operations can be accelerated by using the Matmul API to invoke NPU-assisted computation. Similarly, image scaling operations can be accelerated by invoking the interfaces of RGA.

- Input-output copy time:

  When using the general API, there is a copy time involved between the input/output memory and the NPU memory. This time can be printed when calling the general API. The copy time depends on the performance of DDR and CPU. When the input-output data is small, the copy time is low. However, when the data is large, it's time can't be ignored. Therefore, it is recommended to use zero-copy APIs.

  When using zero-copy APIs, the input-output memory is directly accessed by the NPU. For detailed usage of zero-copy interfaces, please refer to RKNN Runtime Zero-Copy Interface.

- Inference time:

  The time consumed by the NPU for inference is influenced by factors such as the size of the model and the version of the compiler optimization. The reasoning time of RKNN's LOG printing is different at different RKNN_LOG_LEVEL levels because LOG printing takes a certain amount of time.

  Generally, when viewing the time taken for single-frame inference, the LOG level is set to 1, and the average is taken after running multiple times.

5.2.2 Performance Analysis

5.2.2.1 OBTAIN PROFILE INFORMATION

When you need to know the layer-by-layer time consumption of model inference, you can enter the following command before running the program to print detailed information:

```
export RKNN_LOG_LEVEL=4
./run_rknn_test ./test.rknn ./input.jpg
```

If it is on the Android platform, you can obtain detailed logs through the logcat command after running.

If you are using rknn-toolkit2, you can use the following method to get the time consumption of each layer:

```
rknn.init_runtime(target=platform, perf_debug=True)
rknn.eval_perf()
```

The performance analysis report information is as follows (only the performance-related parts are excerpted)

Operator Time Consuming Ranking Table

OpType	CallNumber	CPUTime(us)	GPUTime(us)	NPUTime(us)	TotalTime(us)	TimeRatio(%)
ConvLeakyRelu	7	0	0	4584	4584	53.77%
MaxPool	6	0	0	2273	2273	26.66%
Conv	2	0	0	846	846	9.92%
ConvAdd	1	0	0	511	511	5.99%
Split	1	0	0	152	152	1.78%
LeakyRelu	1	0	0	68	68	0.80%
OutputOperator	1	62	0	0	62	0.73%
InputOperator	1	29	0	0	29	0.34%

Figure 5-3 Performance analysis report

ID	OpType	DataType	Target	InputShape	OutputShape	DDRCycles	NPUCycles	MaxCycles	Time(us)
0	InputOperator	UINT8	CPU	\	(1,3,480,640)	0	0	0	9
1	ConvExSwish	UINT8	NPU	(1,3,480,640),(32,3,3,3),(32)	(1,32,240,320)	794117	1382400	1382400	2805
2	ConvExSwish	INT8	NPU	(1,32,240,320),(64,32,3,3),(64)	(1,64,120,160)	665207	691200	691200	1605
3	ConvExSwish	INT8	NPU	(1,64,120,160),(32,64,1,1),(32)	(1,32,120,160)	332036	153600	332036	832
4	ConvExSwish	INT8	NPU	(1,64,120,160),(32,64,1,1),(32)	(1,32,120,160)	332036	153600	332036	788
5	ConvExSwish	INT8	NPU	(1,32,120,160),(32,32,1,1),(32)	(1,32,120,160)	248988	153600	248988	722
6	ConvExSwish	INT8	NPU	(1,32,120,160),(32,32,3,3),(32)	(1,32,120,160)	250087	345600	345600	768
7	Add	INT8	NPU	(1,32,120,160),(1,32,120,160)	(1,32,120,160)	329574	0	329574	298
8	Concat	INT8	NPU	(1,32,120,160),(1,32,120,160)	(1,64,120,160)	494407	0	494407	453
9	ConvExSwish	INT8	NPU	(1,64,120,160),(64,64,1,1),(64)	(1,64,120,160)	497258	307200	497258	1427
10	ConvExSwish	INT8	NPU	(1,64,120,160),(128,64,3,3),(128)	(1,128,60,80)	401854	691200	691200	962
11	ConvExSwish	INT8	NPU	(1,128,60,80),(64,128,1,1),(64)	(1,64,60,80)	167682	76800	167682	405

Figure 5-4 Performance analysis report

You can quickly locate the desired information from the overall performance profile and the layer-by-layer performance profile, and formulate subsequent optimization strategies with different focuses based on the data. After obtaining the profile, you can perform the following analyses: analyze the layer-by-layer time consumption to find the highly time-consuming operators; analyze the impact of non-NPU operators; analyze the performance bottlenecks of NPU operators. These are discussed in detail below.

5.2.2.2 ANALYZING THE TIME-CONSUMING LAYER BY LAYER

As shown in the figure below, you can find the highly time-consuming operators from the Time column and optimize them first. You can also look at the Op Type column to find out whether the highly time-consuming operators belong to the same OpType, so that they can be optimized uniformly.

ID	OpType	DataType	Target	InputShape	OutputShape	DDRCycles	NPUCycles	MaxCycles	Time(us)
117	Sigmoid	INT8	NPU	(1,83,60,80)	(1,83,60,80)	186625	0	186625	541
118	ConvRelu	INT8	NPU	(1,128,60,80),(128,128,3,3),(128)	(1,128,60,80)	327096	1382400	1382400	1595
119	ConvRelu	INT8	NPU	(1,128,60,80),(128,128,3,3),(128)	(1,128,60,80)	327096	1382400	1382400	1596
120	Conv	INT8	NPU	(1,128,60,80),(4,128,1,1),(4)	(1,4,60,80)	103405	19200	103405	150
121	Conv	INT8	NPU	(1,128,60,80),(1,128,1,1),(1)	(1,1,60,80)	103345	19200	103345	150
122	Sigmoid	INT8	NPU	(1,1,60,80)	(1,1,60,80)	32181	0	32181	142
...									
144	Mul	INT8	NPU	(1,83,60,80),(1,1,60,80)	(1,83,60,80)	319655	0	319655	414
145	ReduceMax	INT8	CPU	(1,83,60,80)	(1,1,60,80)	0	0	0	7441
146	ArgMax	INT8	CPU	(1,83,60,80)	(1,1,60,80)	0	0	0	7389
147	Reshape	INT64	CPU	(1,1,60,80),(3)	(1,60,80)	0	0	0	69
148	OutputOperator	INT64	CPU	(1,60,80)	\	0	0	0	13

Figure 5-5 Performance analysis of highly time-consuming operators

It is worth noting that a highly time-consuming operator is not necessarily an inefficient operator. Some operators consume a large amount of computing power. When the MAC utilization of a highly time-consuming operator is very high, you should consider whether the size/scale of this operator can be reduced to decrease the time consumption. However, when highly time-

consuming operators with very low utilization appear, these operators are the focus of optimization.

5.2.2.3 ANALYZE THE IMPACT OF CPU OPERATORS

As shown in the figure below, you can see that some highly time-consuming operators run on the CPU. Converting these CPU operators to run on the NPU can greatly mitigate the impact of their high time consumption. Generally speaking, most of users' performance optimization problems are solved after converting the CPU operators to NPU. Therefore, pay special attention to the time consumption of CPU operators.

Operator Time Consuming Ranking Table

OpType	CallNumber	CPUTime(us)	GPUTime(us)	NPUTime(us)	TotalTime(us)	TimeRatio(%)
ConvRelu	125	0	0	54613	54613	42.09%
ConvExSwish	35	0	0	20047	20047	15.45%
ReduceMax	7	16104	0	0	16104	12.41%
ArgMax	7	14833	0	0	14833	11.43%
Concat	36	0	0	10464	10464	8.06%
Conv	25	0	0	3685	3685	2.84%
exSoftmax13	1	0	0	1916	1916	1.48%
Resize	11	0	0	1893	1893	1.46%
Sigmoid	12	0	0	1595	1595	1.23%
Reshape	14	388	0	699	1087	0.84%
Mul	7	0	0	1065	1065	0.82%
Add	7	0	0	1046	1046	0.81%
MaxPool	9	0	0	784	784	0.60%
OutputOperator	32	569	0	0	569	0.44%
ConvSigmoid	1	0	0	40	40	0.03%
InputOperator	1	9	0	0	9	0.01%

Figure 5-6 CPU operator performance analysis

Generally speaking, there are several reasons why an operator runs on a non-NPU:

- The operator size exceeds the limit (check the operator size limit in the OpList document)
- The operator does not yet support running on the NPU (check whether OpList supports this operator; you can submit an issue on the Github project)
- NPU hardware limitations make it unsupported (check whether the algorithm can be equivalently replaced with another implementation supported by the NPU)

5.2.2.4 ANALYSIS OF NPU OPERATOR PERFORMANCE BOTTLENECKS

When considering the high time consumption of NPU operators, you can use the three columns DDR Cycles/NPU Cycles/Total Cycles to determine whether the theoretical bottleneck of the operator's time consumption is a bandwidth bottleneck or a computing power bottleneck. The DDR Cycles here is data converted according to the NPU frequency, referring to the number of cycles required, at the NPU frequency, for the operator of that layer to read and write data, so it can be directly compared with the NPU Cycles.

As shown in the figure below:

ID	OpType	DataType	Target	InputShape	OutputShape	DDR Cycles	NPU Cycles	Total Cycles	Time(us)	MacUsage(%	Task Number	Regcmd Size	RV
0	InputOperatc	UINT8	CPU	\	(1,3,300,300)	0	0	0	12	\	0	0	
1	Conv	UINT8	NPU	(1,3,300,300)	(1,3,300,300)	261543	1582	261543	585	0.3	0	0	1
2	Split	INT8	CPU	(1,3,300,300)	(1,1,300,300),(1	0	0	0	6677	\	0	0	
3	Conv	INT8	NPU	(1,1,300,300)	(1,8,150,150)	275553	17226	275553	1556	1.23	0	0	1
4	Conv	INT8	NPU	(1,1,300,300)	(1,8,150,150)	275553	17226	275553	1505	1.27	0	0	1
5	Conv	INT8	NPU	(1,1,300,300)	(1,8,150,150)	275553	17226	275553	1573	1.22	0	0	1
6	Concat	INT8	CPU	(1,8,150,150)	(1,24,150,150)	0	0	0	4684	\	0	0	1
7	Conv	INT8	NPU	(1,24,150,15(	(1,64,150,150)	329723	67500	329723	684	10.96	0	0	2
8	Clip	INT8	NPU	(1,64,150,15(	(1,64,150,150)	438387	0	438387	729	\	0	0	2
9	MaxPool	INT8	NPU	(1,64,150,15(	(1,64,75,75)	275490	0	275490	725	\	0	0	1
10	Conv	INT8	NPU	(1,64,75,75),(	(1,64,75,75)	110676	45000	110676	281	17.79	0	0	
11	Clip	INT8	NPU	(1,64,75,75),(	(1,64,75,75)	109676	0	109676	275	\	0	0	
12	Conv	INT8	NPU	(1,64,75,75),(	(1,192,75,75)	253595	1215000	1215000	1456	92.72	0	0	1
13	Clip	INT8	NPU	(1,192,75,75)	(1,192,75,75)	328817	0	328817	599	\	0	0	2
14	MaxPool	INT8	NPU	(1,192,75,75)	(1,192,38,38)	206599	0	206599	535	\	0	0	1
15	AveragePool	INT8	CPU	(1,192,38,38)	(1,192,38,38)	0	0	0	16159	\	0	0	
16	Conv	INT8	NPU	(1,192,38,38)	(1,32,38,38)	50564	17328	50564	192	10.03	0	0	
17	Clip	INT8	NPU	(1,32,38,38),(	(1,32,38,38)	14169	0	14169	121	\	0	0	
18	Conv	INT8	NPU	(1,192,38,38)	(1,64,38,38)	58609	34656	58609	193	19.95	0	0	
19	Clip	INT8	NPU	(1,64,38,38),(	(1,64,38,38)	28233	0	28233	132	\	0	0	
20	Conv	INT8	NPU	(1,64,38,38),(	(1,96,38,38)	43855	155952	155952	304	57	0	0	
21	Clip	INT8	NPU	(1,96,38,38),(	(1,96,38,38)	42297	0	42297	152	\	0	0	
22	Conv	INT8	NPU	(1,96,38,38),(	(1,96,38,38)	55095	233928	233928	399	65.14	0	0	
23	Clip	INT8	NPU	(1,96,38,38),(	(1,96,38,38)	42297	0	42297	153	\	0	0	
24	Conv	INT8	NPU	(1,192,38,38)	(1,64,38,38)	58609	34656	58609	144	26.74	0	0	

Figure 5-7 NPU operator performance bottleneck analysis

- When the DDR Cycles of the third layer are much larger than the NPU Cycles, it means that the number of Cycles this layer spends reading and writing data is much greater than the number of Cycles required for the operation, so the bottleneck of this Conv comes from bandwidth.
- When the DDR Cycles of the twelfth layer are much smaller than the NPU Cycles, it means that the number of Cycles this layer spends reading and writing data is much less than the number of Cycles required for the operation, so the bottleneck of this Conv comes from computing power.

Currently, the NPU Cycles column mainly displays the Cycles required by Conv; other operator types will be gradually added later.

5.2.3 Quantitative acceleration

Quantization can significantly reduce the calculation amount of the model and save bandwidth consumption. On the other hand, the INT8 computing power of NPU is much greater than that of FP16, so quantization can greatly improve the model running speed. We recommend using quantitative model deployment. For details on how to use model quantification, see Introduction to Quantization.

5.2.4 Graph optimization

Graph optimization of the model is the easiest way to coordinate and optimize the model from an overall perspective. After analyzing the operators or subgraphs that take a high proportion of time, we can transform the graph in many different ways to achieve optimization. Graph optimization mainly aims at saving redundant operators, converting non-NPU OPs into NPUs, and transforming hardware-oriented high-efficiency operators. These goals may sometimes be contradictory. For example, in order to convert non-NPU OP to NPU, a few additional operators may be needed. This may seem to violate the goal of saving redundant operators, but the overall inference performance improvement is beneficial. meaningful.

In the RKNN-Toolkit2 toolchain, the software stack already performs graph optimization to a certain extent during the model conversion process. However, this process is not omnipotent and perfect. Some scenarios that have not been considered will still have redundant operations. Users can perform pre-emptive graph optimization based on some of the ideas introduced in this section. The following is only an introduction to each optimization method. It is not mandatory and needs to be used flexibly in actual scenarios.

5.2.4.1 CONVERT NON-NPU OP TO NPU OP

For non-NPU ops, some equivalent graph transformations can be made to replace them with operators supported by the NPU, so as to achieve the optimization purpose of converting them to NPU.

For example, in the figure below, taking the shufflenetv2_0.5 model as an example, the channel shuffle operation in it is replaced with an approximate convolution. The weight values are 0/1, which can achieve the effect of rearranging data. If the Transpose and Reshape operations cannot be implemented on the NPU, these operators can be consolidated into a Conv operator to achieve the data rearrangement.

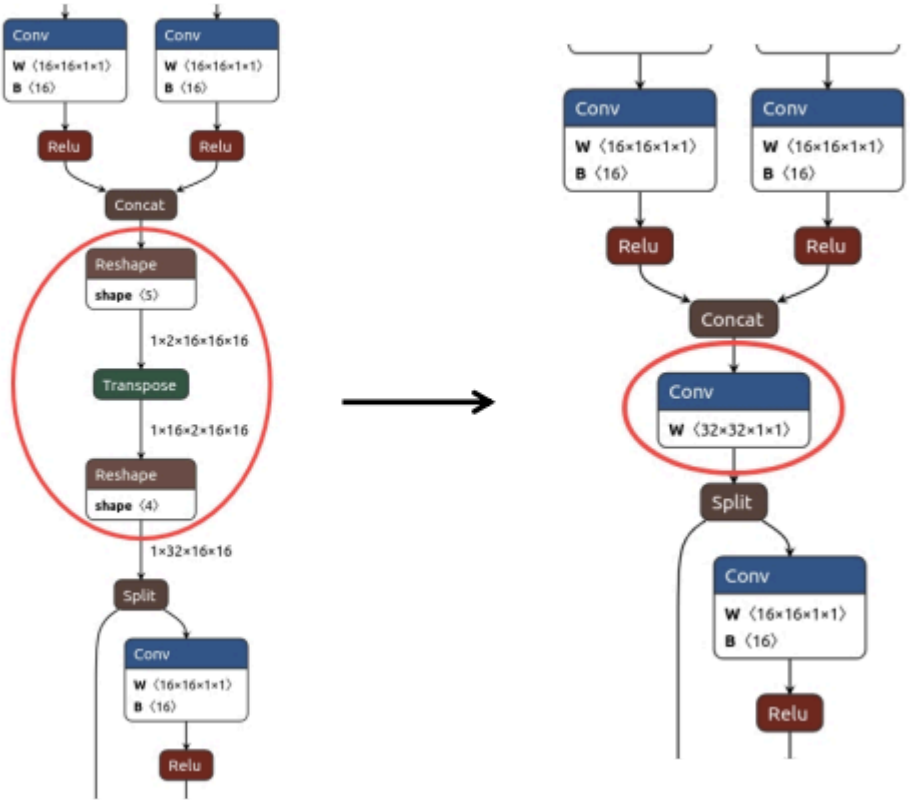


Figure 5-8 Convolution rearrangement

5.2.4.2 USE HARDWARE FUSE FEATURE TO DESIGN NETWORK OR GRAPH OPTIMIZATION

The NPU supports fusing some operator combinations. The operation flow of some operators can be appropriately adjusted to adapt to the NPU fusion rules and achieve operator fusion optimization.

Although the RKNN software stack performs a certain degree of graph optimization, it cannot fully cover all situations. In some special cases, there may be graph structures that could

theoretically be fused and simplified, but that the final graph optimization fails to fuse. Users can manually adjust the algorithm to quickly solve this optimization problem.

For example, in the figure below, without changing the correctness of the calculation, by adjusting the order of the Transpose and Clip operators, Conv and Clip are fused in the operation, improving performance.

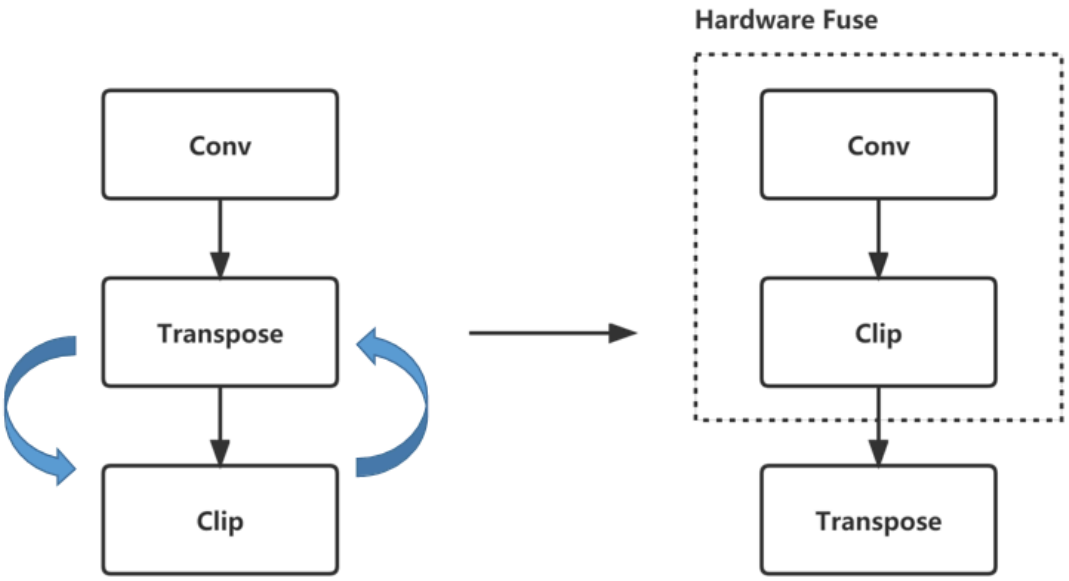


Figure 5-9 Operator graph optimization/fusion

Design with the fusion rules. The fusion rules are as follows:

Supported fusion rules	**Fusion rules supported by future plans**
Conv+Relu	Activation+Add(Mul)
Conv+PRelu(LeakyRelu)	Add(Mul)+Activation
Conv+Clip	Conv+Mul
Conv+Sigmoid(Tanh/Elu/Silu...)	Conv+Activation+Mul
Conv+Add	Conv+Activation+Pooling
Conv+Activation+Add	Conv+Activation+Add(Mul)+Pooling

5.2.4.3 SUBGRAPH TRANSFORMATION

When analyzing a certain graph region, the algorithm design for implementing a certain behavior sometimes pursues clarity of expression without considering the performance impact on the actual deployment, which easily produces complex and redundant graph structures. Through algorithmic equivalence, the subgraph of a certain region can be converted into a single OP, reducing the operator calculation steps and achieving the optimization purpose.

For example, the figure below shows the equivalent graph transformation of Yolov5-nano, which consolidates several complex Slice data extractions into a Conv, forming a new Conv and greatly simplifying the graph structure.

Solution source: https://github.com/ultralytics/yolov5/issues/4825

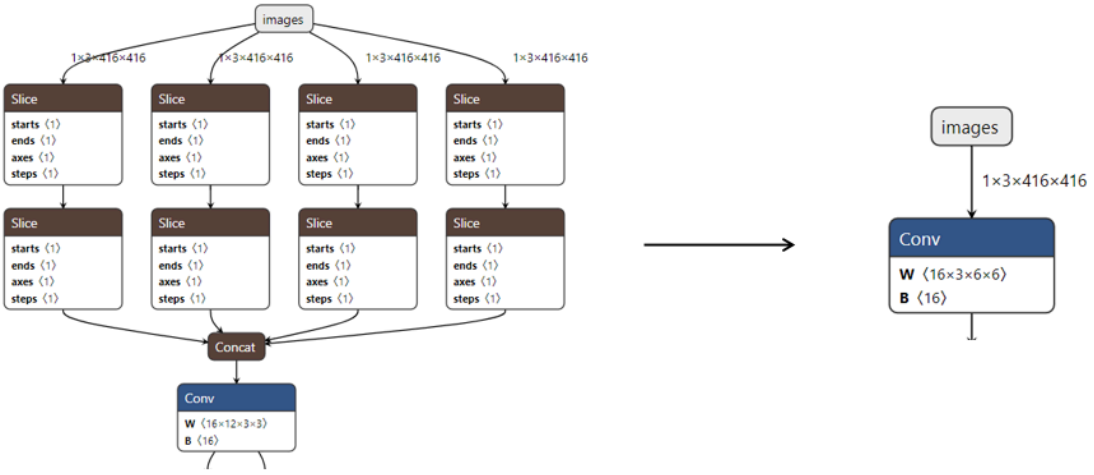


Figure 5-10 Operator equivalent graph transformation

5.2.4.4 "COMBINING SIMILAR TERMS" AND "EXTRACTING COMMON FACTORS"

When certain operators are computed multiple times in succession, they can be simplified and merged into the same operator, such as Reshape, Transpose, Slice, some Add/Mul/Sub/Div, etc.

For example, in the figure below, similar operators can be merged by simply adjusting the order of the graph.

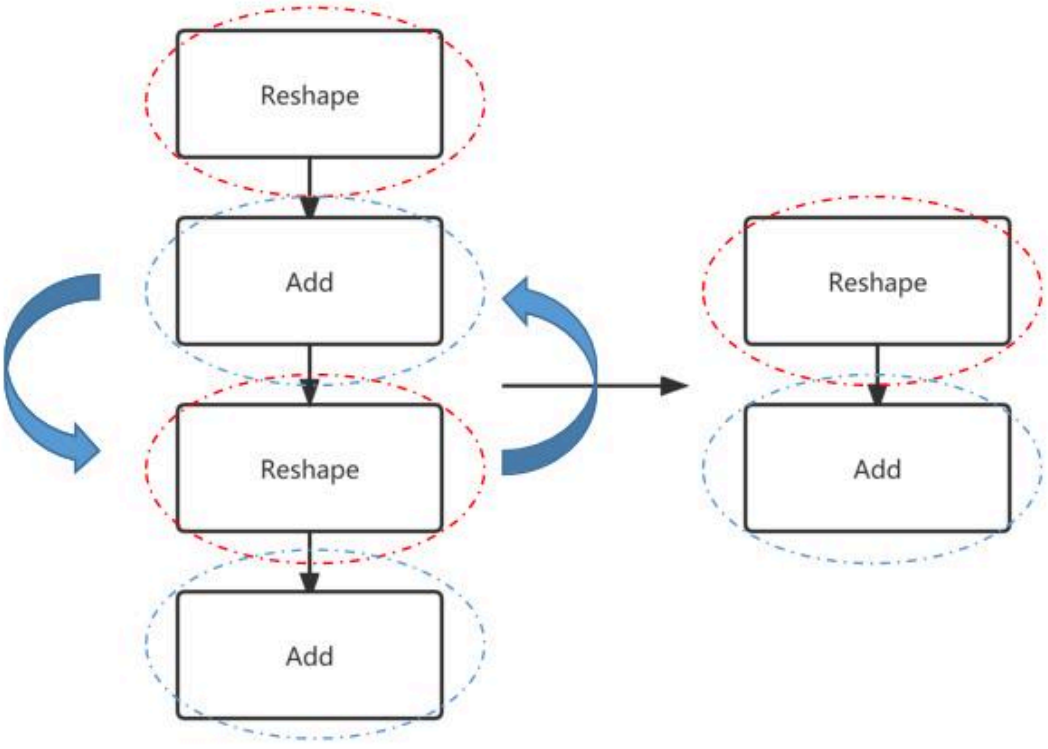


Figure 5-11 Operator merging of similar terms

Some graph structures have common parts with same-type operations that can be reordered and extracted into a single operation.

For example, in the figure below, by adjusting the operator order, the repetitive operators of the same type can be extracted separately so that the operation is executed only once.

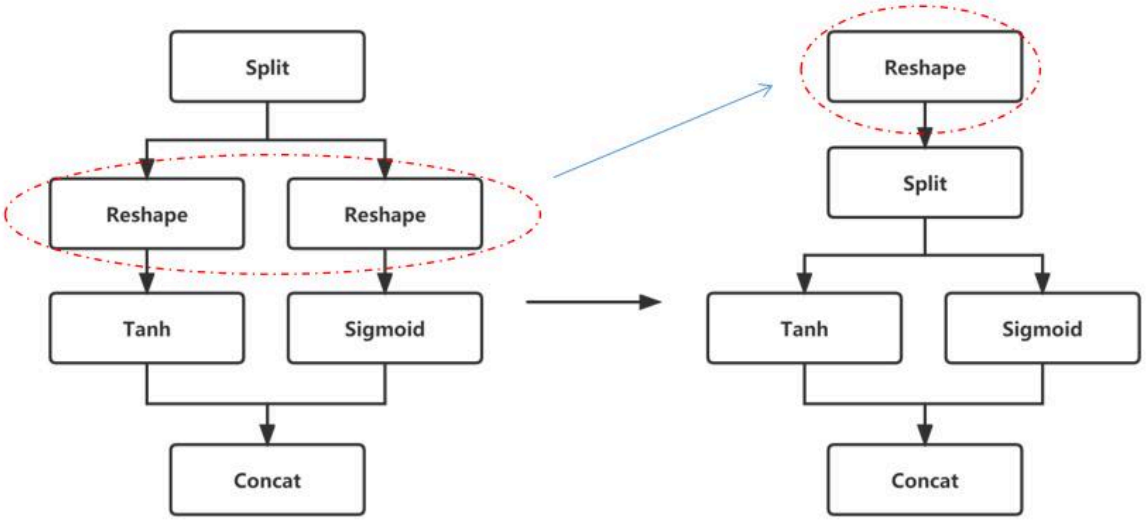


Figure 5-12 Repetitive operator merging

5.2.5 Operator level optimization

Operator-level optimization of a model is a highly targeted fine-grained optimization, involving the specific transformation design of certain operators to further improve performance. Operator optimization is mostly about targeted operator size design, so that operators run at the size specifications with the highest hardware implementation efficiency. For example, for some operators of similar size and scale, the running time consumption of aligned and non-aligned operations may differ greatly. The reason for the difference is that the hardware requires additional redundant operations to guarantee correctness for some operators with non-aligned sizes. Therefore, the size design of operators can also have a great impact on model performance. Users can perform operator optimization in advance according to the following ideas.

5.2.5.1 DDR-FRIENDLY OP SIZE DESIGN (NOT MANDATORY)

Under some aligned sizes, in addition to higher NPU computing efficiency, reading and writing to the DDR is also more friendly. Under the same bandwidth conditions, more friendly reading and writing improves the bandwidth efficiency of the DDR, thereby achieving better performance. The following lists some size rules that are more friendly to DDR reading and writing. These rules are not mandatory.

- **Channel aligned to the alignment amount**

The alignment tables are shown below:

Table 5-1 RK3566/RK3568

	Conv		**Depthwise Conv**	**Other OP**
Dtype	Input Channel	Output Channel	Input & Output Channel	Input & Output Channel
Int8	32	16	32	8
Int16	16	8	16	4

	Conv		Depthwise Conv	Other OP
Float16	16	8	16	4
BFloat16	16	8	16	4

Table 5-2 RK3588/RK3576/RV1126B

	Conv		Depthwise Conv	Other OP
Dtype	Input Channel	Output Channel	Input & Output Channel	Input & Output Channel
Int8	32	32	64	16
Int16	32	16	32	8
Float16	32	16	32	8
BFloat16	32	16	32	8
TFloat32	16	16	16	4

Table 5-3 RV1106/RV1103

	Conv		Depthwise Conv	Other OP
Dtype	Input Channel	Output Channel	Input & Output Channel	Input & Output Channel
Int8	32	16	32	16
Int16	16	16	16	8

Table 5-4 RK3562

	Conv		Depthwise Conv	Other OP
Dtype	Input Channel	Output Channel	Input & Output Channel	Input & Output Channel
Int8	32	16	32	16
Int16	32	8	16	8
Float16	32	8	16	8
BFloat16	32	8	16	8
TFloat32	16	8	8	4

- 4 aligned when Height * Width > 1
- For operators of the same scale, a size with a larger Width and a smaller Height is more friendly to DDR reading and writing. As shown in the figure below, the convolution efficiency of the right figure is higher than that of the left figure

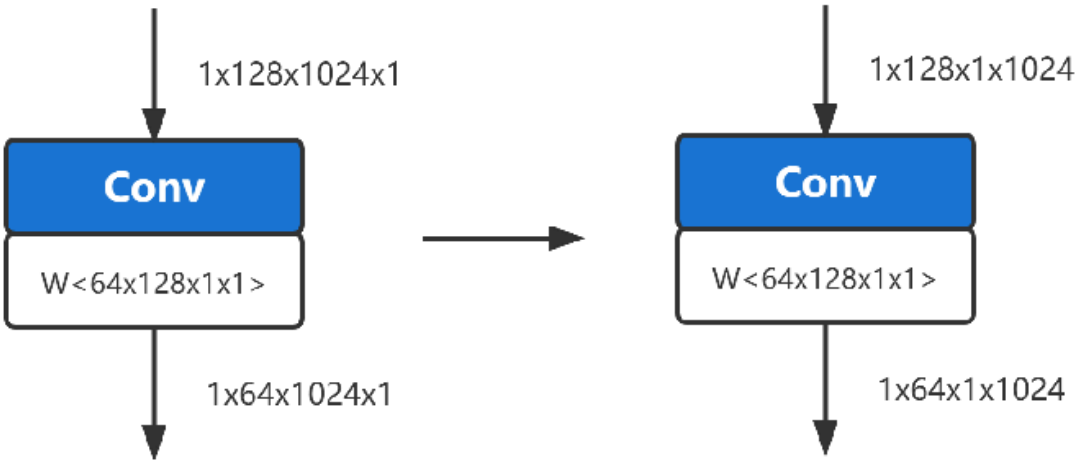


Figure 5-13 Comparison of convolutions of the same specification

5.2.5.2 DESIGN OF HIGH-UTILIZATION OPERATOR

In order to improve the overall utilization of computing power in model design, it is generally necessary to avoid inefficient operators and select operator size designs that are easy to achieve higher utilization. In the following chapters, we will list some inefficient operators that can be avoided as much as possible and discuss the relationship between convolution size and utilization.

- Avoid inefficient operators

  Try to reduce the following three types of operators in the model

  Table 5-5 Three types of inefficient operators

Data transfer	Size transformation	Non-Relu activation function
Transpose	Resize	Sigmoid
Reshape	Tile	Tanh
Split	Pooling	Softplus
Concat	Pad	Hardswish

- Relationship between convolution size and utilization

  Since the performance of convolution will be affected by both computing power and bandwidth, MAC utilization is often used to illustrate the extent of hardware computing power when evaluating performance.

  The relationship between convolution size and utilization is discussed here as a performance reference aid for users to design models. The following is a rough reference based on empirical data:

  Notes on the following nouns: KH (kernel height), KW (kernel width), KC (kernel channel), type_bytes (weight bit width divided by 8), Ksize (KW or KH), Kstride (stride in the KW or KH direction)

  - When the channel of the convolutional input and output Tensor meets the alignment requirements (see aligning table data in 8.4.1), the utilization rate is higher.

- When the channel of the input Tensor is \< 256, the utilization rate is relatively high. When the channel is > 512, the utilization rate will gradually decrease as the channel increases.
- In terms of weight size, the utilization rate is relatively high when KH* KW * KC * type_bytes \< 6K Bytes. Utilization will drop significantly when it exceeds a certain size.
- The larger the ratio of Ksize / Kstride, the higher the utilization rate. For example (Ksize=3, Kstride=1 is better than Ksize=2, Kstride=1)
- The utilization rate decreases when the Height * Width of the output Tensor \< 16.
- The larger the Channel of the output Tensor, the higher the utilization rate.

The above discussion only examines the factors that affect utilization due to independent size. The actual MAC utilization shown by convolution in the actual deployment model is the result of a combination of many factors.

5.2.5.3 SUBGRAPH FUSION

The RKNN software stack will match certain specific graph relationships into custom operators, as shown in the figure below. If they are not fused into the corresponding operators, you can check whether the connection relationships are different, resulting in no match.

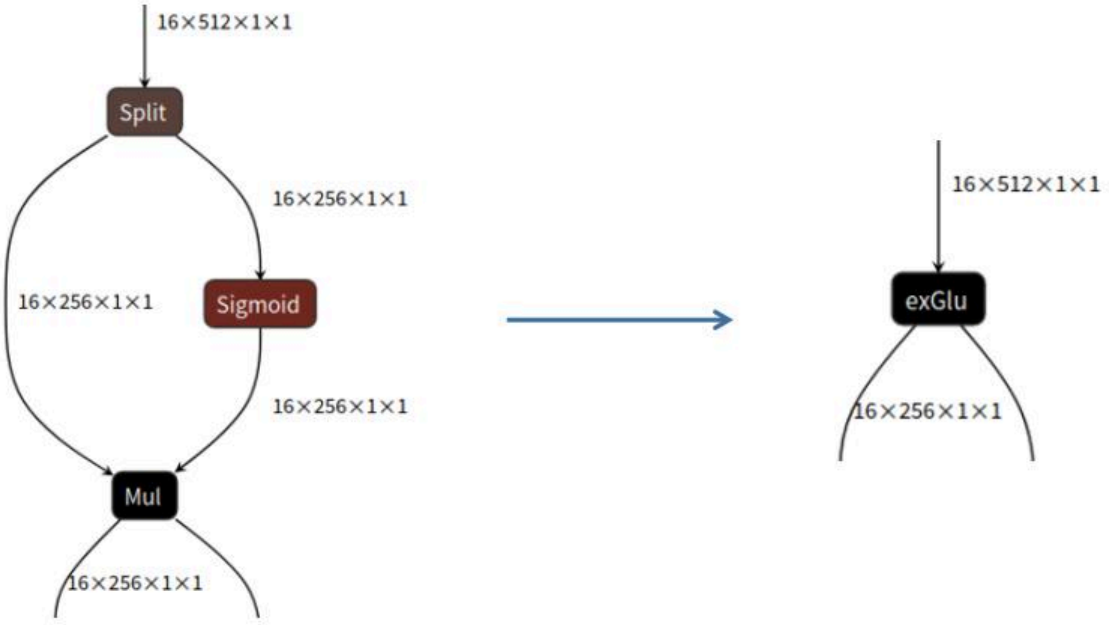


Figure 5-14 Glu subgraph fusion

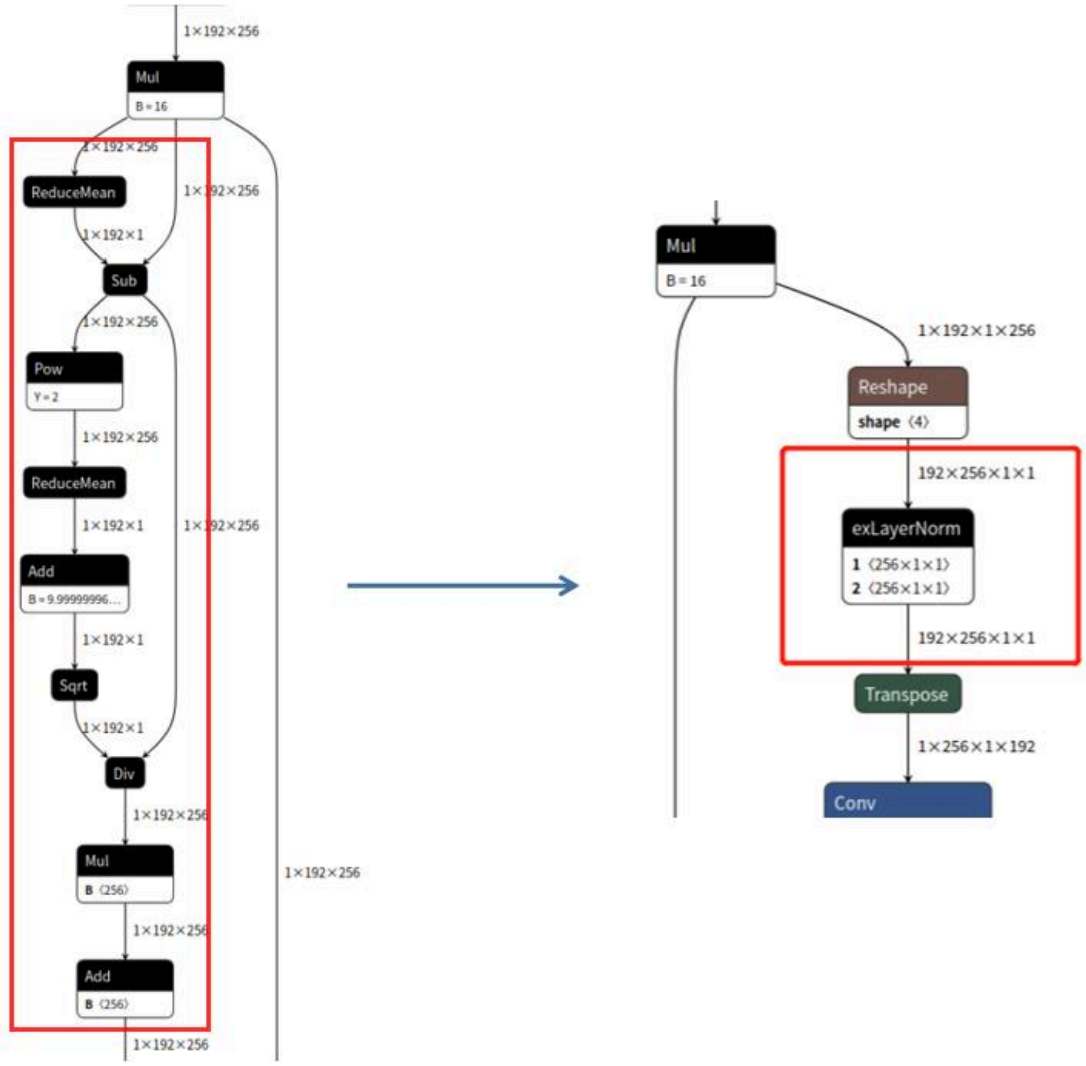


Figure 5-15 LayerNorm subgraph fusion

The currently supported subgraph fusion rules are:

- Split + Sigmoid + Mul -> GLU
- ReduceMean + Sub + Pow + ReduceMean + Add + Sqrt + Div (+ Mul + Add) -> LayerNorm

5.3 Memory Analysis and Optimization

5.3.1 Introduction to memory composition and analysis methods

5.3.1.1 MEMORY COMPOSITION DURING RKNN MODEL RUNTIME

The RKNN model runtime memory mainly consists of four parts: weights and internal tensors, register configurations, input tensors and output tensors. Runtime memory is usually created during rknn_init.

5.3.1.2 MODEL MEMORY ANALYSIS METHOD

After the rknn_init() interface is called, when the user needs to view the memory allocated by the model or needs to allocate model weights externally, call the rknn_query interface and pass in RKNN_QUERY_MEM_SIZE to query the model's weights and internal memory (excluding input and output) , the occupancy of all DMA memory and SRAM memory used for model inference (0 if SRAM is not turned on or does not have this function).

The following is the sample code:

```
  rknn_context ctx = 0;

  // Load RKNN Model
  int ret = rknn_init(&ctx, model_path, 0, NULL, NULL);
  if (ret < 0) {
     printf("rknn_init fail! ret=%d\n", ret);
     return -1;
  }

  // Get weight and internal mem size
  rknn_mem_size mem_size;
  ret = rknn_query(ctx, RKNN_QUERY_MEM_SIZE, &mem_size, sizeof(mem_size));
  if (ret != RKNN_SUCC) {
     printf("rknn_query fail! ret=%d\n", ret);
     return -1;
  }
  printf("total weight size: %d, total internal size: %d\n",
 mem_size.total_weight_size, mem_size.total_internal_size);
```

5.3.2 How to use externally allocated memory

5.3.2.1 EXTERNALLY ALLOCATE INPUT AND OUTPUT MEMORY

According to what is mentioned in section C API Zero-Copy Process, if the user uses the zero-copy API, they can allocate memory externally for the input and output tensors, and then configure it for use by the NPU. For the specific process, please refer to the flow chart in C API Zero-Copy Process. Note that to use the external memory allocation method, you can only use the zero-copy API. This method is mainly applicable to scenarios where users need to manually allocate memory for the NPU to use, instead of letting the NPU allocate memory itself through the rknn_create_mem() interface.

External memory can be recorded with physical addresses and fds, and is mainly created through the following two interfaces:

- rknn_create_mem_from_phys(): Create the rknn_tensor_mem structure through a physical address
- rknn_create_mem_from_fd(): Create the rknn_tensor_mem structure through an fd

Here is an example of creating memory with mpi mmz. This example uses the rknn_create_mem_from_phys() interface to import the physical address of external memory and create an rknn tensor mem structure carrying the physical memory information. The following is the sample code:

```
// Create input tensor memory
rknn_tensor_mem* input_mems[1];
// default input type is int8 (normalize and quantize need compute in outside)
// if set uint8, will fuse normalize and quantize to npu
input_attrs[0].type = input_type;
// default fmt is NHWC, npu only support NHWC in zero copy mode
input_attrs[0].fmt = input_layout;

input_mems[0] = rknn_create_mem_from_phys(ctx, input_phys, input_virt,
input_attrs[0].size_with_stride);

...

// Create output tensor memory
rknn_tensor_mem* output_mems[io_num.n_output];
for (uint32_t i = 0; i < io_num.n_output; ++i) {
    output_mems[i] = rknn_create_mem_from_phys(ctx, output_physs[i],
output_virts[i], output_attrs[i].size);
}

// Set input tensor memory
ret = rknn_set_io_mem(ctx, input_mems[0], &input_attrs[0]);
if (ret < 0) {
    printf("rknn_set_io_mem fail! ret=%d\n", ret);
    return -1;
}

// Set output tensor memory
for (uint32_t i = 0; i < io_num.n_output; ++i) {
    // set output memory and attribute
    ret = rknn_set_io_mem(ctx, output_mems[i], &output_attrs[i]);
    if (ret < 0) {
        printf("rknn_set_io_mem fail! ret=%d\n", ret);
        return -1;
    }
}
```

In addition to referencing the physical address of external memory, externally allocated memory can also be used by referencing an fd. The following is the sample code:

```
int mb_flags = RK_MMZ_ALLOC_TYPE_CMA | RK_MMZ_ALLOC_UNCACHEABLE;

// Allocate weight memory in outside
MB_BLK weight_mb;
rknn_tensor_mem* weight_mem;
ret = RK_MPI_MMZ_Alloc(&weight_mb, mem_size.total_weight_size, mb_flags);
if (ret < 0) {
    printf("RK_MPI_MMZ_Alloc failed, ret: %d\n", ret);
    return ret;
}

void* weight_virt = RK_MPI_MMZ_Handle2VirAddr(weight_mb);
if (weight_virt == NULL) {
    printf("RK_MPI_MMZ_Handle2VirAddr failed!\n");
    return -1;
}

int weight_fd = RK_MPI_MMZ_Handle2Fd(weight_mb);
if (weight_fd < 0) {
    printf("RK_MPI_MMZ_Handle2Fd failed!\n");
    return -1;
}

weight_mem = rknn_create_mem_from_fd(ctx, weight_fd, weight_virt,
mem_size.total_weight_size, 0);
printf("weight mb info: virt = %p, fd = %d, size: %d\n", weight_virt, weight_fd,
mem_size.total_weight_size);

int mb_flags = RK_MMZ_ALLOC_TYPE_CMA | RK_MMZ_ALLOC_UNCACHEABLE;

// Allocate weight memory in outside
MB_BLK weight_mb;
rknn_tensor_mem* weight_mem;
ret = RK_MPI_MMZ_Alloc(&weight_mb, mem_size.total_weight_size, mb_flags);
if (ret < 0) {
    printf("RK_MPI_MMZ_Alloc failed, ret: %d\n", ret);
    return ret;
}

void* weight_virt = RK_MPI_MMZ_Handle2VirAddr(weight_mb);
if (weight_virt == NULL) {
    printf("RK_MPI_MMZ_Handle2VirAddr failed!\n");
    return -1;
}

int weight_fd = RK_MPI_MMZ_Handle2Fd(weight_mb);
if (weight_fd < 0) {
    printf("RK_MPI_MMZ_Handle2Fd failed!\n");
    return -1;
}

weight_mem = rknn_create_mem_from_fd(ctx, weight_fd, weight_virt,
mem_size.total_weight_size, 0);
printf("weight mb info: virt = %p, fd = %d, size: %d\n", weight_virt, weight_fd,
mem_size.total_weight_size);
```

5.3.2.2 EXTERNALLY ALLOCATED MODEL MEMORY

As mentioned in section 9.1, model memory occupancy is divided into two parts: one part is internal memory, and the other part is weight memory. If the application needs to use externally allocated model memory, it can set the memory used by the model weight and internal through the rknn_set_weight_mem() and rknn_set_internal_mem() interfaces. The following is the reference example:

```
// Load RKNN Model
ret = rknn_init(&ctx, model_virt, model_size, RKNN_FLAG_MEM_ALLOC_OUTSIDE, NULL);
TIME_END(rknn_init);
if (ret < 0) {
    printf("rknn_init fail! ret=%d\n", ret);
    return -1;
}

//query and inset input / output tensor
...

// Allocate weight memory in outside
MB_BLK weight_mb;
rknn_tensor_mem* weight_mem;
ret = RK_MPI_MMZ_Alloc(&weight_mb, SIZE_ALIGN_128(mem_size.total_weight_size),
mb_flags);
void* weight_virt = RK_MPI_MMZ_Handle2VirAddr(weight_mb);
int weight_fd = RK_MPI_MMZ_Handle2Fd(weight_mb);
weight_mem = rknn_create_mem_from_fd(ctx, weight_fd, weight_virt,
mem_size.total_weight_size, 0);
ret = rknn_set_weight_mem(ctx, weight_mem);
if (ret < 0) {
    printf("rknn_set_weight_mem fail! ret=%d\n", ret);
    return -1;
}
printf("weight mb info: virt = %p, fd = %d, size: %d\n", weight_virt, weight_fd,
mem_size.total_weight_size);

// Allocate internal memory in outside
MB_BLK internal_mb;
rknn_tensor_mem* internal_mem;
ret = RK_MPI_MMZ_Alloc(&internal_mb, SIZE_ALIGN_128(mem_size.total_internal_size),
mb_flags);
void* internal_virt = RK_MPI_MMZ_Handle2VirAddr(internal_mb);
int internal_fd = RK_MPI_MMZ_Handle2Fd(internal_mb);

internal_mem = rknn_create_mem_from_fd(ctx, internal_fd, internal_virt,
mem_size.total_internal_size, 0);
ret = rknn_set_internal_mem(ctx, internal_mem);
if (ret < 0) {
    printf("rknn_set_internal_mem fail! ret=%d\n", ret);
    return -1;
}
printf("internal mb info: virt = %p, fd = %d, size: %d\n", internal_virt,
internal_fd, mem_size.total_internal_size);
```

5.3.3 Internal Memory Reuse

The RKNN API provides a mechanism for external management of NPU memory. Through the RKNN_FLAG_INTERNAL_ALLOC_OUTSIDE parameter, users can specify that the intermediate feature memory of the model is allocated externally. Typical application scenarios of this function are as follows:

- During deployment, all NPU memory is allocated by the user, which facilitates the overall arrangement of the entire system memory.
- It is used in multiple model serial running scenarios, where the intermediate feature memory is reused across different contexts, especially for devices such as RV1103/RV1106 where memory is extremely tight.

For example, there are two models in the figure below. The Internal Tensor occupancy of model 1 is larger than that of model 2. If model 1 and model 2 are run sequentially, only one block of memory at the address 0x00000000~0x000c4000 needs to be allocated for model 1 and model 2 to share. After the inference of model 1 finishes, this memory can be used by model 2 to read and write Internal Tensor data, thereby saving memory.

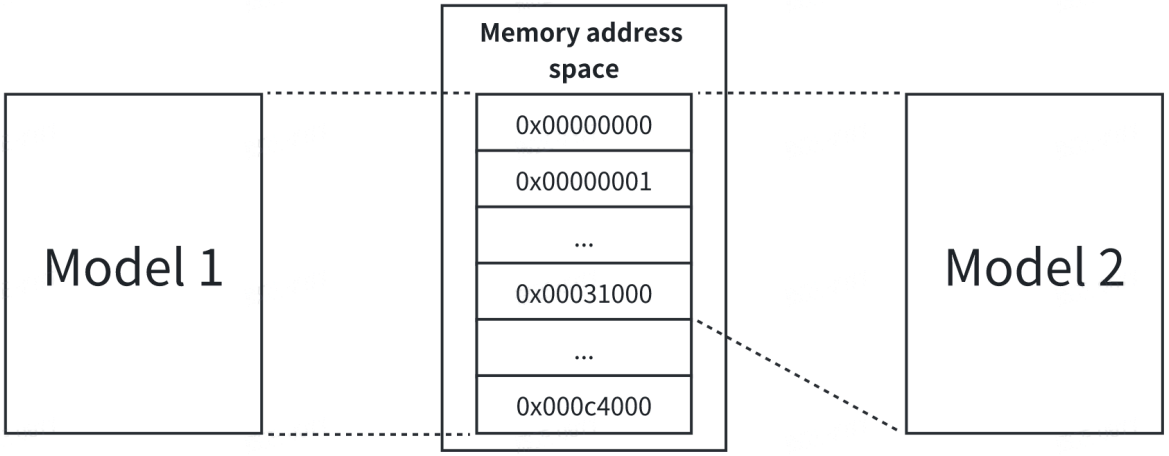


Figure 5-16 Example of two model Internal Tensors sharing the same memory address space

Assume that the path of model 1 is model_path_a and the path of model 2 is model_path_b. The sample code is as follows:

```
rknn_init(&ctx_a, model_path_a, 0, RKNN_FLAG_INTERNAL_ALLOC_OUTSIDE, NULL);
rknn_query(ctx_a, RKNN_QUERY_MEM_SIZE, &mem_size_a, sizeof(mem_size_a));
rknn_init(&ctx_b, model_path_b, 0, RKNN_FLAG_INTERNAL_ALLOC_OUTSIDE, NULL);
rknn_query(ctx_b, RKNN_QUERY_MEM_SIZE, &mem_size_b, sizeof(mem_size_b));

// Get the largest internal size of the two models
max_internal_size = MAX(mem_size_a.total_internal_size,
mem_size_b.total_internal_size);
internal_mem_max = rknn_create_mem(ctx_a, max_internal_size);

// Set the internal memory of model a
internal_mem_a = rknn_create_mem_from_fd(ctx_a, internal_mem_max->fd,
internal_mem_max->virt_addr, mem_size_a.total_internal_size, 0);
rknn_set_internal_mem(ctx_a, internal_mem_a);

// Set the internal memory of model b
internal_mem_b = rknn_create_mem_from_fd(ctx_b, internal_mem_max->fd,
internal_mem_max->virt_addr, mem_size_b.total_internal_size, 0);
rknn_set_internal_mem(ctx_b, internal_mem_b);
```

5.3.4 Multi-threaded Reuse RKNN Context

In a multi-threaded scenario, a model may be executed by multiple threads at the same time. If each thread initializes a context separately, the memory consumption will be very large. Therefore, you can consider sharing a context to avoid repeated construction of data structures and reduce runtime memory usage. The RKNN API provides an interface for reusing contexts. The interface is defined as follows:

```
int rknn_dup_context(rknn_context* context_in,rknn_context* context_out)
```

where context_in is the already initialized context, and context_out is the context that reuses context_in. As shown in the figure below, the model structures of the two contexts are the same, so the contexts can be reused.

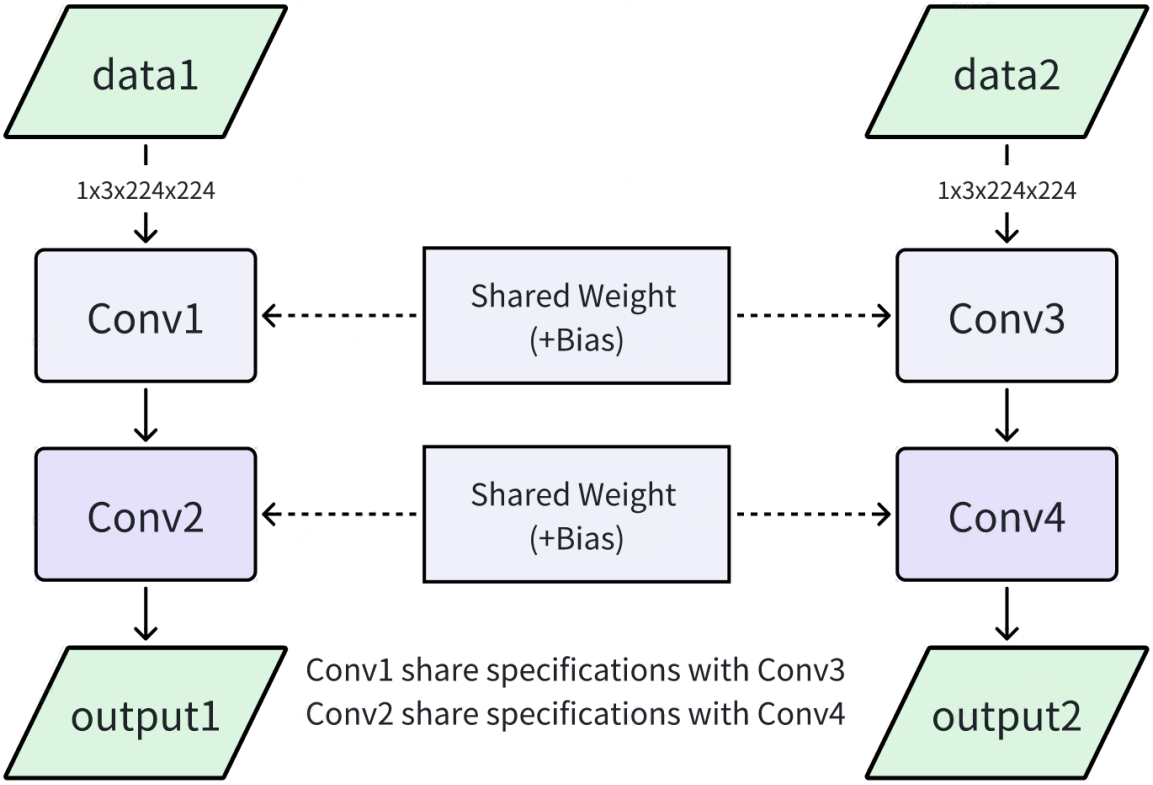


Figure 5-17 Example of two identical model reuse contexts

5.3.5 Multiple resolution models share the same weights

When multiple models with different resolutions have the same weights, they can share the same weights to reduce memory usage. In RKNPU SDK versions <= 1.5.0, this function can realize dynamic switching between models with different resolutions with a smaller memory footprint. After version 1.5.0, this function was replaced by the dynamic shape function.

As shown in the figure below, the weights of model A and model B are exactly the same.

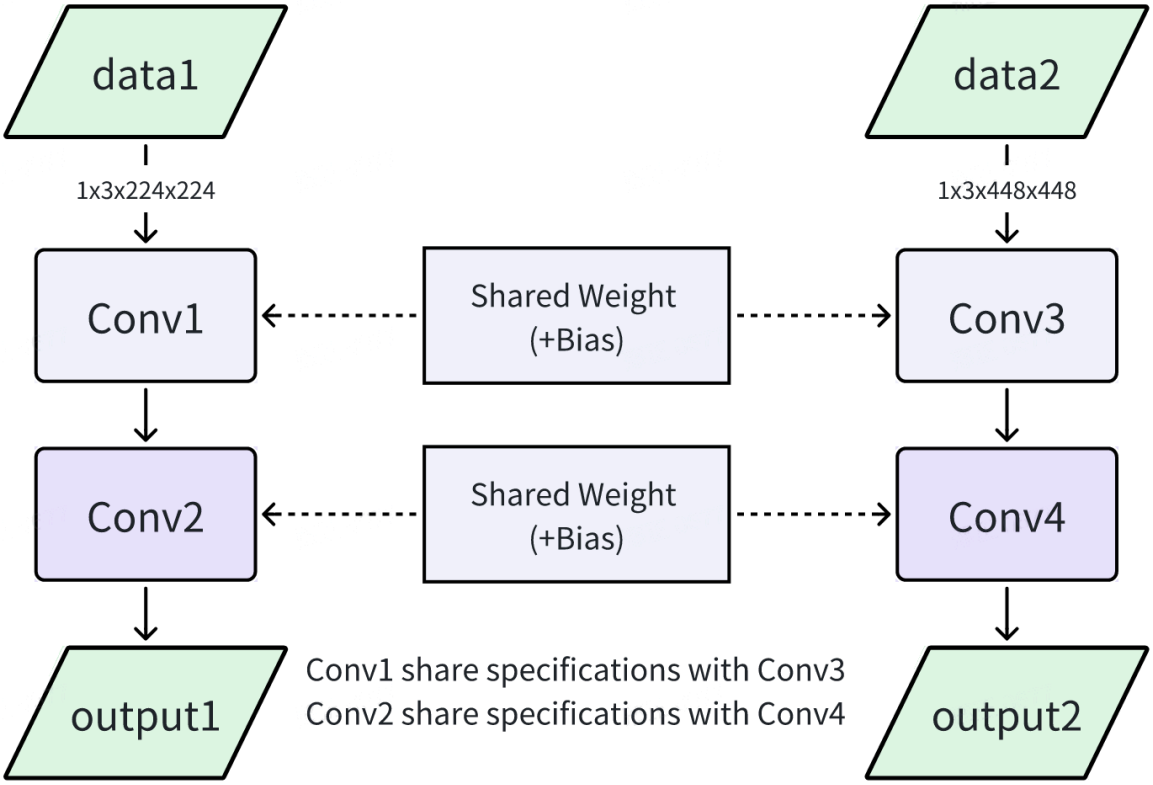


Figure 5-18 Example of two models with different resolutions sharing weights

Multiple resolution models can share the same weights by following the steps below:

1. When converting the RKNN model, one of the models is set as the master model, and the rknn.config interface sets the parameter remove_weight=False; the other model is set as the slave model, with the parameter remove_weight=True set. The master RKNN model contains weights, and the slave RKNN model does not contain convolutional class weights.
2. When deploying, initialize the master RKNN model first, and then initialize the slave RKNN model. When initializing the slave model, use the RKNN_FLAG_SHARE_WEIGHT_MEM flag and add the new parameter rknn_init_extend, whose value is the context of the master model. Assume that the master model path is model_A and the slave model path is model_B. The sample code is as follows:

```
rknn_context context_A;
rknn_context context_B;
ret = rknn_init(&context_A,model_A,0,0,NULL);

...

rknn_init_extend extend;
extend.ctx = context_A;
ret = rknn_init(&context_B, model_B,0,RKNN_FLAG_SHARE_WEIGHT_MEM,&extend);
```

5.4 FAQs and Error Handling

5.4.1 NPU SDK environment problem

- **Version compatibility**
  - NPU kernel driver and Runtime

  It is recommended to upgrade the NPU kernel driver to version 0.9.2 or later. When encountering problems, update to the latest version of the NPU kernel driver first.

  - The compatibility relationship between models exported by RKNN-Toolkit2 and the Runtime version is shown in the table below

Table 5-6 Correspondence between RKNN model and Runtime version

RKNN model version	Runtime version
1.2.0	>=1.2.0 and <=1.5.0
1.3.0	>=1.3.0 and <=1.5.0
1.4.0	>=1.4.0 and <=1.5.0
1.5.0	1.5.0
1.5.2	>=1.5.2
1.6.0	>=1.5.2
2.0.0	>=2.0.0
2.1.0	>=2.0.0

- **How to update the NPU kernel driver**

  It is recommended to upgrade the complete firmware to update the NPU driver. The corresponding firmware can be provided by the vendor.

- **Using NPU in the Docker environment on the board**

  When deploying applications with Docker on the board, if you want to use NPU resources, you need to map the NPU-related resources when starting the container. The reference command is as follows:

  ```
  docker run -t -i --privileged -v /dev/dri/renderD129:/dev/dri/renderD129 -v
  /proc/device-tree/compatible:/proc/device-tree/compatible -v
  /usr/lib/librknnrt.so:/usr/lib/librknnrt.so ai_application:v1.0.0 /bin/bash
  ```

  Pay attention to the following parameters: - **/dev/dri/renderD129**: RK3588 NPU device node. Runtime relies on this node to enable the NPU. - **/proc/device-tree/compatible**: This file records the SOC model. Components such as RKNN-Toolkit Lite2 rely on this file to obtain the current SOC information. - **/usr/lib/librknnrt.so**: The location where the Runtime library is stored. RKNN-Toolkit Lite2 and the RKNPU2 C API rely on this file to use NPU resources. - **ai_application:v1.0.0**: The image name and version used by the container to be started.

5.4.2 RKNN-Toolkit2 installation issue

- **The dependency constraints of RKNN-Toolkit2 are too strict, causing the installation to fail**

  When all dependent libraries have been installed but the versions of some libraries do not match the requirements, you can try adding the "no-deps" parameter after the installation command to skip the environment check when installing the Python library. For example:

  ```
  pip install rknn-toolkit2*.whl --no-deps
  ```

- **PyTorch dependency notes**

  The PyTorch model loading function of RKNN-Toolkit2 depends on PyTorch. PyTorch models are divided into floating-point models and quantized models (including QAT and PTQ quantized models).

  For models exported by PyTorch 1.6.0, it is recommended to downgrade the PyTorch version that RKNN-Toolkit2 depends on to 1.6.0 to avoid loading failures.

  For quantized models (QAT, PTQ), we recommend using PyTorch 1.10~1.13.1 to export the model, and upgrading the PyTorch version that RKNN-Toolkit2 depends on to 1.10~1.13.1.

  In addition, when loading a PyTorch model, it is recommended that the PyTorch version used to export the original model be as consistent as possible with the PyTorch version that RKNN-Toolkit2 depends on.

  The recommended PyTorch versions are 1.6.0, 1.9.0, 1.10, and 1.13.1.

- **TensorFlow dependency notes**

  The TensorFlow model loading function of RKNN-Toolkit2 depends on TensorFlow. Since the compatibility between TensorFlow versions is generally poor, other versions may cause RKNN-Toolkit2 model loading errors. Therefore, when loading a TensorFlow model, it is recommended that the TensorFlow version used to export the original model be consistent with the TensorFlow version that RKNN-Toolkit2 depends on.

  Problems caused by the TensorFlow version are usually reflected in the "rknn.load_tensorflow()" stage, and the error message will point to the dependent TensorFlow path.

  The recommended TensorFlow versions are 2.6.2 or 2.8.0.

- **RKNN-Toolkit2 installation package naming rules** Taking the release artifacts of version 1.5.2 as an example, the RKNN-Toolkit2 wheel package naming rules are as follows:

  ```
  rknn_toolkit2-1.5.2+b642f30c-cp38-cp38-linux_x86_64.whl
  ```

  - rknn_toolkit2: tool name.
  - 1.5.2: version number.
  - b642f30c: commit id.
  - cp\<xx>-cp\<xx>: the applicable Python version, for example, cp38-cp38 means the applicable Python version is 3.8.

- linux_x86_64: system type and CPU architecture.

Please install the corresponding package according to your operating system, CPU architecture, and Python version, otherwise the installation will fail.

- **Is there an ARM Linux version of RKNN-Toolkit2**

  RKNN-Toolkit2 has no ARM Linux version. If you need to use the Python interface for inference on ARM Linux, you can install RKNN-Toolkit-lite2, which can run inference with Python on ARM Linux.

- **The bfloat16 dependency library fails to install**

  An error occurs when installing the bfloat16 dependency library, as follows:

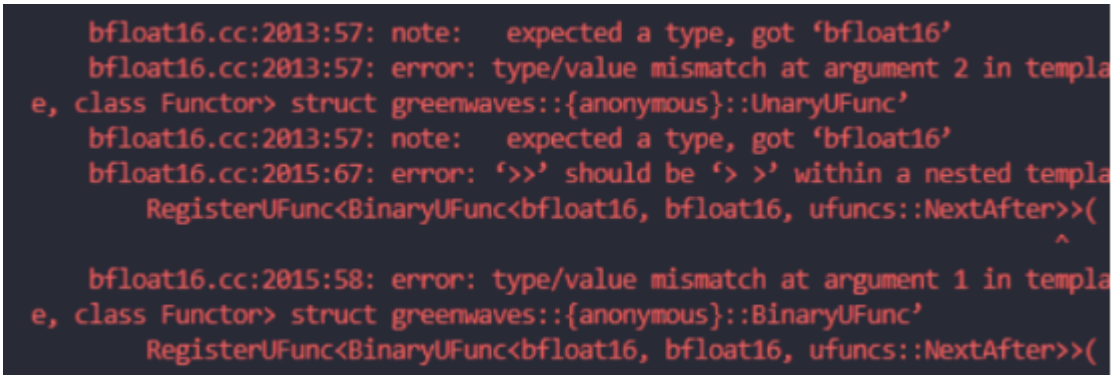


Figure 5-19 bfloat16 dependent library installation failure log

```
Change the pip source to the Alibaba source, or update RKNN-Toolkit2 to version
1.5.0 or later (versions 1.5.0 and later have removed the dependency on the
bfloat16 library).
```

5.4.3 Model conversion parameters description

This section mainly covers the usage of commonly used parameters in the model conversion stage.

- **Parameters determined by the model**

During model conversion, the rknn.config() and rknn.build() interfaces affect the model conversion result.

The input and output nodes specified by rknn.load_onnx() and rknn.load_tensorflow() affect the model conversion result.

The input sizes specified by rknn.load_pytorch() and rknn.load_tensorflow() affect the model conversion result.

You can refer to the following basic steps for model conversion:

1. Prepare quantization data and provide the dataset.txt file.
2. Determine the NPU platform to be used by the model, such as RK3566, RV1106, etc., and fill in the target_platform parameter in the rknn.config() interface.
3. When the input is a 3-channel image and the quantization data is in image format (such as jpg, png), you need to confirm whether the model input is RGB or BGR to determine the

value of the quant_img_RGB2BGR parameter in the rknn.config() interface.

4. Confirm the normalization parameters used during model training to determine the values of the mean_values and std_values parameters in the rknn.config interface.
5. Confirm the input size of the model and fill it into the corresponding parameters of the load interface, such as the input_size_list parameter in the rknn.load_pytorch() interface.
6. Confirm the number of quantization bits of the model to determine the value of the quantized_dtype parameter in the rknn.config() interface. **This step can be ignored when the model is not quantized or an already quantized model is loaded**.
7. Confirm the quantization algorithm to be used for model quantization to determine the value of the quantized_algorithm parameter in the rknn.config() interface. **This step can be ignored when the model is not quantized or an already quantized model is loaded**.
8. Confirm whether the model should be quantized to determine the value of the do_quantization parameter in the rknn.build() interface. When choosing to quantize the model, the dataset parameter in the rknn.build() interface must also be filled in to specify the quantization calibration data.
9. **Cross-platform compatibility of RKNN models**

   For the platform parameter set by target_platform of rknn.config(), the compatibility relationship is as follows:

   - Models used on the RK3566 and RK3568 platforms are compatible with each other.
   - Models used on the RK3588 and RK3588S platforms are compatible with each other.
   - Models used on the RV1103 and RV1106 platforms are compatible with each other.
10. **Format and requirements of quantization calibration data**

    There are two options for the format of quantization calibration data. One is image format (jpg, png), which RKNN-Toolkit2 reads through OpenCV interfaces; the other is npy format, which RKNN-Toolkit2 reads through numpy interfaces.

    For models with non-RGB/BGR image input, it is recommended to provide the quantization data in numpy's npy format.
11. **How to fill in the dataset.txt file for multi-input models**

    Model quantization requires the dataset.txt file to specify the paths of the quantization data. The rule is that one line serves as one set of inputs; when the model has multiple inputs, the multiple inputs are written on the same line and separated by spaces.

    For example, for a single-input model using two sets of quantization data:

    ```
    sampleA.npy
    sampleB.npy
    ```

    For a model with three inputs, two sets of quantization data are filled in as follows:

    ```
    sampleA_in0.npy sampleA_in1.npy sampleA_in2.npy
    sampleB_in0.npy sampleB_in1.npy sampleB_in2.npy
    ```

12. **Confirm the quant_img_RGB2BGR parameter of rknn.config()**

    When using images (jpg, png) as quantization data, you need to consider setting the quant_img_RGB2BGR parameter.

    If the model is trained with RGB images, set the quant_img_RGB2BGR parameter to False or leave it unset. And when performing inference with the Python inference interface or the RKNPU2 C API, input RGB images.

    If the model is trained with BGR images, set the quant_img_RGB2BGR parameter to True. But when performing inference with the Python inference interface or the RKNPU2 C API, you also need to input BGR images (quant_img_RGB2BGR only affects the images read from the quantization calibration set).

    If the quantization data is in numpy's npy format, it is recommended not to use the quant_img_RGB2BGR parameter to avoid confusion in usage.

13. **Calculation order of mean, std and quant_img_RGB2BGR in rknn.config()**

    Because quant_img_RGB2BGR only controls whether channel conversion is performed when reading calibration set images during quantization, and does not affect other steps, both the RKNN-Toolkit2 inference interface and the RKNPU2 C API only perform mean subtraction first and then standard deviation (std) division on the input data, without any channel conversion operation.

14. **Setting mean_values and std_values in rknn.config() when the model has non-3-channel input or multiple inputs**

    The setting formats of mean_values and std_values are the same. Take mean_values as an example.

    Assuming the input has N channels, the value of mean_values is [[channel_1, channel_2, channel_3, channel_4, ..., channel_n]].

    When there are multiple inputs, the value of mean_values is [[channel_1, channel_2, channel_3, channel_4, ..., channel_n], [channel_1, channel_2, channel_3, channel_4, ..., channel_n]].

15. **Selection of the quantization parameter correction algorithm and the number of quantization images**

    RKNN-Toolkit2's quantization algorithm (quantized_algorithm of rknn.config()) provides three algorithms for parameter correction, namely normal, mmse and kl_divergence, with normal used by default. normal is the conventional quantization parameter correction algorithm; mmse iterates over the calculation results of intermediate layers and clips the weight values within a certain range to obtain higher inference accuracy. Using mmse does not necessarily improve quantization accuracy, but compared with the normal method, quantization occupies more memory and takes longer model conversion time; the kl_divergence quantization algorithm takes somewhat more time than normal, but much less than mmse, and can achieve good improvement in some scenarios (when the feature distribution is uneven).

    It is recommended to use the normal algorithm first; if the quantization result is not good, try the mmse or kl_divergence algorithm.

    When using the normal or kl_divergence algorithm, 20-200 sets of data are recommended for quantization. When using mmse, 20-50 sets of data are recommended.

16. **Differences in inference inputs and outputs between quantized and non-quantized models**

    When calling the general RKNPU2 C API (meaning calling the C API without using pass_through or zero_copy), the data type of the input data (such as uint8 data, float data) has nothing to do with whether the model is quantized. The data type of the output data can be automatically processed into float32 format, or you can choose to directly output the model inference result, in which case the data type is the same as that of the output node. Using the Python inference interface is slightly different. The specific relationship is as follows:

Table 5-7 Differences between the Python inference interface and the general C API interface

After model quantization	Python inference (rknn.inference())	C API inference (rknn.run()) (non pass_through, zero_copy)
Whether the input type is restricted	No restrictions. The input of rknn.inference() is a numpy array that carries a data type attribute itself, and the input is automatically converted into the data format required by the RKNN model.	No restrictions. The rknn_tensor_type parameter of rknn_inputs can be set to RKNN_TENSOR_FLOAT32, RKNN_TENSOR_FLOAT16, RKNN_TENSOR_INT8, RKNN_TENSOR_UIN8, RKNN_TENSOR_INT16 according to the actual input. Once set, the input is automatically converted into the data format required by the RKNN model.
Whether the output type changes	No change. Regardless of whether the model is quantized, Python's rknn.inference() interface always returns float output. No other data type can be selected.	Changed. With the rknn outputs attr of the RKNPU2 C API, want_float=1 can be set to get float output. After quantization, want_float=0 can be set, in which case the raw output data of the last node is output, for example int8 data when quantized as i8.
Whether the input format changes (NCHW, NHWC)	No change. Regardless of whether the model is quantized, the data_format parameter of the rknn.inference() interface can be set to nchw or nhwc as needed.	No change. Regardless of whether the model is quantized, the rknn_tensor_format parameter of the rknn inputs structure can be set to NCHW or NHWC as needed.

- **Is there an online pre-compilation mode**

  RKNN-Toolkit2 only supports exporting offline pre-compiled models, and does not support exporting online pre-compiled models (which is supported by RKNN-Toolkit1), so there is no choice between offline pre-compilation and online pre-compilation modes.

- **Can the RKNN model converted by RKNN-Toolkit be used on the RK3566 platform**

  No.

  The RKNN model converted by RKNN-Toolkit is applicable to platforms such as RK1806 / RK1808 / RK3399Pro / RV1109 / RV1126; the RK3566 platform requires the RKNN model converted by RKNN-Toolkit2. The RKNN model converted by RKNN-Toolkit2 is applicable to platforms such as RK3566 / RK3568 / RK3588 / RK3588S / RV1103 / RV1106 / RK3562 / RK3576 / RV1126B.

  For instructions on using the RKNN-Toolkit tool, please refer to the following project:

https://github.com/airockchip/rknn-toolkit

For instructions on using the RKNN-Toolkit2 tool, please refer to the following project:

https://github.com/airockchip/rknn-toolkit2

5.4.4 Deep-learning framework FAQs

5.4.4.1 DEEP LEARNING FRAMEWORKS AND CORRESPONDING VERSIONS SUPPORTED BY RKNN-TOOLKIT2

Please refer to Chapter 2.1.

5.4.4.2 OP SUPPORT LIST FOR EACH FRAMEWORK

RKNN-Toolkit2 has different support levels for different frameworks. For detailed information, please refer to the RKNNToolkit2_OP_Support document in the following link:

https://github.com/airockchip/rknn-toolkit2/blob/master/doc/

5.4.4.3 FAQS ABOUT ONNX MODEL CONVERSION

- **"Error parsing message" appears when loading the model**

For example, converting the `examples/onnx/resnet50v2` model fails and the prompt is as follows:

```
E load_onnx: Catch exception when loading onnx model:
/rknn_resnet_demo/resnet50v2.onnx!

E lod_onnx: Traceback (most recent call last):

E load_onnx: File "rknn/api/rknn_base.py", line 1094, in
rknn.api.rknn_base.RKNNBase.load_onnx

E load_onnx: File "/usr/local/lib/python3.6/dist-packages/onnx/__init__.py", line
115, in load_model

……

E load_onnx: google.protobuf.message.DecoderError: Error parsing message
```

The reason may be that the `resnet50v2.onnx` model is damaged (if the entire model is not downloaded). Re-download the model and ensure that its MD5 value is correct, such as:

```
22ed6e6a8fb9192f0980acca0c941414 resnet50v2.onnx
```

- **Is dynamic input shape supported?**

For example, the input dimension of ONNX is [-1, 3, -1, -1], which means that the batch, height and width dimensions are not fixed. RKNN-Toolkit2 before 1.5.2 version does not support dynamic input shape.

Versions 1.5.2 and later support dynamic input shapes through the dynamic_input parameter of rknn.config(), see Chapter 4.1.3 for details.

- **An error occurs when customizing the output node**

For example, calling `rknn.load_onnx()` with setting outputs parameter and the error occurs as follows.

```
E load_onnx: the '378' in outputs=['378', '439', '500'] is invalied!
```

The log prompts that output node `378` is invalid. It means that the outputs parameter needs to be set to the correct output node name.

5.4.4.4 FAQS ABOUT PYTORCH MODEL CONVERSION

- **An error occurs when loading a Pytorch model: torch._C does not have the _jit_pass_inline attribute**

The error log is as follows:

```
'torch._C' has no attribute '_jit_pass_inline'
```

Please upgrade PyTorch to version 1.6.0 or later.

- **Saving format of Pytorch models**

Currently, only models exported by `torch.jit.trace()` are supported. The `torch.save()` interface only saves the weight parameter dictionary, which lacks network structure information and cannot be normally imported and converted into an RKNN model.

- **PytorchStreamReader failed during the conversion**

The error log is as follows:

```
E Catch exception when loading pytorch model: ./mobilenet0.25_Final.pth!

E Traceback (most recent call last):

……

E cpp_module = torch._C.import_ir_module(cu, f, map_location, extra_files)

E RuntimeError: [enforce fail at inline container.cc:137]. PytorchStreamReader
failed reading zip archive: faild finding central directory frame #0 ……
```

The reason is that the PyTorch model does not have network structure information.

Usually `.pth` file only contains weights but not network structure information. To export a model with the network structure, initialize the corresponding network structure and then use `net.load_state_dict()` to load the `.pth` weight file. Next, use `torch.jit.trace()` interface to solidify the network structure and weight. Finally, use `torch.jit.save()` to save

the traced model in `.pt` format, which can be used for the `rknn.load_pytorch()` interface to convert it into an RKNN model.

- **KeyError encountered during conversion**

The error log is as follows:

```
E Traceback (most recent call last):

……

E KeyError: 'aten::softmax'
```

When an error message in the form of `KeyError: 'aten::xxx'` appears, it means that the current version of the operator does not support it. RKNN-Toolkit2 will fix such bugs every time it is upgraded. Please use the latest version of RKNN-Toolkit2 to solve it.

- **"Syntax error in input! LexToken(xxx)" error encountered during conversion**

The error log is as follows:

```
WARNING: Token 'COMMENT' defined, but not used

WARNING: There is 1 unused token

!!!!! Illegal character '"'

Syntax error in input! LexToken(NAMED_IDENTIFIER, 'fc', 1, 27)

!!!!! Illegal character '"'
```

There are many reasons for this error. Please troubleshoot in the following order:

1)The `torch.nn.module` is not inherited to create the network. Please inherit the `torch.nn.module` class type to create the network, and then use `torch.jit.trace()` to generate the pt file.

2)To update RKNN-Toolkit2 1.4.0 or later version, torch recommends version 1.6.0, 1.9.0, 1.10.0 or 1.13.1.

5.4.4.5 FAQS ABOUT TENSORFLOW MODEL CONVERSION

- **Tensorflow1.x model error**

Loading tensorflow1.x model with rknn.load_tensorflow() interface, an error message appears:

```
E load_tensorflow: Catch exception when loading tensorflow model:
./yolov3_mobilenetv2.pb!

E load_tensorflow: Traceback (most recent call last):

……

E load_tensorflow: tensorflow.python.framework.errors_impl.InvalidArgumentError:
Node 'MobilenetV2/expanded_conv/depthwise/BatchNorm/cond/Switch_1' expects to be
colocated with unknown node
'MobilenetV2/expanded_conv/depthwise/BatchNorm/moving_mean'

E load_tensorflow: During handling of the above exception, another exception
occurred:

E load_tensorflow: Traceback (most recent call last):

E load_tensorflow: File "rknn/api/rknn_base.py", line 990, in
rknn.api.rknn_base.RKNNBase.load_tensorflow

……

E load_tensorflow: return func(*args, **kwargs)

E load_tensorflow: File "/usr/local/lib/python3.6/dist-
packages/tensorflow/python/framework/importer.py", line 431, in import_graph_def

E load_tensorflow: raise ValueError(str(e))

E load_tensorflow: ValueError: Node
'MobilenetV2/expanded_conv/depthwise/BatchNorm/cond/Switch_1' expects to be
colocated with unknown node
'MobilenetV2/expanded_conv/depthwise/BatchNorm/moving_mean'
```

Suggestion:

- If TensorFlow 1.x is currently installed, please update to TensorFlow 2.x.
- Update RKNN-Toolkit2/RKNPU2 to the latest version.
- **TransformGraph error**

The error log is as follows:

```
Traceback (most recent call last):

File "test.py", line 80, in <module>

input_size_list=[[1, 368, 368, 3]])

File "/usr/local/lib/python3.6/site-packages/rknn/api/rknn.py", line 68, in
load_tensorflow

input_size_list=input_size_list, outputs=outputs)

File "rknn/api/rknn_base.py", line 940, in
rknn.api.rknn_base.RKNNBase.load_tensorflow

File "/usr/local/lib/python3.6/dist-
packages/tensorflow/tools/graph_transforms/__init__", line 51, in
TransformGraph.transforms_string, status)

File "/usr/local/lib/python3.6/dist-
packages/tensorflow/python/framework/errors_impl.py". ;ome 548, in __exit__

C_api.TF_GetCode(self.status.status)

Tensorflow.python.framework.error_impl.InvalidArgumentError: Beta input to batch
norm has bad shape: [24]
```

Reason:

1)RKNN-Toolkit2 calls TensorFlow's native TransformGraph function for the model optimization but it fails.

2)One probable reason is that the TensorFlow version used for exporting the model is no longer compatible with the currently installed version.

Suggestion:

Use TensorFlow version 1.14.0 to export the model, or find the same type of model in other deep-learning frameworks.

- **"Shape must be rank 4 but is rank 0" error**

When loading the pb model with the command as follows:

```
    rknn.load_tensorflow(tf_pb='./model.pb',

    inputs=["X","Y"],

    outputs=['generator/xs'],

    input_size_list=1, INPUT_SIZE, INPUT_SIZE, 3)
```

Error occurs:

```
    E load_tensorflow: Catch exception when loading tensorflow model: ./model.pb!

    E load_tensorflow: Traceback (most recent call last):

    E load_tensorflow: File "/usr/local/lib/python3.6/dist-
packages/tensorflow/python/framework/importer.py", line 427, in import_graph_def

    E load_tensorflow: graph._c_graph, serialized, options) # pylint:
disable=protected-access

    E load_tensorflow:
tensorflow.python.framework.errors_impl.InvalidArgumentError: Shape must be rank 4
but is rank 0 for 'generator/conv2d_3/Conv2D' (op: 'Conv2D') with input shapes: [],
[7,7,3,32].
```

The reason may be that the model is a multi-input model and the `input_size_list` of `rknn.load_tensorflow()` is not filled in according to the specifications. Please refer to the following usage in `examples/functions/multi_input_test`:

```
    rknn.load_tensorflow(tf_pb='./conv_128.pb',

    inputs=['input1', 'input2', 'input3', 'input4'],

    outputs=['output'],

    input_size_list=[[1, 128, 128, 3], [1, 128, 128, 3],

    [1, 128, 128, 3], [1, 128, 128, 1]])
```

- **Troubleshooting when loading model errors**

First confirm whether the original deep-learning framework can load the model and invoke it correctly.

Secondly, please upgrade RKNN-Toolkit2 to the latest version. If the model has layers (or OPs) that RKNN-Toolkit2 does not support, the debug log should prompt it. If needed, call `RKNN(verbose=True)` to enable more conversion logs.

If it still cannot be solved, please provide the RKNN-Toolkit2 version and the detailed error log to the Rockchip NPU development team.

5.4.5 Model Quantification Issues

- **Quantization effect on model size**

There are two situations. When the imported model is a quantized model, `do_quantization=False` of the `rknn.build()` interface will use the quantization parameters in the model. When the imported model is a floating-point model, `do_quantization=False` will not perform quantization operations, but will convert the weights from float32 to float16, with almost no accuracy loss. Both cases reduce the size of the model size, thereby making the entire model take up less space.

- **When quantizing the model, does the image need to be consistent with the size of the model input?**

Unnecessary.

RKNN-Toolkit2 will automatically resize these images. However, the resize operation may also change the image information and have a certain impact on the quantization accuracy, so it is best to use images matching the input size.

If the correction data is in a non-image format, such as npy format, it needs to be consistent with the shape input by the model.

- **Whether the quantization correction data needs to be modified according to the rknn_batch_size parameter**

Unnecessary.

The `rknn_batch_size` parameter of `rknn.build()` will only modify the batch dimension of the finally exported RKNN model (from 1 to `rknn_batch_size`), and will not affect the process of the quantization stage. Therefore, the quantization correction data should be prepared like a non-batch model.

- **While quantizing model, the conversion program is stuck or killed after a period of time**

During the model quantization process, RKNN-Toolkit2 will allocate partitioned system memory, which may cause the program to be killed or stuck.

Solution: Increase computer memory or increase virtual memory (swap partition).

5.4.6 Model conversion issues

- **Common conversion bug errors**

  If you encounter a conversion error similar to the following, it is likely caused by a bug in the current version. You can try updating RKNN-Toolkit2 to the latest version.

  - **infer_shapes similar error**

```
(op_type:Mul, name:Where_2466_mul): Inferred elem type differs from
existing elem type: (FLOAT) vs (INT64)
E build: Catch exception when building RKNN model!
E build: Traceback (most recent call last):
E build: File "rknn/api/rknn_base.py", line 1555, in
rknn.api.rknn_base.RKNNBase.build
E build: File "rknn/api/graph_optimizer.py", line 5409, in
rknn.api.graph_optimizer.GraphOptimizer.run
E build: File "rknn/api/graph_optimizer.py", line 5123, in
rknn.api.graph_optimizer.GraphOptimizer._fuse_ops
E build: File "rknn/api/ir_graph.py", line 180, in
rknn.api.ir_graph.IRGraph.rebuild
E build: File "rknn/api/ir_graph.py", line 140, in
rknn.api.ir_graph.IRGraph._clean_model
E build: File "rknn/api/ir_graph.py", line 56, in
rknn.api.ir_graph.IRGraph.infer_shapes
E build: File "/home/anaconda3/envs/rk2/lib/python3.6/site-
packages/onnx/shape_inference.py", line 35, in infer_shapes
E build: inferred_model_str = C.infer_shapes(model_str, check_type)
E build: RuntimeError: Inferred elem type differs from existing elem type:
(FLOAT) vs (INT64)
```

or:

```
E build: Traceback (most recent call last):
E build: File "rknn/api/rknn_base.py", line 1643, in
rknn.api.rknn_base.RKNNBase.build
E build: File "rknn/api/graph_optimizer.py", line 6256, in
rknn.api.graph_optimizer.GraphOptimizer.fuse_ops
E build: File "rknn/api/ir_graph.py", line 285, in
rknn.api.ir_graph.IRGraph.rebuild
E build: File "rknn/api/ir_graph.py", line 149, in
rknn.api.ir_graph.IRGraph._clean_model
E build: File "rknn/api/ir_graph.py", line 62, in
rknn.api.ir_graph.IRGraph.infer_shapes
E build: File "/usr/local/lib/python3.6/dist-packages/onnx/shape_inference.py",
line 35, in infer_shapes
E build: inferred_model_str = C.infer_shapes(model_str, check_type)
E build: RuntimeError: Inferred shape and existing shape differ in rank: (0) vs
(3)
```

or:

```
(op_type:ReduceMax, name:ReduceMax_18): Interred shape and existing shape
differ in rank: (3) vs (0)
E build: Catch exception when building RKNN model!
E build: Traceback (most recent call last):
……
E build: RuntimeError: Interred shape and existing shape differ in rank: (3) vs
(0)
```

- **_p_fuse_two_mul similar error**

```
E build: Catch exception when building RKNN model!
E build: Traceback (most recent call last):
E build: File "rknn/api/rknn_base.py", line 1643, in
rknn.api.rknn_base.RKNNBase.build
E build: File "rknn/api/graph_optimizer.py", line 6197, in
rknn.api.graph_optimizer.GraphOptimizer.fuse_ops
E build: File "rknn/api/graph_optimizer.py", line 204, in
rknn.api.graph_optimizer._p_fuse_two_mul
E build: ValueError: non-broadcastable output operand with shape () doesn't
match the broadcast shape (3,2)
```

- **"Segmentation fault" similar error**

For example, an error occurs when converting the picodet model:

```
I _fold_constant remove nodes = ['Shape_0', 'Gather_4', 'Shape_1', 'Gather_6',
'Unsqueeze_0', 'Concat_8', 'Cast_3']
**Segmentation fault (Core dumped)**
```

- **_p_fuse_mul_into_conv similar error**

```
    E build: Catch exception when building RKNN model:
    ……
    E build: ValueError: non broadcastable output operand whith shape
(1,258,1,256) doesn`t match the broadcast shape (80256,258,1,256)
```

- **How to determine whether an operator is supported by RKNN**

  Directly perform model conversion. If it is not supported, a relevant prompt will be displayed.

  You can also refer to 《06_RKNN2_Supported_Operators_and_Constraints_Reference_V2.5.0》, which contains the operator support list of each framework and detailed RKNN operator specification constraints.

- **Prompt that the Expand operator is not supported during conversion**

  Suggestions:

  1) Newer versions already support Expand on CPU. You can try updating RKNN-Toolkit2 / RKNPU2 to the latest version.

  2) Modify the model and use the repeat operator instead of the expand operator.

- **Prompt "Meet unsupported dims in reducesum" during conversion**

  `Meet unsupported dims in reducesum, dims: 6` appears during model conversion, as follows:

```
D RKNN: [14:54:19.434] >>>>>> start: N4rknn17RKNNInitCastConstE
D RKNN: [14:54:19.434] <<<<<< end: N4rknn17RKNNInitCastConstE
D RKNN: [14:54:19.434] >>>>>> start: N4rknn20RKNNMultiSurfacePassE
D RKNN: [14:54:19.434] <<<<<< end: N4rknn20RKNNMultiSurfacePassE
D RKNN: [14:54:19.434] >>>>>> start: N4rknn14RKNNTilingPassE
D RKNN: [14:54:19.434] <<<<<< end: N4rknn14RKNNTilingPassE
D RKNN: [14:54:19.434] >>>>>> start: N4rknn23RKNNProfileAnalysisPassE
D RKNN: [14:54:19.434] <<<<<< end: N4rknn23RKNNProfileAnalysisPassE
D RKNN: [14:54:19.434] >>>>>> start: OpEmit
E RKNN: [14:54:19.438] Meet unsupported dims in reducesum, dims: 6
Aborted (core dumped)
```

Currently, RKNN does not support 6-dimensional OPs. In most cases, only 4 dimensions are supported.

- **Conversion error caused by post-processing Ops such as NonMaxSuppression**
  - Post-processing Ops such as `NonMaxSuppression` are currently not supported by RKNN.
  - The post-processing subgraph of the graph can be removed, for example:

    ```
    rknn.load_onnx(model='picodet_xxx.onnx', outputs=['concat_4.tmp_0',
    'tmp_16'])
    ```

  - The removed subgraph is processed separately on the CPU side.
  - **"invalid expand shape" similar error**

  For example, the following error occurs when converting `rvm_mobilenetv3_fp32.onnx`:

```
[E:onnxruntime:, sequential_executor.cc:333 Execute] Non-zero status code
returned while running Expand node. Name:'Expand_294' Status Message: invalid
expand shape

E build: Catch exception when building RKNN model!
E build: Traceback (most recent call last):
E build: File "rknn/api/rknn_base.py", line 1638, in
rknn.api.rknn_base.RKNNBase.build
E build: File "rknn/api/graph_optimizer.py", line 5529, in
rknn.api.graph_optimizer.GraphOptimizer.fold_constant
E build: File "rknn/api/session.py", line 69, in rknn.api.session.Session.run
E build: File "/home/cx/work/tools/Anaconda3/envs/rknn/lib/python3.8/site-
packages/onnxruntime/capi/onnxruntime_inference_collection.py", line 124, in
run
E build: return self._sess.run(output_names, input_feed, run_options)
E build: onnxruntime.capi.onnxruntime_pybind11_state.InvalidArgument:
[ONNXRuntimeError] : 2 : INVALID_ARGUMENT : Non-zero status code returned while
running Expand node. Name:'Expand_294' Status Message: invalid expand shape
```

Because the input value of `downsample_ratio` changes the size of the intermediate feature in the model, this kind of graph is essentially a dynamic graph. It is recommended to modify the `downsample_ratio` logic of the model, and not use the input value to control the shape of the intermediate feature. If you need to use the dynamic graph function, after updating RKNN-Toolkit2 to 1.5.2, you can use the dynamic shape function to simulate a

dynamic graph (the `downsample_ratio` logic of the model also needs to be modified, and the input value must not be used to control the shape of the intermediate feature; currently the dynamic shape function only supports the case where the input shape is variable).

- **mean_values error prompt of rknn.config()**

  Set `mean/std` to:

  ```
  rknn.config(mean_values=[128, 128, 128], std_values=[128, 128, 128])
  ```

  Then an error occurs when converting the model:

  ```
  --> Loading model
  transpose_input for input_1: shape must be rank 4, ignored
  E load_tflite: The len of mean_values ([128, 128, 128]) for input 0 is wrong,
  expect 32!
  ```

  The reason may be that the model input is not 3-channel image data (for example, the input shape is 1x32, non-image data). In this case:

  - `mean_values` / `std_values` need to be set according to the number of input channels.
  - If the model does not need `mean/std` specified, `rknn.config()` can omit `mean_values` / `std_values` (`mean/std` is generally only effective for image input).
  - **Error when the model contains Ops with more than 4 dimensions (such as 5 or 6 dimensions)**

  When the model contains Ops with more than 4 dimensions (such as 5 or 6 dimensions), the following error occurs:

  ```
  E build: Catch exception when building RKNN model!
  E build: Traceback (most recent call last):
  E build: File "rknn/api/rknn_base.py", line 1580, in
  rknn.api.rknn_base.RKNNBase.build
  E build: File "rknn/api/rknn_base.py", line 341, in
  rknn.api.rknn_base.RKNNBase._generate_rknn
  E build: File "rknn/api/rknn_base.py", line 307, in
  rknn.api.rknn_base.RKNNBase._biild_rknn
  E build: IndexError: vector::_M_range_check: __n (which is 4) >= this->size()
  (which is 4)
  ```

  RKNN currently does not support OPs with more than 4 dimensions. You can remove these nodes manually.

- **Does RKNN support dynamic convolution**

  Currently, the `RK3588/RK3576/RV1126B` platforms support dynamic convolution with the `group` parameter set to 1. Other platforms are not yet supported.

- **"Not support input data type 'float16'" error**

  For a model trained in PyTorch whose weight type is float16, the following error occurs when converting to RKNN:

```
--> Building model
E build: Not support input data type 'float16'
W build: ==================== WARN(3) =================
E rknn-toolkit2 version: 1.3.0-11912b58
E build: Catch exception when building RKNN model!
E build: Traceback (most recent call last):
E build: File "rknn/api/rknn_base.py", line 1638, in
rknn.api.rknn_base.RKNNBase.build
E build: File "rknn/api/graph_optimizer.py", line 5524, in
rknn.api.graph_optimizer.GraphOptimzer.fold_constant
E build: File "rknn/api/load_checker.py", line 63, in
rknn.api.load_checker.create_random_data
E build: File "rknn/api/rknn_log.py", line 113, in rknn.api.rknn_log.RKNNLog.e
E build: ValueError: Not support input data type 'float16'!
```

Currently, RKNN-Toolkit2 does not yet support PyTorch models with float16 weight type; they need to be changed to float32.

- **Dynamic graph related errors**

  When converting a model, if an error similar to the following occurs:

  ```
  E build: ValueError: The Op of 'NonZero' is not support! it will cause the
  graph to be a dynamic graph!
  ```

  It indicates that the model containing this OP is a dynamic graph, and the model needs to be modified manually: replace the OP with another OP or remove it.

- **RKNN model size issue**

  After model conversion is completed, the converted RKNN model may be larger than the original model, and this may even be related to the input shape of the model. This phenomenon is normal, because the RKNN model contains not only weights and graph structure information, but also a lot of NPU register configuration information. In addition, to improve runtime efficiency, operations such as OP decomposition may be performed, all of which cause the RKNN model to become larger.

5.4.7 Description of inference with simulator or device

- **Explanation of terminology**

Simulator inference: RKNN-Toolkit2 provides a simulator function on the Linux x86_64 platform, which can perform model inference and obtain inference results without a development board. (The output result of this function may not be consistent with the board end. It is more recommended to inference with board-connected or on-board).

Inference with board-connected: When the board is connected to the PC, use the RKNN-Toolkit2 Python API to infer the model and obtain the inference results.

On-board inference: using RKNPU2's C API interface to obtain inference results.

- **The simulator inference results are inconsistent with the board-connected**

When this happens, it may mean that the board-side results are incorrect.

Due to hardware and driver differences, the simulator is not guaranteed to get the exact same results as the board. But if the difference is too large, it is likely to be caused by a bug in the on-board driver, and the problem can be fed back to RK's NPU team for analysis and debugging.

- **How inference with board-connected works**

When using inference with board-connected, RKNN-Toolkit2 will communicate with the rknn_server on the board side. During the communication, the model and model input will be transmitted from the PC side to the board side, and then the RKNPU2 C API will be called to perform model inference. After the board-side inference is completed, the results will be sent to PC.

- **Differences between the results of inference with board-connected and inference on board**

Inference with board-connected is implemented based on RKNPU2 C API. Theoretically, the results will be consistent with the RKNPU2 C API inference results. When there is a big difference between the two, please confirm whether there is any difference in the input preprocessing, data type, and data arrangement (`NCHW` , `NHWC`).

It should be pointed out that if the difference is very small, it is a normal phenomenon. Differences may occur in using different libraries to read images, convert data types, etc.

- **Inference on-board is faster than inference with board-connected**

Since inference with board-connected involves additional data copying and transmission processes, the performance is not as good as the on-board inference with RKNPU2 C API. Therefore, the actual inference performance of NPU is subject to the on-board inference with RKNPU2 C API.

- **Getting the log with more detailed**

When inference with board-connected, the initialization and inference of the model are mainly completed on the development board, and the log information is mainly generated on the board end.

In order to obtain specific board-side debugging information, you can enter the development board operating system through the serial port. Then execute the following two commands to set the environment variables for obtaining logs. Keep the serial port window not closed, and then perform inference with board-connected. At this time, the error message on the board side will be displayed on the serial port window:

```
export RKNN_LOG_LEVEL=5

restart_rknn.sh
```

- **Explanation of Data Layout for Batch Inference**

The input data layout for `rknn.inference` during inference can be configured as either NCHW or NHWC, specified by `1data_format='NCHW'` or `data_format='NHWC'` . The default is NHWC.

In rknn-toolkit2, the ONNX models with 4D inputs are typically in NCHW format by default during conversion. However, when performing inference using RKNN, the default input layout is NHWC, and the toolkit internally converts data from NCHW to NHWC. If data_format='NCHW' is explicitly set during inference, this default conversion is skipped, and the input data is used as-is in NCHW format.For specific examples, refer to the discussion in this GitHub issue“https://github.com/airockchip/rknn-toolkit2/issues/220”.

5.4.8 Common issue about model evaluation

- **Quantized model accuracy is not as good as expected**

  Refer to Chapter 5.1 of this document.

- **Which frameworks of quantized models are supported**

  RKNN-Toolkit2 1.4 and later versions support quantized models from the TensorFlow, TensorFlow Lite and PyTorch frameworks.

- **Failed to connect to the device during board-connected debugging**

  The following error occurs during board-connected accuracy analysis (`rknn.accuracy_analysis()`):

  ```
  E accuracy_analysis: Connect to Device Failure (-1)
  E accuracy_analysis: Catch exception when init runtime!
  E accuracy_analysis: Traceback (most recent call last):
  E accuracy_analysis: File "rknn/api/rknn_base.py", line 2001, in
  rknn.api.rknn_base.RKNNBase.init_runtime
  E accuracy_analysis: File "rknn/api/rknn_runtime.py", line 194, in
  rknn.api.rknn_runtime.RKNNRuntime.__init__
  E accuracy_analysis: File "rknn/api/rknn_platform.py", line 331, in
  rknn.api.rknn_platform.start_ntp_or_adb
  ```

  Or the following error occurs during board-connected inference (`rknn.inference`):

  ```
  I target set by user is: rk3568
  I Starting ntp or adb, target is RK3568
  I Device [0c6a9900ef4871e1] not found in ntb device list.
  I Start adb…
  I Connect to Device success!
  I NPUTransfer: Starting NPU Transfer Client, Transfer version 2.1.0
  (b5861e7@2020-11-23T11:50:36)
  D NPUTransfer: Transfer spec = local:transfer_proxy
  D NPUTransfer: ERROR: socket read fd = 3, n = -1: Connection reset by peer
  D NPUTransfer: Transfer client closed fd = 3

  E RKNNAPI: rknn_init, server connect fail! ret = -9(ERROR_PIPE)!
  E init_runtime: Catch exception when init_runtime!
  E init_runtime: Traceback (most recent call last):
  E init_runtime: File "rknn/api/rknn_base.py", line 2001, in
  rknn.api.rknn_base.RKNNBase.init_runtime
  E init_runtime: File "rknn/api/rknn_runtime.py", line 361, in
  rknn.api.rknn_runtime.RKNNRuntime.build_graph
  E init_runtime: Exception: RKNN init failed. error code:
  RKNN_ERR_DEVICE_UNAVAILABLE
  ```

The cause may be that the RKNN Server service is not started on the board. Please run the RKNN Server service on the board according to the relevant instructions in 1.2.2.

- **During board-connected debugging, rknn_init fails and returns -6 or a model invalid error**

  The error message is as follows:

```
E RKNNAPI: rknn_init, msg_load_ack fail, ack = 1(ACK_FAIL), expect 0(ACK_SUCC)!
D NPUTransfer: Transfer client closed, fd = 4
E init_runtime: Catch exception when init runtime!
E init_runtime: Traceback (most recent call last):
E init_runtime: File "rknn/api/rknn_base.py", line 2011, in
rknn.api.rknn_base.RKNNBase.init_runtime
E init_runtime: File "rknn/api/rknn_runtime.py", line 361, in
rknn.api.rknn_runtime.RKNNRuntime.build_graph
E init_runtime: Exception: RKNN init failed. error code: RKNN_ERR_MODEL_INVALID
```

  This error generally occurs in the following situations:

  - When generating the RKNN model, different versions of RKNN-Toolkit2 and drivers have corresponding relationships. It is recommended to upgrade RKNN-Toolkit2 / RKNPU2 and the firmware of the development board to the latest versions.
  - `target_platform` is not set correctly. For example, when `target_platform` in the `rknn.config()` interface is not set, the generated RKNN model can only run on RK3566/RK3568. To run on other platforms (such as RK3588/RK3588S/RV1103/RV1106/RK3562), you need to set the corresponding `target_platform` when calling the `rknn.config()` interface.
  - If this problem occurs during inference in a Docker container, it may be because the `npu_transfer_proxy` process on the host was not terminated, causing communication exceptions. You can exit the Docker container first, terminate the `npu_transfer_proxy` process on the host, and then enter the container again to execute the inference script.
  - There may also be a problem with the RKNN model itself. In this case, you can connect to the development board via the serial port, execute `restart_rknn.sh` after setting the environment variable `RKNN_LOG_LEVEL=5`, then rerun the program, record the detailed logs generated, and feed them back to the Rockchip NPU team.
  - **During board-connected debugging, rknn_init() fails and returns a device unavailable error**

  The error message is as follows:

```
E RKNNAPI: rknn_init, msg_ioctl_ack fail, data_len = 104985, except 102961!
D NPUTransfer: Transfer client closed, fd = 3
E init_runtime: Catch exception when init_runtime!
E init_runtime: Traceback (most recent call last):
E init_runtime: File "rknn/api/rknn_base.py", line 1961, in
rknn.api.rknn_base.RKNNBase.init_runtime
E init_runtime: File "rknn/api/rknn_runtime.py", line 360, in
rknn.api.rknn_runtime.RKNNRuntime.build_graph
E init_runtime: Exception: RKNN init failed. error code:
RKNN_ERR_DEVICE_UNAVAILABLE
```

The cause of this problem is rather complex. Please troubleshoot as follows:

Make sure that RKNN-Toolkit2 / RKNPU2 and the system firmware of the development board have been upgraded to the latest versions. For how to query the version of each component, refer to 1.2.2.

In addition, make sure the device is visible in both `adb devices` and `rknn.list_devices()`, and that `target` and `device_id` of `rknn.init_runtime()` are set correctly.

- **"Invalid RKNN model version 6" error occurs in Runtime**

  Runtime reports the following error:

  ```
  Loading model …
  E RKNN: [09:13:25.728] 6, 1
  E RKNN: [09:13:25.728] Invalid RKNN model version 6
  E RKNN: [06:28:39.049] rknn_init, load model failed!
  Exception: RKNN init failed. error code: RKNN_ERR_FAIL
  ```

  Cause:

  The model version is incompatible with the RKNN Runtime.

  Suggestion:

  Refer to the version compatibility in section 16.1 to upgrade the corresponding model and Runtime versions, or directly upgrade both to the latest version at the same time.

- **"Invalid RKNN format" error occurs in Runtime**

  Runtime reports the following error:

  ```
  Loading model …
  E RKNN: [06:28:39.048] parseRKNN from buffer: Invalid RKNN format!
  E RKNN: [06:28:39.049] rknn_init, load model failed!
  rknn_init error ret=-1
  ```

  Cause:

  1) The `target_platform` of `rknn.config()` during model conversion may be set incorrectly, or not set (if not set, the default is RK3566).

  2) The Runtime version is incompatible with RKNN-Toolkit2.

  Suggestion:

1) Set the correct `target_platform`.

2) RKNN-Toolkit2 and Runtime should be updated together to the same version.

- **The elapsed time of rknn.inference() is inconsistent with the theoretical speed of rknn.eval_perf()**

  Because `rknn.inference()` performs board-connected inference via PC + adb, there is some fixed data transmission overhead, so it is inconsistent with the theoretical speed of `rknn.eval_perf()`.

  For a more realistic frame rate, it is recommended to test directly on the development board using the RKNPU2 C API.

- **Multi-batch support of rknn.inference()**

  In RKNN-Toolkit2 1.4.0 and later versions, the number of input images can be specified when building the RKNN model. For detailed usage, refer to the description of the `rknn.build` interface in the RKNN-Toolkit2 API manual.

  In addition, when `rknn_batch_size` is greater than 1 (e.g. equal to 4), the inference call in Python should be changed from:

  ```
  outputs = rknn.inference(inputs=[img])
  ```

  to:

  ```
  img = np.expand_dims(img, 0)
  img = np.concatenate((img, img, img, img), axis=0)
  outputs = rknn.inference(inputs=[img])
  ```

  For a complete example, please refer to: `examples/functions/multi_batch/`

- **Running multiple RKNN models**

  When running two or more models, you need to create multiple RKNN objects. One RKNN object corresponds to one model, similar to a context. Each model initializes the model, performs inference, and obtains the inference results in its own context, without interfering with each other. These models run serially on the NPU.

- **Model inference takes a very long time and the results are wrong**

  If the inference takes more than 20s and the results are wrong, this is usually an NPU Hang bug in the NPU. If you encounter this problem, you can try updating RKNN-Toolkit2 / RKNPU2 to the latest version.

- **Board-connected inference results are wrong when the model input is 3-dimensional**

  When the model input is 3-dimensional, it may happen that the Simulator simulation results are correct while the board-connected inference results are wrong. The cause may be that the current NPU's support for 3-dimensional input is not yet complete; 3-dimensional support will be improved later.

  Suggestion:

  - Change the model input to 4-dimensional first.

- Try updating RKNN-Toolkit2 / RKNPU2 to the latest version.
- **Board-connected inference results are wrong and inconsistent each time**

  After an ONNX model is converted to RKNN, the Simulator simulation results are correct and consistent every time, but the board-connected inference results are wrong and inconsistent each time. This kind of problem may be caused by a bug in the NPU kernel driver on the board. In this case, the NPU kernel driver on the board needs to be updated, and the latest RKNN-Toolkit2 / RKNPU2 needs to be updated as well.

- **Accuracy degradation when the model contains many Resize OPs**

  When there are many Resize OPs in the ONNX model, accuracy degrades after conversion to RKNN. Possible causes are:

  1) The accuracy degradation is because the NPU does not currently support hardware-level `Resize` (it will be supported later), and the conversion tool converts `Resize` to `ConvTranspose`, which causes a slight loss of accuracy.

  2) If the model has multiple `Resize` ops in series, too many accumulated errors may cause a larger accuracy drop.

  Suggestion:

  1) At present, try to avoid the use of `Resize` (e.g. change `Resize` to `ConvTranspose` before training)

  2) You can add the `optimization_level=2` parameter to `rknn.config()`. In this case the `Resize Op` runs on the CPU and the accuracy will not drop, but performance will degrade.

- **Inference results are all nan after do_quantization is set to False**

  When `do_quantization` in the `rknn.build()` interface is set to `True`, the inference results are normal, but after it is set to `False`, all inference results become `nan`. The cause may be that when `do_quantization=False`, the computation type of the RKNN model is fp16, but the output range of the model's intermediate layers (such as convolution) may exceed the range of fp16 (65536) (e.g. -51597~75642).

  Suggestion:

  During training, make sure the outputs of the intermediate layers do not exceed the representable range of fp16 (this problem is generally solved by adding a BN layer).

- **The results of the QAT model and the RKNN model are inconsistent**

  A classification model was trained with QAT under the PyTorch framework and converted to an RKNN model. Inferring this model with PyTorch and RKNN respectively gives different results. The cause may be that the PyTorch inference did not set `engine='qnnpack'`, because the RKNN inference method is closer to `qnnpack`.

- **How to get the memory usage when the model is running**

  You can use the `rknn.eval_memory()` interface. There is a Total item in the output log, which is the total memory usage.

- **Difference in performance data when the perf_debug parameter of rknn.init_runtime() is enabled or disabled during performance evaluation**

  When `perf_debug` is enabled, some debugging code is added to collect the information of each layer, and some parallel mechanisms may be disabled, so the elapsed time is longer than when `perf_debug=False`.

The main purpose of enabling `perf_debug` is to check whether the model has layers that account for a large proportion of the elapsed time, and use this as the basis for designing an optimization plan.

- **The environment uses docker. Board-connected inference worked before, but after restarting docker, inference gets stuck at the environment initialization stage?**

  Because when docker restarts, `npu_transfer_proxy` is left in a state similar to an abnormal exit, the RKNN Server on the development board cannot detect that the host connection has been disconnected. In this case, you need to restart the development board to reset the connection state of the RKNN Server.

5.4.9 C API usage FAQ

- **Will rknn_outputs_release() release the rknn_output array?**

`rknn_outputs_release()` is called in conjunction with `rknn_outputs_get()`. It only releases the buffer in the `rknn_output` array. A similar situation exists with `rknn_destroy_mem()`.

- **How does rknn_create_mem create memory with appropriate size?**

For input, the general principle is: if it is a quantized RKNN model, `rknn_create_mem()` uses the `size_with_stride` of `rknn_tensor_attr` to allocate memory; if it is a non-quantized model, `rknn_create_mem()` uses the `n_bytes_type` * `n_elems` to allocate memory.

For output, `rknn_create_mem()` uses the `n_bytes_type` * `n_elems` to allocate memory.

- **How to fill in the input data?**

If you use the general API, for 4-dimensional input, fmt is `NHWC`, that is, the data filling order is [batch, height, width, channel]. For non-4D input, fmt is `UNDEFINED`, which means the data is filled according to the original shape of the model.

If using the zero-copy API, for 4D input, fmt can be `NHWC/NC1HWC2`. For how to fill data when `fmt=NC1HWC2`, please refer to the chapter "RKNN Runtime Zero Copy". For non-4D input, fmt is `UNDEFINED`, which means the data is filled according to the original shape of the model.

- **How to use pass_through?**

The input data format is obtained by the `RKNN_NATIVE_INPUT_ATTR` command of `rknn_query()`. If it is a 4-dimensional shape: For channel values 1, 3, and 4, the layout requires using `NHWC`, and other channel values require using `NC1HWC2`; if it is a non-4-dimensional shape, it is recommended to specify the layout as `UNDEFINED`.

In addition, in pass_through mode, quantized models usually specify the dtype of the input Tensor as INT8, and non-quantized models usually specify the dtype of the input Tensor as FLOAT16.

- **How to deal with the "failed to submit" error?**

If the error occurs in the first layer of convolution and the zero-copy interface is used, the possible cause is insufficient memory allocation for the input tensor. In this case, `size_with_stride` in the tensor attribute should be used to allocate memory.

If the error occurs in the middle NPU layer, the possible reason is that the model configuration is wrong. At this time, you can find the latest SDK link in the error log. It is recommended to upgrade the latest toolchain or assign this layer to the CPU when converting the RKNN model.

- **How to deal with the "Meet unsupport xxx operator" error?**

When a similar error occurs when running the demo on the board end, it is usually because the runtime (librknnrt.so) of the board end does not support this operator. It is recommended that users update the RKNN related toolchain to the latest version first, then convert the model again, and rerun the demo on the board.

If the same error still occurs in the latest toolchain, the user needs to add the implementation of the operator by himself. You can refer to Chapter 4.2.1 to customize the implementation of the operator, or report it to the RKNN team through redmine.

- **Whether the dynamic shape model supports the use of externally allocated memory in the zero-copy process**

Versions before 1.6.0 do not support it. Later versions start to support using the `RKNN_FLAG_MEM_ALLOC_OUTSIDE` flag to initialize the context.

- **Performance accessment for Custom OP on runtime**

This can be achieved via setting `RKNN_LOG_LEVEL=4` or above, then running the testing c api demo, which will print out the performance of each layer including the custom op.

Note: The performance time for custom op involves things need to do during the compute call back function. For example, custom GPU ops would include the time cost for `clImportMemoryARM` function, converting for data layout and data type (both inputs and outputs) and the final GPU OpenCL kernel running time.

5.5 Related Resources

RKNN: https://github.com/airockchip/rknn-toolkit2

RKNN_Model_Zoo: https://github.com/airockchip/rknn_model_zoo

RGA: https://github.com/airockchip/librga